

CINEVERSE

Documentación técnica

Tecnologías

Vue 3 · TypeScript · PHP · MySQL 8 · TMDB API

Documento técnico del proyecto

Autor: Gastón Rizzo

Índice

1. Descripción general de CineVerse	3
2. Tecnologías y herramientas utilizadas	4
3. TMDB: qué es, cómo se obtiene el acceso y cómo lo utiliza CineVerse	6
4. Cómo está organizada CineVerse	14
5. Estructura del proyecto	16
6. Archivos de configuración y variables de entorno	19
7. Base de datos MySQL 8	29
8. Datos de prueba y archivo SQL	32
9. Cómo recorre CineVerse una acción del usuario	34
10. TypeScript en CineVerse	36
11. Cómo está organizado el frontend con Vue	38
12. Cómo se guardan y reutilizan datos en el frontend	41
13. Navegación sin recargar la página: SPA y Vue Router	49
14. Idiomas: vue-i18n y cambio de idioma	54
15. Comunicación entre el frontend, TMDB y el backend	60
16. Explorador de películas y sistema de búsqueda	70
17. Cómo se guardan los filtros y se reflejan en la URL	80
18. Cómo se cargan más películas	81
19. Página de inicio	82
20. Detalle de película	85
21. Trailers, películas similares e imágenes	86
22. Actores y filmografía	88
23. Favoritos	90
24. Cómo está organizado el backend PHP	92
25. Autenticación y cuenta	101
26. Reseñas y perfil	113
27. Validación y seguridad	120
28. Estados de la interfaz y manejo de errores	122
29. Logs: registro y diagnóstico de errores	135
30. Instalación local de CineVerse	141
31. Cómo ejecutar CineVerse en desarrollo	146
32. Cómo publicar CineVerse en un servidor	153
33. Pruebas y verificación del proyecto	156

1. Descripción general de CineVerse

CineVerse es una aplicación web dedicada a brindar información sobre películas y actores. Permite buscar películas, aplicar filtros, consultar información detallada sobre cada título, conocer su reparto y director, ver trailers y acceder a la información y filmografía de los actores.

Además de consultar información, el usuario puede guardar películas como favoritas. También puede crear una cuenta para publicar reseñas, editar o eliminar sus propias reseñas y consultar su actividad desde el perfil. La información sobre películas y actores se obtiene de TMDB, mientras que los datos propios de CineVerse, como usuarios, intentos de inicio de sesión y reseñas, se guardan en MySQL 8.

Idea central

CineVerse no guarda en MySQL una copia completa de las películas y los actores que muestra. Esa información se consulta en TMDB cuando hace falta. MySQL se utiliza para los datos que pertenecen a CineVerse: cuentas de usuario, verificación de correo, recuperación de contraseña, intentos de inicio de sesión y reseñas.

1.1 Qué puede hacer el usuario

- Explorar tendencias, películas en cartelera, próximas, populares y mejor valoradas.
- Buscar por texto y combinar la búsqueda con género, año, puntuación y orden.
- Consultar el detalle de una película, su reparto, director, trailer y películas similares.
- Abrir fichas de actores y recorrer su filmografía.
- Guardar hasta 20 favoritos en el navegador sin necesidad de iniciar sesión.
- Registrarse, verificar el correo, iniciar/cerrar sesión y recuperar la contraseña.
- Crear una única reseña por película, editarla, eliminarla y consultar la actividad del perfil.

1.2 Cómo funciona a grandes rasgos

CineVerse tiene una parte que se ejecuta en el navegador y otra que se ejecuta en el servidor. La parte visible, llamada frontend, está desarrollada con Vue 3 y TypeScript. Allí se muestran las pantallas, los formularios, los filtros, las tarjetas de películas y la navegación.

El backend es el código PHP que se ejecuta en el servidor. Se ocupa de las funciones que necesitan datos propios de CineVerse: registro, inicio y cierre de sesión, verificación de correo, recuperación de contraseña, perfil y reseñas. Cuando necesita guardar o leer esos datos, se comunica con MySQL.

Una API es una forma de comunicación entre programas. CineVerse utiliza dos: la API externa de TMDB para obtener información de películas y actores, y su propia API PHP para conectarse al servidor y trabajar con cuentas y reseñas. El frontend envía solicitudes a una u otra según la información que necesite.

El frontend está desarrollado como una SPA. SPA significa Single Page Application, o aplicación de una sola página. El nombre no significa que CineVerse tenga una sola pantalla: significa que el navegador carga la aplicación inicialmente y, al pasar por ejemplo de Inicio a Películas, al detalle de una película o a un actor, Vue cambia el contenido sin volver a descargar una página HTML completa. Vue Router mantiene la URL sincronizada con la sección que se está visitando.

1.3 Conceptos básicos para leer esta documentación

Antes de continuar conviene fijar un vocabulario mínimo. Estas palabras aparecen muchas veces en los recorridos del proyecto; aquí se explican en forma breve y las secciones posteriores muestran su funcionamiento con mayor profundidad.

Término	Qué significa en CineVerse
API	Forma de comunicación entre programas. CineVerse usa la API externa de TMDB y su propia API PHP.
Endpoint	Ruta concreta dentro de una API que representa una operación o recurso. Ejemplo: /movie/550/videos pide los videos de la película 550.
Request / solicitud	Mensaje HTTP que el frontend envía a una API para pedir información o enviar datos.
Response / respuesta	Mensaje que la API devuelve después de procesar una solicitud; en CineVerse normalmente contiene JSON.
JSON	Formato de texto estructurado en propiedades y valores que CineVerse utiliza para intercambiar datos con APIs.
Query parameter	Dato que viaja en la URL después de ?. Ejemplo: page=2&language=es.
View	Pantalla completa de Vue asociada normalmente a una ruta, por ejemplo MovieDetailView.vue.
Component	Parte reutilizable de una pantalla, por ejemplo MovieHero.vue o MovieCard.vue.
Composable	Función de Vue que reúne lógica reactiva reutilizable. Puede coordinar estado, Vue Query y services, pero no dibuja por sí sola una pantalla.
Service del frontend	Módulo TypeScript que concentra operaciones de acceso a datos y conoce qué endpoint debe utilizar, por ejemplo movies.service.ts.
Store	Estado compartido entre distintas partes del frontend, por ejemplo el usuario autenticado o los favoritos.

No confundir los dos usos de “service”. Un archivo *.service.ts pertenece al frontend y realiza operaciones de acceso a datos por HTTP. Una clase *Service.php pertenece al backend y aplica reglas de negocio. Comparten el término service, pero viven en capas diferentes.

```

Carlos abre una película
-> MovieDetailView.vue      [View]
-> useMovieDetail()         [Composable]
-> getMovieVideos()         [operación del Service]
-> movies.service.ts        [Service del frontend]
-> GET /movie/550/videos    [Endpoint]
-> TMDB                     [API]
-> JSON                     [Response]
-> Vue muestra el resultado

```

2. Tecnologías y herramientas utilizadas

CineVerse utiliza varias tecnologías porque cada parte del proyecto tiene necesidades diferentes. Vue construye la interfaz que ve el usuario; TypeScript ayuda a controlar la forma de los datos; PHP implementa la API propia; MySQL guarda la información de cuentas y reseñas; y TMDB proporciona la información de películas y actores. Las herramientas de esta sección se explican por su función real dentro del proyecto.

Tecnología	Uso real en CineVerse
Vue 3	Construye la interfaz que se ejecuta en el navegador. Permite dividir la pantalla en componentes y actualizar sólo las partes que cambian.
TypeScript 5.9	Añade tipos a JavaScript. CineVerse lo usa para describir, por ejemplo, la forma de una película, una respuesta de TMDB, un usuario o los datos enviados al backend.

Tecnología	Uso real en CineVerse
Vite 7	Inicia el servidor de desarrollo del frontend y genera los archivos optimizados que se publican en producción.
Vue Router	Relaciona cada URL con una pantalla de Vue y permite navegar por la SPA sin recargar un HTML completo.
Pinia	Guarda información que debe estar disponible desde distintas partes del frontend, como el usuario autenticado y los favoritos.
TanStack Vue Query	Organiza las solicitudes de datos que se realizan sin detener la interfaz, reutiliza resultados temporales y controla estados de carga, error y paginación.
Axios	Envía solicitudes HTTP, es decir, mensajes que el frontend manda a una API para pedir o enviar información. CineVerse crea un cliente para TMDB y otro para su propio backend.
vue-i18n	Gestiona los textos de la interfaz en español e inglés.
Tailwind CSS 4	Está instalado y conectado al proceso de Vite, pero no es el sistema principal de estilos. La interfaz de CineVerse se construye principalmente con CSS propio y estilos definidos dentro de componentes Vue.
PHP	Implementa la API propia de CineVerse. No se utiliza Laravel ni otro framework.
PDO	Es la interfaz de PHP utilizada para conectarse con MySQL. Permite ejecutar consultas SQL preparadas, separando la instrucción de los valores recibidos.
MySQL 8	Guarda usuarios, reseñas e intentos de inicio de sesión y aplica reglas para impedir datos inválidos o relaciones inconsistentes.
TMDB API v3	Servicio externo del que se obtiene información de películas y actores: títulos, sinopsis, géneros, reparto, créditos, videos e imágenes.
Monolog	Escribe en archivos los errores técnicos del backend que necesitan diagnóstico.
PHPMailer	Envía los correos de verificación de cuenta y recuperación de contraseña mediante un servidor de correo SMTP.

2.1 Librerías de interfaz relevantes

Paquete	Responsabilidad
pinia-plugin-persistedstate	Extiende Pinia para conservar el store de favoritos en localStorage y restaurarlo después de recargar el navegador.
swiper	Proporciona el motor de carrusel utilizado en la Home. CineVerse combina SwiperSlide, navegación, autoplay y transiciones con lógica propia de carga y estado visual.
vue-multiselect	Proporciona selectores enriquecidos para los filtros, especialmente cuando el usuario necesita elegir opciones como géneros.

Paquete	Responsabilidad
vue-slider-component	Proporciona los controles deslizables utilizados para seleccionar rangos de año y puntuación sin construir esos controles desde cero.
vuedraggable	Permite reordenar mediante arrastrar y soltar los paneles del sistema de filtros; el nuevo orden puede persistirse en el navegador.
lucide-vue-next	Proporciona iconos como componentes Vue reutilizables dentro de botones, avisos y controles de la interfaz.

Estas dependencias no cumplen todas el mismo papel. Vue Router, Pinia, TanStack Vue Query y vue-i18n se integran con la aplicación como infraestructura transversal; pinia-plugin-persistedstate extiende un store de Pinia; Axios se ocupa de las solicitudes HTTP; y Swiper, vue-multiselect, vue-slider-component, vuedraggable y lucide-vue-next resuelven necesidades concretas de la interfaz. La sección 12 explica en profundidad cómo se utiliza cada una dentro de CineVerse.

La instalación actual también contiene @vueuse/core y @vueuse/motion, pero en la versión de código revisada no aparecen importadas dentro de src. Por eso no se presentan como parte funcional de CineVerse. Del mismo modo, Tailwind CSS está conectado a Vite, pero la interfaz está implementada principalmente con CSS propio y estilos de componentes; no es el sistema principal de estilos del proyecto.

2.2 Variables de entorno

CineVerse necesita algunos datos de configuración que no conviene escribir directamente dentro del código. Por ejemplo, el frontend necesita saber dónde está la API de TMDB y cómo llegar al backend; el backend necesita los datos de MySQL y del servidor de correo. Estos valores se guardan en archivos .env. La sección 6 explica dónde están esos archivos, qué contiene cada uno y cómo los leen Vite y PHP.

3. TMDB: qué es, cómo se obtiene el acceso y cómo lo utiliza CineVerse

CineVerse depende de The Movie Database para la información audiovisual, pero TMDB no forma parte del código del proyecto ni se instala como una librería. Es un servicio externo en Internet. Esta sección empieza desde ese punto y sigue el recorrido completo: qué es TMDB, cómo obtiene acceso un desarrollador, qué credencial utiliza CineVerse, qué archivos construyen las solicitudes y cómo la respuesta JSON termina convertida en interfaz Vue.

3.1 Qué es The Movie Database (TMDB)

TMDB significa The Movie Database. Su API permite que otras aplicaciones consulten automáticamente información de películas, series y personas mediante solicitudes HTTP, sin necesidad de que una persona entre al sitio web de TMDB y busque esos datos manualmente. CineVerse utiliza la API v3 para pedir a TMDB la información que necesita en cada pantalla, mientras MySQL conserva únicamente los datos propios de CineVerse, como cuentas, intentos de inicio de sesión y reseñas.

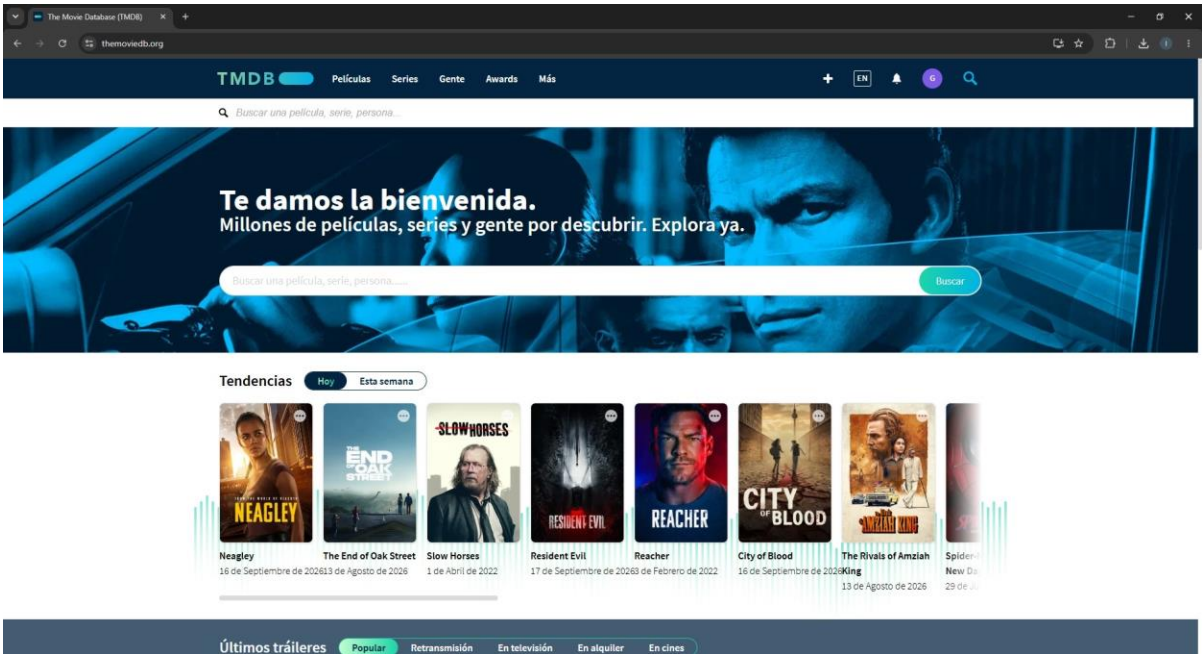


Figura. Sitio público de The Movie Database (TMDB). CineVerse no copia esta página: consulta programáticamente la API que TMDB ofrece a desarrolladores.

Conviene distinguir tres direcciones relacionadas. `www.themoviedb.org` es el sitio web que visita una persona; `api.themoviedb.org` es la API a la que CineVerse envía solicitudes HTTP; e `image.tmdb.org` es la infraestructura utilizada para servir pósters, backdrops y otras imágenes. Son partes del mismo ecosistema, pero cumplen funciones diferentes.

Dirección	Función en CineVerse
<code>www.themoviedb.org</code>	Sitio público de TMDB y configuración de la cuenta del desarrollador.
<code>api.themoviedb.org/3</code>	API v3: recibe solicitudes y devuelve datos en JSON.
<code>image.tmdb.org</code>	Servidor/CDN desde el que el navegador descarga imágenes.

3.2 Qué información obtiene CineVerse de TMDB

TMDB aporta el catálogo externo: títulos, sinopsis, fechas, géneros, puntuaciones, pósters, fondos, reparto, equipo técnico, videos, películas similares y datos de personas. CineVerse selecciona sólo lo que necesita. Por ejemplo, una ficha visual puede combinar el detalle de la película con créditos y videos obtenidos mediante endpoints diferentes.

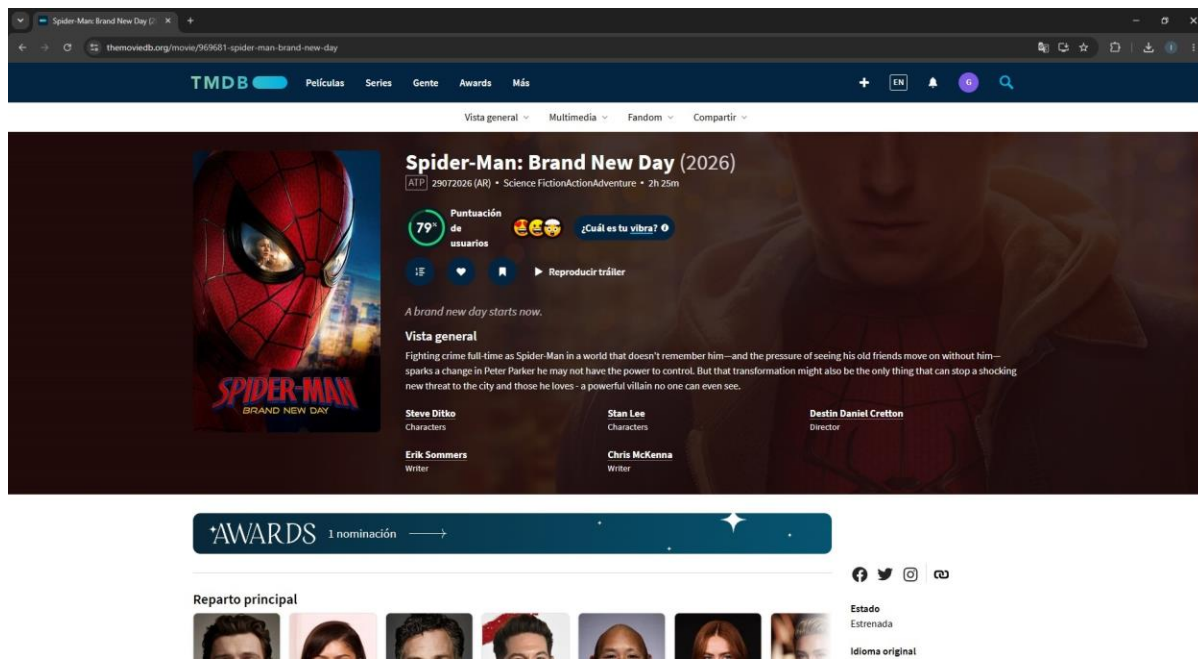


Figura. Ejemplo de una ficha de película en el sitio público de TMDB. La API expone los datos que CineVerse utiliza para construir su propia interfaz; no copia esta página HTML.

La separación con MySQL es intencional: CineVerse no mantiene una tabla movies con una copia completa del catálogo. Una reseña local sólo guarda movie_id y movie_title como referencia suficiente para relacionar la actividad del usuario con una película de TMDB.

3.3 Cómo obtiene acceso un desarrollador

El acceso a la API se administra desde una cuenta de TMDB. Según la documentación oficial, el desarrollador inicia sesión, entra en Account Settings y abre la sección API. Antes de recibir las credenciales debe aceptar las condiciones de uso correspondientes.

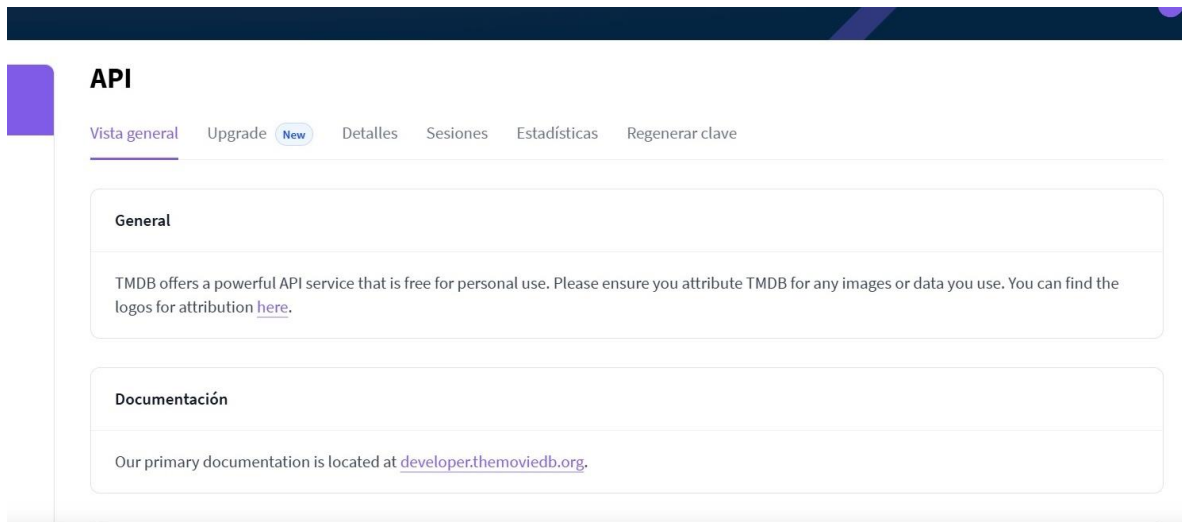


Figura. Sección API de la configuración de una cuenta de TMDB. Desde aquí se administra el acceso para desarrolladores y se accede a la documentación oficial.

Este paso pertenece a la preparación del entorno, no a la ejecución de Vue. Una vez obtenida la credencial, se copia a frontend/.env y Vite la expone al código del frontend mediante import.meta.env.

3.4 API Read Access Token y API Key

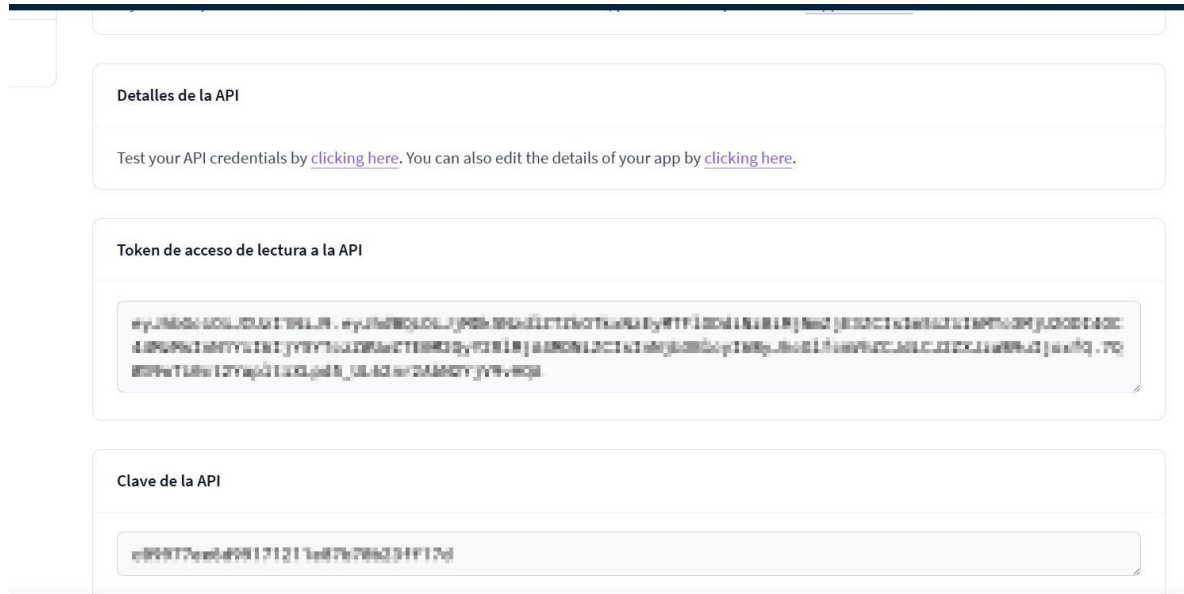


Figura. Credenciales de TMDb. La captura muestra por separado el API Read Access Token y la API Key; los valores sensibles permanecen censurados.

Para APIv3, TMDb admite dos formas de autenticación a nivel de aplicación. La API Key puede enviarse mediante el parámetro `api_key` de la URL. El API Read Access Token puede enviarse en la cabecera Authorization como Bearer. La documentación oficial considera el Bearer token el método predeterminado y señala que el mismo token puede utilizarse también con métodos v4.

Credencial	Cómo se envía	Uso en CineVerse
API Read Access Token	Authorization: Bearer <token>	Sí. Es el valor de VITE_TMDB_TOKEN.
API Key v3	?api_key=<clave>	No en el código de CineVerse; se utiliza sólo en algunas capturas de prueba manual.

Importante. VITE_TMDB_TOKEN no se coloca después de `api_key=`. En CineVerse ese valor es el API Read Access Token y `tmdbApi.ts` lo envía como cabecera Bearer.

3.5 Cómo se configura TMDb dentro de CineVerse

La credencial atraviesa una cadena concreta. El desarrollador la obtiene en TMDb, la guarda en `frontend/.env`, Vite la expone mediante `import.meta.env` y `tmdbApi.ts` crea una instancia Axios que reutilizan los services del proyecto.

API Read Access Token obtenido en TMDb

```
-> frontend/.env
-> VITE_TMDB_TOKEN
-> Vite / import.meta.env
-> tmdbApi.ts
-> Authorization: Bearer <token>
-> api.themoviedb.org/3
```

```
# frontend/.env
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"

// frontend/src/core/api/tmdbApi.ts
const baseUrl = import.meta.env.VITE_TMDB_BASE_URL;
const token = import.meta.env.VITE_TMDB_TOKEN;
```

```
const api = axios.create({
  baseURL,
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});
```

axios.create() no envía todavía una solicitud. Sólo construye un cliente reutilizable. tmdbApi.ts recuerda la dirección base y la credencial; no decide si se va a buscar Batman, cargar una persona o pedir las películas mejor valoradas.

Seguridad de la credencial en este diseño. Las variables VITE_* utilizadas por el frontend quedan incorporadas al JavaScript generado por Vite y pueden observarse desde el navegador. Por eso VITE_TMDB_TOKEN no debe tratarse como un secreto de servidor. Si una credencial necesitara permanecer privada, tendría que utilizarse desde una capa del lado del servidor.

3.6 Qué archivos hablan realmente con TMDB

CineVerse no tiene una clase llamada TMDB. La integración se reparte entre un cliente HTTP compartido, módulos TypeScript que conocen los endpoints y composables que coordinan los datos para las pantallas.

```
View / componente
-> composable
-> movies.service.ts / person.service.ts
-> tmdbApi.ts
-> Axios
-> HTTPS
-> TMDB API
-> JSON
-> Vue Query / composable
-> interfaz
```

Pieza	Responsabilidad
tmdbApi.ts	Crea el cliente Axios con baseURL y Authorization: Bearer.
movies.service.ts	Conoce los endpoints de películas: listas, búsqueda, Discover, géneros, detalle, créditos, videos y similares.
person.service.ts	Consulta personas y filmografía.
composables	Deciden cuándo pedir datos, coordinan Vue Query y preparan la información para la vista.
Views / componentes	Representan visualmente el resultado y reaccionan a las acciones del usuario.

Por ejemplo, useHomeMovies crea tres consultas independientes para tendencias, mejor valoradas y cartelera. useMoviesQuery elige entre listas, búsqueda textual y Discover. useMovieDetail combina detalle, créditos, videos y similares. usePersonDetail obtiene la información de una persona y su filmografía.

3.7 Ejemplo completo: el usuario abre "Mejor valoradas"

El recorrido parte de una acción visible. El usuario entra al explorador y elige Mejor valoradas. useMoviesQuery detecta listType="top_rated", solicita la página correspondiente y Vue Query conserva la respuesta para que MovieResults la represente.

```
Usuario elige "Mejor valoradas"
-> MoviesView.vue mantiene listType="top_rated"
-> useMoviesQuery.ts ejecuta queryFn
-> getMovieList("top_rated", page, language)
-> movies.service.ts
```

```

-> api.get("/movie/topRated", { params: { page, language } })
-> tmdbApi.ts añade baseUrl + Authorization
-> TMDB devuelve page + results + total_pages + total_results
-> Vue Query conserva la página recibida
-> MovieResults une las páginas cargadas
-> MovieCard representa cada película

```

3.7.1 Qué solicitud HTTP construye ese recorrido

```

GET https://api.themoviedb.org/3/movie/top_rated?language=es&page=1
Authorization: Bearer <VITE_TMDB_TOKEN>
Accept: application/json

```

La URL contiene la dirección base, el endpoint y los query params. La credencial no viaja dentro del JSON ni forma parte de la ruta utilizada por CineVerse: se envía en la cabecera Authorization. TMDB responde con JSON y Axios coloca ese contenido en response.data.

3.7.2 Prueba manual de un endpoint en Postman

Las llamadas de TMDB también pueden probarse fuera de CineVerse. La siguiente captura utiliza la otra forma de autenticación admitida por APIv3: la APIKey como parámetro api_key. El valor está censurado. Esta prueba sirve para observar el endpoint, los parámetros, el estado HTTP y el JSON; el código de CineVerse utiliza Bearer, no api_key.

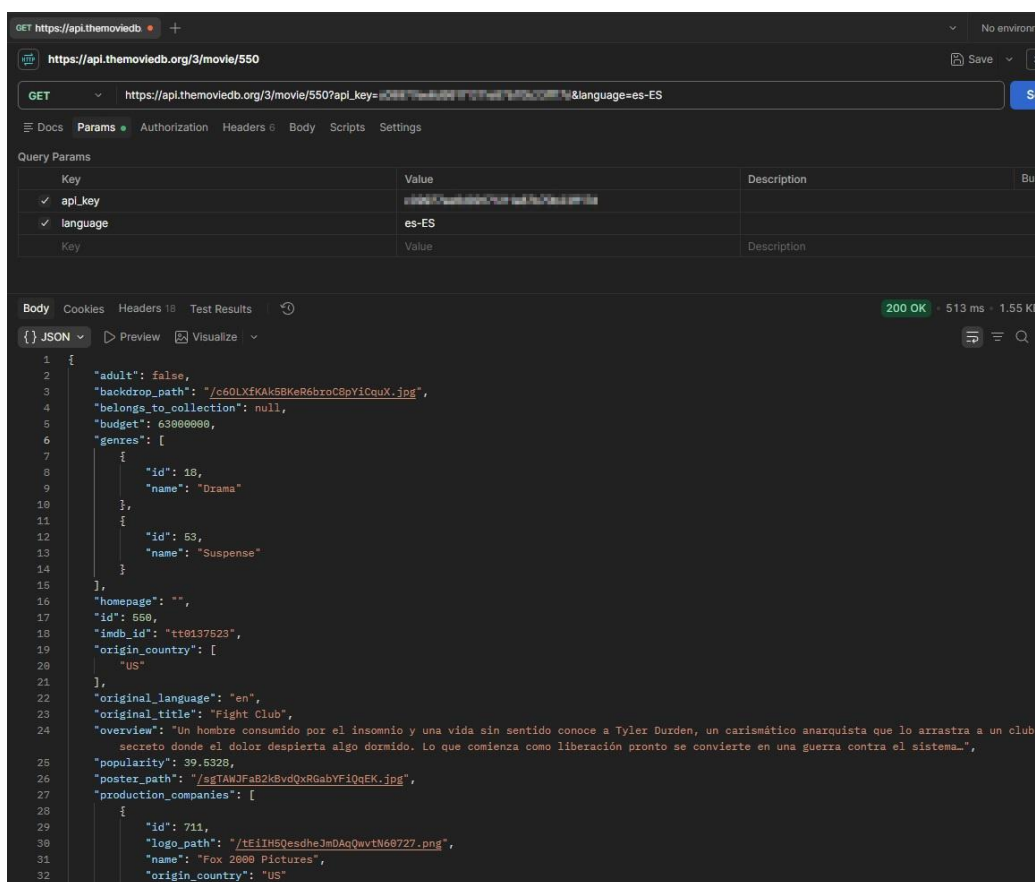


Figura. Prueba manual de GET/3/movie/550 en Postman: parámetros de la solicitud, respuesta 200 OK y comienzo del JSON. La captura utiliza API Key v3; CineVerse utiliza API Read Access Token como Bearer.

3.8 Qué devuelve TMDb y cómo lo interpreta TypeScript

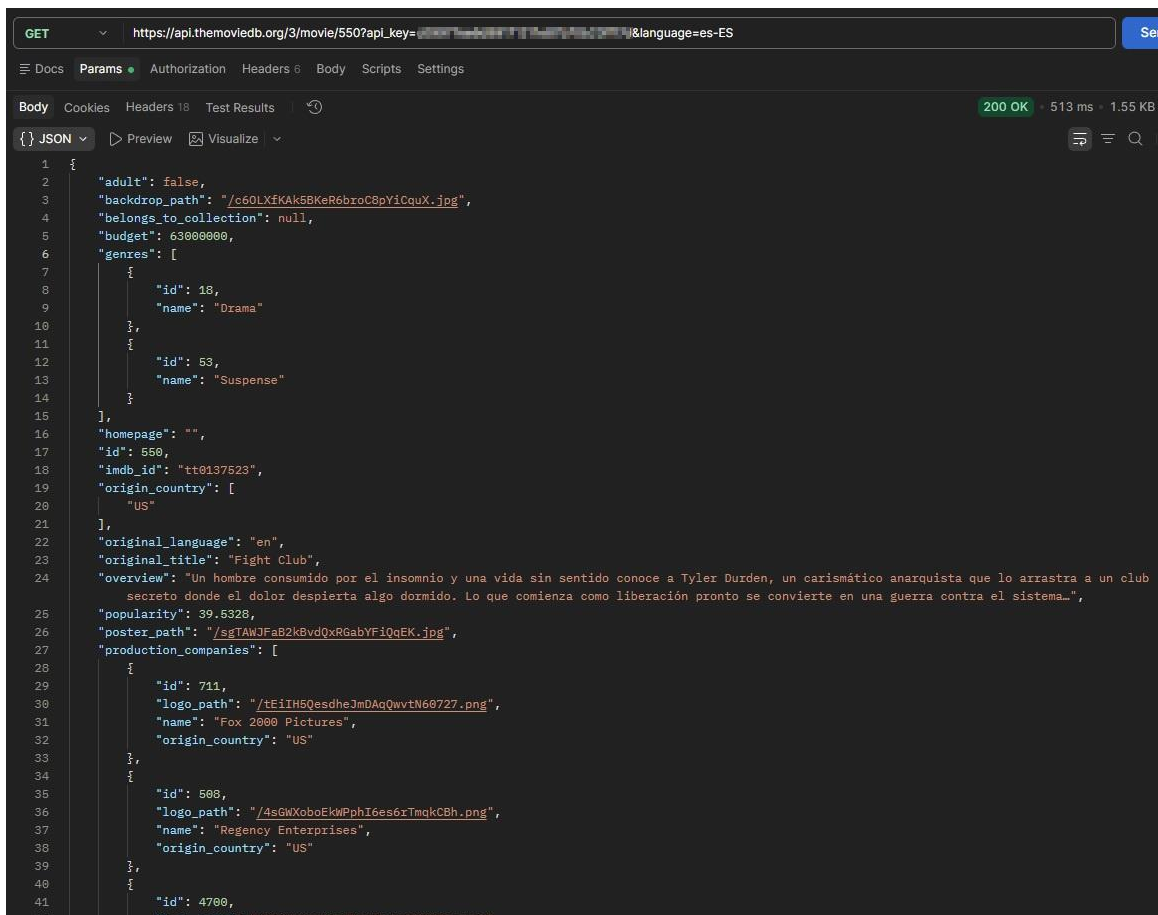


Figura. Respuesta JSON de una película en TMDb. Se observan campos como `id`, `genres`, `overview`, `poster_path` y `production_companies`.

Un endpoint de detalle devuelve un objeto de película; los endpoints paginados, como `topRated` o `search/movie`, devuelven una estructura con `page`, `results`, `total_pages` y `total_results`. CineVerse declara tipos TypeScript para trabajar de forma segura con la estructura que espera.

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}
```

El tipo genérico ayuda durante el desarrollo, pero no valida el JSON recibido en tiempo de ejecución. Si TMDb cambiara inesperadamente la estructura de una respuesta, TypeScript no podría detener el dato que ya llegó por la red; por eso los estados de error y las comprobaciones de la aplicación siguen siendo necesarios.

3.9 Endpoints de TMDb utilizados por el proyecto

Endpoint TMDb	Uso en CineVerse
<code>/trending/movie/day</code>	Tendencias de la Home.
<code>/movie/popular</code>	Catálogo popular.

Endpoint TMDB	Uso en CineVerse
/movie/now_playing	Películas en cartelera.
/movie/upcoming	Próximos estrenos.
/movie/top Rated	Mejor valoradas.
/search/movie	Búsqueda textual.
/discover/movie	Filtros avanzados cuando no existe texto de búsqueda.
/genre/movie/list	Catálogo de géneros.
/movie/{id}	Detalle principal de una película.
/movie/{id}/credits	Reparto y equipo técnico.
/movie/{id}/videos	Videos y trailers.
/movie/{id}/similar	Películas similares.
/person/{id}?append_to_response=movie_credits	Detalle de una persona + filmografía.

No todos los endpoints se utilizan del mismo modo. /search/movie está orientado a texto; /discover/movie acepta filtros estructurados. La sección 16 explica cómo CineVerse elige entre esos caminos y qué limitación aparece cuando combina búsqueda textual con filtros locales.

3.10 Cómo se construyen las imágenes

TMDB no envía normalmente el archivo de imagen dentro del JSON. Los objetos incluyen rutas como poster_path o backdrop_path. El navegador construye después una URL de imagen combinando el servidor de imágenes, un tamaño y el file_path devuelto por la API.

Respuesta JSON:

```
"poster_path": "/abc123.jpg"
```

URL construida:

```
https://image.tmdb.org/t/p/w500/abc123.jpg
```

La documentación oficial de TMDB describe la URL de imagen como la combinación de base_url + file_size + file_path. También ofrece el endpoint /configuration para consultar tamaños y datos de configuración. CineVerse utiliza tamaños concretos como w185, w500, w1280 u original según el lugar de la interfaz.

3.11 Idioma y mecanismos de respaldo

TMDB admite localización en buena parte de sus metadatos, aunque no todos los campos tienen traducciones completas. CineVerse incorpora fallbacks para intentar mantener una ficha útil cuando la primera respuesta llega vacía.

- getMovieDetail vuelve a pedir el otro idioma sólo si poster y backdrop están ambos vacíos.
- getMovieCredits vuelve a pedir el otro idioma si cast y crew vienen ambos vacíos.
- getMovieVideos vuelve a pedir /videos sin language cuando no hay resultados.
- getSimilarMovies utiliza actualmente language="en-US" fijo, independientemente del idioma de la interfaz.

TMDB documenta ISO 639-1 para idiomas y utiliza habitualmente combinaciones con códigos de país, como en-US o pt-BR. CineVerse mantiene además su propio idioma de interfaz mediante vue-i18n; traducir los botones y localizar los datos de TMDB son problemas relacionados pero distintos.

3.12 Personas, `append_to_response` y cuentas de usuario

Para una persona, `person.service.ts` utiliza `/person/{id}` junto con `append_to_response=movie_credits`. `append_to_response` es una capacidad de TMDB que permite incluir determinados subrecursos dentro de una misma respuesta y evitar solicitudes separadas. La sección 22 muestra el recorrido completo de `PersonDetailView` y la filmografía.

Cuenta CineVerse != cuenta TMDB. El API Read Access Token autentica a la aplicación frente a TMDB. Los usuarios que se registran, verifican el correo, inician sesión y escriben reseñas pertenecen al backend PHP/MySQL de CineVerse. TMDB también ofrece autenticación de usuarios para sus propias cuentas, pero CineVerse no utiliza ese flujo.

3.13 Recursos oficiales de TMDB

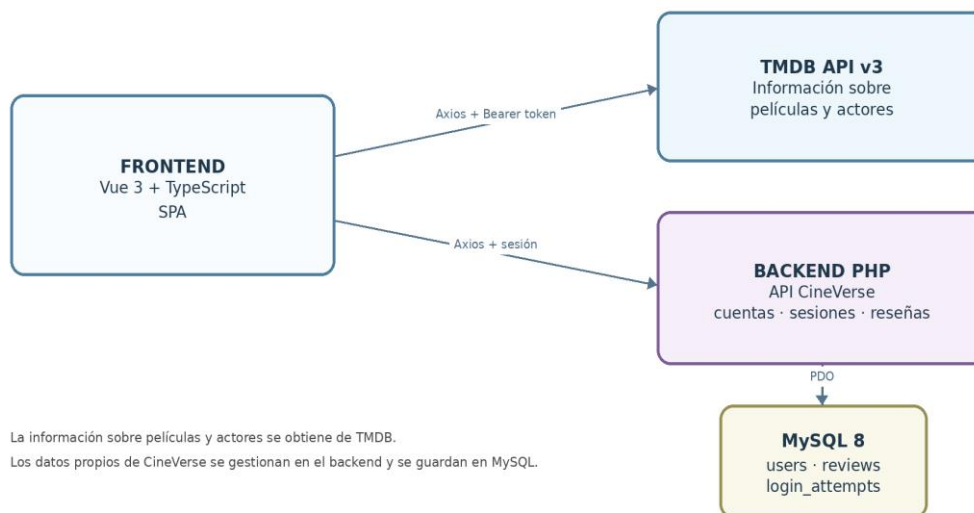
Los enlaces siguientes son las fuentes oficiales recomendadas para consultar cambios en autenticación, endpoints, imágenes y condiciones de uso:

- [Sitio público de TMDB](#)
- [TMDB Developer - Getting Started](#)
- [TMDB API Reference](#)
- [Autenticación de aplicaciones](#)
- [Construcción de URLs de imágenes](#)
- [FAQ, atribución y uso de la API](#)

4. Cómo está organizada CineVerse

CineVerse obtiene información de dos lugares diferentes. Cuando el usuario busca una película, abre su detalle o consulta un actor, el frontend pide esos datos directamente a TMDB. Cuando el usuario registra una cuenta, inicia sesión, publica una reseña o consulta su perfil, el frontend se comunica con el backend PHP de CineVerse. El backend utiliza MySQL cuando necesita guardar o recuperar información propia de la aplicación.

Arquitectura general de CineVerse



Arquitectura general: el frontend Vue obtiene información de TMDB y utiliza el backend PHP para las funciones propias de CineVerse.

Antes de separar responsabilidades conviene ver dos recorridos reales. En ambos casos Vue muestra la interfaz, pero el destino de los datos cambia según lo que necesita el usuario.

```

Usuario abre "Mejor valoradas"
-> MoviesView / useMoviesQuery
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> respuesta JSON
-> Vue muestra las películas

```

```

Usuario inicia sesión
-> LoginView
-> auth.service.ts
-> backendApi.ts
-> backend PHP
-> MySQL
-> respuesta JSON
-> Pinia actualiza el estado de autenticación

```

El primer recorrido no pasa por PHP: el catálogo se consulta directamente en TMDB. El segundo sí utiliza la API propia porque la sesión y las cuentas pertenecen a CineVerse.

En estos recorridos, los archivos terminados en `.service.ts` son módulos TypeScript del frontend que concentran operaciones de acceso a datos. `tmdbApi.ts` y `backendApi.ts` son también archivos propios de CineVerse: crean clientes Axios configurados para comunicarse, respectivamente, con TMDB y con la API PHP. Aquí sólo se los muestra para ubicar el recorrido general; las secciones 11.2.2 y 15 explican después qué hace cada tipo de archivo, dónde está y cómo se utiliza.

4.1 Responsabilidades por sistema

Sistema	Qué hace	Ejemplos
Frontend Vue	Muestra la interfaz, gestiona la navegación y coordina la información que necesita cada pantalla.	Vistas, componentes, filtros, navegación y estado compartido.
TMDB	Servicio externo del que CineVerse obtiene información sobre películas y actores.	Películas, actores, géneros, créditos, videos e imágenes.
Backend PHP	Gestiona las funciones propias de CineVerse que requieren seguridad o datos de usuario.	Registro, sesión, reseñas, perfil, seguridad y envío de correo.
MySQL 8	Guarda de forma permanente los datos propios de CineVerse y aplica restricciones de integridad.	users, reviews y login_attempts.

4.2 Por qué el backend no actúa como proxy de TMDB

En la implementación actual, los archivos `movies.service.ts` y `person.service.ts` realizan las solicitudes a TMDB directamente desde el frontend mediante `tmdbApi.ts`. El backend PHP no queda en el medio de esas consultas. A un servidor que recibe una solicitud para reenviarla a otro servicio se lo suele llamar proxy; CineVerse no utiliza su backend como proxy de TMDB.

Por eso existen dos formas de identificación distintas. El Bearer token de TMDB es la credencial que el frontend envía al servicio externo para identificar a la aplicación. La cookie de sesión PHP es un pequeño identificador que el navegador conserva para relacionar las siguientes solicitudes con el usuario que inició sesión en CineVerse. Una credencial no reemplaza a la otra.

5. Estructura del proyecto

El proyecto se divide en dos partes: frontend y backend. El frontend agrupa los archivos según la función de CineVerse a la que pertenecen. Por ejemplo, auth reúne el registro y el inicio de sesión; movies reúne la búsqueda, los filtros y el detalle de películas; person contiene la pantalla de actores; y profile reúne el perfil. Estas agrupaciones se encuentran dentro de la carpeta features, nombre que el proyecto utiliza para reunir cada funcionalidad en un mismo lugar.

5.1 Frontend

Dentro de cada funcionalidad también se separan responsabilidades. Una view es una pantalla completa; un component es una parte de una pantalla; un service contiene las solicitudes a TMDB o al backend; un composable reúne lógica de Vue que puede reutilizarse; un store mantiene datos compartidos; y types describe con TypeScript la forma que deben tener determinados datos.

```

frontend/
├── public/
├── src/
│   ├── assets/
│   ├── core/
│   │   ├── api/
│   │   ├── composables/
│   │   ├── i18n/
│   │   └── router/
│   ├── features/
│   │   ├── auth/
│   │   ├── home/
│   │   ├── movies/
│   │   ├── person/
│   │   └── profile/
│   ├── layouts/
│   ├── shared/
│   ├── views/
│   ├── App.vue
│   ├── main.ts
│   └── style.css
├── index.html
├── package.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
└── vite.config.ts
  
```

Ubicación	Responsabilidad
src/core/api	Configuración común de Axios: un cliente para TMDB y otro para el backend.
src/core/router	Define las URLs de la aplicación, el idioma, las protecciones de navegación y la pantalla 404.
src/features/*/views	Pantallas completas asociadas a las rutas de Vue Router.
src/features/*/components	Partes de interfaz que pertenecen a una funcionalidad concreta.
src/features/*/services	Funciones que envían solicitudes HTTP a TMDB o al backend.

Ubicación	Responsabilidad
src/features/*/composables	Funciones de Vue que reúnen lógica reutilizable, por ejemplo filtros o carga de datos.
stores	Datos compartidos entre distintas pantallas, principalmente autenticación y favoritos.
types	Definiciones TypeScript que describen la estructura de los datos.

Ejemplo: qué contiene la carpeta movies

```

features/movies/
├── components/
│   └── movie-detail/
├── composables/
│   └── movie-detail/
├── services/
├── stores/
├── types/
│   └── movie-detail/
├── utils/
└── views/

```

La carpeta `movies` reúne en un mismo lugar todo lo relacionado con películas: tipos TypeScript, servicios que consultan TMDb o el backend, filtros, carga de resultados, favoritos, detalle de película y componentes visuales. Así es más sencillo encontrar los archivos de una funcionalidad sin buscarlos entre carpetas generales sin contexto.

5.2 Backend

El backend separa el trabajo de otra manera. Cuando llega una solicitud HTTP, cada tipo de clase cumple una tarea: los `middleware` realizan comprobaciones previas; un `controller` recibe los datos y decide qué respuesta HTTP devolver; un `validator` comprueba el formato; un `service` aplica las reglas de CineVerse; y un `model` ejecuta las consultas a MySQL. Entre esas comprobaciones puede estar el token CSRF, un valor de seguridad que se exige en operaciones que modifican datos. Esta organización fue programada directamente en PHP, sin Laravel ni otro framework.

```

backend/
├── config/
├── controllers/
├── core/
│   ├── auth/
│   ├── http/
│   ├── logging/
│   ├── middleware/
│   └── validation/
├── database/
├── models/
├── public/
│   ├── index.php
│   └── .htaccess
├── routes/
│   └── api.php
├── services/
├── storage/
│   └── logs/
├── templates/
│   └── emails/
├── validators/
├── composer.json
└── composer.lock

```

Capa	Qué hace	Qué tarea no le corresponde
Controller	Recibe la solicitud HTTP, obtiene los datos enviados, llama a las clases necesarias y construye la respuesta.	No debería escribir consultas SQL directamente.
Service	Aplica las reglas de la aplicación y coordina varias operaciones cuando una acción lo necesita.	No debería contener detalles de presentación del frontend.
Model	Lee y modifica la base de datos mediante consultas preparadas de PDO.	No debería decidir qué código HTTP recibe el navegador.
Validator	Comprueba que los datos recibidos tengan el formato, longitud y rango permitidos.	No consulta MySQL para aplicar reglas de negocio.
Middleware	Realiza comprobaciones antes de llegar al controller, por ejemplo sesión válida o token CSRF.	No ejecuta la funcionalidad final solicitada por el usuario.

5.3 Cómo orientarse dentro del proyecto

No hace falta memorizar cada archivo. La forma más rápida de orientarse es identificar si se está buscando infraestructura compartida, una funcionalidad del frontend o una capa del backend.

Zona	Qué reúne y cuándo conviene buscar ahí
src/core/api	Los dos clientes Axios compartidos. Aquí se define cómo el frontend se conecta con TMDB y con el backend PHP.
src/features	Funcionalidades del usuario. auth, movies, person y profile reúnen sus vistas, servicios, composables, stores y tipos.
src/layouts y src/shared	Piezas de interfaz o lógica reutilizable que no pertenecen a una sola funcionalidad.
backend/routes y controllers	Puntos de entrada HTTP: qué URL existe y qué controller recibe la solicitud.
backend/services y validators	Reglas de negocio y validación de los datos antes de guardar o modificar información.
backend/models y database	Acceso a MySQL mediante PDO y consultas preparadas.
backend/core	Infraestructura común: sesión, HTTP, middleware, logging, bootstrap y validación base.

5.4 Dos recorridos entre carpetas

```

Explorar películas
MoviesView.vue
  -> useMoviesQuery.ts
  -> movies.service.ts
  -> core/api/tmdbApi.ts
  -> TMDB

Iniciar sesión
LoginView.vue
  -> auth.service.ts
  -> core/api/backendApi.ts
  -> routes/api.php
  -> AuthController.php
  -> AuthService.php
  -> User.php / PDO / MySQL

```

Estos recorridos son una guía de lectura del repositorio. Las secciones posteriores explican cada paso con más detalle.

6. Archivos de configuración y variables de entorno

CineVerse necesita algunos datos para saber con qué servicios debe comunicarse y cómo hacerlo. El frontend necesita, por ejemplo, la dirección de TMDB, el token que autoriza las consultas a su API y la dirección del backend. El backend necesita los datos de conexión a MySQL, la configuración del servidor de correo y la carpeta donde guarda los logs. En lugar de escribir esos valores dentro de cada archivo TypeScript o PHP, el proyecto los guarda en archivos .env.

Una variable de entorno es un nombre asociado a un valor de configuración. Por ejemplo, DB_NAME=cineverse significa que la variable DB_NAME contiene el nombre de la base de datos que debe utilizar el backend. Cuando el código necesita ese dato, lee la variable en lugar de tener el nombre cineverse escrito directamente dentro de Database.php.

La idea de esta sección es seguir cada valor hasta el lugar donde produce un efecto. En el frontend, Vite expone las variables VITE_ mediante import.meta.env. En el backend, phpdotenv carga backend/.env y deja sus valores disponibles en \$_ENV para clases como Database, AppLogger y MailService.

6.1 Mapa de los archivos de configuración del frontend

Antes de estudiar una variable o un comando conviene ubicar físicamente las piezas. En CineVerse, los archivos que controlan la instalación y el arranque del frontend están en la raíz de la carpeta frontend. No son pantallas Vue: son archivos que npm, Vite y TypeScript leen para saber qué instalar, cómo iniciar el proyecto y qué valores de configuración exponer al código.

```

cineverse/
├── frontend/
│   ├── package.json
│   ├── package-lock.json
│   ├── vite.config.ts
│   ├── .env
│   ├── .env.example
│   ├── tsconfig.json
│   ├── tsconfig.app.json
│   ├── tsconfig.node.json
│   └── src/

```

Archivo / carpeta	Qué es	Quién lo utiliza
frontend/package.json	Manifiesto del proyecto JavaScript: declara paquetes y comandos de npm.	npm.
frontend/package-lock.json	Registro de las versiones concretas que npm resolvió.	npm durante instalaciones reproducibles.
frontend/node_modules/	Carpeta generada por npm con los paquetes instalados.	Node.js, Vite y el código que importa dependencias.
frontend/vite.config.ts	Configuración propia de Vite para este proyecto.	Vite, ejecutado con Node.js.
frontend/.env	Valores locales de configuración del frontend.	Vite.
frontend/.env.example	Plantilla con los nombres de variables esperadas.	El desarrollador; Vite no la usa como sustituto automático de .env.
frontend/tsconfig*.json	Reglas de TypeScript para la aplicación y las herramientas.	TypeScript / vue-tsc.

Este mapa evita una confusión frecuente: package.json no contiene Axios, vite.config.ts no contiene las variables VITE_* y .env no instala paquetes. Cada pieza tiene una responsabilidad distinta y se conecta con la siguiente durante el arranque.

6.2 package.json, npm, package-lock.json y node_modules

Node.js es el entorno que CineVerse utiliza para ejecutar herramientas de JavaScript fuera del navegador. npm es el gestor de paquetes que se instala junto con Node.js. El archivo que npm consulta en este proyecto es frontend/package.json.

6.2.1 Qué es package.json y dónde está

package.json es un archivo JSON que describe el proyecto del frontend. Está ubicado en frontend/package.json. No es una pantalla, no se importa desde los componentes Vue y el navegador no lo lee directamente. npm lo utiliza para saber qué dependencias necesita CineVerse y qué comandos puede ejecutar.

```

// frontend/package.json (fragmento real)
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@tanstack/vue-query": "^5.92.9",

```

```

    "axios": "^1.13.6",
    "pinia": "^3.0.4",
    "vue": "^3.5.25",
    "vue-i18n": "^11.2.8",
    "vue-router": "^5.0.3"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^6.0.2",
    "typescript": "~5.9.3",
    "vite": "^7.3.1",
    "vue-tsc": "^3.1.5"
  }
}

```

dependencies contiene paquetes que utiliza la aplicación, como Vue, Pinia, Axios y Vue Router. devDependencies contiene herramientas necesarias para desarrollar, comprobar o compilar el proyecto, como Vite, TypeScript y vue-tsc. La separación describe el propósito del paquete; npm install instala ambos grupos en un entorno normal de desarrollo.

Los símbolos que aparecen delante de algunas versiones son rangos de compatibilidad utilizados por npm. En versiones mayores que cero, ^1.13.6 permite instalar una versión compatible desde 1.13.6 hasta antes de 2.0.0; ~5.9.3 limita normalmente la actualización a parches de la serie 5.9, es decir, desde 5.9.3 hasta antes de 5.10.0. package.json declara esos rangos; package-lock.json registra después las versiones concretas que npm resolvió para una instalación reproducible.

6.2.2 Cómo los scripts de package.json se convierten en comandos

```

npm run dev
  -> npm abre frontend/package.json
  -> busca scripts.dev
  -> encuentra "vite"
  -> ejecuta Vite

npm run build
  -> npm busca scripts.build
  -> ejecuta "vite build"
  -> Vite genera la versión publicable

```

Comando	Qué hace en CineVerse
npm install	Lee package.json y package-lock.json e instala las dependencias dentro de node_modules.
npm run dev	Ejecuta el script dev y pone en marcha el servidor de desarrollo de Vite.
npm run build	Ejecuta vite build y genera frontend/dist para producción.
npm run preview	Sirve localmente el build generado para comprobarlo.

6.2.3 package-lock.json y node_modules: declaración, resolución e instalación

package.json declara qué paquetes y rangos de versión necesita el proyecto. package-lock.json registra las versiones concretas y dependencias transitivas que npm resolvió. node_modules es la carpeta física que npm crea al instalar esos paquetes. En CineVerse, node_modules no debe editarse para configurar Axios, Vue Router o Vite: el código propio se mantiene fuera de esa carpeta.

```

frontend/package.json
  "axios": "^1.13.6"
    |
    | npm install
    v
frontend/node_modules/axios/
  |
  | import axios from "axios"
  v
código TypeScript de CineVerse

```

6.3 Vite y vite.config.ts

Vite es la herramienta que inicia el frontend durante el desarrollo y genera el build de producción. Vite llega al proyecto porque está declarado en frontend/package.json y npm lo instala en node_modules. Cuando npm run dev ejecuta el comando vite, Vite carga su archivo de configuración propio: frontend/vite.config.ts.

6.3.1 Qué es vite.config.ts, dónde está y cuándo se usa

vite.config.ts es un archivo TypeScript propio de la configuración del frontend. Está ubicado en frontend/vite.config.ts. No lo ejecuta directamente el navegador y no es una vista Vue. Vite lo lee mediante Node.js al iniciar el servidor de desarrollo o al construir la aplicación.

```
npm run dev
-> package.json: scripts.dev = "vite"
-> Vite inicia
-> Vite lee frontend/vite.config.ts
-> configura plugins, alias y proxy
-> sirve el frontend
```

6.3.2 Código fuente de vite.config.ts

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import tailwindcss from '@tailwindcss/vite'
import { fileURLToPath, URL } from 'node:url'

export default defineConfig({
  base: '/',
  plugins: [
    vue(),
    tailwindcss(),
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(
        new URL('./src', import.meta.url)
      )
    }
  },
  server: {
    proxy: {
      '/api': {
        target: 'http://cineverse.local',
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})
```

Parte	Qué significa en CineVerse
base: "/"	El build supone que el frontend se publica desde la raíz del sitio.
vue()	Permite que Vite procese componentes .vue.
tailwindcss()	Conecta Tailwind con el proceso de Vite; la interfaz sigue usando principalmente CSS propio.
alias "@"	Hace que @ represente frontend/src y evita rutas relativas largas.
server.proxy["/api"]	Intercepta en desarrollo las solicitudes cuyo camino comienza con /api.
target	Define el backend local al que Vite reenviará esas solicitudes: http://cineverse.local.
rewrite	Elimina /api antes de entregar la ruta al backend.

defineConfig(...) es una función auxiliar de Vite que permite escribir la configuración con mejor soporte de tipos y autocompletado; no inicia por sí sola el servidor. La clase URL y import.meta.url permiten construir una dirección relativa al propio archivo vite.config.ts, y fileURLToPath(...) convierte esa dirección de archivo en una ruta normal.

del sistema operativo. Con esas piezas, el alias @ queda asociado a frontend/src. Por eso un import como import backendApi from "@core/api/backendApi" apunta realmente a frontend/src/core/api/backendApi.ts y evita escribir rutas relativas largas.

Dentro de server.proxy["/api"], target indica el servidor de destino del proxy. changeOrigin: true hace que Vite adapte la cabecera Host al destino reenviado; no autentica al usuario ni modifica la sesión. secure: false controla la verificación de certificados cuando el destino utiliza HTTPS; como el target actual es http://cineverse.local, no interviene una validación TLS en este entorno. rewrite elimina el prefijo /api antes de entregar la ruta a Apache, de modo que /api/user se convierte en /user.

6.3.3 Ejemplo real del proxy de desarrollo

En este ejemplo, baseUrl significa la parte común de una dirección que Axios antepone a los endpoints relativos. Si backendApi tiene baseUrl="/api" y el código solicita el endpoint "/user", la ruta resultante en el navegador es "/api/user".

```
backendApi.get("/user")
  -> baseUrl = "/api"
  -> navegador pide http://localhost:5173/api/user
  -> Vite detecta /api
  -> rewrite elimina /api
  -> proxy reenvía a http://cineverse.local/user
  -> Apache / PHP responden
```

Este proxy existe durante el desarrollo porque el navegador está usando el frontend servido por Vite en localhost:5173 y el backend PHP vive en cineverse.local. En producción, VITE_BACKEND_URL puede apuntar directamente a la URL pública del backend y el proxy de desarrollo deja de ser el camino principal.

6.4 frontend/.env, .env.example y variables VITE_

frontend/.env es el archivo de valores locales del frontend. Está en la raíz de frontend y puede cambiar entre computadoras o entornos. frontend/.env.example es una plantilla que documenta qué variables deben existir, pero Vite no la toma automáticamente como reemplazo de .env.

```
frontend/
├── .env           <- valores activos de esa instalación
├── .env.example  <- plantilla para crear .env
├── package.json
└── vite.config.ts
```

No confundir frontend/.env con backend/.env

frontend/.env lo procesa Vite y sus variables VITE_* terminan disponibles para el código que llega al navegador. backend/.env lo carga PHP y contiene valores del servidor, como la conexión MySQL o SMTP. Son archivos diferentes y los leen sistemas diferentes.

6.4.1 Cómo Vite expone las variables VITE_ al código

Cuando Vite inicia o compila el frontend, lee frontend/.env. Para que una variable personalizada quede disponible en el código del navegador, su nombre debe comenzar con VITE_. Vite la expone mediante import.meta.env. El prefijo no lo exige TMDB ni PHP: es la convención de Vite para variables accesibles desde el frontend.

```
frontend/.env
VITE_BACKEND_URL="/api"
|
| Vite lee .env
v
import.meta.env.VITE_BACKEND_URL
|
v
código TypeScript del frontend
```

6.4.2 VITE_TMDB_BASE_URL: dirección base de TMDB

VITE_TMDB_BASE_URL es una variable de entorno del frontend. Se define en frontend/.env y su plantilla aparece en frontend/.env.example. Su valor indica la dirección común de la API v3 de TMDB. En Axios, baseUrl es precisamente esa parte común de la dirección: se antepone a cada endpoint relativo que solicita el código. El archivo frontend/src/core/api/tmdbApi.ts lee VITE_TMDB_BASE_URL directamente y la utiliza como baseUrl de la instancia Axios compartida.

```
# frontend/.env
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"

// frontend/src/core/api/tmdbApi.ts
const baseUrl = import.meta.env.VITE_TMDB_BASE_URL;
```

6.4.3 VITE_TMDB_TOKEN: credencial Bearer de TMDB

VITE_TMDB_TOKEN también se define en frontend/.env. Contiene el APIRead Access Token que TMDB entrega en la configuración de API de la cuenta. CineVerse lo lee desde import.meta.env y tmdbApi.ts lo coloca en la cabecera Authorization de la instancia Axios compartida con el formato Bearer <token>. La sección 3 explica desde cero qué es TMDB, cómo se obtiene esta credencial y cómo recorre el frontend hasta llegar a la API.

```
# frontend/.env
VITE_TMDB_TOKEN="<token proporcionado por TMDB>"

// tmdbApi.ts
const token = import.meta.env.VITE_TMDB_TOKEN;
Authorization: `Bearer ${token}`
```

Importante sobre VITE_TMDB_TOKEN

Una variable VITE_ termina incorporada al JavaScript que se entrega al navegador. Por eso este token no debe tratarse como un secreto privado del servidor. Una credencial que necesite permanecer secreta debería utilizarse desde una capa del backend.

6.4.4 VITE_BACKEND_URL: cómo el frontend encuentra la API PHP

VITE_BACKEND_URL es una variable de entorno del frontend que indica qué dirección base debe utilizar CineVerse para hablar con su propia API PHP. Se define en frontend/.env y aparece como ejemplo en frontend/.env.example. En desarrollo vale /api. Cuando backendApi usa ese valor como baseUrl, Axios lo antepone a endpoints relativos como /login, /user o /profile; por ejemplo, baseUrl=/api más el endpoint /user produce /api/user.

```
# frontend/.env
VITE_BACKEND_URL="/api"
```

La variable no es una función y no es una URL que Axios inventa. backendApi.ts la lee directamente al crear la instancia Axios del backend. Después, auth.service.ts, reviews.service.ts y profile.service.ts reutilizan backendApi y por eso reciben esa configuración de forma indirecta.

```
frontend/.env
VITE_BACKEND_URL="/api"
|
| Vite expone import.meta.env
v
frontend/src/core/api/backendApi.ts
axios.create({ baseUrl: "/api" })
|
|--> auth.service.ts
|--> reviews.service.ts
|--> profile.service.ts
|
v
API PHP
```

6.4.5 Resumen: quién usa cada variable del frontend

Variable	Ubicación	Quién la lee directamente	Quién la aprovecha después
VITE_TMDB_BASE_URL	frontend/.env	tmdbApi.ts	movies.service.ts y person.service.ts mediante api.
VITE_TMDB_TOKEN	frontend/.env	tmdbApi.ts	movies.service.ts y person.service.ts mediante api.
VITE_BACKEND_URL	frontend/.env	backendApi.ts	auth.service.ts, reviews.service.ts y profile.service.ts mediante backendApi.

La sección 15 retoma estas piezas desde el punto de vista de ejecución: muestra cómo tmdbApi.ts y backendApi.ts crean clientes Axios y cómo los archivos *.service.ts utilizan esos clientes para enviar solicitudes reales.

6.5 Configuración del backend: backend/.env

El backend tiene su propio archivo backend/.env. PHP no lee ese archivo por sí solo. CineVerse utiliza una librería llamada vlucas/phpdotenv para abrirlo y convertir sus valores en variables que el código PHP puede consultar.

La librería phpdotenv está declarada en backend/composer.json. Composer es el gestor de dependencias del backend: lee composer.json, instala las librerías necesarias al ejecutar composer install y genera vendor/autoload.php para que PHP pueda cargarlas.

```
"require": {
  "vlucas/phpdotenv": "^5.6",
  "monolog/monolog": "^3.10",
  "phpmailer/phpmailer": "^7.1"
}
```

La secuencia es: composer.json declara phpdotenv; composer install la instala dentro de vendor/; vendor/autoload.php permite cargarla; y core/bootstrap.php la utiliza para leer backend/.env. Después, los valores quedan disponibles mediante \$_ENV.

6.5.1 Variables del backend: qué controla cada una

```
APP_ENV=local
APP_DEBUG=true

DB_HOST=localhost
DB_PORT=3306
DB_NAME=cineverse
DB_USER=usuario
DB_PASSWORD=contraseña
DB_CHARSET=utf8mb4

CINEVERSE_LOG_PATH=storage/logs

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=usuario
MAIL_PASSWORD=contraseña
MAIL_FROM=no-reply@example.com
MAIL_FROM_NAME=CineVerse

FRONTEND_URL=http://localhost:5173
```

Variable	Qué controla	Dónde interviene en CineVerse
APP_ENV	Identifica el entorno declarado, por ejemplo local o production.	core/bootstrap.php comprueba que exista. El código actual no cambia por sí solo las respuestas únicamente por este valor.

Variable	Qué controla	Dónde interviene en CineVerse
APP_DEBUG	Guarda la bandera de depuración de la instalación.	bootstrap.php valida el valor; actualmente no activa automáticamente trazas o mensajes distintos para el navegador.
DB_HOST	Dirección del servidor donde se ejecuta MySQL.	Database::connect() la incorpora al DSN utilizado por PDO.
DB_PORT	Puerto por el que se accede a MySQL.	Database::connect() lo agrega al DSN junto con host, base y charset.
DB_NAME	Nombre de la base que contiene users, reviews y login_attempts.	Database::connect() lo utiliza como dbname en la conexión PDO.
DB_USER	Usuario de MySQL con el que se abre la conexión.	Se entrega como segundo argumento al crear PDO.
DB_PASSWORD	Contraseña del usuario de MySQL.	Se entrega a PDO al abrir la conexión; puede estar vacía sólo si esa instalación de MySQL está configurada de esa manera.
DB_CHARSET	Juego de caracteres de la conexión; el proyecto usa utf8mb4.	Forma parte del DSN para que PDO y MySQL intercambien correctamente texto Unicode.
CINEVERSE_LOG_PATH	Carpeta donde se escriben los logs del backend.	AppLogger la lee para crear los archivos rotativos de Monolog.
MAIL_HOST	Servidor SMTP al que se conecta PHPMailer.	MailService lo asigna al host de la conexión de correo.
MAIL_PORT	Puerto del servidor SMTP.	MailService lo utiliza al configurar PHPMailer.
MAIL_USERNAME	Cuenta con la que CineVerse se autentica ante SMTP.	MailService la utiliza como usuario de autenticación.
MAIL_PASSWORD	Contraseña de la cuenta SMTP.	MailService la utiliza al autenticarse antes de enviar el mensaje.
MAIL_FROM	Dirección que recibe el destinatario como remitente.	MailService la configura con setFrom(); no necesariamente tiene que ser igual a MAIL_USERNAME.
MAIL_FROM_NAME	Nombre visible del remitente.	MailService lo muestra junto a MAIL_FROM en los correos de CineVerse.

Variable	Qué controla	Dónde interviene en CineVerse
FRONTEND_URL	Dirección pública del frontend.	MailService la usa para construir los enlaces de verificación y recuperación que vuelven a Vue.

bootstrap.php exige DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_CHARSET, APP_ENV, APP_DEBUG y CINEVERSE_LOG_PATH. Las variables de correo y FRONTEND_URL no se comprueban durante el arranque; si faltan o están mal configuradas, el problema puede aparecer recién cuando una acción intenta enviar un correo.

6.6 core/bootstrap.php: cómo se carga backend/.env

public/index.php carga core/bootstrap.php al comienzo de cada solicitud. bootstrap.php prepara lo que el backend necesita antes de procesar una ruta: carga las librerías instaladas por Composer, fija la zona horaria y utiliza phpdotenv para leer backend/.env.

```
require_once __DIR__ . '/../vendor/autoload.php';

date_default_timezone_set('America/Argentina/Buenos_Aires');

$dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/..');
$dotenv->load();
```

vendor/autoload.php es el archivo generado por Composer que permite cargar las clases de las librerías y las clases del proyecto sin escribir un require_once diferente para cada archivo. Dotenv::createImmutable(__DIR__ . "/..") indica que phpdotenv debe buscar .env en la raíz del backend. Después de load(), el código puede leer valores como \$_ENV["DB_HOST"] o \$_ENV["FRONTEND_URL"].

createImmutable() evita que el archivo .env reemplace una variable que ya hubiera sido definida por el sistema operativo o por el servidor antes de iniciar PHP.

6.7 composer.json y carga automática de clases

Además de instalar dependencias, Composer configura la carga automática de las clases de CineVerse. El backend utiliza la convención PSR-4, que relaciona el namespace de una clase con una carpeta del proyecto. Namespace es el nombre jerárquico que PHP utiliza para organizar clases y evitar conflictos de nombres.

```
"autoload": {
    "psr-4": {
        "App\\Controllers\\": "controllers/",
        "App\\Core\\": "core/",
        "App\\Database\\": "database/",
        "App\\Models\\": "models/",
        "App\\Services\\": "services/",
        "App\\Validators\\": "validators/"
    }
}
```

Por ejemplo, cuando PHP necesita App\\Services\\MailService, Composer sabe que debe buscar la clase dentro de services/MailService.php. A este mecanismo se lo llama autoload o carga automática. composer.lock conserva las versiones concretas de las librerías resueltas por Composer.

6.8 Conexión a MySQL con PDO

Database.php está ubicado en backend/database/Database.php. Es una clase propia de CineVerse que se encarga de crear y devolver la conexión PDO que utiliza el backend para comunicarse con MySQL.

public/index.php llama a Database::connect() al comenzar cada petición; después, el mismo objeto PDO se entrega a las capas que necesitan consultar o modificar MySQL. Database::connect() toma de \$_ENV los datos DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD y DB_CHARSET. Con ellos construye un DSN, es decir, una cadena que reúne los datos necesarios para abrir la conexión, y crea el objeto PDO.

```
$dsn = "mysql:host={$host};port={$port};dbname={$dbName};charset={$charset}";

return new PDO(
    $dsn,
    $user,
    $password,
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]
);
```

PDO es la interfaz de PHP utilizada para comunicarse con MySQL. `ERRMODE_EXCEPTION` hace que un error de MySQL se convierta en una excepción que el backend puede capturar. `FETCH_ASSOC` hace que una fila se devuelva como un array cuyos índices son los nombres de las columnas; por ejemplo, `["id" => 15, "username" => "usuario"]`.

`config/config.php` también puede devolver una conexión PDO, pero la entrada principal de la API no depende de ese archivo: `public/index.php` carga `bootstrap.php` y llama directamente a `Database::connect()` dentro de un bloque que permite detectar una caída de MySQL y responder con HTTP 503. El código 503 indica que el servicio no está disponible temporalmente, en este caso porque la base de datos no pudo utilizarse.

6.9 Configuración de los logs

La ubicación de los logs se configura desde `backend/.env` mediante `CINEVERSE_LOG_PATH`. Esta variable pertenece a CineVerse: no es una opción incorporada de PHP ni de Monolog. `core/bootstrap.php` carga `backend/.env` con `phpdotenv` y deja el valor disponible en `$_ENV` para que `AppLogger` pueda consultarlo.

```
# backend/.env
CINEVERSE_LOG_PATH=storage/logs
```

Con el valor relativo `storage/logs`, `AppLogger` resuelve normalmente la carpeta `backend/storage/logs`. El archivo base utilizado por el código es `cineverse-backend.log` y Monolog agrega la fecha al realizar la rotación diaria, generando nombres como `cineverse-backend-2026-09-07.log`.

Esta sección sólo describe la configuración. La sección 29 desarrolla la implementación completa: cómo `AppLogger` obtiene la ruta, cómo crea el Logger de Monolog, cómo configura `RotatingFileHandler` y `LineFormatter` y cómo `AppLogger::error()` termina escribiendo una entrada en el archivo.

6.10 Configuración del correo y FRONTEND_URL

`MailService` utiliza `PHPMailer` para enviar los correos de verificación de cuenta y recuperación de contraseña. `PHPMailer` se conecta a un servidor SMTP, que es el servicio encargado de enviar el mensaje. Los datos de esa conexión se obtienen de `backend/.env`.

```
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=no-reply@example.com
MAIL_PASSWORD=<contraseña>
MAIL_FROM=no-reply@example.com
MAIL_FROM_NAME=CineVerse
FRONTEND_URL=http://localhost:5173
```

`MAIL_USERNAME` es la cuenta utilizada para autenticarse ante el servidor SMTP. `MAIL_FROM` es la dirección que verá el destinatario como remitente; pueden coincidir, pero cumplen funciones distintas. `FRONTEND_URL` indica la dirección del frontend y se utiliza para construir los enlaces incluidos en los correos.

Ejemplo: con `FRONTEND_URL=http://localhost:5173`, idioma es y token `abc123`, `MailService` puede construir `http://localhost:5173/es/verify-email?token=abc123`. En producción `FRONTEND_URL` debe apuntar al dominio HTTPS real del frontend.

CORS (Cross-Origin Resource Sharing) es una política del navegador que decide qué sitio web puede comunicarse con una API cuando ambos están en direcciones diferentes. Cambiar `FRONTEND_URL` no modifica

automáticamente esa regla. CorsMiddleware todavía contiene `http://localhost:5173` escrito directamente en el código, por lo que el origen permitido debe revisarse por separado cuando se publique el proyecto.

La configuración anterior indica con qué servidor se enviará el correo. El recorrido completo -idioma, construcción del enlace, plantilla HTML, PHPMailer, conexión SMTP, envío y tratamiento de fallos- se desarrolla en la sección 25.

6.11 Valores que todavía están escritos directamente en el código

No toda la configuración de CineVerse se obtiene desde `.env`. Algunos valores continúan escritos directamente dentro de archivos PHP o TypeScript. Funcionan en el entorno local actual, pero deben revisarse si cambia la dirección del frontend, la ubicación del backend o la forma de publicar la aplicación.

Configuración	Dónde está	Valor o comportamiento actual
Zona horaria	<code>backend/core/bootstrap.php</code>	America/Argentina/Buenos_Aires.
Origen permitido por CORS	<code>backend/core/middleware/CorsMiddleware.php</code>	<code>http://localhost:5173</code> .
Métodos y headers permitidos por CORS	<code>CorsMiddleware.php</code>	GET, POST, PUT, PATCH, DELETE, OPTIONS; Content-Type y X-CSRF-Token.
Tiempo de inactividad de la sesión	<code>backend/core/auth/Session.php</code>	1800 segundos / 30 minutos.
Cookie de sesión	<code>Session.php</code>	<code>lifetime=0, path=/, HttpOnly=true, SameSite=Lax; Secure si PHP detecta HTTPS.</code>
Seguridad SMTP	<code>backend/services/MailService.php</code>	Autenticación SMTP, STARTTLS (cifrado de la conexión) y UTF-8.
Rotación de logs	<code>backend/core/logging/AppLogger.php</code>	30 archivos; nivel mínimo ERROR.
Prefijo base de las rutas	<code>backend/public/index.php</code>	<code>/cineverse/backend/public</code> .
Destino del proxy de desarrollo	<code>frontend/vite.config.ts</code>	<code>http://cineverse.local</code> .

Que un valor esté escrito directamente en el código no significa que el proyecto falle en desarrollo. El punto importante es saber que esos lugares también deben revisarse al mover CineVerse a otro servidor. Por ejemplo, un dominio distinto obliga a revisar CORS y una subcarpeta diferente puede exigir cambiar el prefijo base de las rutas.

7. Base de datos MySQL 8

CineVerse utiliza MySQL para guardar la información que pertenece a la propia aplicación. Las películas, actores, géneros, imágenes y créditos no se copian a esta base: se consultan en TMDb cuando hacen falta. El esquema local tiene tres tablas y cada una cumple una responsabilidad diferente.

Esta separación permite que MySQL conserve cuentas y actividad propia de CineVerse sin intentar duplicar el catálogo externo de TMDb.

Tabla	Para qué se usa en CineVerse
users	Guarda las cuentas: username, email, password_hash, verificación de correo y datos temporales de recuperación de contraseña.
reviews	Guarda las reseñas escritas por los usuarios y las relaciona con una película mediante movie_id de TMDB.
login_attempts	Mantiene un contador temporal de intentos fallidos por combinación IP + email para aplicar el límite de login.

Tres ejemplos sencillos

Registro correcto

-> se crea una fila en users

Usuario publica una reseña

-> se crea una fila en reviews

-> user_id apunta a users.id

-> movie_id identifica la película en TMDB

Contraseña incorrecta

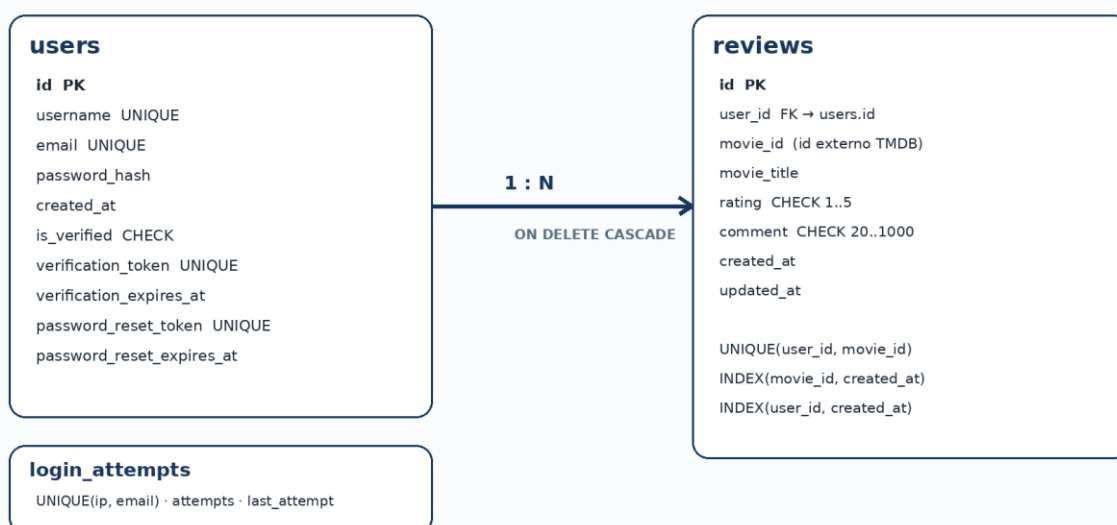
-> se crea o incrementa login_attempts para esa IP + email

A partir de esa función práctica se pueden entender las reglas técnicas del esquema. Una tabla reúne registros del mismo tipo; cada fila es un registro y cada columna representa un dato.

MySQL utiliza PRIMARY KEY para identificar una fila; AUTO_INCREMENT para generar ids; UNIQUE para impedir duplicados; FOREIGN KEY para relacionar tablas; CHECK para limitar valores; e INDEX para acelerar determinadas búsquedas. INT UNSIGNED, VARCHAR, TEXT y DATETIME indican qué clase de dato puede guardar cada columna.

El archivo SQL crea las tablas con InnoDB y usa utf8mb4. InnoDB permite trabajar con transacciones y claves foráneas; utf8mb4 permite almacenar texto Unicode, incluidos acentos y otros caracteres. El diagrama siguiente resume la relación real entre las tres tablas.

Modelo de datos real - MySQL 8



Relación real entre las tres tablas definidas por el archivo SQL.

7.1 Tabla users

Columna	Tipo	Regla / función
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT.
username	VARCHAR(20)	NOT NULL, UNIQUE. Nombre público y control de duplicados.
email	VARCHAR(80)	NOT NULL, UNIQUE. Login, verificación y recuperación.
password_hash	VARCHAR(255)	Resultado generado a partir de la contraseña; nunca se almacena la contraseña original.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP.
is_verified	TINYINT(1)	DEFAULT 0 y CHECK IN (0,1).
verification_token	VARCHAR(255)	UNIQUE; puede ser NULL. Token de confirmación.
verification_expires_at	DATETIME	Vigencia del token de verificación.
password_reset_token	VARCHAR(255)	UNIQUE; puede ser NULL. Token de recuperación.
password_reset_expires_at	DATETIME	Vigencia del token de recuperación.

7.2 Tabla reviews

Columna	Tipo	Regla / función
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT.
user_id	INT UNSIGNED	FOREIGN KEY hacia users.id; ON DELETE CASCADE elimina sus reseñas si se elimina el usuario.
movie_id	INT UNSIGNED	ID externo de la película en TMDB.
movie_title	VARCHAR(255)	Título persistido junto a la reseña.
rating	TINYINT UNSIGNED	CHECK entre 1 y 5.
comment	TEXT	CHECK exige entre 20 y 1000 caracteres.
created_at	DATETIME	Fecha de creación.
updated_at	DATETIME	Se actualiza automáticamente al modificar la fila.

Integridad importante

UNIQUE(user_id, movie_id) hace cumplir en MySQL la regla de una reseña por usuario y película. La aplicación comprueba esa regla antes del INSERT, pero la restricción de MySQL es la garantía final incluso ante dos solicitudes concurrentes.

Índices y relación con users

- idx_reviews_movie_created (movie_id, created_at): favorece el listado de reseñas de una película ordenadas por fecha.
- idx_reviews_user_created (user_id, created_at): favorece el perfil y las reseñas propias del usuario.
- fk_reviews_user: garantiza que user_id exista y elimina sus reseñas si el usuario se elimina.

7.3 Tabla login_attempts

Columna	Tipo	Uso
id	INT UNSIGNED	Identificador interno.
ip	VARCHAR(45)	Dirección IP; longitud compatible con representaciones IPv6.
email	VARCHAR(80)	Email asociado al intento.
attempts	TINYINT UNSIGNED	Cantidad de fallos acumulados.
last_attempt	DATETIME	Se actualiza con cada intento y permite limpiar registros antiguos.

login_attempts no es un historial general de accesos. Es un contador temporal utilizado por AuthService para limitar fallos de login. La combinación IP + email es única, por lo que sólo existe una fila por pareja; last_attempt permite localizar registros que ya quedaron fuera de la ventana de 15 minutos.

7.4 Por qué no existe una tabla movies

CineVerse no crea una tabla movies porque la información completa de las películas sigue perteneciendo a TMDB. Cada reseña guarda movie_id para saber a qué película de TMDB corresponde y movie_title para poder mostrar el título guardado junto a la actividad del usuario sin tener que almacenar una copia completa de la película en MySQL.

7.5 Ejemplo concreto: qué queda guardado al escribir una reseña

La tabla reviews permite ver con claridad qué datos pertenecen a TMDB y cuáles pertenecen a CineVerse. Cuando un usuario escribe una reseña, el backend no guarda una copia completa de la película. Guarda solamente la referencia necesaria para relacionar la reseña con la película y el contenido escrito por el usuario.

Ejemplo conceptual de una fila creada en reviews:

```
id           = 8301
user_id      = 125
movie_id     = 550
movie_title  = "Fight Club"
rating       = 5
comment      = "Una reseña escrita por el usuario..."
created_at   = 2026-09-08 13:20:00
updated_at   = 2026-09-08 13:20:00
```

user_id se obtiene de la sesión PHP y relaciona la fila con users.id. movie_id y movie_title llegan desde la película que el frontend obtuvo de TMDB. rating y comment son los valores que escribió el usuario. created_at y updated_at los mantiene MySQL. La sección 26 sigue este recorrido completo desde el formulario Vue hasta este INSERT.

Si más adelante el usuario elimina su cuenta, ON DELETE CASCADE en la clave foránea de reviews.user_id hace que MySQL elimine también sus reseñas. Si elimina una reseña individual, sólo desaparece esa fila; la película continúa existiendo en TMDB porque CineVerse no es dueño de ese catálogo.

8. Datos de prueba y archivo SQL

El archivo SQL está preparado para crear la base de CineVerse y cargar los datos de prueba en una sola ejecución. Primero configura MySQL, crea la base si no existe, elimina las tablas anteriores que correspondan, crea las tres tablas actuales y finalmente inserta los usuarios y las reseñas.

No es una migración incremental. Una migración suele aplicar un cambio pequeño sobre una base que ya existe, procurando conservar sus datos. Este archivo, en cambio, contiene DROP TABLE IF EXISTS y está pensado para recrear un entorno controlado desde cero.

Antes de cargar los usuarios y las reseñas, el script ejecuta `START TRANSACTION` para agrupar la carga masiva de datos. Al finalizar también ajusta los siguientes valores `AUTO_INCREMENT` de `users` y `reviews` para que los nuevos registros continúen después del conjunto precargado. Como MySQL puede realizar commits implícitos al ejecutar determinadas instrucciones de definición, como `ALTER TABLE`, el archivo completo no debe interpretarse como una única transacción reversible de principio a fin.

Elemento	Cantidad / estado
Usuarios sintéticos	5.000
Usuarios verificados	4.800
Usuarios no verificados	200
Reseñas	24.322
Comentarios distintos	24.322
Contraseña común de los datos de prueba	password (sólo pruebas)

Advertencia de importación

El script contiene `DROP TABLE IF EXISTS` y elimina también tablas heredadas que ya no usa el backend PHP puro. Debe tratarse como un script de recreación/carga para un entorno controlado, no como una operación inocua sobre una base productiva.

Esquema general del script

```
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS `reviews`;
DROP TABLE IF EXISTS `login_attempts`;
...
DROP TABLE IF EXISTS `users`;

CREATE TABLE `users` (...);
CREATE TABLE `login_attempts` (...);
CREATE TABLE `reviews` (...);

SET FOREIGN_KEY_CHECKS = 1;
START TRANSACTION;
-- INSERTs de usuarios y reseñas
COMMIT;
```

La contraseña común `password` pertenece únicamente al conjunto de datos de prueba. No representa la regla de contraseñas del formulario real. Al registrar una cuenta nueva, CineVerse exige entre 8 y 32 caracteres e incluye comprobaciones de mayúscula, minúscula, número y carácter especial.

8.1 Orden real de ejecución del archivo SQL

El archivo no es solamente una colección de `INSERT`. Está organizado para poder reconstruir un entorno de pruebas completo sin que las relaciones entre tablas interrumpen el proceso.

```

SET FOREIGN_KEY_CHECKS = 0
    -> permite eliminar tablas relacionadas durante la reconstrucción
DROP TABLE IF EXISTS ...
    -> elimina el esquema anterior contemplado por el script
CREATE TABLE users / login_attempts / reviews
    -> crea el esquema activo
SET FOREIGN_KEY_CHECKS = 1
    -> reactiva las comprobaciones de claves foráneas
START TRANSACTION
    -> comienza la carga masiva de datos de prueba
INSERT ...
    -> carga usuarios y reseñas
ALTER TABLE users / reviews AUTO_INCREMENT = ...
    -> fija los próximos identificadores; esta operación DDL provoca un commit implícito
COMMIT
    -> cierre explícito final del script

```

Desactivar `FOREIGN_KEY_CHECKS` es útil durante la recreación porque `reviews` depende de `users`. No significa que la base quede sin relaciones: después de crear nuevamente las tablas se reactivan las comprobaciones y las restricciones `FOREIGN KEY`, `UNIQUE` y `CHECK` vuelven a actuar normalmente.

9. Cómo recorre CineVerse una acción del usuario

Con el vocabulario básico de la sección 1.3, conviene seguir qué ocurre desde que el usuario realiza una acción hasta que la pantalla recibe el resultado. Los ejemplos usan solicitudes y respuestas HTTP, JSON y endpoints ya introducidos allí. Cuando aparece CSRF, se trata de un token de seguridad que protege las operaciones que modifican datos.

Los tres ejemplos siguientes muestran recorridos diferentes: consultar una película usa TMDB; iniciar sesión usa el backend y MySQL; publicar una reseña combina información de TMDB con una cuenta de CineVerse.

9.1 Abrir una película

1. El usuario abre la URL de una película. Vue Router reconoce el id numérico y muestra `MovieDetailView`.
2. La vista utiliza `useMovieDetail`, que llama a `getMovieDetail(id, language)` para pedir la información principal.
3. `movies.service.ts` envía la solicitud a TMDB mediante `tmdbApi.ts`. El cliente añade el Bearer token automáticamente.
4. Si TMDB devuelve la película, CineVerse pide créditos, videos y películas similares al mismo tiempo para no esperar una consulta detrás de otra.
5. `useMovieDetail` prepara los datos que necesita la pantalla: director, reparto, trailer y películas similares.
6. Las reseñas no vienen de TMDB. La pestaña Reparto es la pestaña inicial; la API PHP de reseñas se consulta cuando el usuario abre por primera vez la pestaña Reseñas y se monta `MovieReviews`.

9.2 Iniciar sesión

1. El usuario completa el formulario. `LoginView` realiza las comprobaciones básicas antes de enviarlo.
2. `auth.service.ts` envía los datos al backend con una solicitud `POST /login` mediante `backendApi.ts`.
3. `AuthController` recibe los datos, comprueba su forma básica y pasa el trabajo a `AuthService`.
4. `AuthService` aplica las reglas de inicio de sesión: revisa intentos previos, busca el usuario, verifica la contraseña y comprueba que el correo esté verificado.
5. Si el login es correcto, `Session` regenera el identificador de sesión, guarda `user_id` y crea un token CSRF.
6. El backend devuelve los datos del usuario y el token CSRF. El store de Pinia actualiza el estado de autenticación del frontend.

9.3 Publicar una reseña

1. La pantalla ya conoce el id y el título de la película porque fueron obtenidos de TMDB.
2. El usuario elige una puntuación y escribe su comentario.

3. `reviews.service.ts` envía POST `/reviews` al backend e incluye X-CSRF-Token para demostrar que la solicitud pertenece a la sesión activa.
4. `AuthMiddleware` y `CsrfMiddleware` comprueban la sesión y el token antes de permitir que la solicitud llegue al controller.
5. `ReviewController` recibe los datos y `ReviewValidator` limpia el comentario y comprueba sus reglas de longitud, título de la película y puntuación.
6. `ReviewService` aplica la regla de una sola reseña por usuario y película.
7. Si todo es válido, `Review::create()` ejecuta la consulta preparada que inserta la reseña en MySQL.
8. El backend devuelve una respuesta JSON y el frontend actualiza las estadísticas y las reseñas mostradas.

Cómo se relacionan TMDB y MySQL en una reseña
`movie_id` y `movie_title` identifican la película obtenida de TMDB. `user_id` identifica al usuario de la sesión de CineVerse. `rating` y `comment` son los datos que escribe el usuario y se guardan en MySQL.

9.4 Guardar una película como favorita: ejemplo sin backend

No todas las acciones recorren frontend, backend y MySQL. Favoritos es un buen contraste porque todo el recorrido ocurre dentro del navegador. El usuario puede hacerlo incluso sin una cuenta.

```

Usuario pulsa el corazón de una MovieCard
-> MovieCard llama onFavoriteClick(movie)
-> useFavoritesStore() obtiene el store compartido de Pinia
-> isFavorite(movie.id) comprueba si ya existe
-> addFavorite(movie) agrega el objeto Movie al array favorites
-> Vue detecta el cambio reactivo y actualiza el corazón/contador
-> pinia-plugin-persistedstate persiste el estado en localStorage

```

```

No hay request HTTP
No se ejecuta PHP
No se modifica MySQL

```

Si el usuario recarga la página, el plugin de persistencia restaura el estado guardado y el store vuelve a disponer de esas películas. La sección 23 desarrolla el store, el límite de 20, la eliminación, la vista Favoritos y el comportamiento de la interfaz.

9.5 Escribir una reseña: recorrido completo resumido

Publicar una reseña sí cruza todas las capas propias de CineVerse. El frontend ya conoce `movie_id` y `movie_title` porque antes obtuvo la película de TMDB; después combina esos datos con la puntuación y el comentario que escribe el usuario.

```

MovieReviews.vue
-> POST /reviews + cookie de sesión + X-CSRF-Token
-> Router
-> AuthMiddleware
-> CsrfMiddleware
-> ReviewController::create()
-> Request::json()
-> Validator + ReviewValidator
-> ReviewService::create()
-> Review::existsReviewByUserIdAndMovieId()
-> Review::create()
-> PDO INSERT INTO reviews
-> MySQL
-> Response::success("REVIEW_CREATED")
-> frontend vuelve a pedir la página 1 de reseñas
-> aparecen la reseña propia, el nuevo promedio y el nuevo total

```

La sección 26 desarrolla cada paso, muestra qué se envía, qué columna recibe cada dato y qué ocurre al editar, borrar o cargar más reseñas.

10. TypeScript en CineVerse

TypeScript es una extensión de JavaScript que permite indicar qué tipo de dato se espera en cada parte del código. Por ejemplo, CineVerse puede declarar que el `id` de una película es un número, que el `título` es texto y que `poster_path` puede ser una ruta de imagen o `null` cuando TMDB no proporciona un póster. Estas comprobaciones ayudan a detectar errores mientras se desarrolla, antes de abrir la aplicación en el navegador.

En CineVerse los tipos también describen respuestas HTTP, usuarios, reseñas, filtros, propiedades de componentes y valores devueltos por composables. El proyecto activa `strict: true`, por lo que TypeScript realiza comprobaciones más exigentes que en una configuración básica.

10.1 Configuración estricta

```
{
  "compilerOptions": {
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true
  }
}
```

Con estas opciones el compilador puede avisar, por ejemplo, que se intenta leer una propiedad que no existe, que un valor podría ser `null`, que se pasó un `string` donde se esperaba un número o que quedó una variable declarada pero sin uso.

10.2 Tipos de dominio

Ejemplo: Movie

```
export interface Movie {
  id: number;
  title: string;
  release_date: string;
  poster_path: string | null;
  backdrop_path: string | null;
  vote_average: number;
  popularity: number;
  genre_ids?: number[];
  overview: string;
}
```

`interface` es una de las formas que ofrece TypeScript para describir la estructura de un objeto. En `Movie`, `poster_path: string | null` indica que el valor puede ser una ruta de texto o puede no existir. La interfaz debe contemplar ambos casos en lugar de asumir que todas las películas tienen póster.

10.3 Genéricos: `PaginatedResponse<T>`

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}

const { data } = await api.get<PaginatedResponse<Movie>>({
  "/search/movie",
  { params: { query, page, language } }
});
```

`PaginatedResponse<T>` es un tipo genérico: `T` representa el tipo concreto de los elementos que contendrá `results`. Cuando se usa `PaginatedResponse<Movie>`, TypeScript sabe que `results` es `Movie[]` y que `page`, `total_pages` y `total_results` son números. La misma estructura de paginación puede reutilizarse con otros tipos sin copiarla.

10.4 Reutilización de tipos en los filtros

```
export interface BaseFilters {
  genres?: number[];
  yearFrom?: number;
  yearTo?: number;
  minRating?: number;
  maxRating?: number;
  sortBy?: string;
}

export interface DiscoverFilters extends BaseFilters {}

export interface SearchWithTextAndFilters extends BaseFilters {
  query: string;
}
```

`BaseFilters` reúne las propiedades compartidas por varios tipos de búsqueda. `DiscoverFilters` reutiliza esas propiedades y `SearchWithTextAndFilters` añade `query`. Así género, años, puntuación y orden se definen una sola vez en lugar de duplicarse en cada tipo.

10.5 Obtener un tipo a partir de un composable

```
export type MovieFilters = ReturnType<typeof useMoviesFilters>;

export function useMoviesQuery(
  filters: MovieFilters,
  language: Ref<string>
) { ... }
```

`ReturnType` es una utilidad de TypeScript que obtiene automáticamente el tipo que devuelve una función. `MovieFilters` se obtiene a partir de `useMoviesFilters`; si ese composable cambia lo que devuelve, el tipo se actualiza junto con él y reduce la posibilidad de que ambas definiciones se separen.

En el mismo ejemplo aparece `Ref<string>`. `Ref` es un contenedor reactivo de Vue: guarda un valor que puede cambiar y permite que Vue detecte ese cambio. `Ref<string>` indica que ese valor reactivo contiene texto; en `useMoviesQuery` representa el idioma activo.

10.6 TypeScript no reemplaza la validación del backend

Que TypeScript indique `rating: number` protege al propio código durante el desarrollo, pero no convierte al navegador en una barrera de seguridad. Alguien puede enviar manualmente una solicitud con `rating = 999` sin utilizar la interfaz. Por eso el backend vuelve a validar los datos recibidos y MySQL mantiene `CHECK(rating BETWEEN 1 AND 5)`.

Cada capa cumple una función distinta

TypeScript ayuda a detectar errores mientras se programa. Los Validadores de PHP comprueban los datos que realmente llegan por HTTP. Las consultas preparadas de PDO protegen el acceso a MySQL. Las restricciones `UNIQUE`, `FOREIGN KEY` y `CHECK` mantienen la integridad de los datos guardados.

10.7 Zonas donde el tipado puede endurecerse

El proyecto utiliza una configuración estricta, pero todavía existen zonas menos tipadas. Hay `Record<string, any>`, errores capturados como `any` y algunos arrays de créditos con `any[]`. `any` indica que TypeScript deja de comprobar con precisión ese valor. También debe recordarse que un tipo de `Axios` describe lo que el código espera recibir, pero no valida durante la ejecución que TMDb o el backend hayan enviado exactamente esa estructura.

10.8 Props y eventos tipados entre componentes

TypeScript también protege la comunicación entre componentes Vue. Las props son valores que un componente padre entrega a un hijo. Los eventos permiten el camino inverso: el hijo avisa que ocurrió algo y el padre decide qué hacer.

```
// MovieHero recibe datos del padre
defineProps<{
  movie: MovieDetail;
  cast: CastMember[];
  director: { name: string } | null;
  hasTrailer: boolean;
}>();

// MovieHero avisa al padre
const emit = defineEmits(["openTrailer", "ready"]);
```

Con estas definiciones, TypeScript puede avisar si el padre intenta pasar un tipo incorrecto. En `MovieDetailView`, `openTrailer` se conecta con el evento emitido por `MovieHero` y `ready` se utiliza para retirar el estado de carga cuando el hero terminó de preparar su imagen.

10.9 Tipos de las respuestas del backend

El mismo criterio se utiliza con la API PHP. `reviews.service.ts` declara que `getMovieReviews()` devuelve `ReviewsResponse` y que `getUserReviews()` devuelve `UserReviewsResponse`. Esto permite que `MovieReviews` sepa que `data` contiene `stats`, `pagination`, `user_review` y `reviews` sin trabajar con objetos completamente desconocidos.

El tipo ayuda durante el desarrollo, pero sigue sin validar el JSON real en tiempo de ejecución. Si un servidor devolviera una estructura distinta, TypeScript no podría detener esa respuesta de red; por eso los estados de error y las validaciones del backend siguen siendo necesarios.

11. Cómo está organizado el frontend con Vue

`main.ts` es el archivo que pone en marcha el frontend. Crea la aplicación Vue, conecta Pinia para el estado compartido, Vue Router para la navegación, Vue Query para las consultas de datos e `i18n` para los idiomas. También importa los estilos globales e inicia la comprobación de la sesión del usuario.

Montaje principal

```
const app = createApp(App);
app.use(pinia);

authStore.initialize(); // se dispara, pero no se espera

app
  .use(router)
  .use(VueQueryPlugin)
  .use(i18n)
  .mount("#app");
```

11.1 Comprobación de la sesión al iniciar la aplicación

Al arrancar, `useAuthStore.initialize()` consulta `GET /user` para saber si el navegador ya tiene una sesión válida. `main.ts` inicia esa comprobación en segundo plano para que las pantallas públicas, como Inicio o Películas, puedan mostrarse sin esperar al backend. Las rutas que sí dependen de la sesión esperan el resultado antes de decidir si permiten la navegación.

Ventaja

Home y Movies no quedan bloqueadas por una consulta a `/user`. Las rutas que sí necesitan conocer el estado de sesión esperan el resultado de forma selectiva.

Una Promise es un objeto de JavaScript que representa una operación asíncrona que puede terminar más adelante. La implementación reutiliza una única Promise para evitar que `main.ts` y un guard de Vue Router envíen dos `GET /user` al mismo tiempo. Actualmente `initializePromise` queda guardada incluso después de resolverse. Si la primera consulta falla porque el backend estaba temporalmente caído, otra llamada durante la misma vida de la SPA reutiliza esa promesa ya terminada en lugar de realizar automáticamente un nuevo intento.

Detalle de implementación actual. `main.ts` emite el evento `app:ready` después de montar Vue, pero `index.html` no registra ningún listener para ese evento. La pantalla inicial de carga se oculta actualmente mediante `window.onload`, por lo que `app:ready` no produce un efecto visible.

11.2 Organización de archivos por funcionalidades (features)

La carpeta `features` agrupa archivos por funcionalidad. `auth` contiene registro, login y recuperación; `movies` reúne exploración, filtros, detalle, reseñas y favoritos; `person` contiene la información de actores; y `profile` reúne las pantallas del perfil. `core` queda reservado para código común utilizado por varias funcionalidades, como Axios, el router y los idiomas.

11.2.1 Qué tipos de archivo puede contener una feature

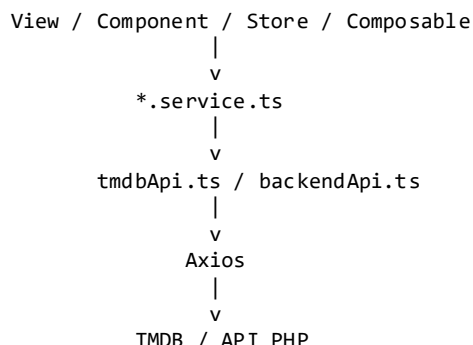
Una feature reúne archivos que participan en una misma funcionalidad, pero cada tipo de archivo tiene una responsabilidad diferente. El nombre de la carpeta no cambia cómo funciona Vue; es una decisión de organización del proyecto para que el código relacionado quede cerca.

Tipo	Responsabilidad en CineVerse	Ejemplo
view	Pantalla completa asociada normalmente a una ruta.	LoginView.vue, MoviesView.vue, ProfileView.vue
component	Parte visual reutilizable de una pantalla.	MovieCard.vue, MovieReviews.vue
composable	Función que reúne lógica reactiva reutilizable y puede integrar Vue Query.	useMoviesQuery.ts, useMovieDetail.ts
store	Estado compartido disponible desde distintas partes de la SPA.	useAuthStore.ts, useFavoritesStore.ts
service	Operaciones de acceso a datos: prepara y ejecuta llamadas hacia TMDB o el backend mediante un cliente HTTP compartido.	movies.service.ts, auth.service.ts
types	Tipos TypeScript que describen payloads, respuestas y entidades.	movie.ts, auth.ts, profile.ts

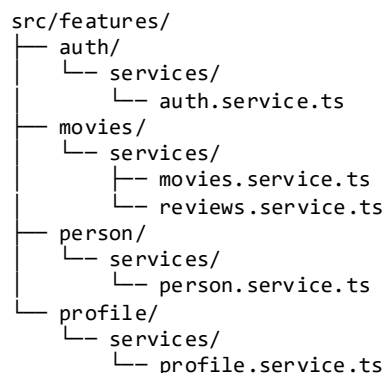
11.2.2 Qué significa *.service.ts en CineVerse

El sufijo `.service.ts` es una convención propia de organización: Vue, TypeScript y Axios no obligan a utilizar ese nombre. En CineVerse identifica módulos que concentran operaciones relacionadas con una fuente de datos. Un service suele recibir parámetros, elegir un endpoint, preparar query params, body o cabeceras, invocar un cliente Axios y devolver los datos recibidos.

Como se adelantó en la sección 1.3, un endpoint es la ruta concreta de una API que representa una operación o recurso, por ejemplo `/movie/top Rated` o `/login`. Los query params son valores que viajan en la URL después de `?`, como `page=2&language=es`. El body es el conjunto de datos enviado dentro de la solicitud, habitual en POST o PUT. Las cabeceras o headers son información adicional que acompaña a la solicitud, como Authorization o X-CSRF-Token.



Un service no dibuja la interfaz, no registra rutas, no mantiene por sí mismo un store reactivo y no reemplaza a Vue Query. Por ejemplo, `useMoviesQuery` administra `queryKey`, paginación y estados de carga; `movies.service.ts` se ocupa de las operaciones concretas que obtienen películas.



La sección 15 explica estos cinco services uno por uno: ubicación, cliente HTTP utilizado, funciones exportadas, código fuente y ejemplos de quién los invoca.

11.2.3 Mantenibilidad de componentes grandes

La organización por features es modular, pero algunos componentes han crecido considerablemente y concentran plantilla, estado, eventos, validación y estilos en un mismo archivo. Como evolución futura conviene extraer lógica de formularios, filtros o efectos del DOM a composables y componentes menores cuando esa división reduzca complejidad real, sin fragmentar la interfaz sólo por tamaño.

11.3 Componentes y composables

Una vista representa una pantalla completa. Los componentes dividen esa pantalla en partes más pequeñas y reciben datos o emiten eventos. Los composables son funciones que reúnen lógica de Vue que puede reutilizarse: `useMoviesFilters` controla los filtros, `useMoviesQuery` carga las páginas del explorador, `useMovieDetail` combina los datos de una película y `usePersonDetail` obtiene la información de un actor.

11.4 Cómo funciona la reactividad de Vue en CineVerse

Vue mantiene valores reactivos para que la interfaz se actualice cuando cambian los datos sin reconstruir manualmente el HTML. CineVerse usa principalmente `ref`, `computed`, `reactive` y `watch`.

- `ref(value)` guarda un valor reactivo. Ejemplo: `MovieReviews` usa `formRating`, `formComment`, `isSubmitting` y `reviews` como refs.
- `computed(...)` calcula un valor a partir de otros valores reactivos. `favoritesCount` se recalcula a partir de `favorites.length` y `canSubmitForm` depende del rating y la longitud del comentario.

- `reactive({...})` agrupa varias propiedades reactivas en un objeto; `useMoviesFilters` lo utiliza para `listType`, texto, géneros, años, puntuación y orden.
- `watch(...)` ejecuta lógica cuando cambia un valor. CineVerse lo usa, por ejemplo, para aplicar `debounce` a la búsqueda, bloquear el scroll al abrir un modal o resetear la grilla al cambiar filtros.

Ejemplo: al ejecutar `favorites.value.push(movie)`, no hace falta buscar el contador en el DOM y modificarlo. `favoritesCount` depende de `favorites`, Vue detecta el cambio y las expresiones del template que usan ese contador se vuelven a evaluar automáticamente.

11.5 Componentes padre, props y emits

Los componentes no comparten variables locales de forma directa. El padre entrega datos mediante props y el hijo puede emitir eventos. `MovieTabs` recibe `cast`, `similar`, `movieId` y `movieTitle` desde `MovieDetailView`; luego entrega `movieId` y `movieTitle` a `MovieReviews`. `MovieHero`, en cambio, emite `openTrailer` para que `MovieDetailView` sea quien abra el modal.

```
MovieDetailView
-> props -> MovieHero
<- emit "openTrailer" - MovieHero

MovieDetailView
-> props movieId/movieTitle -> MovieTabs
-> props movieId/movieTitle -> MovieReviews
```

11.6 Ciclo de vida: `onMounted` y `onUnmounted`

`onMounted` se ejecuta cuando el componente ya fue creado y colocado en el DOM. `MovieReviews` lo utiliza para pedir la primera página de reseñas. `ProfileView` lo usa para cargar el perfil. `MovieDetailView` registra allí el listener de la tecla `Escape` para cerrar el trailer.

`onUnmounted` se ejecuta cuando el componente se destruye. `MovieDetailView` elimina el listener global de teclado y `MovieResults` desconecta su `IntersectionObserver`. Esta limpieza evita que queden funciones activas después de abandonar una pantalla.

11.7 `App.vue`, `RouterView` y cuándo se recrea una vista

`App.vue` mantiene `AppHeader` y `AppFooter` fuera de `RouterView` para que el marco general permanezca mientras cambia la pantalla central. `RouterView` recibe el componente asociado a la URL actual.

`routeKey(route)` construye una key con el nombre de la ruta y sus parámetros, pero excluye `lang`. Por eso pasar de `/es/movies/550` a `/es/movies/272` cambia el id y fuerza a recrear la vista, mientras cambiar sólo `/es` por `/en` conserva la instancia y deja que la reactividad del idioma y `Vue Query` actualicen los datos.

12. Cómo se guardan y reutilizan datos en el frontend

En el frontend existen tres necesidades diferentes que conviene separar. El estado compartido representa información que varias partes de la interfaz necesitan conocer al mismo tiempo; los datos remotos provienen de APIs y tienen un ciclo de carga, error, caché y actualización; y la persistencia conserva determinados valores aunque la página se recargue. CineVerse utiliza una herramienta distinta para cada problema en lugar de concentrarlo todo en un único store.

Necesidad	Herramienta principal	Ejemplo en CineVerse
Estado compartido	Pinia	Usuario autenticado, token CSRF y favoritos.
Datos obtenidos de APIs	TanStack Vue Query	Home, explorador, detalle de película y actores.
Solicitudes HTTP	Axios	Cliente de TMDB y cliente del backend PHP.

Necesidad	Herramienta principal	Ejemplo en CineVerse
Persistencia local	localStorage / sessionStorage	Favoritos y preferencias visuales; último estado de filtros.

Idea clave

Pinia y Vue Query no compiten entre sí. Pinia conserva estado propio del frontend; Vue Query administra datos que llegan desde una API. Axios es quien realiza la solicitud HTTP real.

12.1 Pinia: estado compartido de la aplicación

Pinia es la biblioteca de gestión de estado utilizada por CineVerse. Un componente Vue puede guardar datos con `ref()` o `reactive()`, pero ese estado local pertenece a esa instancia. Cuando el Header, el Router, una pantalla de perfil y una vista de reseñas necesitan conocer al mismo usuario autenticado, mantener copias independientes sería difícil de sincronizar. Un store de Pinia ofrece un único punto compartido al que pueden acceder distintas partes de la aplicación.

12.1.1 Cómo se instala y qué significa un store compartido

`main.ts` crea una sola instancia de Pinia y la registra en Vue. A partir de ese momento, `useAuthStore()` y `useFavoritesStore()` obtienen acceso a stores identificados por un nombre estable. Llamar `useAuthStore()` desde varios componentes no crea varios usuarios: todos observan el mismo estado reactivo mientras la aplicación está montada.

```
const pinia = createPinia();
pinia.use(piniaPluginPersistedState);

const app = createApp(App);
app.use(pinia);
```

12.1.2 Anatomía de un store: estado, valores derivados y acciones

Los stores actuales utilizan el estilo Setup Store. Dentro de `defineStore` se crean refs para los datos modificables, `computed` para valores derivados y funciones que actúan como acciones. El componente que consume el store no necesita conocer cómo se coordinan internamente esas piezas.

```
export const useAuthStore = defineStore("auth", () => {
  const user = ref<AuthUser | null>(null);
  const csrfToken = ref<string | null>(null);
  const isInitialized = ref(false);

  const isAuthenticated = computed(() => user.value !== null);

  function clear() {
    user.value = null;
    csrfToken.value = null;
  }

  return { user, csrfToken, isInitialized, isAuthenticated, clear };
});
```

`user`, `csrfToken` e `isInitialized` son estado. `isAuthenticated` no necesita guardarse por separado: se calcula a partir de `user`. `clear()` concentra una modificación relacionada del estado para que los consumidores no tengan que recordar qué propiedades deben vaciar.

12.2 useAuthStore: representación de la sesión en el frontend

El store de autenticación mantiene el usuario actual, el token CSRF y el estado de inicialización. Sin embargo, Pinia no autentica al usuario y no es la fuente de verdad de la sesión. La sesión real vive en PHP y se relaciona con el navegador mediante una cookie. Pinia conserva una representación reactiva de ese estado para que la interfaz pueda responder inmediatamente.

```

Cookie de sesión PHP
  ↓
GET /user
  ↓
useAuthStore.initialize()
  ↓
user + csrfToken
  ↓
Header / Router / Perfil / Reseñas

```

12.2.1 Qué ocurre después de un login correcto

LoginView envía las credenciales al backend. Si PHP responde LOGIN_SUCCESS, el frontend recibe user y csrf_token y actualiza el store. Como user es reactivo, isAuthenticated cambia automáticamente y cualquier componente que dependa de ese valor vuelve a evaluarse.

```

login correcto
-> setUser(response.data.user)
-> setCsrftoken(response.data.csrf_token)
-> isAuthenticated pasa a true
-> Header y rutas protegidas observan el nuevo estado

```

12.2.2 initialize(): reconstruir la sesión al iniciar la SPA

Al recargar el navegador, el estado en memoria de Pinia desaparece. La cookie de sesión puede seguir siendo válida, por lo que initialize() consulta GET /user. Si el backend reconoce la cookie, devuelve nuevamente el usuario y un token CSRF; si no existe una sesión válida, el store queda sin usuario. La implementación reutiliza una initializePromise para evitar que main.ts y un guard de Vue Router lancen dos GET /user simultáneos.

La Promise compartida es útil para deduplicar la inicialización, aunque la implementación actual conserva esa Promise incluso después de finalizar. Si el primer intento falló por una caída temporal del backend, una nueva llamada durante la misma vida de la SPA no fuerza automáticamente otra consulta.

12.2.3 Por qué el store de autenticación no se persiste en localStorage

Guardar user en localStorage no convertiría ese valor en una sesión válida. Un usuario podría conservarse visualmente aunque la sesión PHP hubiera expirado. Por eso la autenticación se reconstruye consultando al servidor. La persistencia local se reserva para datos que pueden pertenecer al navegador sin representar una credencial de acceso, como los favoritos.

12.3 useFavoritesStore y persistencia con pinia-plugin-persistedstate

Los favoritos son un caso distinto. El store guarda objetos Movie, evita duplicados y aplica MAX_FAVORITES = 20. No se envían al backend y no están vinculados a una cuenta de MySQL. Por esa razón sí resulta adecuado conservarlos en el navegador.

```

const favorites = ref<Movie[]>([]);
const favoritesCount = computed(() => favorites.value.length);

function isFavorite(id: number) { ... }
function addFavorite(movie: Movie) { ... }
function removeFavorite(id: number) { ... }
function clearFavorites() { ... }

// configuración del store
persist: true

```

12.3.1 Qué aporta pinia-plugin-persistedstate

Pinia por sí mismo mantiene estado en memoria. pinia-plugin-persistedstate observa los stores configurados con persist: true, serializa su estado en localStorage y lo restaura cuando la aplicación se crea de nuevo. Así, un F5 no borra la lista de favoritos.

```

addFavorite(movie)
  -> favorites cambia
  -> Vue actualiza corazones y contador
  -> pinia-plugin-persistedstate serializa el store
  -> localStorage

F5
  -> se crea Pinia
  -> se instancia el store
  -> el plugin restaura el estado
  -> favorites vuelve a contener las películas guardadas

```

Implicación

Los favoritos pertenecen al navegador, no al usuario autenticado. Dos cuentas que utilicen el mismo perfil del navegador comparten esa lista local; cambiar de navegador o borrar localStorage puede eliminarla.

12.4 TanStack Vue Query: datos remotos, caché y ciclo de carga

TanStack Vue Query administra los datos que la aplicación obtiene de forma asíncrona, especialmente desde TMDB. A este tipo de información se lo suele llamar server state: CineVerse no controla su origen, debe pedirlo por HTTP y necesita saber si está cargando, si falló, si puede reutilizar un resultado previo o si todavía existen páginas por obtener.

Sin Vue Query, cada vista tendría que programar manualmente variables para data, loading y error, decidir cuándo repetir una solicitud, conservar páginas anteriores y evitar consultas innecesarias. Vue Query se encarga de gestionar ese proceso, mientras Axios sigue realizando las peticiones HTTP y Pinia continúa administrando el estado global de la aplicación.

```

Componente / composable
  ↓
TanStack Vue Query
  queryKey + queryFn + estados
  ↓
service del frontend
  ↓
Axios
  ↓
TMDB / API

```

12.4.1 VueQueryPlugin y el acceso desde los composables

main.ts registra VueQueryPlugin mediante app.use(VueQueryPlugin). Esa instalación incorpora la infraestructura de TanStack Query a la aplicación Vue. Después, composables como useHomeMovies, useMoviesQuery, useMovieDetail y usePersonDetail pueden crear consultas con useQuery() o useInfiniteQuery().

```

app
  .use(router)
  .use(VueQueryPlugin)
  .use(i18n)
  .mount("#app");

```

12.4.2 queryKey: la identidad de los datos

Cada consulta tiene una queryKey. No es una etiqueta decorativa: identifica qué información representa el resultado almacenado por Vue Query. Si cambia una parte material de la key, Vue Query trata la nueva combinación como datos diferentes.

```

["movies", "top_rated", "es"]
  !=
["movies", "top_rated", "en"]

["search", "Batman", [28], "es"]
  !=
["search", "Batman", [18], "es"]

```

El idioma forma parte de las claves relevantes. Por eso cambiar /es por /en no se limita a traducir botones: la key cambia y las consultas pueden solicitar a TMDB la versión localizada correspondiente.

12.4.3 queryFn: qué trabajo se ejecuta cuando la consulta necesita datos

queryFn es la función que Vue Query ejecuta para obtener la información. En CineVerse normalmente no contiene toda la lógica HTTP: llama a un service, y ese service utiliza la instancia de Axios adecuada. Esta separación mantiene al composable concentrado en estado de consulta y al service concentrado en endpoints y parámetros.

12.4.4 useQuery(): consultas que representan un resultado

useQuery() se utiliza cuando la consulta representa esencialmente un resultado identificable: una fila de la Home, el detalle de una película, el detalle de un actor o el catálogo de géneros. Expone data y estados como isLoading o isError y puede volver a ejecutar la consulta según su configuración.

12.4.5 useInfiniteQuery(): resultados formados por varias páginas

El explorador necesita acumular páginas. useInfiniteQuery() conserva cada respuesta dentro de data.pages y ofrece fetchNextPage(), hasNextPage e isFetchingNextPage. getNextPageParam() decide qué página corresponde pedir después y devuelve undefined cuando ya no existen páginas nuevas.

```
getNextPageParam: (lastPage) => {
  if (lastPage.page < lastPage.total_pages) {
    return lastPage.page + 1;
  }
  return undefined;
}
```

MovieResults une las páginas con flatMap y después aplica sus reglas visuales, como descartar películas sin póster. De esta manera el usuario ve una lista continua aunque TMDB siga entregando datos por páginas.

12.4.6 Frescura y permanencia: staleTime y gcTime

staleTime indica durante cuánto tiempo un resultado se considera reciente antes de quedar marcado como stale. gcTime indica cuánto tiempo puede conservarse una consulta sin consumidores antes de que Vue Query la elimine de memoria. Son decisiones distintas: conservar un dato no significa necesariamente considerarlo fresco.

Área	Configuración / intención observada
Home	Tres consultas independientes. Los datos se consideran recientes durante 5 minutos y se conserva temporalmente la versión anterior al cambiar el idioma.
Explorador	useInfiniteQuery; staleTime y gcTime en 0 para priorizar resultados actuales y no conservar la consulta inactiva.
Detalle de película	Key por id e idioma; retry=false; primero obtiene el detalle y después créditos, videos y similares.
Actor	Key por id e idioma; retry=false; la respuesta incluye movie_credits.

12.4.7 keepPreviousData y continuidad visual

En la Home, al cambiar el idioma cambia la queryKey. Mientras llega la nueva respuesta, keepPreviousData permite conservar temporalmente el resultado anterior en vez de vaciar de inmediato el carrusel. No evita la nueva solicitud: evita un salto visual innecesario mientras esa solicitud termina.

12.4.8 enabled, reintentos y refetch

Las opciones de cada consulta permiten decidir cuándo puede ejecutarse y cuándo conviene repetirla. enabled puede impedir una consulta hasta que exista un id o una condición válida; retry controla los reintentos

automáticos; `refetchOnWindowFocus` y `refetchOnMount` determinan si el regreso a una ventana o el montaje de un componente deben provocar una nueva solicitud.

Detalle de la implementación actual

En `useMoviesQuery.ts`, la expresión de `enabled` termina con `|| true`. Por eso actualmente queda siempre habilitada y las condiciones anteriores no deciden el resultado. Es un detalle de implementación que puede simplificarse o corregirse sin cambiar el papel general de `Vue Query`.

12.4.9 Ejemplo real: las tres consultas de la Home

La Home no espera a reunir todos los datos en una única operación. Crea consultas independientes para tendencias, mejor valoradas y cartelera. Si tendencias responde primero, esa fila puede mostrar contenido mientras las otras continúan cargando. Cada `queryKey` incluye el idioma, por lo que ES y EN mantienen resultados diferenciados.

```

HomeView
-> useHomeMovies(locale)
  -> useQuery(trending)
  -> useQuery(top_rated)
  -> useQuery(now_playing)
-> cada consulta actualiza su propia data
-> HomeRow / HomeCarousel

```

12.4.10 Pinia, Vue Query y Axios: quién hace qué

Herramienta	Responsabilidad en CineVerse	No debe confundirse con
Pinia	Estado compartido propio de la SPA.	Caché de respuestas remotas.
Vue Query	Ciclo de vida, caché, paginación y estados de datos remotos.	El cliente HTTP que envía la request.
Axios	Construye y envía solicitudes HTTP a TMDB o al backend PHP.	Un store global o un sistema de caché.
Persistedstate	Persiste stores seleccionados de Pinia en el navegador.	La sesión autenticada del servidor.

12.5 sessionStorage y localStorage

Dato	Dónde se guarda	Duración / propósito
Último query de filtros	sessionStorage	Vive en la pestaña actual y permite reconstruir filtros al volver al explorador.
Favoritos	Pinia + localStorage	Sobrevive a recargas y nuevas sesiones del navegador.
Banner de Firefox descartado	localStorage	Evita mostrar nuevamente el aviso después de cerrarlo.
Orden y estado visual de paneles	localStorage	Recupera preferencias de la interfaz entre visitas.

`sessionStorage` y `localStorage` son mecanismos del navegador, no bases de datos del backend. `sessionStorage` se aísla por pestaña; `localStorage` puede permanecer después de cerrar y volver a abrir el sitio. Ninguno de los dos reemplaza MySQL para información que debe pertenecer a una cuenta o garantizar integridad en el servidor.

12.6 Librerías especializadas de la interfaz

Además de Pinia, Vue Query, Router, i18n y Axios, CineVerse utiliza librerías especializadas para resolver controles concretos de la interfaz. No todas se registran globalmente con `app.use()`: Swiper, vue-multiselect, vue-slider-component, vuedraggable y lucide-vue-next se importan únicamente en los componentes que las necesitan. La idea es delegar una interacción específica a una librería y mantener en CineVerse la lógica de negocio y el estado de la pantalla.

12.6.1 Swiper: carruseles de la Home

Swiper proporciona el motor del carrusel principal. HomeCarousel.vue importa los componentes Swiper y SwiperSlide y los módulos Navigation, Autoplay y EffectFade. La librería resuelve el movimiento entre slides, el autoplay y la transición; CineVerse conserva el control de cuándo el carrusel está realmente listo y de qué película se muestra.

```
import { Swiper, SwiperSlide } from "swiper/vue";
import { Navigation, Autoplay, EffectFade } from "swiper/modules";

<Swiper
  :modules="[Navigation, Autoplay, EffectFade]"
  effect="fade"
  :autoplay="{ delay: 3500, disableOnInteraction: false }"
  @swiper="onSwiper"
  @slideChange="onSlideChange"
>
  <SwiperSlide v-for="(movie, i) in movies" :key="movie.id">
    <MovieCardCarousel
      :movie="movie"
      :priority="i === 0"
      @loaded="i === 0 && onImageLoaded()"
    />
  </SwiperSlide>
</Swiper>
```

La propiedad `:modules` indica qué módulos de Swiper necesita esta instancia. `effect="fade"` cambia la transición a fundido y `autoplay` hace avanzar automáticamente el carrusel. Los eventos `@swiper` y `@slideChange` vuelven a entrar en código propio de CineVerse. HomeCarousel guarda la instancia recibida, calcula los extremos mediante `activeIndex` y emite `ready` sólo cuando Swiper terminó de inicializarse y la primera imagen visible ya fue pintada. Por eso Swiper controla el carrusel, pero no sustituye la lógica de carga de la Home.

12.6.2 vue-multiselect: selección de opciones en los filtros

MovieFilters.vue utiliza vue-multiselect para los selectores de tipo de catálogo y ordenamiento. El componente recibe una lista de objetos con `label` y `value`, muestra el texto traducido y devuelve el objeto elegido mediante `@select`. CineVerse transforma después esa elección en el valor reactivo que utilizan `useMoviesFilters` y las consultas.

```
import Multiselect from "vue-multiselect";

<Multiselect
  :model-value="listTypesOptions.find((o) => o.value === listType)"
  :options="listTypesOptions"
  label="label"
  track-by="value"
  :searchable="false"
  @select="(o: ListTypeOption) => (listType = o.value)"
  @clear="listType = 'discover'"
/>
```

`options` contiene las alternativas disponibles; `label` indica qué propiedad se muestra al usuario y `track-by` identifica cada opción por `value`. En lugar de guardar el objeto completo del selector, el evento `@select` toma `o.value` y actualiza `listType`. Ese valor termina participando en la URL, en la `queryKey` de Vue Query y en la decisión de qué endpoint de TMDb consultar. La librería resuelve el control visual; el significado del valor sigue perteneciendo a CineVerse.

12.6.3 vue-slider-component: rangos de año y puntuación

Los rangos de año y rating utilizan vue-slider-component. MovieFilters mantiene valores temporales para el movimiento del control y sólo copia esos valores a los filtros reales cuando termina el arrastre. De esta manera, mover el mouse dentro del slider no provoca una consulta nueva por cada cambio intermedio.

```
import VueSlider from "vue-slider-component";

const sliderRating = ref([
  ratingRange.value?.[0] ?? 0,
  ratingRange.value?.[1] ?? 10,
]);

function applyRatingFilter() {
  const [min, max] = sliderRating.value;
  ratingRange.value = [min ?? 0, max ?? 10];
}

<VueSlider
  v-model="sliderRating"
  :min="0"
  :max="10"
  :interval="0.5"
  :marks="ratingMarks"
  :clickable="false"
  @drag-end="applyRatingFilter"
/>
```

v-model mantiene sliderRating sincronizado con la posición visual del control. @drag-end se ejecuta al terminar el movimiento y recién entonces applyRatingFilter() actualiza ratingRange, que es el filtro utilizado por el resto de la aplicación. El slider de años aplica el mismo patrón con sliderYears, yearFrom y yearTo.

12.6.4 vuedraggable: reordenamiento y persistencia de paneles

vuedraggable se utiliza para permitir que el usuario cambie el orden de los paneles del sidebar. La librería trabaja directamente con el array reactivo panels. Al arrastrar un elemento, v-model modifica el orden del array; como Vue renderiza los paneles a partir de ese mismo array, la interfaz se reordena sin reconstruir manualmente el DOM.

```
import draggable from "vuedraggable";

const panels = ref<PanelId[]>(
  readPanelsOrder() ?? DEFAULT_PANELS_ORDER
);

<draggable
  v-model="panels"
  item-key="id"
  :disabled="!customizingPanels"
  ghost-class="drag-ghost"
  animation="200"
>
  ...
</draggable>

watch(panels, (newOrder) => {
  writePanelsOrder(newOrder);
}, { deep: true });
```

item-key permite identificar cada panel y :disabled evita el drag & drop fuera del modo de personalización. Después del cambio, un watch llama writePanelsOrder(), que serializa el array en localStorage con la clave moviesFilters:panelsOrder. Por eso el nuevo orden no sólo cambia visualmente en ese momento: también puede recuperarse cuando el usuario vuelve al explorador.

12.6.5 lucide-vue-next: iconos como componentes Vue

lucide-vue-next proporciona iconos SVG como componentes Vue. CineVerse los importa por nombre dentro de las pantallas que los necesitan, por lo que no hace falta mantener archivos SVG separados ni repetir el marcado completo de cada icono.

```
import {
  MessageSquareText,
  Pencil,
  Trash2,
  AlertTriangle,
} from "lucide-vue-next";

<AlertTriangle :size="28" />
<Pencil :size="15" />
<Trash2 :size="15" />
```

La prop `size` modifica el tamaño del SVG generado por el componente. El texto del botón, los eventos y los estilos continúan perteneciendo a CineVerse. Por ejemplo, `MovieReviews` combina `Pencil` y `Trash2` con las acciones de editar y eliminar; `lucide-vue-next` sólo aporta la representación gráfica.

12.6.6 Qué responsabilidad conserva CineVerse

Estas librerías no reemplazan la arquitectura de la aplicación. `Swiper` no decide qué películas pedir; `Multiselect` y `VueSlider` no construyen las consultas a TMDB; `vuedraggable` no define qué significa un panel; y `Lucide` no ejecuta acciones. Los componentes Vue conectan cada librería con refs, props, eventos, composables, stores y servicios. Esa separación permite cambiar un control visual sin trasladar la lógica de negocio al paquete externo.

13. Navegación sin recargar la página: SPA y Vue Router

Como CineVerse es una SPA (Single Page Application), el navegador carga la aplicación una vez y Vue cambia la vista visible sin descargar un documento HTML completo en cada navegación. Vue Router es la biblioteca que mantiene sincronizadas la URL y la pantalla que debe mostrar la aplicación.

Dentro del proyecto, Vue Router no es una idea abstracta ni una función del backend. Está instalado como dependencia del frontend, se configura en `src/core/router/index.ts`, se registra globalmente en `main.ts` y después los componentes o composables lo utilizan para leer la ruta actual o iniciar una navegación.

Pieza	Ubicación / función
Dependencia	frontend/package.json → vue-router
Configuración central	frontend/src/core/router/index.ts
Registro global	frontend/src/main.ts → app.use(router)
Lugar donde se muestra la vista	frontend/src/App.vue → RouterView
Uso desde componentes	useRouter(), useRoute(), router.push()
Actualización de query string	router.replace() en useMoviesFilters.ts

13.1 Dónde está Vue Router y cómo entra en la aplicación

`package.json` declara `vue-router` como dependencia del frontend. `npm install` instala el paquete dentro de `node_modules` y TypeScript puede importarlo por el nombre del paquete. La configuración propia de CineVerse no vive en `node_modules`: está en `src/core/router/index.ts`.

```
"vue-router": "^5.0.3"
```

`main.ts` importa la instancia ya configurada y la registra con `app.use(router)`. Ese registro conecta Vue Router con toda la aplicación; a partir de ese momento `RouterView`, `useRouter()`, `useRoute()` y los componentes de navegación pueden trabajar con la misma instancia.

```
import router from "../core/router";

app
  .use(router)
  .use(VueQueryPlugin)
  .use(i18n)
  .mount("#app");
```

13.2 createRouter(), createWebHistory() y la instancia central

src/core/router/index.ts importa createRouter y createWebHistory desde vue-router y crea una única instancia central. createRouter() recibe la configuración completa del enrutador. createWebHistory(import.meta.env.BASE_URL) habilita URLs normales, sin #, y respeta la raíz configurada por Vite. scrollBehavior() devuelve { top: 0 } para que cada navegación comience desde la parte superior de la nueva vista.

```
import { createRouter, createWebHistory } from "vue-router";

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),

  scrollBehavior() {
    return { top: 0 };
  },

  routes: [
    // ... rutas de CineVerse
  ],
});

export default router;
```

Idea clave

createRouter() crea y configura el enrutador; no navega por sí solo. Las navegaciones ocurren después, cuando cambia la URL, el usuario utiliza un RouterLink o el código llama a router.push() o router.replace().

13.3 Cómo se declara una ruta: path, name, component, params y meta

Cada objeto dentro de routes relaciona una URL con una vista. path define el patrón que debe coincidir, name permite referirse a la ruta sin escribir manualmente la URL y component indica qué componente Vue debe mostrar RouterView. Los parámetros dinámicos comienzan con :, y meta guarda información adicional que el guard puede consultar.

```
{
  path: "/:lang(en|es)/movies",
  name: "Movies",
  component: MoviesView,
},
{
  path: "/:lang(en|es)/movies/:id(\\d+)",
  name: "MovieDetail",
  component: MovieDetailView
},
{
  path: "/:lang(en|es)/profile",
  name: "Profile",
  component: ProfileView,
  meta: { requiresAuth: true }
}
```

/:lang(en|es) restringe el idioma a en o es. En la ruta de detalle, :id(\\d+) acepta únicamente ids numéricos. meta: { requiresAuth: true } no bloquea la ruta por sí mismo: marca la ruta para que el guard global aplique la comprobación de autenticación antes de completarla.

Ruta	Vista / propósito
/:lang	HomeView
/:lang/movies	Explorador de películas
/:lang/movies/:id	Detalle de película; id sólo numérico
/:lang/person/:id	Detalle de actor; id sólo numérico
/:lang/favorites	Favoritos locales
/:lang/profile	Perfil; requiresAuth
/:lang/profile/reviews	Mis reseñas; requiresAuth
/:lang/login	Logín; requiresGuest
/:lang/register	Registro; requiresGuest
/:lang/forgot-password	Recuperación; requiresGuest
/:lang/reset-password	Nueva contraseña

La ruta raíz tiene un comportamiento distinto: cuando el usuario entra a /, la función `redirect` consulta `navigator.language`. Si el idioma del navegador empieza por `es`, redirige a `/es`; en cualquier otro caso utiliza `/en`.

```
{
  path: "/",
  redirect: () => {
    const browserLang = navigator.language.toLowerCase();

    if (browserLang.startsWith("es")) {
      return "/es";
    }

    return "/en";
  },
}
```

13.4 Guards: cómo `requiresAuth` y `requiresGuest` se convierten en una decisión

Un guard es una función que se ejecuta antes de completar una navegación. CineVerse utiliza un guard global con `router.beforeEach()`. Primero sincroniza `:lang` con `vue-i18n`. Después sólo espera la reconstrucción de sesión cuando la ruta necesita conocer el estado de autenticación: `requiresAuth` protege `Profile` y `MyReviews`; `requiresGuest` evita mostrar `Login`, `Register` o `ForgotPassword` a un usuario que ya tiene sesión.

```

router.beforeEach(async (to) => {
  const lang = to.params.lang as string;

  if (lang === "en" || lang === "es") {
    i18n.global.locale.value = lang;
  }

  if (to.meta.requiresAuth || to.meta.requiresGuest) {
    const authStore = useAuthStore();

    if (!authStore.isInitialized) {
      await authStore.initialize();
    }

    if (to.meta.requiresAuth && !authStore.isAuthenticated) {
      return {
        name: "Login",
        params: { lang: lang || i18n.global.locale.value }
      };
    }

    if (to.meta.requiresGuest && authStore.isAuthenticated) {
      return {
        name: "Home",
        params: { lang: lang || i18n.global.locale.value }
      };
    }
  }

  return true;
});

```

to representa la ruta que se intenta abrir. to.meta contiene los flags definidos en la declaración de la ruta. Si la sesión todavía no fue reconstruida, el guard espera authStore.initialize(); si la decisión permite continuar devuelve true, y si no devuelve un objeto de navegación hacia Login o Home.

```

ruta con meta.requiresAuth
    ↓
router.beforeEach()
    ↓
useAuthStore()
    ↓
authStore.initialize() si hace falta
    ↓
authStore.isAuthenticated
    ↓
permitir navegación o redirigir a Login

```

13.5 Rutas 404 y parámetros inválidos

Las rutas de captura general aparecen después de las rutas válidas. `/:lang(en|es)/:pathMatch(.*)*` conserva el idioma cuando una dirección con `/es` o `/en` no coincide con ninguna pantalla; `/:pathMatch(.*)*` captura el resto. Como las rutas de película y persona aceptan únicamente ids numéricos, una dirección como `/es/movies/abc` llega a `NotFoundView` sin enviar una consulta innecesaria a TMDB.

13.6 Cómo un componente invoca Vue Router: `useRouter()`, `useRoute()` y `router.push()`

La configuración central sólo define las reglas. Los componentes utilizan esas reglas mediante composables de Vue Router. `MovieCard.vue` importa `useRouter()` para navegar y `useRoute()` para leer el idioma de la URL actual. Cuando el usuario pulsa una tarjeta, `goToMovie()` navega por nombre de ruta y completa los parámetros `lang` e `id`.

```
import { useRoute, useRouter } from "vue-router";

const router = useRouter();
const route = useRoute();

function goToMovie(id: number) {
  router.push({
    name: "MovieDetail",
    params: {
      lang: route.params.lang,
      id,
    },
  });
}
```

`useRouter()` devuelve la instancia que permite iniciar una navegación. `useRoute()` expone la ruta activa y sus parámetros. `router.push()` agrega una nueva entrada al historial del navegador. Usar `name: "MovieDetail"` evita construir manualmente una URL y deja que Vue Router aplique el patrón declarado en `router/index.ts`.

Los filtros utilizan otra operación: `router.replace({ query })`. `replace` cambia la URL actual sin agregar una nueva entrada al historial. Por eso mover varias veces un slider no obliga al usuario a pulsar Atrás una vez por cada valor intermedio.

```
// useMoviesFilters.ts
router.replace({ query });
```

13.7 RouterView y routeKey: dónde aparece la vista seleccionada

`App.vue` contiene `RouterView`, que es el punto donde Vue Router entrega el componente correspondiente a la URL actual. CineVerse utiliza el slot de `RouterView` para obtener `Component` y `route`, y aplica una `key` calculada. `routeKey()` excluye `lang`: cambiar sólo el idioma reutiliza la vista actual; cambiar un `id` produce una `key` diferente y fuerza un nuevo montaje del componente.

```
function routeKey(route: RouteLocationNormalizedLoaded) {
  const { lang, ...rest } = route.params;
  return `${route.name as string}-${JSON.stringify(rest)}`;
}

<RouterView v-slot="{ Component, route }">
  <component
    :is="Component"
    :key="routeKey(route)"
  />
</RouterView>
```

Esto evita conservar accidentalmente los datos de una película al pasar, por ejemplo, de `/es/movies/550` a `/es/movies/272`. En cambio, `/es/movies/550` → `/en/movies/550` mantiene la misma `key` y permite que `vue-i18n` y las consultas reactivas actualicen el idioma sin destruir toda la vista.

13.8 Ejemplo completo: desde MovieCard hasta MovieDetailView

```
Usuario pulsa MovieCard con id 550
-> @click ejecuta goToMovie(550)
-> router.push({ name: "MovieDetail", params: { lang, id: 550 } })
-> beforeEach sincroniza el idioma y aplica guards si correspondiera
-> Router resuelve /es/movies/550
-> RouterView recibe MovieDetailView
-> routeKey contiene id=550
-> Vue monta la vista
-> useMovieDetail(550, locale)
-> service / Axios / TMDB
```

En este recorrido Vue Router no obtiene los datos de la película. Su responsabilidad termina en resolver la navegación y mostrar `MovieDetailView`. La carga de información se delega después al composible, `Vue Query`, los `services` y `Axios`.

14. Idiomas: vue-i18n y cambio de idioma

i18n es una abreviatura utilizada en desarrollo para internationalization, es decir, internacionalización. CineVerse puede mostrar su interfaz en español o inglés, pero el cambio de idioma no afecta únicamente palabras visibles como “Inicio” o “Películas”. También modifica el parámetro lang de la URL, cambia el idioma utilizado en determinadas consultas a TMDB, adapta el formato de algunas fechas y determina el idioma de los correos de verificación y recuperación enviados por el backend.

CineVerse utiliza vue-i18n para los textos propios de la interfaz. Las traducciones en español están en frontend/src/core/i18n/es.json y las traducciones en inglés en frontend/src/core/i18n/en.json. En los ejemplos siguientes se sigue el recorrido completo desde lo que hace Carlos hasta el archivo y la función que producen el cambio.

14.1 Ejemplo guiado: Carlos cambia el menú de español a inglés

Carlos está utilizando CineVerse en español. En el encabezado ve Inicio, Películas y Favoritos; el buscador muestra “Buscar una película” y el selector de idioma tiene ES activo.



Interfaz en español antes del cambio de idioma: navegación, buscador y selector ES / EN.

Carlos pulsa EN. La primera pieza que recibe esa acción es AppHeader.vue: el botón no traduce textos por sí mismo, sino que llama a una función que pide a Vue Router navegar a la misma pantalla utilizando otro valor para lang.

Ubicación en el proyecto: frontend/src/layouts/components/AppHeader.vue · Component · selector ES / EN y comienzo del cambio de idioma

```
<button
  @click="switchLang('en')"
  :class="{ active: locale === 'en' }"
  :disabled="locale === 'en'"
>
  EN
</button>
```

La directiva @click indica qué debe ejecutar Vue cuando el usuario pulsa el botón. En este caso se invoca switchLang("en"). El botón además se marca como activo cuando locale vale "en" y queda deshabilitado mientras ese idioma ya esté seleccionado.

Ubicación en el proyecto: frontend/src/layouts/components/AppHeader.vue · Component · función switchLang()

```
function switchLang(lang: string) {
  router.push({
    name: route.name as string,
    params: { ...route.params, lang },
  });
}
```

switchLang() conserva el nombre de la ruta y los parámetros actuales —por ejemplo, el id de una película o de una persona— y reemplaza lang. Así, una dirección como /es/person/1892 pasa a /en/person/1892 sin enviar a Carlos a otra funcionalidad.

Carlos pulsa EN

- > AppHeader ejecuta switchLang("en")
- > router.push(...) conserva la ruta y cambia lang
- > /es/... pasa a /en/...
- > el guard global del Router lee lang="en"
- > i18n.global.locale.value pasa a "en"
- > vue-i18n vuelve a resolver las claves t(...)
- > Vue actualiza los textos visibles

Ubicación en el proyecto: frontend/src/core/router/index.ts · Router · sincronización entre el parámetro lang de la URL y vue-i18n

```
router.beforeEach(async (to) => {
  const lang = to.params.lang as string;

  if (lang === "en" || lang === "es") {
    i18n.global.locale.value = lang;
  }

  // ... guards de autenticación
});
```

beforeEach se ejecuta antes de cada navegación. Si el parámetro de la ruta es es o en, el Router copia ese valor al locale global de vue-i18n. Como locale es reactivo, los componentes que utilizan traducciones se vuelven a evaluar automáticamente.



Resultado después de pulsar EN: Home, Movies, Favorites y Search for a movie aparecen en inglés.

Carlos sigue dentro de CineVerse y en la misma clase de pantalla. No se descarga otra página HTML completa: la SPA conserva la navegación, cambia el idioma activo y Vue vuelve a representar las partes que dependen de ese valor.

14.2 Cómo vue-i18n obtiene “Películas” o “Movies”

Los componentes no escriben dos versiones de cada frase. En lugar de decidir manualmente si deben mostrar “Películas” o “Movies”, solicitan una clave mediante la función t(). vue-i18n busca esa clave dentro del diccionario correspondiente al locale activo.

Ubicación en el proyecto: frontend/src/layouts/components/AppHeader.vue · Component · uso de claves de traducción dentro del encabezado

```
{{ t("nav.home") }}
{{ t("nav.movies") }}
{{ t("nav.favorites") }}

:placeholder=t('header.searchPlaceholder')
```

t("nav.movies") no contiene la palabra visible. Sólo identifica qué texto necesita el componente. El valor concreto está en los archivos de idioma.

Ubicación en el proyecto: frontend/src/core/i18n/es.json · Diccionario · textos propios de la interfaz en español

```
"nav": {
  "home": "Inicio",
  "movies": "Películas",
  "favorites": "Favoritos"
},
"header": {
  "searchPlaceholder": "Buscar una película"
}
```

Ubicación en el proyecto: frontend/src/core/i18n/en.json · Diccionario · textos propios de la interfaz en inglés

```
"nav": {
  "home": "Home",
  "movies": "Movies",
  "favorites": "Favorites"
},
"header": {
  "searchPlaceholder": "Search for a movie"
}

t("nav.movies")
-> locale = "es" -> es.json -> "Películas"
-> locale = "en" -> en.json -> "Movies"
```

Ubicación en el proyecto: frontend/src/core/i18n/index.ts · Configuración · creación de la instancia global de vue-i18n

```
const i18n = createI18n({
  legacy: false,
  locale: "es",
  fallbackLocale: "en",
  messages: {
    en,
    es,
  },
});
```

locale indica el idioma activo. messages registra los dos diccionarios disponibles. fallbackLocale: "en" establece el inglés como idioma de respaldo si una clave no existe en el idioma activo. legacy: false habilita la API moderna de vue-i18n utilizada con Vue 3 y useI18n().

En la revisión actual del código, es.json contiene 210 claves terminales y en.json 208. En inglés faltan auth.success.resendSuccessTitle y auth.success.resendSuccessSubtitle, utilizadas por el flujo exitoso de reenvío de verificación. El fallback ayuda cuando existe una traducción alternativa, pero no sustituye la necesidad de mantener ambos diccionarios sincronizados.

14.3 Ejemplo guiado: Carlos cambia el idioma de la ficha de Matt Damon

El cambio de idioma también puede modificar contenido que no pertenece a los diccionarios de CineVerse. Carlos abre la ficha de Matt Damon con ES activo. La navegación está en español y la biografía recibida desde TMDB aparece en español.



Ficha de Matt Damon con ES activo: navegación y biografía mostradas en español.

Carlos pulsa EN sin abandonar la ficha. La ruta cambia de idioma y PersonDetailView sigue utilizando el mismo id de persona, pero ahora recibe locale="en". Ese locale no sólo afecta t(...): también forma parte de la consulta de datos de la persona.

La biografía no sale de en.json. Los diccionarios traducen textos propios de CineVerse; la biografía pertenece a los datos externos de TMDB y se vuelve a solicitar con otro valor de language.

Ubicación en el proyecto: frontend/src/features/person/views/PersonDetailView.vue · View · obtención del locale y llamada al composable de detalle de persona

```
const { t, locale } = useI18n();

const { data, isError } = usePersonDetail(id, locale);
```

PersonDetailView no traduce la biografía. Su responsabilidad es obtener el id desde la ruta, acceder al locale activo y entregar ambos valores a usePersonDetail().

Ubicación en el proyecto: frontend/src/features/person/composables/usePersonDetail.ts · Composable · consulta reactiva identificada por persona e idioma

```
return useQuery<PersonDetail>({
  queryKey: ["person-detail", id, language],
  enabled: Number.isFinite(id) && id > 0,
  retry: false,
  refetchOnWindowFocus: false,
  queryFn: async () => {
    return await getFullPersonDetail(id, language.value);
  },
});
```

La queryKey incluye el id y el idioma. Cuando Carlos cambia ES por EN, el valor reactivo de language cambia y Vue Query trata el detalle en inglés como una consulta distinta. Entonces queryFn vuelve a ejecutar getFullPersonDetail() con el nuevo idioma.

Ubicación en el proyecto: frontend/src/features/person/services/person.service.ts · Service del frontend · solicitud HTTP del detalle de persona a TMDB

```
const { data } = await api.get<PersonDetail>(`/person/${personId}`, {
  params: {
    language,
    append_to_response: "movie_credits",
  },
});
```

person.service.ts envía language como parámetro de la solicitud a TMDB. append_to_response="movie_credits" pide además que la filmografía se incluya en la misma respuesta. Cuando TMDB responde, PersonHero recibe el nuevo objeto person y muestra person.biography.

Carlos pulsa EN

```
-> locale cambia de "es" a "en"
-> PersonDetailView entrega el nuevo locale a usePersonDetail
-> cambia la queryKey ["person-detail", id, language]
-> Vue Query ejecuta getFullPersonDetail(id, "en")
-> person.service.ts envía GET /person/{id}?language=en
-> TMDB devuelve nuevamente los datos localizados
-> PersonHero muestra person.biography
-> Carlos ve la biografía en inglés
```



La misma ficha después de pulsar EN: el menú cambia por vue-i18n y la biografía vuelve desde TMDB en inglés.

14.4 Traducir la interfaz y localizar datos no es lo mismo

Los ejemplos anteriores usan un único idioma activo, pero no todos los textos provienen del mismo lugar. La interfaz propia se resuelve con `vue-i18n`; el contenido externo se solicita a TMDB con `language`; las fechas se formatean con `Intl.DateTimeFormat`; y los correos utilizan plantillas PHP del backend.

Qué cambia	De dónde sale	Mecanismo
Inicio → Home	es.json / en.json	vue-i18n resuelve t("nav.home") según locale.
Películas → Movies	es.json / en.json	vue-i18n resuelve t("nav.movies").
Buscar una película → Search for a movie	es.json / en.json	El placeholder usa t("header.searchPlaceholder").
Biografía de Matt Damon	TMDB	PersonDetail vuelve a consultarse con language="es" o "en".
Datos localizados de películas y personas	TMDB	Los services envían el idioma activo como parámetro language.
Formato visible de una fecha	Intl.DateTimeFormat	useLocalizedDate mapea es → es-AR y en → en-US.
Correo de verificación o recuperación	Backend PHP	MailService elige la plantilla .es.php o .en.php según language.

Esta separación es importante para leer el proyecto. Cambiar en.json no modifica la biografía que devuelve TMDB, y cambiar language en una solicitud a TMDB no traduce por sí solo los botones escritos por CineVerse.

14.5 El idioma también cambia el formato de las fechas

La fecha visible de una persona tampoco se escribe manualmente con un orden fijo. PersonHero utiliza useLocalizedDate(), un composable común que observa el locale actual y crea un formateador regional.

Ubicación en el proyecto: frontend/src/features/person/components/PersonHero.vue · Component · visualización de la fecha de nacimiento

```
{{ formatDate(person.birthday) }}
```

Ubicación en el proyecto: frontend/src/core/composables/useLocalizedDate.ts · Composable · mapeo del idioma de CineVerse a un locale de Intl.DateTimeFormat

```
const LOCALE_MAP: Record<string, string> = {
  es: "es-AR",
  en: "en-US"
};

const dateFormatter = computed(() => {
  const resolvedLocale = LOCALE_MAP[locale.value] ?? "es-AR";

  return new Intl.DateTimeFormat(resolvedLocale, {
    day: "numeric",
    month: "numeric",
    year: "numeric"
  });
});
```

Con ES activo, CineVerse utiliza es-AR; con EN activo, utiliza en-US. Por eso la misma fecha puede presentarse con orden día/mes/año o mes/día/año. Esto muestra que internacionalizar una interfaz no consiste solamente en reemplazar palabras: también puede adaptar la representación de valores según las convenciones regionales.

Detalle de implementación actual. formatDate() construye un objeto Date a partir del string recibido. Cuando TMDB entrega una fecha sin hora, JavaScript puede interpretarla en UTC y después convertirla a la zona horaria local. En las capturas de Matt Damon se observa el efecto: la biografía menciona el 8 de octubre, mientras la etiqueta de fecha aparece como 7/10/1970 en español y 10/7/1970 en inglés. El cambio de orden entre ambas etiquetas sí corresponde a es-AR frente a en-US; el desplazamiento de un día es un efecto independiente del parseo de fecha de la implementación actual.

14.6 El idioma también llega al backend y a los correos

El mismo locale se reutiliza en operaciones del frontend que pueden producir un correo. Supongamos que Carlos tiene CineVerse en inglés y crea una cuenta: RegisterView incluye `language="en"` dentro del JSON enviado a `POST /register`.

Ubicación en el proyecto: `frontend/src/features/auth/views/RegisterView.vue` · View · envío del idioma activo durante el registro

```
const lang = locale.value as "es" | "en";

await registerUser({
  username,
  email,
  password,
  password_confirmation,
  language: lang
});
```

Carlos usa CineVerse en inglés

```
-> locale.value = "en"
-> RegisterView incluye language: "en"
-> POST /register
-> AuthController
-> AuthValidator comprueba que sea "es" o "en"
-> AuthService
-> MailService
-> verify-account.en.php
-> Carlos recibe el correo de verificación en inglés
```

El backend no intenta adivinar el idioma a partir del navegador, del email ni de la ubicación del usuario. Utiliza el valor que recibió explícitamente del frontend. El mismo criterio se aplica al reenvío de verificación y a la recuperación de contraseña, donde MailService elige entre las plantillas `.es.php` y `.en.php`. La sección 25 desarrolla el proceso completo de autenticación y correo.

14.7 Detalles de implementación actual

La internacionalización funciona en las áreas principales, pero el código actual conserva algunos comportamientos que conviene distinguir de la arquitectura general.

- `switchLang()` conserva `route.params` —por ejemplo el id de una película o persona—, pero no copia `route.query`. Si Carlos cambia de idioma desde una URL del explorador que contiene búsqueda, géneros, años u otros filtros, esos parámetros de consulta pueden perderse. Para conservarlos, la navegación tendría que incluir también `query: route.query`.

Ubicación en el proyecto: `frontend/src/features/person/components/PersonHero.vue` · Component · texto del botón de expansión de la biografía

```
{{ expanded ? 'Leer menos' : 'Leer más' }}
```

- “Leer más” y “Leer menos” están escritos directamente en español dentro de `PersonHero.vue`. Por eso la captura en inglés todavía muestra “Leer más”. En cambio, el mensaje de biografía ausente sí utiliza `t("personDetail.biographyNotAvailable")`. Para internacionalizar completamente el botón habría que mover ambos textos a `es.json` y `en.json` y resolverlos mediante `t(...)`.
- El formato regional de fechas y el desplazamiento de un día descrito en 14.5 son problemas distintos: `Intl.DateTimeFormat` cambia el orden visible según el locale, mientras el posible desplazamiento proviene de cómo se construye `Date` a partir de una fecha sin hora.

15. Comunicación entre el frontend, TMDB y el backend

CineVerse mantiene separadas tres comunicaciones: el frontend Vue consulta TMDB para obtener catálogo y personas; el mismo frontend consulta la API PHP para las funciones de cuenta y reseñas; y el backend PHP utiliza PDO para leer o escribir MySQL.

El navegador elige el camino según la acción del usuario. Las películas no pasan por PHP, mientras que una sesión, una reseña o el perfil sí pasan por el backend propio.

15.1 De Vue a TMDB: recorrido resumido

Cuando una pantalla necesita películas o actores, un service del frontend utiliza `tmdbApi.ts` para consultar TMDB directamente. Ese cliente añade la dirección base y el Bearer token; la respuesta vuelve al frontend para que Vue la represente. La sección 3 desarrolla desde cero qué es TMDB, cómo obtiene acceso un desarrollador y cómo una operación viaja desde una View hasta sus servidores.

```
Pantalla Vue
-> service de movies o person
-> tmdbApi.ts
-> TMDB
-> respuesta JSON
-> Vue Query / vista
```

En este camino PHP y MySQL no intervienen: el Bearer token identifica a CineVerse frente a TMDB y la respuesta vuelve directamente al frontend.

15.2 De Vue al backend PHP: ejemplo con el inicio de sesión

Cuando el usuario envía el formulario de login, `LoginView` llama al servicio de autenticación. `backendApi` envía `POST /login` con las credenciales y permite que el navegador gestione la cookie de sesión. El backend valida la entrada, consulta `users` en MySQL y devuelve el usuario y el token CSRF si el acceso es correcto.

```
Usuario pulsa "Ingresar"
-> LoginView.vue
-> auth.service.ts
-> backendApi.ts
-> POST /login
-> Router / AuthController
-> AuthService / User
-> PDO / MySQL
-> LOGIN_SUCCESS { user, csrf_token }
-> useAuthStore / Pinia
-> interfaz autenticada
```

Aquí sí intervienen la API propia, MySQL y la sesión PHP. Las operaciones posteriores que modifican datos combinan la cookie de sesión con X-CSRF-Token.

15.3 Axios, clientes HTTP y capa de services

Esta sección une las piezas que en secciones anteriores se presentaron por separado. Axios es la biblioteca que ejecuta solicitudes HTTP; `tmdbApi.ts` y `backendApi.ts` son archivos propios de CineVerse que crean clientes Axios configurados; y los archivos `*.service.ts` utilizan esos clientes para realizar operaciones concretas.

```
View / Component / Store / Composable
|
v
service
|
v
tmdbApi / backendApi
|
v
Axios
|
v
TMDB / API PHP
```

15.3.1 Axios: qué es, dónde se instala y qué responsabilidad tiene

Axios es una biblioteca JavaScript/TypeScript para realizar solicitudes HTTP. En CineVerse está declarada como dependencia en `frontend/package.json`. Al ejecutar `npm install`, `npm` instala el paquete dentro de `frontend/node_modules/axios`. El código propio puede entonces importarlo con `import axios from "axios"`.

```

frontend/package.json
"axios": "^1.13.6"
  |
  | npm install
  v
frontend/node_modules/axios/
  |
  v
import axios from "axios";

```

Axios no decide qué película buscar, no guarda la caché de Vue Query y no conoce las reglas de negocio del backend. Su responsabilidad es construir y enviar requests HTTP y entregar al código la respuesta o un error.

15.3.2 tmdbApi.ts: cliente Axios de CineVerse para TMDB

tmdbApi.ts es un archivo TypeScript propio del frontend de CineVerse; no forma parte de Axios ni de TMDB. Está ubicado en frontend/src/core/api/tmdbApi.ts. Su función es crear una única instancia Axios preparada para hablar con la API externa de TMDB y exportarla para que otros módulos la reutilicen.

El archivo lee VITE_TMDB_BASE_URL y VITE_TMDB_TOKEN desde import.meta.env. De esa manera concentra la URL base y la autenticación Bearer en un solo lugar. movies.service.ts y person.service.ts importan la instancia api y no necesitan repetir esas opciones en cada función.

```

// frontend/src/core/api/tmdbApi.ts (fragmento ejecutable real)
import axios from "axios";

const baseUrl = import.meta.env.VITE_TMDB_BASE_URL;
const token = import.meta.env.VITE_TMDB_TOKEN;

const api = axios.create({
  baseUrl: baseUrl,
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});

export default api;

```

axios.create() no envía todavía ninguna request. Crea una instancia reutilizable. baseUrl se antepone a los endpoints relativos y Authorization se agrega por defecto a las solicitudes realizadas con api. Content-Type: application/json establece JSON como tipo de contenido de las solicitudes que envían un cuerpo; no valida por sí mismo la estructura de la respuesta.

```

tmdbApi.ts
-> export default api
  |
  +--> movies.service.ts
  +--> person.service.ts
  |
  v
  TMDB

```

15.3.3 backendApi.ts: cliente Axios de CineVerse para el backend PHP

backendApi.ts también es un archivo TypeScript propio de CineVerse. Está ubicado en frontend/src/core/api/backendApi.ts. Su función es crear y exportar la instancia Axios utilizada para la API PHP del proyecto. A diferencia de tmdbApi.ts, no utiliza el token Bearer de TMDB: toma VITE_BACKEND_URL como dirección base y habilita withCredentials para que el navegador gestione la cookie de sesión.

```

// frontend/src/core/api/backendApi.ts (fragmento ejecutable real)
import axios from "axios";

const backendApi = axios.create({
  baseUrl: import.meta.env.VITE_BACKEND_URL,
  headers: {
    "Content-Type": "application/json"
  },
});

```

```

    withCredentials: true
  });

  export default backendApi;

```

withCredentials: true permite que las solicitudes de este cliente incluyan credenciales administradas por el navegador, como la cookie de sesión PHP. El token CSRF es otra pieza: las operaciones protegidas lo agregan explícitamente como X-CSRF-Token en la request correspondiente.

```

backendApi.ts
  -> export default backendApi
      |
      +--> auth.service.ts
      +--> reviews.service.ts
      +--> profile.service.ts
      |
      v
  backend PHP

```

15.3.4 La capa *.service.ts: para qué existe entre Vue y Axios

Un service es un módulo TypeScript propio de CineVerse que reúne operaciones relacionadas con una fuente de datos. El service no importa siempre axios directamente: normalmente importa api o backendApi, es decir, una instancia que ya contiene la configuración común. Esto evita repetir baseUrl, token Bearer o withCredentials en cada operación.

Service	Ubicación	Ciente utilizado	Destino
movies.service.ts	src/features/movies/services/	api de tmdbApi.ts	TMDB
person.service.ts	src/features/person/services/	api de tmdbApi.ts	TMDB
auth.service.ts	src/features/auth/services/	backendApi	API PHP
reviews.service.ts	src/features/movies/services/	backendApi	API PHP
profile.service.ts	src/features/profile/services/	backendApi	API PHP

El patrón habitual es: recibir argumentos -> construir endpoint, params, body o headers -> esperar la Promise de Axios -> extraer data -> devolver el resultado. El código que llama al service puede concentrarse entonces en la interfaz, el estado o la caché.

15.3.5 movies.service.ts: catálogo, búsqueda y detalle de películas

movies.service.ts es uno de los archivos centrales del frontend. Está ubicado en frontend/src/features/movies/services/movies.service.ts. Reúne las operaciones que obtienen películas desde TMDB: tendencias, listas predefinidas, búsqueda, filtros, géneros y datos del detalle. Importa api desde tmdbApi.ts, por lo que trabaja con un cliente que ya conoce la URL base y el Bearer token.

```
import api from "../../core/api/tmdbApi";
```

Función exportada	Responsabilidad	Quién la utiliza
getTrendingMovies()	Tendencias diarias.	useHomesMovies.ts
getMovieList()	Listas popular, now_playing, upcoming o top_rated.	useHomesMovies.ts y useMoviesQuery.ts
searchByText()	Busca títulos mediante /search/movie.	La usa internamente searchTextAndFilters().
searchByFilters()	Construye /discover/movie con género, fechas, rating y orden.	useMoviesQuery.ts
searchByTextAndFilters()	Busca texto y luego aplica filtros/orden localmente sobre la página recibida.	useMoviesQuery.ts
getMovieGenres()	Obtiene /genre/movie/list.	useMoviesGenresQuery.ts
getMovieDetail()	Obtiene el detalle y aplica fallback de imágenes entre idiomas.	useMovieDetail.ts
getMovieCredits()	Obtiene cast y crew con fallback.	useMovieDetail.ts

Función exportada	Responsabilidad	Quién la utiliza
getMovieVideos()	Obtiene videos y reintentará sin idioma si no encuentra resultados.	useMovieDetail.ts
getSimilarMovies()	Obtiene películas similares.	useMovieDetail.ts

Ejemplo 1: un GET con endpoint dinámico y parámetros

```
export const getMovieList = async (
  listType: MovieListType,
  page: number,
  language: string
): Promise<PaginatedResponse<Movie>> => {
  const { data } = await api.get<PaginatedResponse<Movie>>`(
    `/movie/${listType}`,
    {
      params: {
        page,
        language,
      },
    }
  );

  if (listType === "upcoming") {
    const today = new Date().toISOString().split("T")[0]!;
    data.results = data.results.filter(
      (movie) => movie.release_date >= today
    );
  }

  return data;
};
```

api es la instancia creada por tmdbApi.ts. .get() selecciona HTTP GET. /movie/\${listType} permite cambiar el endpoint según el listado solicitado. params se transforma en query string. El service además aplica una regla local para upcoming: elimina resultados cuya fecha ya pasó.

Ejemplo 2: convertir filtros de la interfaz en parámetros que entiende TMDB

```
export const searchByFilters = async (
  page: number,
  language: string,
  filters: DiscoverFilters = {}
): Promise<PaginatedResponse<Movie>> => {
  const params: Record<string, any> = { page, language };

  if (filters.genres?.length) {
    params.with_genres = filters.genres.join(",");
  }

  if (filters.yearFrom) {
    params["primary_release_date.gte"] = `${filters.yearFrom}-01-01`;
  }

  if (filters.minRating !== undefined) {
    params["vote_average.gte"] = filters.minRating;
  }

  const { data } = await api.get<PaginatedResponse<Movie>>`(
    "/discover/movie",
    { params }
  );

  return data;
};
```

Este ejemplo muestra por qué el service es más que una línea de Axios: traduce valores de la interfaz a los nombres y formatos que espera TMDB. Los géneros pasan de un array a un string separado por comas y el año se convierte en una fecha de inicio.

Ejemplo 3: un service también puede transformar la respuesta

```
const data = await searchText(
  filters.query,
  page,
  language
);

let results = data.results;

// ... filtros y orden aplicados en el frontend ...

return {
  ...data,
  results,
  total_results: results.length,
};
```

`searchByTextAndFilters()` existe porque `/search/movie` no permite combinar todas las opciones avanzadas del explorador. Primero obtiene una página desde TMDB y luego aplica sobre esa página los filtros adicionales. Por eso `total_results` pasa a representar la cantidad filtrada de esa página, mientras `total_pages` sigue proveniente de TMDB; la sección 16 explica esta limitación con más detalle.

Quién invoca `movies.service.ts`

```
MoviesView
-> useMoviesQuery.ts
  -> getMovieList()
  -> searchByFilters()
  -> searchTextAndFilters()
-> movies.service.ts
-> api de tmdbApi.ts
-> Axios
-> TMDB
```

`useMoviesQuery.ts` decide cuál de esas operaciones necesita según el texto, el tipo de listado y los filtros. Vue Query administra `queryKey`, paginación, `loading` y `cache`; `movies.service.ts` se ocupa de la operación de datos concreta.

15.3.6 auth.service.ts: autenticación, cuenta y recuperación de contraseña

`auth.service.ts` está ubicado en `frontend/src/features/auth/services/auth.service.ts`. Es el service que concentra las llamadas del ciclo de cuenta y autenticación hacia la API propia de CineVerse. Importa `backendApi` y no se comunica con TMDB.

```
import backendApi from "@core/api/backendApi";
```

Función	Operación	Consumidor principal
<code>registerUser()</code>	POST <code>/register</code>	<code>RegisterView.vue</code>
<code>verifyEmail()</code>	GET <code>/verify-email?token=...</code>	<code>VerifyEmailView.vue</code>
<code>loginUser()</code>	POST <code>/login</code>	<code>LoginView.vue</code>
<code>getAuthenticatedUser()</code>	GET <code>/user</code>	<code>useAuthStore.ts</code>
<code>resendVerificationEmail()</code>	POST <code>/resend-verification-email</code>	<code>LoginView.vue</code>
<code>forgotPassword()</code>	POST <code>/forgot-password</code>	<code>ForgotPasswordView.vue</code>
<code>validateResetToken()</code>	GET <code>/reset-password/validate</code>	<code>ResetPasswordView.vue</code>
<code>resetPassword()</code>	POST <code>/reset-password</code>	<code>ResetPasswordView.vue</code>
<code>logoutUser()</code>	POST <code>/logout</code> + X-CSRF-Token	<code>UserMenu.vue</code>

Ejemplo: `login`

```
export const loginUser = async (
  payload: LoginPayload
): Promise<LoginResponse> => {
  const { data } = await backendApi.post<LoginResponse>(
    "/login",
    payload
  );
};
```

```
    return data;
  };
```

backendApi.post() recibe el endpoint y el payload. Axios serializa el objeto como body JSON. La cookie de sesión queda bajo administración del navegador gracias a withCredentials, mientras la respuesta JSON puede contener user y csrf_token.

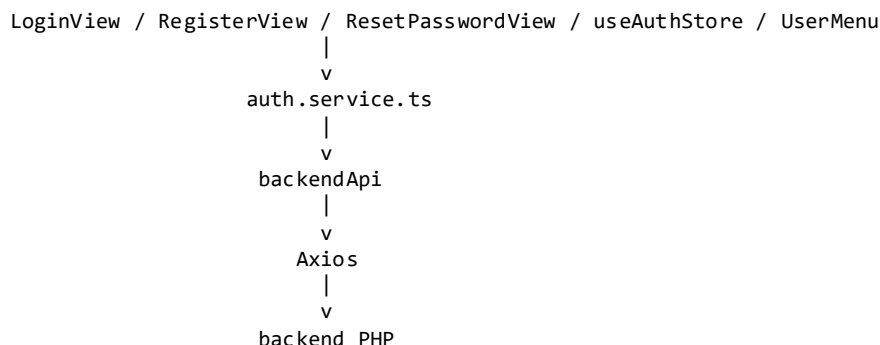
Ejemplo: reconstruir la sesión al iniciar la SPA

```
export const getAuthenticatedUser = async (
): Promise<AuthenticatedUserResponse> => {
  const { data } = await backendApi.get<AuthenticatedUserResponse>(
    "/user"
  );
  return data;
};
```

useAuthStore.initialize() llama a getAuthenticatedUser(). El service sólo envía GET /user y devuelve la respuesta; el store es quien actualiza user, csrfToken e isAuthenticated. Así se separa comunicación HTTP de estado global.

Ejemplo: logout protegido por CSRF

```
export const logoutUser = async (
  csrfToken: string
): Promise<LogoutResponse> => {
  const { data } = await backendApi.post<LogoutResponse>(
    "/logout",
    {},
    { headers: { "X-CSRF-Token": csrfToken } }
  );
  return data;
};
```



15.3.7 reviews.service.ts: lectura y escritura de reseñas

reviews.service.ts está ubicado en frontend/src/features/movies/services/reviews.service.ts. Aunque pertenece a la feature movies, no consulta TMDB: utiliza backendApi porque las reseñas son datos propios de CineVerse guardados en MySQL. Este servicio se encarga de las operaciones relacionadas con las reseñas, como obtener el listado, crear una nueva, actualizarla o eliminarla.

Función	HTTP	Uso
createReview()	POST /reviews	Crear una reseña desde MovieReviews.vue.
updateReview()	PUT /reviews/{id}	Editar una reseña propia.
deleteReview()	DELETE /reviews/{id}	Eliminar una reseña propia.
getUserReviews()	GET /reviews?page=...	Cargar el historial en MyReviewsView.vue.
getMovieReviews()	GET /movies/{movieId}/reviews?page=...	Cargar reseñas y estadísticas en MovieReviews.vue.

```
export const createReview = async (
  payload: CreateReviewPayload,
  csrfToken: string
) => {
  const { data } = await backendApi.post(
    "/reviews",
```

```

    payload,
    {
      headers: {
        "X-CSRF-Token": csrfToken,
      },
    }
  );
  return data;
};

```

En `post(url, body, config)`, el primer argumento identifica el endpoint, el segundo es el body JSON y el tercero permite agregar opciones específicas de esa request. CineVerse coloca X-CSRF-Token sólo en operaciones protegidas que modifican datos.

```

// actualizar
backendApi.put(`/reviews/${reviewId}`, payload, {
  headers: { "X-CSRF-Token": csrfToken }
});

// eliminar
backendApi.delete(`/reviews/${reviewId}`, {
  headers: { "X-CSRF-Token": csrfToken }
});

```

No confundir frontend y backend

`reviews.service.ts` es un módulo TypeScript del frontend que envía HTTP. `ReviewService.php` es una clase del backend que aplica reglas de negocio después de que la request llegó al servidor. Comparten la palabra `service`, pero viven en capas diferentes y tienen responsabilidades diferentes.

15.3.8 person.service.ts: actores y filmografía desde TMDB

`person.service.ts` está ubicado en `frontend/src/features/person/services/person.service.ts`. Utiliza la instancia `api` de `tmdbApi.ts` y exporta una operación principal: `getFullPersonDetail()`. El composable `usePersonDetail.ts` la invoca mediante `Vue Query`.

```

import api from "@core/api/tmdbApi";

export const getFullPersonDetail = async (
  personId: number,
  language: string
): Promise<PersonDetail> => {
  const { data } = await api.get<PersonDetail>(
    `/person/${personId}`,
    {
      params: {
        language,
        append_to_response: "movie_credits",
      },
    }
  );
  return data;
};

```

`append_to_response=movie_credits` le pide a TMDB que incluya la filmografía en la misma respuesta del detalle de la persona. Esto evita una segunda llamada separada para los créditos de películas.

```

PersonDetailView
-> usePersonDetail.ts
-> getFullPersonDetail()
-> person.service.ts
-> api / tmdbApi.ts
-> Axios
-> TMDB

```

15.3.9 profile.service.ts: resumen del perfil autenticado

profile.service.ts está ubicado en frontend/src/features/profile/services/profile.service.ts. Es un service pequeño dedicado a obtener del backend el resumen que necesita ProfileView: cantidad total de reseñas y últimas reseñas. Usa backendApi, por lo que la cookie de sesión viaja con la solicitud.

```
import backendApi from "@core/api/backendApi";

import type {
  ProfileResponse
} from "../types/profile";

export const getProfile = async (
): Promise<ProfileResponse> => {
  const { data } = await backendApi.get<ProfileResponse>(
    "/profile"
  );
  return data;
};
```

ProfileView.vue importa getProfile() y lo ejecuta dentro de loadProfile(). El service devuelve la respuesta; la vista decide cómo copiar reviews_count y latest_reviews a sus refs, cómo mostrar loading y qué hacer si recibe AUTH_REQUIRED.

```
ProfileView.vue
-> loadProfile()
-> getProfile()
-> profile.service.ts
-> backendApi.get("/profile")
-> Axios
-> backend PHP
-> ProfileResponse
```

15.3.10 Cómo leer una llamada Axios en los services

Una vez conocidas las piezas anteriores, las llamadas de los services pueden leerse de forma sistemática. El método Axios indica el verbo HTTP; el primer argumento es el endpoint relativo; params produce query string; en POST o PUT el segundo argumento suele ser el body; y config permite agregar cabeceras u otras opciones.

Código	Significado
api.get(url, { params })	GET a TMDb; params se convierte en query string.
backendApi.get(url, { params })	GET al backend propio.
backendApi.post(url, body, config)	POST con body JSON y configuración opcional.
backendApi.put(url, body, config)	PUT para actualizar recursos.
backendApi.delete(url, config)	DELETE; el config puede contener X-CSRF-Token.
const { data } = await ...	Espera la Promise y extrae response.data del objeto de respuesta Axios.
api.get<T>(...)	T es el tipo que TypeScript espera; no valida el JSON real en runtime.

Los services suelen devolver data y no el objeto Axios completo. Por eso la vista o el composable trabaja con la respuesta de negocio y queda menos acoplado a detalles como status o headers. Cuando Axios recibe una respuesta HTTP de error, rechaza la Promise; la sección 15.7 muestra cómo ese error llega al catch de una vista.

15.4 Proxy de Vite durante el desarrollo

En desarrollo, VITE_BACKEND_URL=/api hace que backendApi construya rutas relativas bajo /api. La variable se define y se sigue paso a paso en 5.4.4. El servidor de desarrollo de Vite aplica entonces la regla de proxy de frontend/vite.config.ts: reenvía esas rutas a http://cineverse.local y elimina el prefijo /api antes de entregarlas a Apache.

Estado actual

Los dos clientes Axios no tienen un tiempo máximo de espera global ni un interceptor global de errores. Un interceptor es una función común que puede revisar todas las solicitudes o respuestas antes de que lleguen al resto del código. Actualmente cada service o composable recibe el error y decide cómo tratarlo en su propio flujo.

15.5 Qué partes tiene una solicitud en CineVerse

Una solicitud puede transportar información en lugares diferentes. CineVerse utiliza los tres mecanismos siguientes y cada uno cumple una función distinta.

- **Parámetros de URL:** en GET /movies/550/reviews?page=2, page=2 viaja después del signo ?. Request::query() lo lee mediante \$_GET.
- **Cuerpo JSON:** POST /reviews envía movie_id, movie_title, rating y comment dentro del body. Request::json() lee php://input y ejecuta json_decode().
- **Cabeceras:** Authorization lleva el Bearer token a TMDB; X-CSRF-Token lleva el token CSRF en escrituras protegidas.

La cookie de sesión es distinta: no forma parte del JSON ni del token CSRF. El navegador la administra como cookie HTTP y backendApi tiene withCredentials: true para permitir su envío al backend.

15.6 Cookie, CSRF y Bearer no son lo mismo

TMDB

Authorization: Bearer <token TMDB>
-> identifica la aplicación frente a TMDB

Backend CineVerse

Cookie de sesión PHP
-> relaciona el navegador con \$_SESSION

Operación protegida del backend

Cookie de sesión + X-CSRF-Token
-> autenticación + prueba adicional contra CSRF

Por ejemplo, GET /profile necesita sesión pero no CSRF porque sólo lee información. POST /reviews necesita ambos porque crea datos. GET /movies/{movieId}/reviews es público: no exige sesión ni CSRF, aunque si existe una cookie el backend puede detectar al usuario y devolver su reseña propia separada.

15.7 Cómo llega un error de Axios a una vista

Cuando el backend responde con un estado HTTP de error, Axios rechaza la Promise y el control pasa al catch. En las vistas de CineVerse se consulta error.response?.data?.code para recuperar el código técnico enviado por la API y convertirlo en un mensaje o error de formulario.

```

try {
  const response = await loginUser(form);

  if (response.success) {
    authStore.setUser(response.data.user);
    authStore.setCsrfToken(response.data.csrf_token);
    await router.push({
      name: "Home",
      params: { lang: locale.value }
    });
  }
}
catch (error: any) {
  const code = error.response?.data?.code as string;

  const fieldError =
    authFieldErrorMap[
      code as keyof typeof authFieldErrorMap
    ];

  // ... el código se convierte en un error de la interfaz
}

```

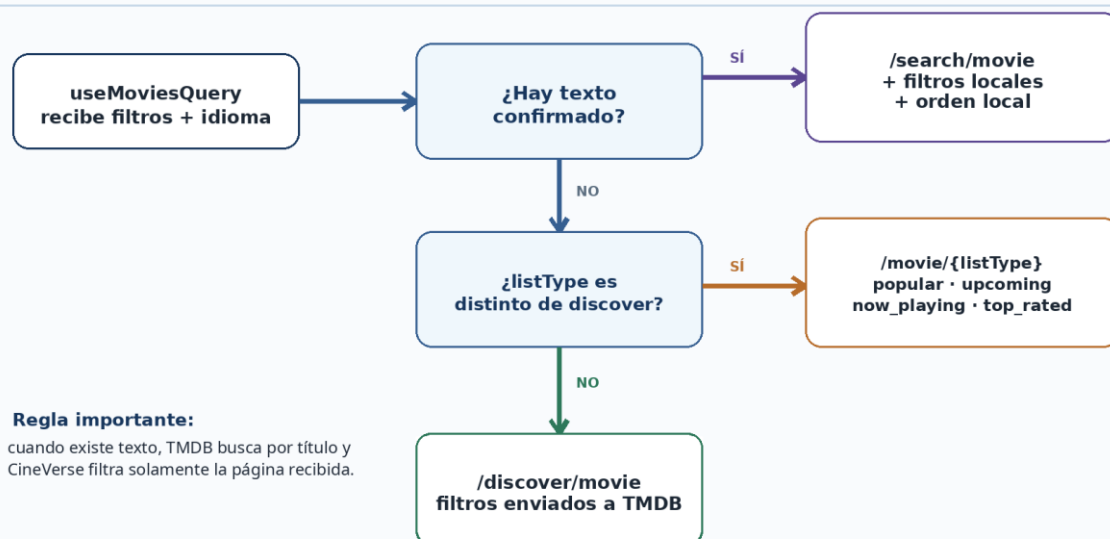
`error.response` existe cuando Axios recibió una respuesta HTTP que representa un error. `data` contiene el JSON de esa respuesta y `code` es el identificador estable producido por el backend, por ejemplo `INVALID_CREDENTIALS` o `REVIEW_ALREADY_EXISTS`. La sección 28 desarrolla la clasificación completa de errores esperados e inesperados.

16. Explorador de películas y sistema de búsqueda

El explorador combina el panel de filtros, la navegación de la SPA y las consultas a TMDB. La idea importante no es memorizar funciones aisladas, sino seguir qué ocurre desde que el usuario cambia una opción hasta que aparecen nuevas tarjetas en la grilla.

Idea clave. El explorador tiene tres caminos de consulta. El texto confirmado tiene prioridad; si no hay texto se utiliza una lista predefinida o Discover. `useMoviesQuery` decide qué camino corresponde y `movies.service.ts` ejecuta la petición concreta.

Cómo decide CineVerse qué búsqueda realizar



Decisión interna de useMoviesQuery: búsqueda por texto, catálogo predefinido o Discover.

16.1 Qué piezas intervienen

Ubicación en el proyecto: frontend/src/features/movies/views/MoviesView.vue · View · coordina filtros y resultados

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesFilters.ts · Composable · mantiene el estado de búsqueda y filtros

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable · decide qué consulta ejecutar y gestiona la paginación

Ubicación en el proyecto: frontend/src/features/movies/services/movies.service.ts · Service · construye las peticiones a TMDB

Ubicación en el proyecto: frontend/src/features/movies/components/MovieResults.vue · Component · convierte las páginas recibidas en tarjetas visibles

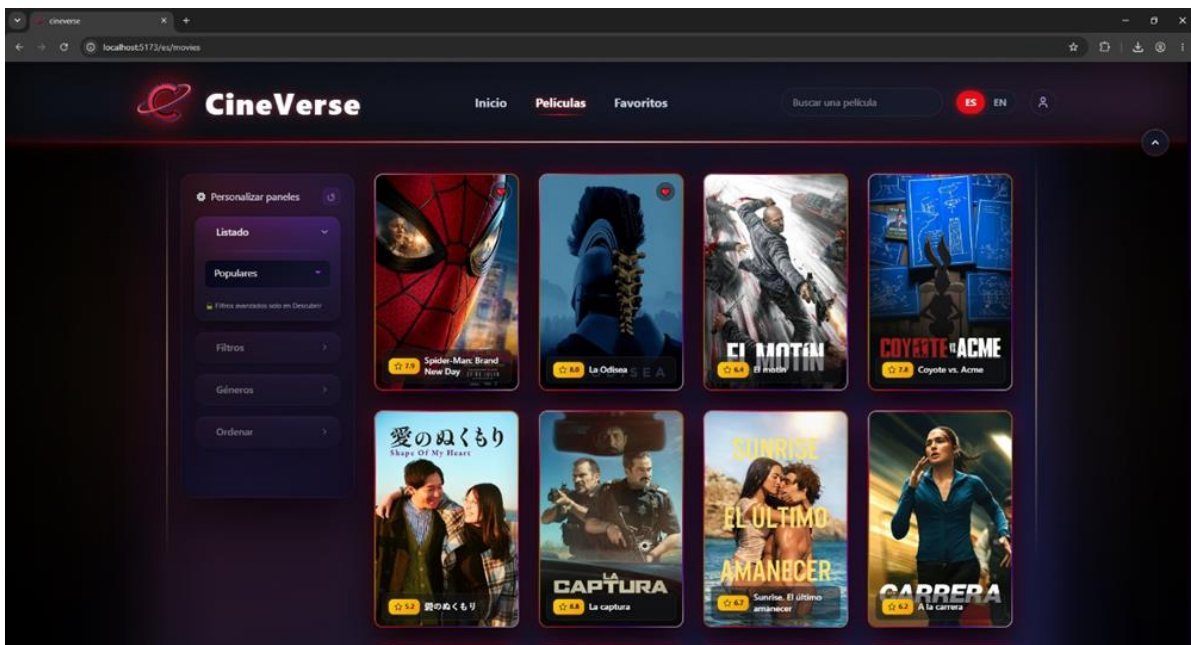
Ubicación en el proyecto: frontend/src/features/movies/components/MovieCard.vue · Component · representa cada película y contiene el overlay de hover

Ubicación en el proyecto: frontend/src/layouts/components/AppHeader.vue · Component · inicia la búsqueda global desde el encabezado

MoviesView no conoce los endpoints de TMDB y MovieResults no decide qué buscar. Cada pieza recibe el resultado de la anterior y cumple una responsabilidad concreta.

16.2 Ejemplo guiado: Carlos entra a Películas y se construye la primera grilla

Carlos entra desde el menú a **/es/movies**. Para seguir el recorrido desde cero, en este ejemplo suponemos que la URL no trae filtros y que la pestaña no tiene una búsqueda previa guardada en sessionStorage. En ese caso, useMoviesFilters utiliza sus valores por defecto y el listado inicial es popular.



Primera grilla del explorador: panel lateral y tarjetas creadas a partir de los resultados recibidos de TMDB.

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesFilters.ts · Composable · decide el estado inicial de los filtros

```
function parseListType(value: unknown): MovieListType {
  const valid: MovieListType[] = [
    "discover", "popular", "now_playing", "upcoming", "top_rated",
  ];
  return valid.includes(value as MovieListType)
    ? (value as MovieListType)
    : "popular";
}

const hasUrlParams = Object.keys(route.query).length > 0;
const initialQuery = hasUrlParams ? route.query : readStoredQuery();
```

Si la URL trae parámetros, esos valores tienen prioridad. Si está limpia, el composable intenta recuperar el último estado guardado en sessionStorage. Sólo cuando tampoco existe ese estado previo se cae en los valores por defecto. parseListType() devuelve "popular" cuando no recibe un listado válido.

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable · inicia la consulta paginada

```
initialPageParam: 1,

if (filters.listType.value !== "discover") {
  return getMovieList(
    filters.listType.value as MovieListType,
    page,
    language.value,
  );
}
```

Como en este ejemplo no existe texto confirmado y listType vale "popular", la primera ejecución utiliza page=1 y entra en la rama de getMovieList(). Vue Query mantiene además isLoading, data, error y la información necesaria para solicitar páginas posteriores.

Ubicación en el proyecto: frontend/src/features/movies/services/movies.service.ts · Service · construye la solicitud de la lista popular

```
const { data } = await api.get<PaginatedResponse<Movie>>(
  `/movie/${listType}`,
  { params: { page, language } }
);
```

Con listType="popular", page=1 e idioma español, el endpoint efectivo es /movie/popular con page=1 y language=es. tmdbApi.ts aporta la base <https://api.themoviedb.org/3> y la cabecera Authorization. TMDB devuelve un objeto paginado con page, results, total_pages y total_results.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieResults.vue · Component · estado de carga y procesamiento de data.pages

```
<div v-if="isLoading" class="cinema-loading">
  ...
  {{ t('common.searchingMovies') }}
</div>
```

Mientras la primera solicitud está pendiente, MovieResults no intenta dibujar tarjetas: muestra el estado de carga. Cuando Vue Query entrega data.pages, el componente procesa los resultados y reemplaza ese estado por la grilla.

```
const pages = data?.pages;
const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);
```

flatMap() convierte los results de todas las páginas cargadas en una sola lista. Después se eliminan las películas sin poster_path utilizable, porque la tarjeta necesita una imagen válida para formar parte de esta grilla.

Detalle importante. MovieResults define MIN_MOVIES = 20. Si después de descartar películas sin póster quedan menos de 20 y TMDb informa que existe otra página, el componente puede emitir load-more automáticamente para intentar completar una primera grilla razonable. Esto es anterior e independiente del scroll infinito que Carlos activa más adelante con “Mostrar más”.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieResults.vue · Component · creación de MovieCard y distribución con CSS Grid

```
<div class="movies-grid">
  <MovieCard
    v-for="(movie, index) in validMovies"
    :key="movie.id"
    :movie="movie"
    :delay="index * 30"
  />
</div>
```

v-for recorre validMovies. En cada vuelta Vue crea una instancia de MovieCard y entrega el objeto movie mediante la prop :movie. Si validMovies contiene 20 películas, el componente intenta representar 20 tarjetas; la cantidad de columnas es una decisión visual separada.

```
.movies-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 32px;
}
```

En escritorio la grilla usa cuatro columnas. Los media queries del mismo componente pasan a tres columnas hasta 1024 px, dos hasta 768 px y una hasta 480 px. Por lo tanto, “cuántas películas existen” y “cuántas se ven por fila” son conceptos distintos.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieCard.vue · Component · carga visual del póster y animación de entrada de cada tarjeta

```
<div
  class="movie-card"
  :style="{ animationDelay: delay + 'ms' }"
  @click="goToMovie(movie.id)"
>
  ...
  <div v-if="!imageLoaded" class="poster-skeleton"></div>

  
</div>
```

Que el objeto Movie ya haya llegado no significa que el póster esté descargado. Cada MovieCard empieza con imageLoaded=false y muestra poster-skeleton mientras el navegador obtiene la imagen desde image.tmdb.org. loading="lazy" permite diferir la descarga de imágenes que todavía están lejos de la zona visible. Cuando la etiqueta dispara @load, onPosterLoad() cambia imageLoaded a true y la clase loaded hace aparecer el póster mediante una transición de opacidad.

La prop delay también tiene un efecto exclusivamente visual: MovieResults entrega $\text{index} \times 30$ milisegundos y MovieCard lo aplica como animationDelay. Así las tarjetas no entran todas exactamente al mismo tiempo, sino de forma ligeramente escalonada. Este retraso no modifica el orden de los datos ni retrasa la consulta a TMDB.

```

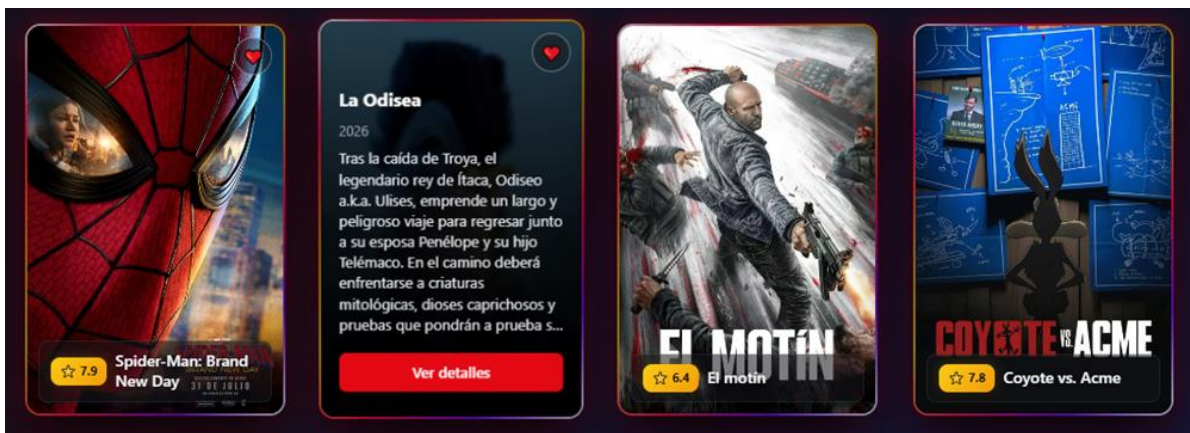
TMDB ya devolvió el objeto Movie
-> MovieCard se monta con imageLoaded = false
-> aparece poster-skeleton
-> el navegador solicita el póster (loading="lazy")
-> <img> termina de cargar y dispara @load
-> onPosterLoad() pone imageLoaded = true
-> la clase loaded hace visible el póster
-> animationDelay = index x 30 ms escalona la entrada visual

Carlos entra a Películas
-> MoviesView
-> useMoviesFilters
-> useMoviesQuery(page 1)
-> getMovieList("popular")
-> movies.service.ts
-> TMDB /movie/popular
-> Vue Query guarda data.pages
-> MovieResults hace flatMap + filtra poster_path
-> validMovies
-> v-for crea MovieCard x N
-> cada MovieCard carga su póster y oculta el skeleton al terminar
-> CSS Grid distribuye las tarjetas
-> Carlos ve la grilla

```

16.3 Ejemplo guiado: Carlos pasa el mouse sobre una película

Con la grilla ya cargada, Carlos coloca el cursor sobre La Odisea. La tarjeta deja de mostrar únicamente el póster y revela el título, el año, la sinopsis y el botón Ver detalles.



Hover sobre una MovieCard: La Odisea revela información que ya estaba dentro del objeto movie recibido por la tarjeta.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieCard.vue · Component · datos mostrados dentro del overlay

```

<h3 class="overlay-title">{{ movie.title }}</h3>
<p class="year">{{ movie.release_date?.slice(0, 4) }}</p>
<p class="overview">
  {{ movie.overview || t("movies.noDescription") }}
</p>
<button class="details-btn">
  {{ t("movies.viewDetails") }}
</button>

```

La descripción no se pide al servidor cuando comienza el hover. movie.overview llegó junto con el resto del objeto Movie en la respuesta que ya construyó la grilla. MovieCard sólo decide cuándo hacerlo visible; si overview está vacío utiliza la traducción movies.noDescription.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieCard.vue · Component · CSS que revela el overlay

```
.poster-overlay {
  opacity: 0;
}

.movie-card:hover .poster-overlay {
  opacity: 1;
}

.overlay-content {
  opacity: 0;
  transform: translateY(12px);
}

.movie-card:hover .overlay-content {
  opacity: 1;
  transform: translateY(0);
}
```

El selector `:hover` pertenece a CSS. Al entrar el puntero en `.movie-card`, el overlay pasa de transparente a visible y su contenido sube suavemente desde 12 px. Este efecto visual no modifica el objeto `movie` ni dispara una consulta nueva.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieCard.vue · Component · clic sobre “Ver detalles” y navegación al detalle

```
<div
  class="movie-card"
  @click="goToMovie(movie.id)"
>
  ...
  <button class="details-btn">
    {{ t("movies.viewDetails") }}
  </button>

  <button
    class="btn-favorite"
    @click.stop="onFavoriteClick(movie)"
  >
    ...
  </button>
</div>
```

El botón Ver detalles no tiene un `@click` propio. Cuando Carlos lo pulsa, el evento de clic se propaga hasta el contenedor `.movie-card`, cuyo `@click` ejecuta `goToMovie(movie.id)`. Esa función usa Vue Router para abrir `MovieDetail` conservando el idioma actual. El recorrido completo `MovieCard` → Router → `MovieDetailView` se desarrolla en la sección 13.

El botón de favorito muestra el contraste: utiliza `@click.stop`, por lo que Vue detiene la propagación del evento. Carlos puede pulsar el corazón sin que ese mismo clic abra accidentalmente el detalle de la película.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieResults.vue · Component · seguimiento del mouse para tilt 3D y glare

```
<div class="movies-grid"
  @mousemove="handleGridMouseMove"
  @mouseleave="resetTiltedCard">
```

En paralelo al overlay de CSS, `MovieResults` escucha `mousemove` sobre toda la grilla. `handleGridMouseMove()` usa `closest(".movie-card")` para identificar la tarjeta activa, `getBoundingClientRect()` para conocer su posición y `requestAnimationFrame()` para limitar los cálculos a aproximadamente una actualización por frame. Con esa información modifica `rotateX`, `rotateY` y la posición del glare de la capa `.tilt-layer`.

```

Carlos mueve el mouse sobre La Odisea
-> CSS :hover revela poster-overlay
-> MovieCard muestra title + release_date + overview
-> al mismo tiempo MovieResults recibe mousemove
-> handleGridMouseMove() identifica la card
-> requestAnimationFrame() calcula tilt/glare
-> Carlos ve información + efecto 3D

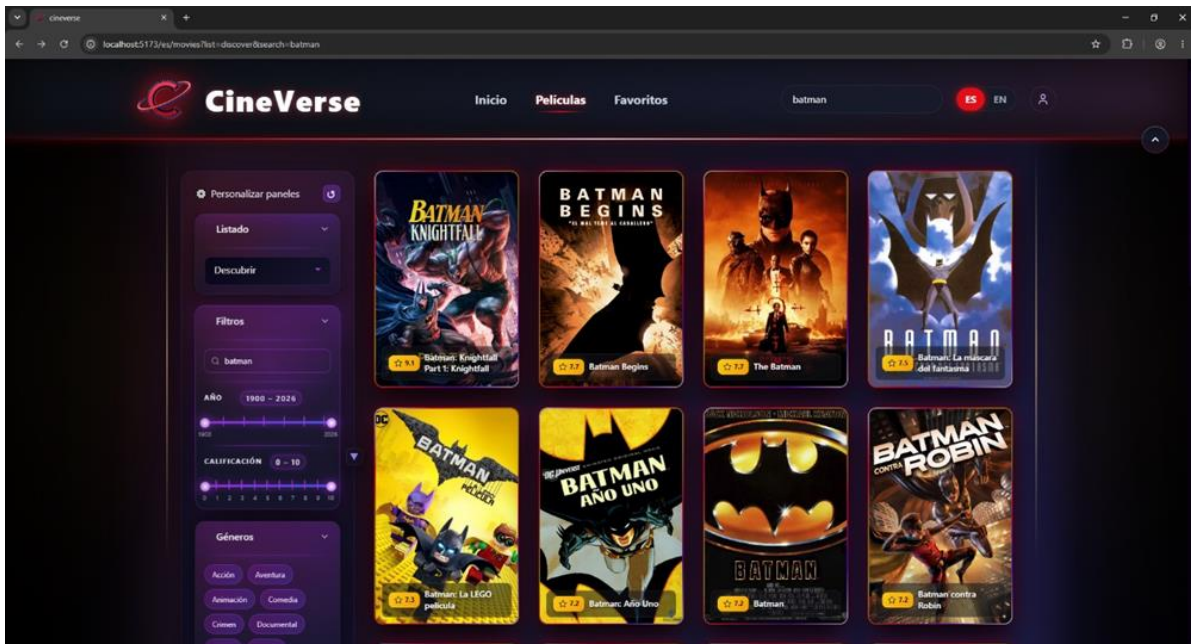
```

No hay una nueva request a TMDB.

Comportamiento responsive. En pantallas de hasta 768 px el CSS de MovieCard fuerza el overlay y su contenido a permanecer visibles, porque en un dispositivo táctil no existe el mismo hover de un mouse.

16.4 Ejemplo guiado: Carlos busca “Batman” desde el encabezado

Carlos escribe Batman en el buscador superior y envía el formulario. Este camino no comienza en MovieFilters: primero AppHeader modifica la URL mediante Vue Router y después el explorador reconstruye su estado a partir de esa URL.



Resultado de buscar “batman” desde el encabezado: la URL lleva el criterio y el panel lateral lo refleja al reconstruir sus filtros.

Ubicación en el proyecto: frontend/src/layouts/components/AppHeader.vue · Component · handleSearch() del buscador superior

```
function handleSearch() {
  const query = searchQuery.value.trim();
  const current = route.query.search;

  if (!query) return;
  if (query.length < 3) return;

  if (current === query) {
    router.go(0);
    return;
  }

  router.push({
    name: "Movies",
    params: { lang: route.params.lang },
    query: {
      search: query,
      list: "discover",
      sort: "vote_average.desc",
    },
  });
}
```

handleSearch() limpia espacios y exige al menos tres caracteres. Si la búsqueda es nueva, Vue Router navega a Movies con search, list="discover" y el orden por rating. Si la misma búsqueda ya está en la URL, el comportamiento actual utiliza router.go(0) para forzar una recarga completa porque la URL no cambiaría.

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesFilters.ts · Composable · lectura de route.query después de la navegación

```
const search = typeof query.search === "string"
  ? query.search.trim()
  : "";

filters.txtMovie = search;
filters.debounceQuery = search.length >= 3 ? search : "";
```

Como Batman llegó desde la URL, useMoviesFilters lo considera un valor ya confirmado: actualiza txtMovie y debounceQuery directamente. No espera los 500 ms del debounce que sí se utiliza cuando Carlos escribe dentro del campo lateral. Por eso el panel lateral de la captura también muestra batman.

No son el mismo input. AppHeader mantiene su propio searchQuery local y el panel lateral utiliza el estado de useMoviesFilters. En esta captura ambos muestran "batman" porque el encabezado inició la navegación y el panel leyó search desde route.query; no existe un único ref compartido y bidireccional entre los dos campos.

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable · la nueva queryKey activa la búsqueda por texto

```
if (filters.debounceQuery.value) {
  return searchByTextAndFilters(page, language.value, {
    query: filters.debounceQuery.value,
    ...filterOptions,
  });
}
```

debounceQuery forma parte de la queryKey. Al pasar a "batman", TanStack Vue Query trata esa combinación como otra consulta y ejecuta searchByTextAndFilters() desde la página 1.

Ubicación en el proyecto: frontend/src/features/movies/services/movies.service.ts · Service · búsqueda textual en TMDB

```
const { data } = await api.get<PaginatedResponse<Movie>>(
  "/search/movie",
  { params: { query, page, language } }
);
```

TMDB recibe query=batman, page=1 y el idioma activo. Después, searchByTextAndFilters() puede aplicar en el frontend los filtros adicionales que /search/movie no admite directamente.

Ubicación en el proyecto: frontend/src/features/movies/views/MoviesView.vue · View · reset de la grilla cuando cambia queryKey

```
watch(
  () => filters.queryKey.value,
  () => {
    (document.activeElement as HTMLElement)?.blur();
    resetSignal.value++;
  }
);
```

MoviesView incrementa resetSignal cuando cambia la identidad de los filtros. MovieResults observa esa señal, limpia las películas anteriores, desactiva temporalmente el scroll infinito y vuelve al principio. Así las tarjetas populares de la primera captura no se mezclan con los resultados de Batman.

```
Carlos escribe Batman en AppHeader
-> handleSearch()
-> router.push() escribe search en la URL
-> useMoviesFilters lee route.query
-> txtMovie y debouncedQuery = "batman"
-> cambia queryKey
-> MoviesView incrementa resetSignal
-> useMoviesQuery usa searchByTextAndFilters()
-> movies.service.ts llama /search/movie
-> TMDB devuelve results[]
-> MovieResults construye una nueva validMovies
-> v-for crea nuevas MovieCard
-> Carlos ve la grilla de Batman
```

Dos entradas, un mismo motor de resultados. El buscador superior navega primero mediante la URL; el campo del panel lateral modifica primero el estado reactivo y espera 500 ms. Ambos caminos terminan alimentando debouncedQuery, useMoviesQuery, movies.service.ts y MovieResults.

16.5 Los tres caminos de búsqueda

Ubicación en el proyecto: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable · queryFn de useInfiniteQuery

```
if (filters.debouncedQuery.value) {
  return searchByTextAndFilters(page, language.value, {
    query: filters.debouncedQuery.value,
    ...filterOptions,
  });
}

if (filters.listType.value !== "discover") {
  return getMovieList(
    filters.listType.value as MovieListType,
    page,
    language.value,
  );
}

return searchByFilters(page, language.value, filterOptions);
```

La lectura es de arriba hacia abajo. Si existe texto confirmado, no se llega a las otras ramas. Sin texto, una lista como popular o top_rated utiliza el catálogo preparado por TMDB. Discover queda como tercer camino y envía los filtros estructurados al endpoint /discover/movie.

Situación del usuario	Función utilizada	Qué recibe TMDB
Escribe texto	searchByTextAndFilters()	/search/movie con texto, página e idioma
Elige Popular / Cartelera / Próximas / Mejor valoradas	getMovieList()	/movie/{listType}
Usa Discover sin texto	searchByFilters()	/discover/movie con género, fechas, rating y orden

Caso especial de Próximas. Cuando listType es "upcoming", getMovieList() recibe la página de TMDB y elimina las películas cuya release_date es anterior a hoy. Es una corrección local del proyecto porque TMDB puede incluir títulos que ya se estrenaron dentro de ese listado.

16.6 Ejemplo guiado: Carlos busca “Batman” desde el panel lateral

Carlos ya está dentro de Películas y utiliza el campo de búsqueda del panel lateral. A diferencia del buscador superior, este camino comienza en el estado reactivo de MovieFilters y aplica un debounce antes de confirmar el texto. El recorrido interno es el siguiente:

Aquí sí interviene el debounce de escritura: Carlos está tipeando dentro del panel, no llegando desde un query param ya confirmado.

- MovieFilters actualiza el texto reactivo que mantiene useMoviesFilters.
- El watcher de búsqueda espera 500 ms desde la última tecla. Como “Batman” tiene al menos tres caracteres, lo copia a debouncedQuery.
- debouncedQuery forma parte de queryKey. Al cambiar esa identidad, TanStack Vue Query vuelve a ejecutar la query del explorador.
- useMoviesQuery detecta que existe texto confirmado y llama a searchByTextAndFilters().
- movies.service.ts ejecuta primero GET /search/movie con query=Batman, page=1 y el idioma activo.
- tmdbApi.ts aporta la base URL y la cabecera Authorization. TMDB devuelve una PaginatedResponse<Movie>.
- Si Carlos había elegido género, año, puntuación u orden, searchByTextAndFilters() procesa únicamente results de esa página en el navegador.
- Vue Query conserva la página procesada. MoviesView entrega data a MovieResults y MovieResults crea las tarjetas visibles.

Ubicación en el proyecto: frontend/src/features/movies/services/movies.service.ts · Service · searchByTextAndFilters()

```
const data = await searchByText(filters.query, page, language);
let results = data.results;

if (filters.genres?.length) {
  results = results.filter((movie) =>
    movie.genre_ids?.some((id) => filters.genres!.includes(id))
  );
}
```

Este fragmento muestra por qué “texto + filtros” es diferente de Discover. Primero llega una página de /search/movie y después JavaScript elimina de esa página las películas que no cumplen el filtro. CineVerse no está filtrando todo el universo de coincidencias de TMDB.

Limitación actual. En una búsqueda textual, total_results se reemplaza por la cantidad que sobrevivió al filtrado de esa página, mientras total_pages conserva la paginación original informada por TMDB. Por eso una página puede quedar corta y aun existir páginas siguientes.

16.7 Géneros: Discover y texto no significan exactamente lo mismo

En Discover, `movies.service.ts` convierte `[28, 12]` en `with_genres="28,12"` y TMDb interpreta esa combinación. En la búsqueda textual, el código local usa `some(...)`: basta con que una película tenga alguno de los IDs elegidos para conservarla. El mismo panel visual termina por tanto en dos mecanismos distintos según exista o no texto de búsqueda.

17. Cómo se guardan los filtros y se reflejan en la URL

`useMoviesFilters` no pide películas. Su función es mantener el estado que representa lo elegido por el usuario y sincronizarlo con la URL. Esto permite que el botón Atrás/Adelante, F5 y una URL compartida reconstruyan el mismo estado del explorador.

17.1 Tres lugares con responsabilidades distintas

Lugar	Qué conserva	Para qué sirve
Estado reactivo de Vue	texto, catálogo, géneros, años, rating y orden	Representar lo que está seleccionado ahora
<code>route.query</code> / URL	los filtros que difieren de los valores por defecto	Compartir, recargar y navegar con un estado reproducible
<code>sessionStorage</code>	<code>moviesFilters:lastQuery</code>	Recuperar el último query cuando se vuelve a Películas con una URL limpia

17.2 Ejemplo guiado: Carlos selecciona filtros y la URL cambia

Carlos selecciona Discover, Acción y Aventura, año desde 2000 y puntuación mínima 7. El estado cambia primero dentro de Vue. Un watcher espera 300 ms para no escribir una URL por cada pequeño movimiento del slider y construye sólo los parámetros necesarios.

Ubicación en el proyecto: `frontend/src/features/movies/composables/useMoviesFilters.ts` · Composable ·
watch de escritura estado → URL

```
if (filters.listType !== "popular") query.list = filters.listType;
if (filters.selectedGenres.length) query.genres = filters.selectedGenres.join(",");
if (filters.yearFrom !== null) query.yearFrom = String(filters.yearFrom);
if (filters.ratingRange[0] > 0) query.ratingMin = String(filters.ratingRange[0]);
if (filters.debouncedQuery) query.search = filters.debouncedQuery;

writeStoredQuery(query);
router.replace({ query });
```

El resultado puede quedar así:

```
/es/movies?list=discover&genres=28,12&yearFrom=2000&ratingMin=7
```

Se utiliza `router.replace` y no `router.push`. Así el usuario no acumula una entrada de historial por cada movimiento intermedio de un slider.

17.3 Qué ocurre si Carlos copia esa URL y la abre de nuevo

En la dirección inversa, el watcher de `route.query` reconstruye el estado: `genres="28,12"` se convierte en `[28, 12]`, `yearFrom` pasa a número y `search` se considera ya confirmado. Como el valor llegó desde una URL, no espera el debounce de 500 ms que se utiliza mientras una persona escribe.

Ubicación en el proyecto: `frontend/src/features/movies/composables/useMoviesFilters.ts` · Composable ·
watch de lectura URL → estado

```
const newGenres = parseGenresParam(query.genres);
if (!arraysEqual(filters.selectedGenres, newGenres)) {
  filters.selectedGenres = newGenres;
}
```

```
filters.yearFrom = parseIntParam(query.yearFrom);
const search = typeof query.search === "string" ? query.search.trim() : "";
filters.debouncedQuery = search.length >= 3 ? search : "";
```

El flag `syncingFromRoute` evita un ciclo: leer URL → modificar filtros → watcher de filtros vuelve a escribir URL → volver a leer. Durante una sincronización nacida en la ruta, el watcher de escritura se corta y deja terminar el recorrido.

17.4 Qué tiene prioridad al entrar al explorador

- Si la URL trae query params, la URL es la fuente de verdad.
- Si la URL está limpia, se intenta recuperar `sessionStorage["moviesFilters:lastQuery"]`.
- Si tampoco existe un query guardado, se utilizan los valores por defecto.

No confundir persistencias. El query de búsqueda se guarda en `sessionStorage`. El orden o apertura de paneles pertenece a preferencias visuales del filtro y se maneja por separado. No todo el estado de la pantalla se guarda en el mismo lugar.

18. Cómo se cargan más películas

TMDB devuelve resultados paginados. CineVerse no reemplaza la primera página cuando llega la segunda: TanStack Vue Query conserva las páginas ya obtenidas y MovieResults las une para formar una sola grilla continua.

18.1 Ejemplo guiado: Carlos pulsa “Mostrar más”

- MovieResults ejecuta `enableScroll()`: habilita el scroll automático y emite `load-more`.
- MoviesView escucha `@load-more` y llama directamente a `fetchNextPage()`, función entregada por `useInfiniteQuery`.
- `getNextPageParam` mira la última respuesta. Si `lastPage.page` es menor que `lastPage.total_pages`, devuelve `page + 1`.
- La query vuelve a ejecutar el mismo camino de búsqueda, ahora con `pageParam=2`.
- Cuando llega la página 2, `data.pages` contiene la página 1 y la página 2; la primera no se pierde.

Ubicación en el proyecto: `frontend/src/features/movies/views/MoviesView.vue` · View · conexión entre el evento del componente y Vue Query

```
<MovieResults
  :data="data"
  :isLoading="isLoading"
  :isFetching="isFetching"
  :error="error"
  :hasNextPage="hasNextPage"
  :isFetchingNextPage="isFetchingNextPage"
  :resetSignal="resetSignal"
  @load-more="fetchNextPage"
/>
```

Ubicación en el proyecto: `frontend/src/features/movies/composables/useMoviesQuery.ts` · Composable · `getNextPageParam`

```
getNextPageParam: (lastPage) => {
  if (lastPage.page < lastPage.total_pages) {
    return lastPage.page + 1;
  }
  return undefined;
}
```

18.2 Cómo dos páginas se convierten en una sola grilla

Ubicación en el proyecto: `frontend/src/features/movies/components/MovieResults.vue` · Component · procesamiento de `data.pages`

```
const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);
```

Ejemplo conceptual: si la primera página trae 20 películas y la segunda otras 20, flatMap genera una lista de 40. Si tres no tienen un poster_path utilizable, validMovies queda con 37 y ésta es la colección que recorre el v-for de tarjetas.

MIN_MOVIES vale 20. Si al procesar las páginas cargadas quedan menos de 20 tarjetas utilizables y todavía existe una página siguiente, MovieResults puede emitir otra carga para completar una primera grilla razonable.

18.3 Del botón al scroll infinito

El scroll infinito no empieza automáticamente. Primero Carlos pulsa Mostrar más; desde ese momento autoScrollEnabled queda activo. Un IntersectionObserver observa un elemento invisible al final de la grilla.

Ubicación en el proyecto: frontend/src/features/movies/components/MovieResults.vue · Component · IntersectionObserver

```
if (
  entry.isIntersecting &&
  autoScrollEnabled.value &&
  !props.isFetchingNextPage &&
  !loadingMore.value &&
  props.hasNextPage
) {
  loadingMore.value = true;
  emit("load-more");
}
```

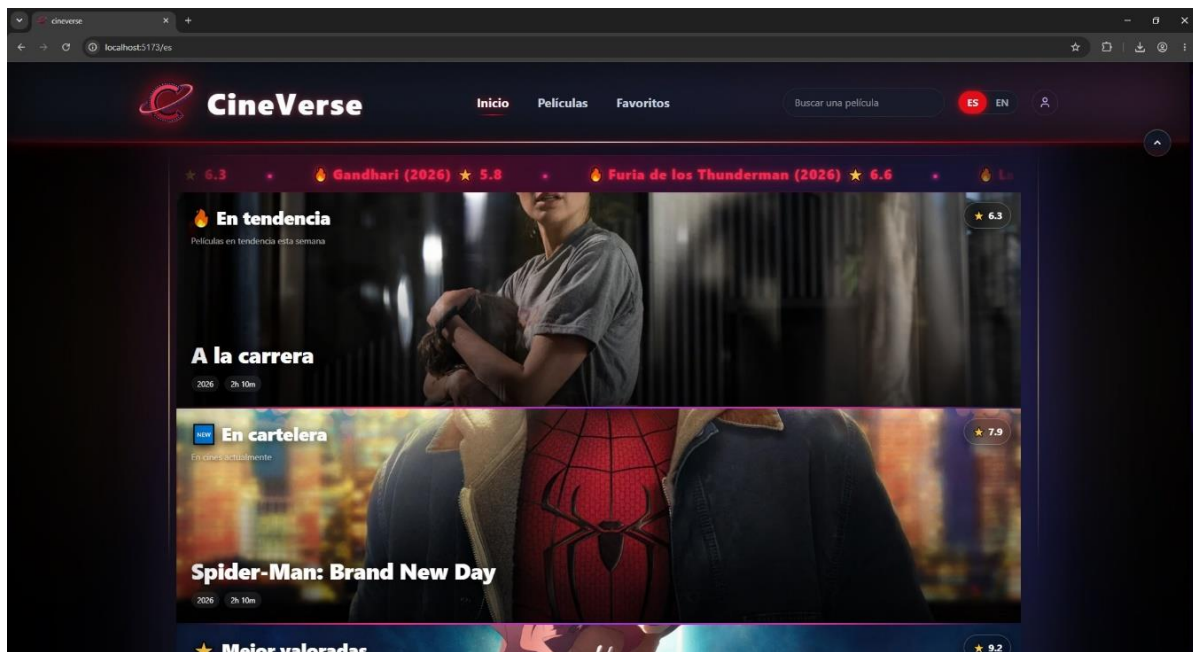
El observer utiliza rootMargin: "2500px". Esto hace que la siguiente carga pueda comenzar antes de que el usuario llegue físicamente al final, reduciendo la posibilidad de encontrar un hueco vacío mientras espera.

18.4 Qué ocurre cuando cambia la búsqueda

MoviesView incrementa resetSignal cada vez que cambia queryKey. MovieResults observa esa señal, vacía validMovies, desactiva temporalmente el scroll infinito y vuelve al principio. Después las páginas de la nueva query vuelven a poblar la grilla. Así no se mezclan resultados de dos búsquedas distintas.

19. Página de inicio

La Home utiliza tres consultas independientes para tendencias, cartelera y mejor valoradas. La independencia es importante: una fila puede estar lista y mostrarse mientras las otras todavía esperan a TMDB.



Página de inicio: tendencias, cartelera y mejor valoradas se muestran en carruseles independientes.

19.1 Qué piezas intervienen

Ubicación en el proyecto: `frontend/src/features/home/views/HomeView.vue` · View · compone las filas de la página

Ubicación en el proyecto: `frontend/src/features/home/composables/useHomesMovies.ts` · Composable · crea las tres queries de la Home; la función exportada se llama `useHomeMovies()`

Ubicación en el proyecto: `frontend/src/features/home/components/HomeRow.vue` · Component · gestiona título, overlay y estado `ready` de una fila

Ubicación en el proyecto: `frontend/src/features/home/components/HomeCarousel.vue` · Component · inicializa Swiper y avisa cuándo la primera imagen está lista

Ubicación en el proyecto: `frontend/src/features/home/components/MovieCardCarousel.vue` · Component · representa cada película del carrusel

19.2 Ejemplo guiado: Carlos entra a Inicio

Al montar `HomeView` se ejecuta `useHomeMovies(locale)`. El composable crea tres `useQuery` diferentes; ninguna espera a que termine la anterior.

Ubicación en el proyecto: `frontend/src/features/home/composables/useHomesMovies.ts` · Composable · tres queries independientes

```
const trending = useQuery({
  queryKey: ["home-movies", "trending", language],
  queryFn: async () => (await getTrendingMovies(1, language.value)).results,
  ...HOME_QUERY_OPTIONS,
});

const topRated = useQuery({
  queryKey: ["home-movies", "top Rated", language],
  queryFn: async () => (await getMovieList("top Rated", 1, language.value)).results,
  ...HOME_QUERY_OPTIONS,
});

const nowPlaying = useQuery({
  queryKey: ["home-movies", "now_playing", language],
  queryFn: async () => (await getMovieList("now_playing", 1, language.value)).results,
```

```
...HOME_QUERY_OPTIONS,
});
```

Por ejemplo, si Tendencias responde antes, esa data ya puede alimentar su HomeRow aunque Top Rated siga cargando. Esta concurrencia no depende de un Promise.all: surge de tres queries independientes que se montan al mismo tiempo.

19.3 Reutilización temporal de datos

Ubicación en el proyecto: frontend/src/features/home/composables/useHomesMovies.ts · Composable · HOME_QUERY_OPTIONS

```
const HOME_QUERY_OPTIONS = {
  staleTime: 1000 * 60 * 5,
  refetchOnWindowFocus: false,
  placeholderData: keepPreviousData,
} as const;
```

Durante cinco minutos los resultados se consideran recientes. Al cambiar el idioma cambia la queryKey; keepPreviousData permite conservar temporalmente lo anterior mientras llega la nueva versión localizada, en lugar de vaciar de golpe la fila.

19.4 Cuándo desaparece el loader de una fila

Recibir el JSON no es suficiente para que HomeRow quite el overlay. HomeCarousel espera dos condiciones: que Swiper esté inicializado y que la primera imagen visible haya cargado. La carga de esa imagen se confirma después de dos requestAnimationFrame para darle al navegador oportunidad de pintarla antes de emitir ready.

Ubicación en el proyecto: frontend/src/features/home/components/HomeRow.vue · Component · handleReady()

```
function handleReady() {
  isReady.value = true

  if (!alreadyLoaded) {
    sessionStorage.setItem("home-loaded", "true")
  }

  setTimeout(() => {
    showOverlay.value = false
  }, 50)
}
```

sessionStorage["home-loaded"] evita repetir el mismo loader inicial si la Home ya se mostró durante esa sesión de la pestaña.

Detalle actual. MovieCardCarousel muestra “2h 10m” como texto fijo. Las filas de Home no piden /movie/{id} para conocer el runtime real; la duración real sí está disponible en el detalle de la película.

19.5 Cómo se construye el ticker superior

Ubicación en el proyecto: frontend/src/features/home/views/HomeView.vue · View · computed tickerItems

```
for (const movie of trend.slice(0, 5)) {
  if (!movie) continue;

  const title = movie.title;
  const year = movie.release_date?.slice(0, 4) || "----";
  const rating = movie.vote_average?.toFixed(1) || "0.0";

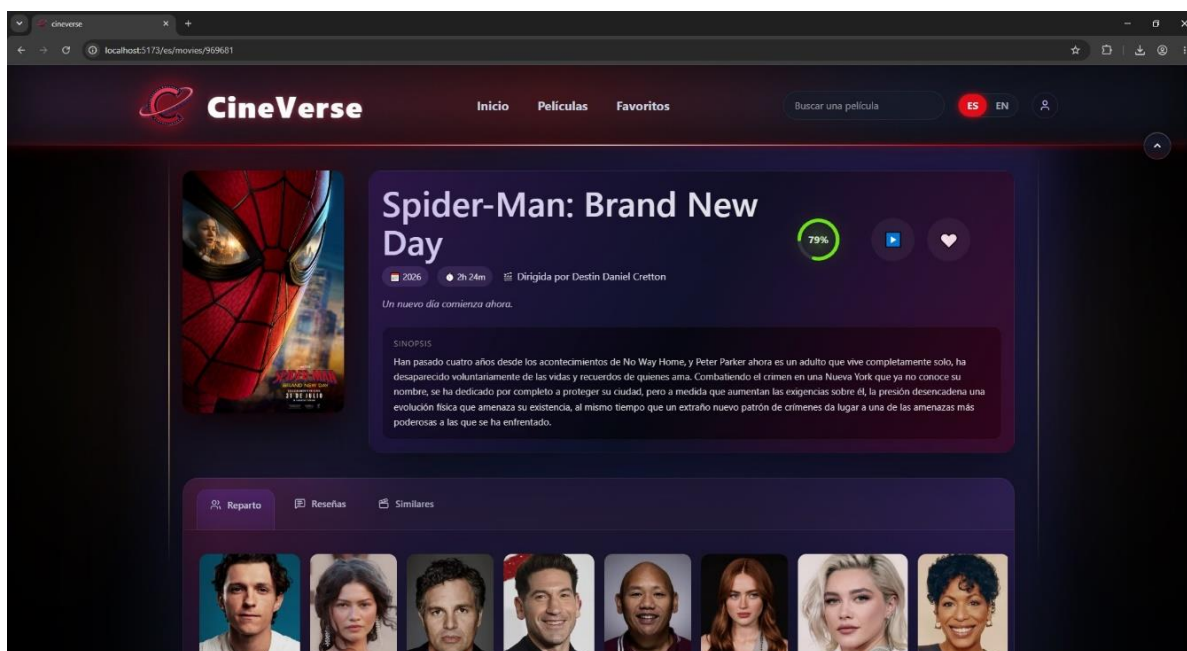
  result.push(`🔥 ${title} (${year}) ☆ ${rating}`);
}

return [...result, ...result];
```

El ticker toma las primeras cinco películas de Tendencias y construye una frase con título, año y puntuación. Antes del return, HomeView agrega además tres mensajes traducidos de la interfaz. Finalmente duplica el array completo para que la animación pueda desplazarse de forma continua sin dejar un hueco entre el final y el comienzo.

20. Detalle de película

El detalle es un buen ejemplo de coordinación entre Router, Vue Query, TMDb y componentes. Primero se confirma la película principal; después se obtienen en paralelo créditos, videos y similares; por último, la vista reparte la información entre MovieHero y MovieTabs.



Detalle de película: ficha principal, trailer, favorito y pestañas de reparto, reseñas y similares.

20.1 Ejemplo guiado: Carlos abre /es/movies/550

- Vue Router reconoce la ruta de detalle y monta MovieDetailView.vue.
- MovieDetailView toma route.params.id, lo convierte a número y llama useMovieDetail(550, locale).
- useMovieDetail solicita primero getMovieDetail(550, idioma). Si esa petición falla, la query entra en estado de error y no se lanzan las consultas secundarias.
- Si la película existe, el composible lanza créditos, videos y similares dentro de Promise.all.
- Con las respuestas, limita el reparto a diez, busca crew.job === "Director", selecciona un trailer de YouTube y conserva results de similares.
- El resultado procesado vuelve a MovieDetailView. La vista extrae movie, cast, director, trailer y similar con computed.
- MovieHero recibe la información principal. MovieTabs recibe reparto, similares, movieId y movieTitle.

Ubicación en el proyecto: frontend/src/features/movies/composables/movie-detail/useMovieDetail.ts · Composable · queryFn

```
const movieData = await getMovieDetail(id, language.value);

if (!movieData) {
  throw new Error("Movie not found");
}

const [creditsData, videosData, similarData] =
  await Promise.all([
    getMovieCredits(id, language.value),
    getMovieVideos(id, language.value),
```

```
getSimilarMovies(id, 1),
]);
```

La primera petición no forma parte del Promise.all: CineVerse espera el detalle principal antes de lanzar las secundarias. Una vez confirmada la película, las otras tres sí se esperan en conjunto y no una detrás de otra.

20.2 Qué devuelve useMovieDetail a la pantalla

```
return {
  movie: movieData,
  cast: creditsData.cast.slice(0, 10),
  director: director ? { name: director.name } : null,
  trailer: trailerData ? trailerData.key : null,
  similar: similarData.results,
};
```

Este objeto MovieDetailResult es una estructura preparada para la interfaz. MovieDetailView no necesita volver a buscar al director dentro de crew ni seleccionar el trailer; esas transformaciones ya se hicieron en el composabile.

20.3 Por qué “llegó el JSON” no significa “ya se ve el detalle”

MovieDetailView mantiene heroReady separado de isLoading. La query puede haber terminado, pero la vista conserva el overlay hasta que MovieHero confirme que el póster está listo.

Ubicación en el proyecto: frontend/src/features/movies/components/movie-detail/MovieHero.vue · Component
· checkReady()

```
function checkReady() {
  if (posterLoaded.value) {
    setTimeout(() => {
      isReady.value = true;
      intro.value = true;
      emit("ready");
    }, 150);
  }
}
```

MovieDetailView recibe ese evento con onHeroReady(). Sólo cuando heroReady es true monta MovieTabs. Esto evita que el contenido completo aparezca mientras el hero todavía está entrando visualmente.

20.4 Reparto, Reseñas y Similares no se cargan de la misma forma

Ubicación en el proyecto: frontend/src/features/movies/components/movie-detail/MovieTabs.vue · Component
· selección de pestaña con KeepAlive

```
<KeepAlive>
  <MovieCast v-if="activeTab === TABS.CAST" :cast="cast" />
  <SimilarMovies v-else-if="activeTab === TABS.SIMILAR" :movies="similar" />
  <MovieReviews v-else :movie-id="movieId" :movie-title="movieTitle" />
</KeepAlive>
```

Reparto es la pestaña inicial. Las películas similares ya fueron obtenidas por useMovieDetail. Reseñas es distinta: MovieReviews se crea cuando Carlos entra por primera vez a esa pestaña; su onMounted carga las reseñas desde la API PHP y MySQL. KeepAlive permite conservar la instancia después de haberla creado.

21. Trailers, películas similares e imágenes

Esta sección aplica los conceptos definidos al inicio del documento a tres datos secundarios del detalle: la key del trailer, las películas similares y las rutas de imagen. En cada caso se sigue el recorrido desde lo que recibe CineVerse hasta lo que termina viendo Carlos.

21.1 Ejemplo guiado: Carlos abre el trailer

Carlos ya está viendo el detalle de una película. useMovieDetail obtuvo previamente los videos de TMDB y debe decidir qué elemento utilizar como trailer antes de que MovieHero pueda ofrecer el botón correspondiente.

Ubicación en el proyecto: frontend/src/features/movies/composables/movie-detail/useMovieDetail.ts · Composable · selección del trailer

```
const trailerData = videosData.results.find(
  (v) => v.type === "Trailer" && v.site === "YouTube"
);

trailer: trailerData ? trailerData.key : null
```

TMDB no entrega el archivo de video dentro de esa respuesta: entrega metadata y una key. El código toma el primer elemento cuyo type sea Trailer y cuyo site sea YouTube. No comprueba la propiedad official; por eso la descripción correcta es “primer trailer de YouTube encontrado”.

```
Carlos abre el detalle
-> useMovieDetail recibe videosData.results[]
-> busca type="Trailer" + site="YouTube"
-> conserva trailerData.key
-> MovieHero recibe que existe trailer
-> Carlos pulsa "Trailer"
-> MovieHero emite openTrailer
-> MovieDetailView cambia isTrailerOpen = true
-> Vue monta el modal y su iframe
-> Carlos ve reproducirse el trailer
```

Ubicación en el proyecto: frontend/src/features/movies/views/MovieDetailView.vue · View · iframe del modal

```
<iframe
  :src="`https://www.youtube.com/embed/${trailer}?autoplay=1`"
  title="Trailer"
  allow="autoplay; encrypted-media"
  allowfullscreen
></iframe>
```

MovieHero no abre directamente el modal: emite el evento openTrailer y MovieDetailView, que es el componente padre que controla el modal, reacciona al evento. Al cerrar el modal, el iframe se desmonta y la reproducción se detiene. La tecla Escape también ejecuta el cierre; mientras el modal está abierto, el scroll del body permanece bloqueado.

21.2 Ejemplo guiado: Carlos abre la pestaña Similares

La petición de películas similares no comienza cuando Carlos pulsa la pestaña. useMovieDetail ya obtiene esos datos durante la carga del detalle. El clic posterior sólo cambia qué componente de MovieTabs se muestra en pantalla.

Ubicación en el proyecto: frontend/src/features/movies/services/movies.service.ts · Service del frontend · getSimilarMovies()

```
const { data } = await api.get(`/movie/${movieId}/similar`, {
  params: {
    page,
    language: "en-US",
  },
});
```

Recorrido paso a paso:

```
Carlos abre el detalle
-> useMovieDetail llama getSimilarMovies(movieId, 1)
-> movies.service.ts
-> GET /movie/{id}/similar
-> TMDB devuelve results[]
-> useMovieDetail conserva el array similar
-> MovieDetailView lo entrega a MovieTabs
-> Carlos pulsa la pestaña "Similares"
-> MovieTabs activa TABS.SIMILAR
-> SimilarMovies recibe :movies="similar"
-> Vue crea las tarjetas
-> Carlos ve las películas similares
```

La llamada utiliza actualmente `language="en-US"` fijo en lugar del idioma activo. Por eso Carlos puede estar usando CineVerse en español y, aun así, encontrar títulos o textos de películas similares en inglés. Esta es una característica de la implementación actual, no del componente visual.

21.3 Ejemplo guiado: cómo Carlos termina viendo un póster

TMDB normalmente no envía los bytes de una imagen dentro del JSON. Devuelve una ruta como `/abc123.jpg`. CineVerse combina esa ruta con el servidor de imágenes de TMDB y con un tamaño apropiado para cada contexto; después el navegador realiza otra solicitud para descargar la imagen.

```
poster_path = "/abc123.jpg"

https://image.tmdb.org/t/p/w500/abc123.jpg
https://image.tmdb.org/t/p/original/abc123.jpg
```

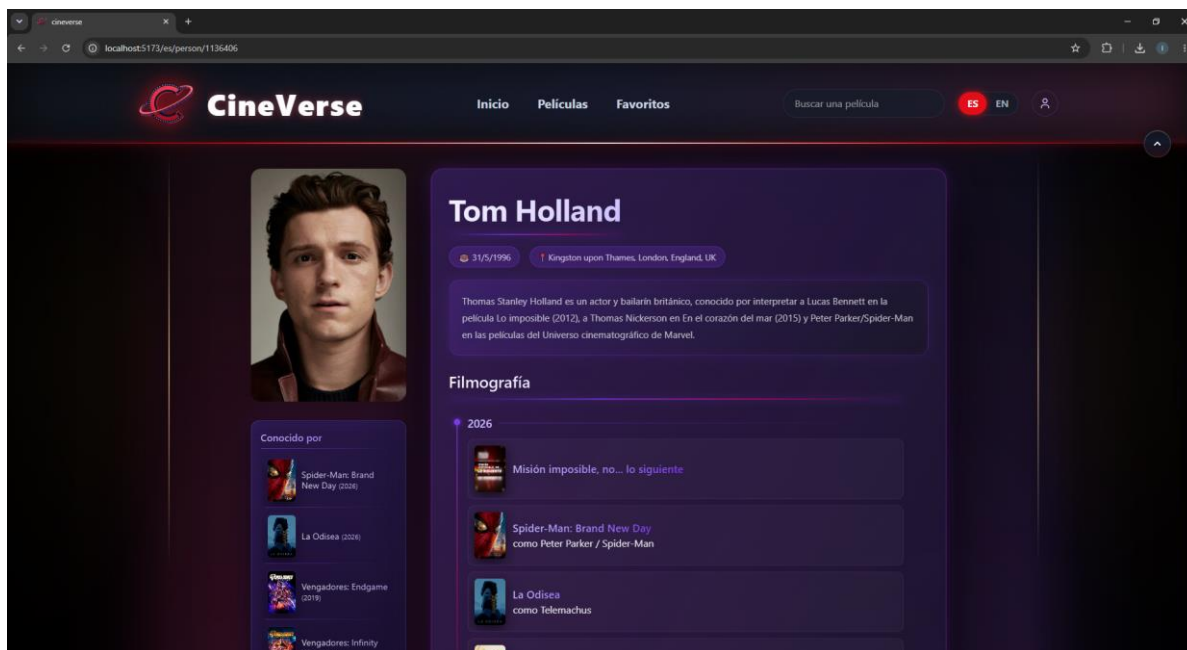
Recorrido paso a paso:

```
TMDB responde JSON con poster_path
-> el componente recibe la ruta
-> combina host + tamaño + poster_path
-> obtiene una URL completa de image.tmdb.org
-> el navegador solicita esa imagen
-> <img> la renderiza
-> Carlos ve el póster
```

El tamaño depende del contexto: MovieHero utiliza original para una imagen grande; SimilarMovies utiliza w300 para sus tarjetas; MovieCast utiliza w185 para los retratos del reparto. Los fallbacks tampoco son idénticos: MovieHero puede mostrar `no-image-poster.jpg`; MovieCast y la ficha de persona usan `no-image-avatar.jpg`; MovieResults, en cambio, descarta de la grilla las películas que no tienen un `poster_path` utilizable.

22. Actores y filmografía

El detalle de una persona comienza en el reparto de una película. Desde ese clic, Vue Router cambia de vista y una única petición a TMDB trae tanto los datos de la persona como sus `movie_credits`.



Detalle de persona en CineVerse: hero, "Conocido por" y filmografía.

22.1 Ejemplo guiado: Carlos pulsa un actor del reparto

- MovieCast.vue muestra las tarjetas del reparto. Al hacer clic ejecuta `goToPerson(actor.id)`.

- `goToPerson` usa `router.push` con `name="Person"`, mantiene `lang` y coloca el `id` de la persona en la nueva ruta.
- `PersonDetailView.vue` obtiene `route.params.id` y llama a `usePersonDetail(id, locale)`.
- `usePersonDetail` crea una query identificada por el `id` de la persona y el idioma.
- `person.service.ts` ejecuta `GET /person/{id}` con `language` y `append_to_response=movie_credits`.
- `TMDB` devuelve `PersonDetail` y `movie_credits` dentro de la misma respuesta. `Vue Query` conserva el resultado y `PersonDetailView` lo pasa a `PersonHero`.

Ubicación en el proyecto: `frontend/src/features/movies/components/movie-detail/MovieCast.vue` · Component
· `goToPerson()`

```
function goToPerson(id: number) {
  router.push({
    name: "Person",
    params: { lang: route.params.lang, id },
  });
}
```

Ubicación en el proyecto: `frontend/src/features/person/services/person.service.ts` · Service
`getFullPersonDetail()`

```
const { data } = await api.get<PersonDetail>(`/person/${personId}`, {
  params: {
    language,
    append_to_response: "movie_credits",
  },
});
```

`append_to_response` evita hacer una segunda petición separada sólo para la filmografía: `movie_credits` llega incorporado dentro de la respuesta de la persona.

22.2 Qué construye `PersonHero` con la respuesta

`PersonHero` muestra fotografía, nombre, cumpleaños, lugar de nacimiento y biografía. Si `profile_path` no es válido utiliza `no-image-avatar.jpg`. La biografía se muestra resumida hasta `MAX_CHARS = 600` y, cuando es más larga, aparece el control `Leer más / Leer menos`.

Dos usos relevantes en el código fuente: `frontend/src/features/person/components/PersonHero.vue` · Component
· distribución de `movie_credits`

```
<PersonKnownFor :credits="person.movie_credits?.cast || []" />
<PersonFilmography :credits="person.movie_credits?.cast || []"/>
```

Detalle de implementación. Las dos secciones reciben actualmente `movie_credits.cast`. Por lo tanto, la filmografía visual se construye con créditos de actuación; no mezcla automáticamente los créditos de crew.

22.3 Cómo se construye “Conocido por”

Ubicación en el proyecto: `frontend/src/features/person/components/PersonKnownFor.vue` · Component
`knownFor`

```
const knownFor = computed(() =>
  [...props.credits]
    .sort((a, b) => b.popularity - a.popularity)
    .slice(0, 8)
);
```

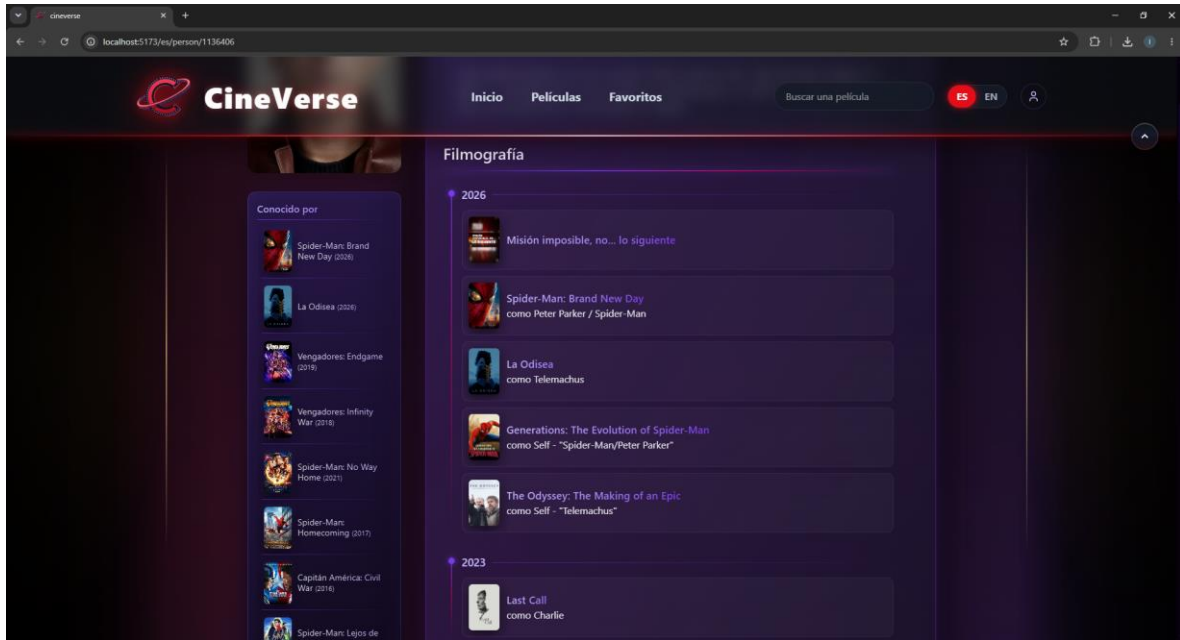
El array se copia antes de ordenar para no modificar directamente la prop recibida. Después se ordena por `popularity` de mayor a menor y se conservan ocho películas. Al pulsar una de esas tarjetas, el componente vuelve a navegar a `MovieDetail` manteniendo el idioma actual.

22.4 Cómo se organiza la filmografía

Ubicación en el proyecto: `frontend/src/features/person/components/PersonFilmography.vue` · Component
`sortedCredits` y `groupedByYear`

```
const sortedCredits = computed(() =>
  [...props.credits]
    .filter(m => m.release_date)
    .sort((a, b) =>
      new Date(b.release_date).getTime() -
      new Date(a.release_date).getTime()
    )
);
```

PersonFilmography descarta créditos sin `release_date`, ordena del más reciente al más antiguo y después agrupa las películas por el año extraído de YYYY-MM-DD. El template recorre esos grupos y construye una timeline; hacer clic en una película vuelve a llevar al detalle de ese título.



Filmografía en CineVerse: créditos de actuación ordenados por fecha y agrupados por año.

23. Favoritos

Los favoritos se guardan únicamente en el navegador. No requieren una cuenta, no se envían al backend y no crean registros en MySQL. El store de Pinia conserva objetos `Movie` y `pinia-plugin-persistedstate` los guarda en `localStorage`.

Regla	Implementación
Sin duplicados	<code>isFavorite()</code> y <code>addFavorite()</code> comparan <code>movie.id</code> .
Máximo	<code>MAX_FAVORITES = 20</code> .
Eliminar	<code>removeFavorite(id)</code> .
Vaciar	<code>clearFavorites()</code> .
Persistencia	<code>persist: true</code> en el store.

Implicación práctica

Los favoritos pertenecen al navegador actual, no a la cuenta de CineVerse. Si el usuario cambia de navegador, utiliza otro perfil o borra el almacenamiento local, la lista puede desaparecer. Las reseñas no se ven afectadas porque sí están guardadas en MySQL y asociadas a la cuenta.

23.1 Pinia aplicado a favoritos

La arquitectura general de Pinia se explica en la sección 12. En Favoritos interesa su aplicación concreta: MovieCard, MovieHero, AppHeader y MoviesFavoritesView consultan el mismo useFavoritesStore(), por lo que todos observan una única lista reactiva durante la vida de la SPA.

El store "favorites" guarda la lista de películas favoritas y permite agregar, eliminar o consultar elementos. La persistencia se utiliza únicamente para conservar este estado local en el navegador; no sustituye la sesión PHP ni guarda datos en MySQL.

23.2 Cómo está construido useFavoritesStore

```
const favorites = ref<Movie[]>([]);

const favoritesCount = computed(() =>
  favorites.value.length
);

isFavorite(id)
addFavorite(movie)
removeFavorite(id)
clearFavorites()

persist: true
```

favorites es el estado principal y contiene objetos Movie completos, no solamente ids. favoritesCount es derivado: se recalcula a partir de favorites.length. isFavorite() usa some() para buscar una película por id. addFavorite() devuelve true si pudo agregarla y false si ya existía o si se alcanzó el máximo de 20.

removeFavorite(id) construye un nuevo array excluyendo ese id. clearFavorites() reemplaza el array por uno vacío. Como favorites es reactivo, cada cambio se refleja inmediatamente en todos los componentes que consumen el store.

23.3 Ejemplo completo: Carlos guarda Fight Club como favorito

Carlos encuentra Fight Club y pulsa el icono del corazón. La película ya está cargada en la tarjeta, de modo que CineVerse no necesita consultar otra API para guardarla como favorita.

MovieCard recibe un objeto Movie:

```
{
  id: 550,
  title: "Fight Club",
  poster_path: "/...",
  release_date: "1999-10-15",
  vote_average: ...,
  ...
}
```

Carlos pulsa el corazón

```
-> onFavoriteClick(movie)
-> isFavorite(550) == false
-> addFavorite(movie)
-> favorites.value.push(movie)
-> devuelve true
-> Vue actualiza el corazón y favoritesCount
-> el plugin de persistencia guarda el nuevo estado en localStorage
```

No se envía POST al backend y no se crea ninguna fila en MySQL. Tampoco se vuelve a pedir el detalle de la película para guardarla: se persiste el objeto Movie que el componente ya tenía disponible.

23.4 Persistencia después de recargar

Después de F5, pinia-plugin-persistedstate restaura desde localStorage el estado del store favorites. El mecanismo general de serialización y rehidratación se desarrolla en la sección 12.3; aquí la consecuencia funcional es que la lista vuelve a estar disponible sin consultar al backend.

La persistencia pertenece al navegador, no al usuario autenticado. Iniciar sesión con otra cuenta en el mismo perfil del navegador no crea otra lista independiente. Usar otro navegador, otro perfil o borrar localStorage puede hacer desaparecer esos favoritos.

23.5 Límite de 20 y comportamiento de los botones

MAX_FAVORITES vale 20. addFavorite() comprueba primero duplicados y luego favorites.length >= MAX_FAVORITES. En MovieCard se utiliza el booleano devuelto: si addFavorite() devuelve false por el límite, la tarjeta muestra temporalmente un tooltip durante 1500 ms.

MovieHero también alterna favoritos, pero actualmente no utiliza el valor false devuelto por addFavorite(). Por eso el límite se respeta igualmente, aunque desde el hero no aparece el mismo aviso visual de límite alcanzado. Es una diferencia actual entre ambas interfaces.

23.6 Eliminar un favorito

```

usuario pulsa el corazón de una película ya guardada
-> isFavorite(id) == true
-> removeFavorite(id)
-> filter(m => m.id !== id)
-> favorites cambia
-> Vue actualiza la interfaz
-> el estado persistido también queda actualizado

```

23.7 Página Favoritos y "Limpiar todo"

MoviesFavoritesView obtiene directamente favorites y favoritesCount del store. Muestra el contador actual sobre 20 y crea una MovieCard por cada objeto guardado. Si la lista está vacía, en lugar de la grilla muestra un estado vacío con un enlace para volver al explorador.

El botón Limpiar todo no borra inmediatamente. openConfirm() abre un modal, un watch bloquea el scroll del body y confirmClear() llama clearFavorites(). Al vaciarse el store, la grilla desaparece, favoritesCount pasa a 0 y el plugin persiste el estado vacío.

23.8 Consecuencia de guardar el objeto Movie completo

MoviesFavoritesView reutiliza los objetos Movie guardados para construir las tarjetas. La vista no recorre favoritos haciendo una consulta nueva a TMDB por cada película. Esto hace que abrir Favoritos sea inmediato y funcione con los datos persistidos, pero también significa que esos datos son una copia local del momento en que se guardó cada película.

24. Cómo está organizado el backend PHP

El backend de CineVerse está escrito en PHP puro, sin Laravel, Symfony ni otro framework. Por eso el propio proyecto implementa el recorrido completo de una petición: prepara el entorno, reconoce la ruta, aplica middleware, lee los datos HTTP, valida la entrada, ejecuta reglas de negocio, consulta MySQL y construye la respuesta JSON.

La separación en clases evita concentrar todo ese trabajo dentro de un único archivo. Cada capa responde una pregunta diferente: el Router decide qué endpoint corresponde; los middleware deciden si la petición puede continuar; el Controller coordina HTTP; los Validator comprueban la entrada; el Service aplica reglas de CineVerse; el Model habla con MySQL mediante PDO; y Response construye la salida.

Solicitud HTTP

- > public/index.php
- > bootstrap.php / PDO / Session / CORS
- > routes/api.php
- > Router
- > Middleware
- > Controller
 - > Request (lee body / query string)
 - > Validator
- > Service
- > Model
- > PDO / MySQL
- > Response JSON

Mapa de las clases propias del backend

La versión actual del backend contiene 23 clases programadas específicamente para CineVerse, sin contar las librerías instaladas dentro de vendor. El mapa siguiente permite ubicar cada clase antes de estudiar el recorrido de una petición. No todas necesitan la misma profundidad: algunas se desarrollan aquí y otras, como MailService y AppLogger, tienen secciones específicas más adelante.

Capa	Clase y ubicación dentro de backend/	Responsabilidad principal
Controller	AuthController controllers/AuthController.php	Recibe las operaciones HTTP de registro, login, verificación, recuperación, logout y consulta del usuario autenticado.
Controller	ProfileController controllers/ProfileController.php	Atiende GET /profile y coordina Session, ProfileService y Response.
Controller	ReviewController controllers/ReviewController.php	Atiende creación, edición, eliminación y lectura de reseñas.
Core / auth	Session core/auth/Session.php	Agrupar sesión PHP, usuario autenticado, regeneración, logout y token CSRF.
Core / HTTP	Router core/http/Router.php	Resuelve método + ruta, ejecuta middleware y llama al Controller configurado.
Core / HTTP	Request core/http/Request.php	Lee el body JSON y los parámetros de la query string.
Core / HTTP	Response core/http/Response.php	Construye respuestas JSON uniformes y establece el estado HTTP.
Core / logging	AppLogger core/logging/AppLogger.php	Se encarga de registrar errores y eventos técnicos del backend mediante Monolog, guardando la información en los archivos de log configurados por la aplicación.
Middleware	AuthMiddleware core/middleware/AuthMiddleware.php	Impide continuar cuando una ruta exige un usuario autenticado y no existe sesión válida.
Middleware	CsrfMiddleware core/middleware/CsrfMiddleware.php	Valida la cabecera X-CSRF-Token en operaciones protegidas.
Middleware	CorsMiddleware core/middleware/CorsMiddleware.php	Configura CORS para el frontend y responde las solicitudes OPTIONS de comprobación previa.
Validación base	Validator core/validation/Validator.php	Convierte y valida tipos básicos recibidos desde HTTP.
Base de datos	Database database/Database.php	Crea y devuelve la conexión PDO configurada con las variables del backend.
Model	User models/User.php	Agrupar las consultas SQL sobre cuentas, verificación y recuperación de contraseña.
Model	LoginAttempt models/LoginAttempt.php	Lee y modifica la tabla login_attempts utilizada por el límite de intentos de login.
Model	Review models/Review.php	Ejecuta las consultas SQL relacionadas con reseñas y sus estadísticas.
Model	Profile models/Profile.php	Ejecuta las consultas que construyen el resumen de actividad del perfil.
Service	AuthService services/AuthService.php	Aplica las reglas de autenticación, verificación, recuperación y manejo de intentos de login.
Service	MailService services/MailService.php	Construye y envía los correos de verificación y recuperación mediante PHPMailer.

Capa	Clase y ubicación dentro de backend/	Responsabilidad principal
Service	ReviewService services/ReviewService.php	Aplica las reglas de negocio de las reseñas y delega persistencia en Review.
Service	ProfileService services/ProfileService.php	Define la regla de seis reseñas recientes y delega las consultas en Profile.
Validator específico	AuthValidator validators/AuthValidator.php	Aplica las reglas propias de username, email, password e idioma.
Validator específico	ReviewValidator validators/ReviewValidator.php	Aplica las reglas propias de movie_id, título, rating y comentario.

Las secciones siguientes siguen la misma separación: infraestructura HTTP primero, autenticación y cuenta después, y finalmente reseñas y perfil. El objetivo es que una clase pueda localizarse tanto por su archivo como por su responsabilidad dentro del recorrido.

24.1 public/index.php: punto de entrada de la API

backend/public/index.php es el punto de entrada principal de la API propia de CineVerse. Apache dirige hacia este archivo las solicitudes destinadas al backend, por lo que rutas como /login, /reviews o /reviews/15 terminan allí aunque no exista un archivo PHP con ese nombre. Las consultas que el frontend envía directamente a TMDB no pasan por index.php. Su función no es implementar cada caso de uso: carga el entorno común, abre PDO, inicia la sesión, aplica CORS, carga las rutas y finalmente entrega el método HTTP y el path al Router, que decide qué Controller debe ejecutarse.

```
require_once __DIR__ . '/../core/bootstrap.php';

try {
    $pdo = Database::connect();
} catch (\PDOException $exception) {
    AppLogger::error($exception->getMessage(), [
        'controller' => 'Application',
        'method' => 'bootstrap',
        'status' => 503,
        'pdo_code' => $exception->getCode(),
    ]);

    Response::error('DATABASE_UNAVAILABLE', 503);
    exit;
}

Session::start();
CorsMiddleware::handle();
```

El require_once ejecuta el bootstrap antes de procesar la petición. Database::connect() crea la conexión PDO; si MySQL no está disponible, la excepción se captura en este nivel porque todavía no existe un Controller capaz de manejarla. Después se inicia la sesión y se aplica CORS. Este orden explica por qué una caída temprana de MySQL puede producir la respuesta 503 antes de que el Router intervenga.

```
$routes = require __DIR__ . '/../routes/api.php';

$method = $_SERVER['REQUEST_METHOD'];
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

$basePath = '/cineverse/backend/public';
$path = str_replace($basePath, '', $path);

$router = new Router($routes);
$router->dispatch($method, $path, $pdo);
```

El método HTTP y el path se obtienen de la petición real. parse_url(..., PHP_URL_PATH) elimina la query string para que el Router compare sólo la ruta. El basePath actual permite quitar un prefijo de instalación concreto y, finalmente, dispatch() recibe método, ruta y conexión PDO.

24.2 bootstrap.php: preparar PHP antes de atender la ruta

bootstrap.php no decide qué endpoint se ejecutará. Su responsabilidad es preparar el entorno común que cualquier endpoint puede necesitar. Esta sección lo presenta dentro del flujo; la configuración de .env y Composer se desarrolla en la sección 6.

```
require_once __DIR__ . '/../vendor/autoload.php';

date_default_timezone_set('America/Argentina/Buenos_Aires');

$dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/../');
$dotenv->load();

$dotenv->required([
    'DB_HOST', 'DB_PORT', 'DB_NAME', 'DB_USER',
    'DB_CHARSET', 'APP_ENV', 'APP_DEBUG',
    'CINEVERSE_LOG_PATH'
]);
```

vendor/autoload.php permite utilizar tanto las clases propias configuradas mediante PSR-4 como las dependencias instaladas por Composer. phpdotenv lee backend/.env y coloca los valores en \$_ENV. required() comprueba que las variables imprescindibles existan antes de que el backend continúe.

24.3 routes/api.php: declarar qué endpoints existen

routes/api.php devuelve un array organizado por método HTTP. Cada entrada describe una ruta, el método del Controller que debe ejecutarse y dos requisitos de seguridad. Declarar estas reglas en un único archivo permite que el Router sea genérico: no necesita conocer la lógica específica de login, perfil o reseñas.

```
"POST" => [
    "/reviews" => [
        "handler" => [ReviewController::class, "create"],
        "auth" => true,
        "csrf" => true
    ]
],

"GET" => [
    "/user" => [
        "handler" => [AuthController::class, "getAuthenticatedUser"],
        "auth" => true,
        "csrf" => false
    ]
],

"PUT" => [
    "/reviews/{id}" => [
        "handler" => [ReviewController::class, "update"],
        "auth" => true,
        "csrf" => true
    ]
]
```

handler contiene la clase y el método que resolverán la operación. auth indica si antes debe existir user_id en la sesión. csrf agrega la comprobación del token para operaciones protegidas que modifican estado. Así, POST /reviews y PUT /reviews/{id} requieren sesión y CSRF, mientras GET /user necesita sesión pero no CSRF porque sólo consulta información.

24.4 Router: encontrar la ruta y ejecutar el handler

Router.php está ubicado en backend/core/http/Router.php. public/index.php crea esta clase con la tabla de rutas cargada desde routes/api.php y luego llama a Router::dispatch(). El método recibe el método HTTP, el path y la conexión PDO; selecciona únicamente las rutas declaradas para ese método, recorre las alternativas y transforma parámetros dinámicos como {id} en grupos numéricos de una expresión regular.

```

$methodRoutes = $this->routes[$method] ?? [];

foreach ($methodRoutes as $route => $routeData) {
    $pattern = preg_replace(
        '#\[a-zA-Z_]+\#',
        '([0-9]+)',
        $route
    );

    $pattern = '#^' . $pattern . '$#';

    if (preg_match($pattern, $path, $matches)) {
        ...
    }
}

```

Por ejemplo, `/reviews/{id}` se convierte en un patrón que acepta `/reviews/15` pero no `/reviews/15/edit`. Los símbolos `^` y `$` obligan a que la coincidencia abarque toda la ruta. El grupo `([0-9]+)` captura el número para entregarlo después al método del Controller.

24.4.1 Cómo el Router invoca los middleware

```

if ($routeData["auth"]) {
    if (!$authMiddleware->requireAuth()) {
        return;
    }
}

if ($routeData["csrf"]) {
    if (!$csrfMiddleware->validateCsrf()) {
        return;
    }
}

```

En este punto interesa la responsabilidad del Router: lee los flags `auth` y `csrf` declarados en `routes/api.php` y, cuando corresponde, invoca los middleware antes de crear el Controller. Si uno devuelve `false`, `dispatch()` termina inmediatamente. Por eso una petición sin sesión o con CSRF inválido no entra en `ReviewController::create()` ni llega a ejecutar reglas de negocio o SQL. El funcionamiento interno de `AuthMiddleware` y `CsrfMiddleware` se desarrolla en la sección 24.6.

24.4.2 Parámetros dinámicos y llamada al Controller

```

array_shift($matches);

$matches = array_map(
    static fn (string $value) => ctype_digit($value)
        ? (int) $value
        : $value,
    $matches
);

[$controllerClass, $method] = $routeData["handler"];
$controller = new $controllerClass($pdo);
$controller->$method(...$matches);

```

`preg_match()` guarda la URL completa en `$matches[0]`, por eso `array_shift()` la elimina. Los valores capturados llegan como strings y se convierten a `int` cuando contienen únicamente dígitos. Después el Router toma el handler declarado en `routes/api.php`, crea el Controller con la conexión PDO y utiliza `...$matches` para pasar los parámetros de la URL. Para `PUT /reviews/15`, la llamada final equivale a `$controller->update(15)`. Si ninguna ruta coincide, el Router responde `ROUTE_NOT_FOUND` con HTTP 404.

24.5 Request: leer JSON y parámetros de la URL

`Request.php` está ubicado en `backend/core/http/Request.php`. Es una clase propia que encapsula los datos que entran desde HTTP y ofrece a los Controllers una forma controlada de acceder al body JSON y a los parámetros

de la query string. No aplica reglas de negocio y tampoco es una etapa que el Router ejecute antes del Controller: en el recorrido real, el propio Controller utiliza Request cuando necesita leer esos datos.

```
private ?array $body = null;

private function getJson(): array
{
    $rawBody = file_get_contents("php://input");
    $data = json_decode($rawBody, true);

    if (!is_array($data)) {
        throw new RuntimeException("INVALID_JSON_BODY");
    }

    return $data;
}

public function json(): array
{
    if ($this->body === null) {
        $this->body = $this->getJson();
    }
    return $this->body;
}

public function query(string $key, mixed $default = null): mixed
{
    return $_GET[$key] ?? $default;
}
```

php://input contiene el body crudo enviado por el cliente. json_decode(..., true) lo convierte en un array asociativo. La propiedad \$body actúa como caché dentro del objeto Request: el JSON se procesa la primera vez que se llama json() y después se reutiliza. query() no lee el body; consulta \$_GET, por ejemplo para obtener page en /reviews?page=2.

24.6 Middleware: autenticación, CSRF y CORS

Los middleware propios del backend están en backend/core/middleware/. AuthMiddleware.php exige una sesión válida cuando la ruta tiene auth=true; CsrfMiddleware.php valida el token de las operaciones con csrf=true; y CorsMiddleware.php configura qué origen, métodos y cabeceras puede utilizar el frontend. Son capas previas al Controller: si una comprobación falla, la funcionalidad final no se ejecuta.

```
public function requireAuth(): bool
{
    if (!Session::has("user_id")) {
        Response::error("AUTH_REQUIRED", 401);
        return false;
    }
    return true;
}

public function validateCsrf(): bool
{
    $token = $_SERVER["HTTP_X_CSRF_TOKEN"] ?? "";

    if (!Session::validateCsrfToken($token)) {
        Response::error("INVALID_CSRF_TOKEN", 403);
        return false;
    }
    return true;
}
```

AuthMiddleware comprueba la identidad asociada a la sesión. CsrfMiddleware lee la cabecera X-CSRF-Token y delega la comparación en Session. Ambos pueden construir la respuesta de error y devolver false, por lo que el Router corta el recorrido antes de ejecutar el handler. La razón de seguridad de estas comprobaciones se desarrolla en la sección 27.

CorsMiddleware.php: permitir la comunicación del frontend

CorsMiddleware.php, ubicado en backend/core/middleware/CorsMiddleware.php, es una clase estática que public/index.php ejecuta antes de crear el Router. En el entorno documentado permite el origen http://localhost:5173, habilita credenciales y declara los métodos y cabeceras admitidos por la API.

```
public static function handle(): void
{
    header("Access-Control-Allow-Origin: http://localhost:5173");
    header("Access-Control-Allow-Credentials: true");
    header("Access-Control-Allow-Headers: Content-Type, X-CSRF-Token");
    header("Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS");

    if ($_SERVER["REQUEST_METHOD"] === "OPTIONS") {
        http_response_code(204);
        exit;
    }
}
```

Las solicitudes OPTIONS son comprobaciones previas que el navegador puede enviar antes de la operación real. El middleware responde 204 y termina la ejecución porque en ese caso no existe un Controller que deba atender una funcionalidad. El origen permitido está escrito directamente en el código actual; al publicar CineVerse en otro dominio debe revisarse junto con la configuración de producción.

24.7 Controller: coordinar la petición HTTP

Las clases controladoras se encuentran en backend/controllers/. La implementación actual contiene AuthController.php, ProfileController.php y ReviewController.php. El Router crea la clase indicada por el handler de la ruta y le entrega la conexión PDO mediante su constructor. Un Controller trabaja en la frontera entre HTTP y la lógica de la aplicación: lee la entrada, consulta la sesión cuando corresponde, llama a validadores y Services y utiliza Response para devolver JSON. No debería ejecutar consultas SQL ni concentrar reglas de negocio. El fragmento siguiente utiliza ReviewController::create() como ejemplo completo de esa responsabilidad.

```
public function create(): void
{
    $userId = Session::getUserId();
    $data = $this->request->json();
    $movieId = Validator::int($data["movie_id"]);
    $movieTitle = Validator::stringTrim(
        $data["movie_title"] ?? ""
    );
    $rating = Validator::int($data["rating"]);
    $comment = ReviewValidator::normalizeComment(
        Validator::stringTrim($data["comment"] ?? "")
    );
    ReviewValidator::validateCreate(
        $movieId, $movieTitle, $rating, $comment
    );
    $this->reviewService->create(
        $userId, $movieId, $movieTitle, $rating, $comment
    );
    Response::success("REVIEW_CREATED");
}
```

El fragmento muestra el núcleo real de create() con los bloques catch omitidos para concentrarse en responsabilidades. movie_id y rating se leen actualmente mediante acceso directo al array recibido; si una llamada manual omite alguna de esas claves, PHP puede producir un aviso antes de que Validator::int() aplique el flujo controlado. Utilizar \$data["movie_id"] ?? null y \$data["rating"] ?? null sería una mejora de endurecimiento. El Controller conoce la entrada HTTP y la respuesta final, pero el INSERT y la regla de una reseña por usuario y película pertenecen a capas posteriores.

24.8 Validator y ReviewValidator: comprobar la entrada

CineVerse separa validaciones generales de reglas específicas de una funcionalidad. Validator convierte y comprueba tipos básicos; ReviewValidator conoce límites concretos de una reseña.

```

public static function int(mixed $value): int
{
    if (is_string($value) && ctype_digit($value)) {
        return (int) $value;
    }

    if (!is_int($value)) {
        throw new InvalidArgumentException("MUST_BE_INTEGER");
    }

    return $value;
}

public static function validateCreate(
    int $movieId,
    string $movieTitle,
    int $rating,
    string $comment
): void {
    if ($movieId <= 0) {
        throw new InvalidArgumentException("INVALID_MOVIE_ID");
    }

    self::validateMovieTitle($movieTitle);
    self::validateContent($rating, $comment);
}

```

Validator::int() acepta un entero PHP o un string formado únicamente por dígitos y devuelve siempre un int; si no puede hacerlo lanza MUST_BE_INTEGER. ReviewValidator trabaja después con valores ya tipados y aplica reglas del dominio, como movie_id mayor que cero, rating entre 1 y 5 y comentario entre 20 y 1000 caracteres.

24.9 Service: aplicar reglas de negocio

El Service responde preguntas propias de CineVerse. ReviewService::create() no se ocupa de leer JSON ni de elegir un código HTTP. Comprueba la regla de una única reseña por usuario y película y, si se cumple, delega la escritura al Model.

```

public function create(
    int $userId,
    int $movieId,
    string $movieTitle,
    int $rating,
    string $comment
): void {
    $existingReview = Review::existsReviewByUserIdAndMovieId(
        $this->pdo, $userId, $movieId
    );

    if ($existingReview) {
        throw new RuntimeException("REVIEW_ALREADY_EXISTS");
    }

    Review::create(
        $this->pdo, $userId, $movieId,
        $movieTitle, $rating, $comment
    );
}

```

Si la regla falla, el Service lanza REVIEW_ALREADY_EXISTS. El Controller decide después que ese caso corresponde a HTTP 409. Esta separación permite que la regla exista en la capa de negocio sin acoplarla a la forma exacta de una respuesta HTTP.

24.10 Model y PDO: ejecutar SQL contra MySQL

El Model contiene el acceso a datos. Review::create() recibe la conexión PDO y ejecuta un INSERT preparado. Aquí sí aparecen nombres de tabla, columnas y parámetros SQL.

```

$stmt = $pdo->prepare("
    INSERT INTO reviews (
        user_id, movie_id, movie_title, rating, comment
    )
    VALUES (
        :user_id, :movie_id, :movie_title, :rating, :comment
    )
");

$stmt->execute([
    ":user_id" => $userId,
    ":movie_id" => $movieId,
    ":movie_title" => $movieTitle,
    ":rating" => $rating,
    ":comment" => $comment
]);

```

`prepare()` separa la estructura SQL de los valores. Los marcadores `:user_id`, `:movie_id` y los demás se enlazan al ejecutar la consulta. El Model no decide si el usuario estaba autenticado ni qué HTTP debe recibir el navegador; se concentra en leer o modificar MySQL.

24.11 Response: construir una salida JSON consistente

`Response.php` está ubicado en `backend/core/http/Response.php`. Se encarga de construir las respuestas HTTP en formato JSON para que los Controllers no tengan que repetir `http_response_code()` y `json_encode()` en cada método. Cumple la función opuesta a `Request`: toma un resultado interno del backend y lo transforma en una respuesta uniforme mediante `success()` o `error()`.

```

public static function success(
    string $code,
    array $data = [],
    int $statusCode = 200
): void {
    self::json($statusCode, true, $code, $data);
}

public static function error(
    string $code,
    int $statusCode
): void {
    self::json($statusCode, false, $code);
}

// salida JSON resultante
{
    "success": true,
    "code": "REVIEW_CREATED"
}

```

`Response::json()` establece el estado HTTP, crea las claves `success` y `code`, agrega data sólo cuando contiene información y finalmente serializa la respuesta con `json_encode(..., JSON_UNESCAPED_UNICODE)`. De esta manera, el frontend recibe respuestas JSON con la misma estructura básica tanto cuando una operación termina correctamente como cuando ocurre un error. En la implementación actual, el header `Content-Type: application/json; charset=utf-8` se establece desde los Controllers antes de enviar la respuesta.

24.12 Ejemplo completo: PUT /reviews/15 desde Apache hasta MySQL

Después de conocer cada pieza, el recorrido completo puede leerse sin introducir conceptos nuevos. Supongamos que un usuario autenticado edita su reseña número 15.

```

PUT /reviews/15
-> Apache / .htaccess
-> public/index.php
-> bootstrap.php
-> Database::connect() / Session::start() / CORS
-> routes/api.php encuentra PUT /reviews/{id}
-> Router convierte {id} en patrón numérico
-> AuthMiddleware::requireAuth()
-> CsrfMiddleware::validateCsrf()
-> Router extrae id=15 y crea ReviewController
-> ReviewController::update(15)
-> Request::json()
-> Validator + ReviewValidator
-> ReviewService::update()
-> Review::findReviewById(15)
-> comprobar review.user_id === usuario de sesión
-> Review::update()
-> PDO UPDATE reviews
-> MySQL
-> Response::success("REVIEW_UPDATED")
-> JSON vuelve al frontend

```

La utilidad de separar capas se ve en este recorrido. El Router no conoce la regla de propiedad; ReviewService no sabe cómo se extrajo {id} de la URL; Review no conoce CSRF; y Response no ejecuta SQL. Cada pieza puede leerse y probarse con una responsabilidad concreta. Las secciones 25 y 26 aplican esta arquitectura a autenticación, cuenta, perfil y reseñas.

25. Autenticación y cuenta

Autenticar significa comprobar quién es el usuario y mantener esa identidad durante las siguientes solicitudes. CineVerse combina validaciones, contraseñas almacenadas como hash, sesiones PHP, cookies, tokens aleatorios y correo electrónico. Una cuenta nueva debe verificar su dirección de email antes de poder iniciar sesión.

Un hash de contraseña es un resultado irreversible generado a partir de la contraseña; el backend guarda ese hash en lugar del texto original. Un token es un valor aleatorio difícil de adivinar que se utiliza, por ejemplo, para verificar un email o restablecer una contraseña. Una sesión guarda en el servidor que un navegador ya inició sesión, mientras la cookie permite relacionar las siguientes solicitudes de ese navegador con la sesión correcta.

Cómo se reparten las clases de autenticación

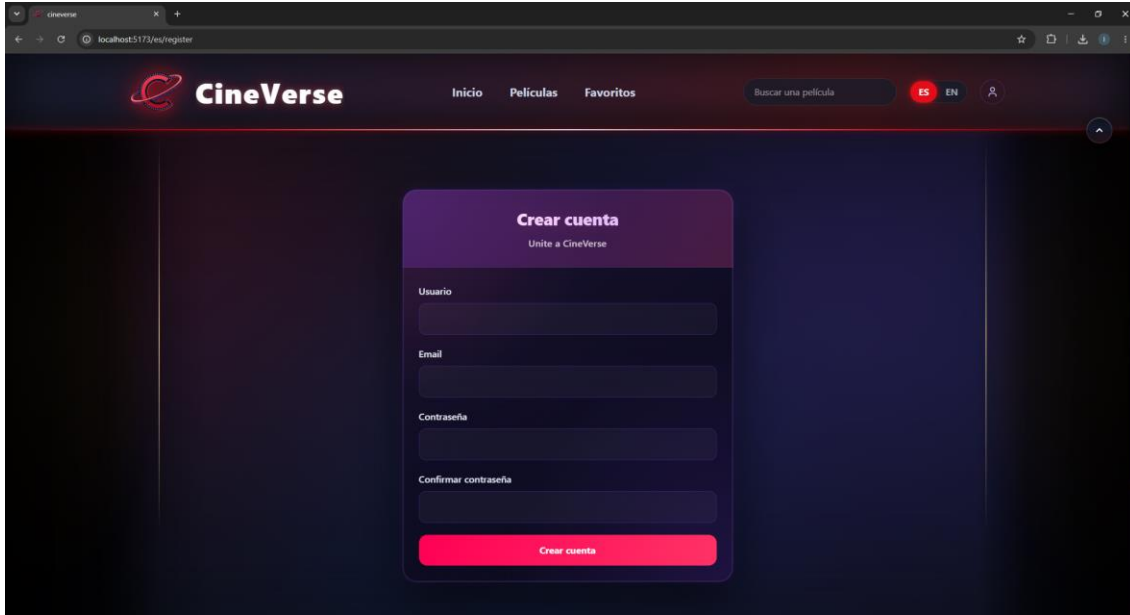
La autenticación no vive en una única clase. AuthController recibe la operación HTTP; AuthValidator valida los datos de entrada; AuthService aplica las reglas del proceso; User y LoginAttempt acceden a MySQL; Session mantiene la identidad autenticada y el token CSRF; y MailService interviene cuando el flujo necesita enviar verificación o recuperación por correo.

```

AuthController
-> AuthValidator
-> AuthService
    -> User
    -> LoginAttempt
    -> Session
    -> MailService cuando hay correo
-> Response

```

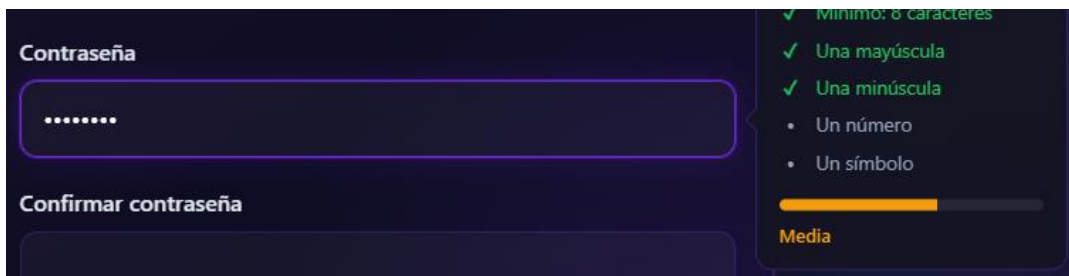
25.1 Registro



Pantalla de registro de CineVerse. Desde aquí comienza el proceso de creación y verificación de una nueva cuenta.

Ejemplo práctico: Carlos crea una cuenta

Carlos todavía no tiene una cuenta. Completa usuario, email, contraseña y confirmación. RegisterView.vue valida el formulario en el navegador antes de llamar al backend; si encuentra un error, corta el envío y lo muestra en la propia pantalla.



Validación visual de la contraseña durante el registro. La interfaz marca en tiempo real qué requisitos ya se cumplen y resume la fortaleza actual.

Frontend y backend validan por motivos diferentes. RegisterView.vue utiliza las funciones de register.validation.ts para dar feedback inmediato: mínimo de 8 caracteres, mayúscula, minúscula, número y símbolo; además valida el máximo de 32 caracteres y que la confirmación coincida. Al pulsar Crear cuenta, AuthValidator.php vuelve a comprobar en el servidor las reglas que realmente deben cumplirse, sin confiar en lo enviado por el navegador.

Piezas principales

Frontend: RegisterView.vue -> auth.service.ts

Backend: AuthController.php -> AuthService.php -> User.php -> MailService.php

```

Carlos pulsa "Crear cuenta"
-> RegisterView.register()
-> validateRegisterForm()
-> registerUser()
-> POST /register mediante backendApi
-> Router
-> AuthController::register()
-> AuthValidator valida username, email, password e idioma
-> AuthService comprueba username/email duplicados
-> password_hash(PASSWORD_DEFAULT)
  
```

```

-> genera verification_token con vencimiento de 24 horas
-> beginTransaction()
-> User::create() inserta la cuenta con is_verified = 0
-> MailService construye el correo
-> PHPMailer lo entrega al servidor SMTP
-> COMMIT
-> RegisterView muestra que debe revisar su correo

```

Si el correo no puede enviarse antes del COMMIT, AuthService ejecuta ROLLBACK y el alta no queda confirmada.

Si el envío termina correctamente, Carlos recibe el enlace de verificación y continúa con la sección 25.2.

Campo	Reglas del backend
username	3-20 caracteres; letras, números y _.
email	Formato válido; máximo 80.
password	8-32; mayúscula, minúscula, número y carácter especial.
language	Sólo es o en.

AuthValidator.php: reglas específicas de autenticación

AuthValidator.php está ubicado en backend/validators/AuthValidator.php. Es una clase de validación específica de autenticación: comprueba username, email, password e idioma, pero no consulta MySQL ni decide si una cuenta ya existe. Cuando un valor incumple una regla lanza `InvalidArgumentException` con un código estable que el Controller puede convertir en una respuesta HTTP.

```

public static function validateRegister(
    string $username,
    string $email,
    string $password
): void {
    self::validateUsername($username);
    self::validateEmail($email);
    self::validatePassword($password);
}

public static function validateLanguage(string $language): void
{
    if (!in_array($language, ["es", "en"], true)) {
        throw new InvalidArgumentException("INVALID_LANGUAGE");
    }
}

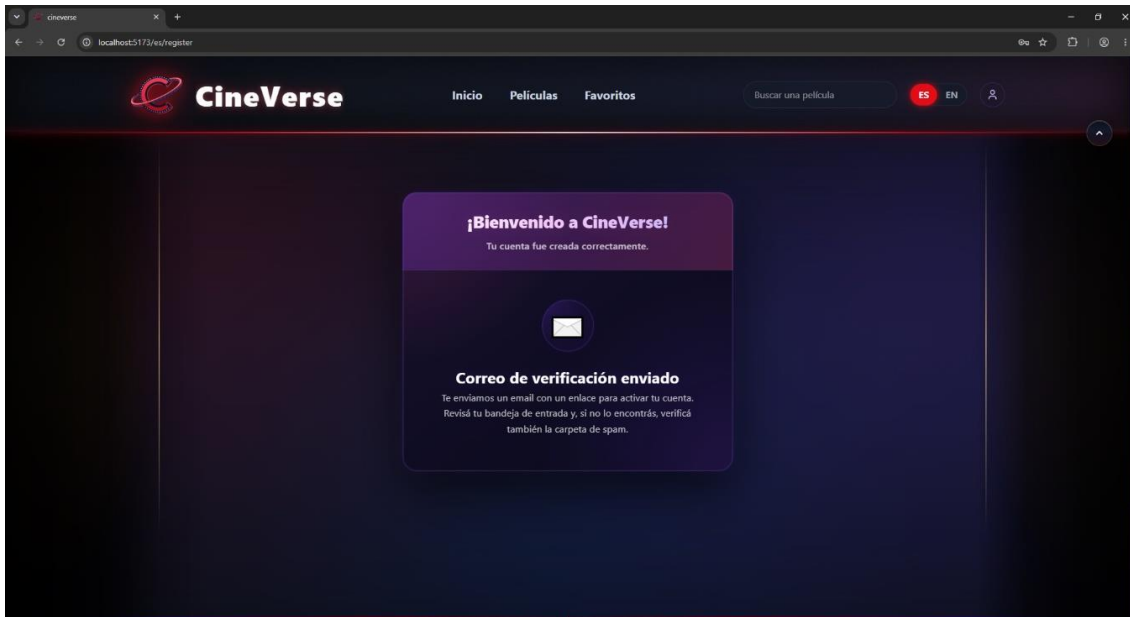
```

El primer método encadena reglas privadas de formato y complejidad. `validateLanguage()` limita los correos a los dos idiomas que realmente tienen plantillas. La comprobación de duplicados, el hash de la contraseña y la creación del usuario ocurren después, en `AuthService` y `User`.

1. `AuthController` recibe el JSON enviado por el formulario y comprueba que los campos básicos tengan el tipo esperado.
2. `AuthValidator` comprueba las reglas de username, email, password e idioma.
3. `AuthService` consulta MySQL para evitar usuarios o emails duplicados.
4. `password_hash(PASSWORD_DEFAULT)` transforma la contraseña en el hash que se guardará en users.
5. `random_bytes(32)` genera un token de verificación difícil de predecir y se guarda una fecha de expiración de 24 horas.
6. El alta del usuario y el envío del correo se realizan dentro de una transacción. El usuario se inserta como no verificado y `MailService` envía el enlace de confirmación.
7. Si todo termina correctamente se ejecuta COMMIT. Si ocurre una excepción antes de completar el proceso, se ejecuta ROLLBACK para deshacer los cambios de esa transacción.

Detalle heredado del frontend. `RegisterView.vue` todavía utiliza el nombre `laravelErrors` y contempla `password_confirmation`; el archivo `auth.ts` conserva además un comentario que menciona Laravel. El backend

actual de CineVerse no utiliza Laravel, por lo que esos nombres son restos de una implementación anterior y conviene eliminarlos o renombrarlos para evitar confusión.



Pantalla posterior al registro: el usuario debe verificar el correo antes del login.

25.1.1 Cómo se decide el idioma del correo

El backend no detecta automáticamente si el usuario quiere un correo en español o en inglés. El idioma ya está activo en vue-i18n y el frontend lo incluye explícitamente en las operaciones que pueden enviar un mensaje.

```
RegisterView.vue
locale.value = "es"
    |
    v
POST /register
{
  "username": "usuario",
  "email": "usuario@example.com",
  "password": "...",
  "language": "es"
}
    |
    v
AuthController -> AuthValidator -> AuthService -> MailService
```

AuthController lee language del JSON con Validator::stringTrim() y AuthValidator::validateLanguage() sólo acepta es o en. Después AuthService recibe ese valor y lo pasa sin reinterpretarlo a generateVerificationEmail() o generatePasswordResetEmail().

25.1.2 Cómo MailService construye el correo de verificación

MailService es una clase propia de CineVerse. PHPMailer es la librería externa que realiza el envío. Primero MailService construye el contenido; recién después delega la transmisión a PHPMailer.

generateVerificationEmail(email, token, language) realiza este recorrido:

1. Construye verificationUrl con FRONTEND_URL, el idioma y el token. El token pasa por urlencode() antes de incorporarse a la URL.
2. Forma la ruta de plantilla templates/emails/verify-account.<idioma>.php. Para es utiliza verify-account.es.php; para en, verify-account.en.php.
3. ob_start() abre un buffer de salida. require ejecuta la plantilla PHP, que utiliza \$verificationUrl para crear el botón y el enlace visible del correo.

4. `ob_get_clean()` recupera todo el HTML generado por la plantilla y lo guarda en `$html`.
5. El asunto también depende de `language`: en usa "CineVerse Account Verification" y es usa "Verificación de cuenta | CineVerse".
6. Finalmente llama al método privado `send(email, subject, html)`.

```

FRONTEND_URL + "/" + language + "/verify-email?token=" + token
    |
    v
verify-account.es.php / verify-account.en.php
    |
    v
HTML completo
    |
    v
send(destinatario, asunto, html)

```

Si la plantilla calculada no existe, `MailService` lanza `RuntimeException("EMAIL_TEMPLATE_NOT_FOUND")`. Como `AuthValidator` ya limita `language` a `es` o `en`, en el flujo normal sólo deberían utilizarse las cuatro plantillas incluidas en el proyecto.

25.1.3 Cómo PHPMailer envía realmente el mensaje

`send()` crea `new PHPMailer(true)`. El `true` indica que `PHPMailer` debe lanzar excepciones cuando el envío falla. Después configura una conexión SMTP con los valores de `backend/.env`.

```

$mail = new PHPMailer(true);
$mail->isSMTP();
$mail->Host = $_ENV["MAIL_HOST"];
$mail->Port = (int) $_ENV["MAIL_PORT"];
$mail->SMTPAuth = true;
$mail->Username = $_ENV["MAIL_USERNAME"];
$mail->Password = $_ENV["MAIL_PASSWORD"];
$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
$mail->CharSet = "UTF-8";

$mail->setFrom($_ENV["MAIL_FROM"], $_ENV["MAIL_FROM_NAME"]);
$mail->addAddress($to);
$mail->isHTML(true);
$mail->Subject = $subject;
$mail->Body = $html;
$mail->send();

```

`isSMTP()` indica que el mensaje se enviará mediante SMTP. `Host` y `Port` identifican el servidor. `SMTPAuth` activa la autenticación y `Username/Password` aportan las credenciales. `STARTTLS` cifra la conexión SMTP después de iniciarla. `setFrom()` establece el remitente, `addAddress()` el destinatario, `isHTML(true)` indica que `Body` contiene HTML, y `Subject/Body` establecen el contenido final.

Cuando `$mail->send()` termina sin excepción, `PHPMailer` entregó el mensaje al servidor SMTP configurado y `MailService` vuelve a `AuthService`. Eso no garantiza por sí solo que el destinatario ya lo haya leído; significa que el envío SMTP solicitado por la aplicación terminó correctamente.

25.1.4 Qué ocurre si el envío del correo falla

El bloque `try/catch` de `send()` envuelve específicamente la llamada a `$mail->send()`. Si esa llamada lanza un `Throwable`, `MailService` lo transforma en una `RuntimeException` con el código interno `EMAIL_DELIVERY_FAILED`. La excepción original queda guardada como `previous`, de modo que `AuthController` puede registrar el mensaje técnico real sin enviarlo al navegador. Un error que ocurriera antes, durante la configuración de `PHPMailer`, seguiría el manejo general de excepciones del backend.

```
$mail->send() lanza un Throwable
-> el try/catch de send() lo captura
-> RuntimeException("EMAIL_DELIVERY_FAILED", previous = error SMTP)
-> AuthController
-> AppLogger registra el detalle SMTP real
-> Response::error("EMAIL_DELIVERY_FAILED", 422)
-> frontend
```

En el registro, AuthService ya había abierto una transacción antes de insertar el usuario. Si el correo falla, el catch ejecuta rollBack() y vuelve a lanzar la excepción; por lo tanto, el alta no se confirma. Sólo después de enviar correctamente el correo se ejecuta commit(). El reenvío de verificación utiliza el mismo patrón para que el nuevo token no quede confirmado si el mensaje no pudo enviarse.

El código actual de AuthController registra los fallos EMAIL_DELIVERY_FAILED mediante AppLogger y responde HTTP 422. Esa clasificación se documenta como comportamiento actual; un código 5xx sería más representativo de una caída del servicio SMTP. La sección 28 detalla además qué mensaje termina viendo el usuario en cada pantalla.

25.1.5 Qué se guarda en users durante el registro

User::create() ejecuta un INSERT en users con username, email, password_hash, is_verified=0, verification_token y verification_expires_at. La contraseña original no entra en la tabla: antes AuthService la transforma con password_hash(PASSWORD_DEFAULT).

```
Registro correcto, antes de verificar el correo:
username           = "usuario"
email              = "usuario@example.com"
password_hash      = "<hash generado por PHP>"
is_verified        = 0
verification_token  = "<token aleatorio>"
verification_expires_at = "<24 horas>"
```

Ese INSERT y el envío de email están dentro de la misma transacción. Si el correo falla antes de commit(), rollBack() deshace el alta y la fila no queda confirmada.

25.2 Verificación y reenvío

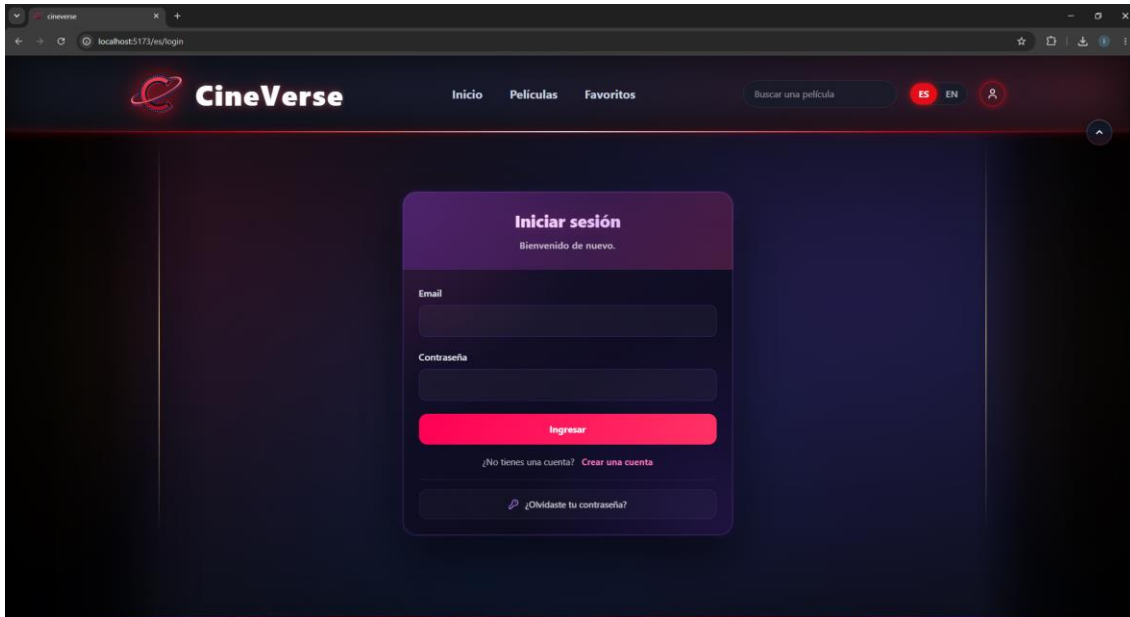
Carlos abre el correo recibido durante el registro y pulsa el enlace de verificación. La URL contiene el token generado durante el alta. GET /verify-email busca ese token, comprueba que no haya vencido y activa la cuenta. Después se borran el token y su fecha de expiración para que el mismo enlace no pueda reutilizarse. El reenvío utiliza respuestas neutras cuando el email no existe o ya está verificado para no revelar qué cuentas están registradas.

25.2.1 Qué ocurre al abrir el enlace de verificación

```
correo -> /es/verify-email?token=...
-> VerifyEmailView lee token de la URL
-> GET /verify-email?token=...
-> AuthController
-> AuthService::verifyEmail(token)
-> User::findByVerificationToken()
-> comprobar expiración
-> User::activateEmail()
-> UPDATE users:
    is_verified = 1
    verification_token = NULL
    verification_expires_at = NULL
-> EMAIL_VERIFIED
```

El token queda invalidado al activar la cuenta. El reenvío sigue otra rama: si la cuenta existe y sigue sin verificar, AuthService genera un token nuevo, actualiza las dos columnas de verificación y sólo confirma la transacción si MailService pudo enviar el nuevo mensaje.

25.3 Inicio de sesión y límite de intentos



Pantalla de inicio de sesión. Desde aquí comienza el proceso que termina creando la sesión PHP y actualizando el estado de autenticación del frontend.

25.3.1 Ejemplo práctico: Carlos inicia sesión

Carlos ya verificó su cuenta. Escribe email y contraseña y pulsa Ingresar. LoginView.vue valida primero el formulario; recién después auth.service.ts envía las credenciales a la API propia de CineVerse.

Piezas principales

Frontend: LoginView.vue -> auth.service.ts

Backend: AuthController.php -> AuthService.php -> User.php / LoginAttempt.php -> Session.php

```
Carlos pulsa "Ingresar"
-> LoginView.login()
-> loginUser()
-> POST /login
-> AuthController::login()
-> AuthValidator::validateLogin()
-> AuthService::login()
-> checkLoginAttempts(IP + email)
-> User::findByEmail()
-> password_verify()
-> comprobar_is_verified
-> limpiar intentos fallidos si existían
-> Session::regenerate()
-> Session::set("user_id", id)
-> Session::generateCsrfToken()
-> LOGIN_SUCCESS { user, csrf_token }
-> authStore.setUser(...)
-> authStore.setCsrfToken(...)
-> router.push(Home)
-> Carlos queda autenticado
```

Antes de comprobar la contraseña, AuthService revisa los intentos fallidos asociados a la combinación IP + email. Los tres primeros intentos incorrectos se registran como fallos. Cuando esa combinación ya tiene tres intentos acumulados, el intento siguiente se bloquea antes de comprobar las credenciales y el backend responde LOGIN_TEMPORARILY_BLOCKED. Los registros antiguos se eliminan después de 15 minutos. A este control se lo conoce como rate limiting o límite de intentos.

Si el usuario no existe o password_verify() determina que la contraseña no coincide con el hash guardado, se incrementa el contador de fallos. Si la contraseña es correcta pero la cuenta todavía no está verificada, el

backend devuelve EMAIL_NOT_VERIFIED. En un login correcto se limpia el contador, se regenera el id de sesión y se genera un token CSRF.

AuthController.php: recibir el POST /login

AuthController.php está ubicado en backend/controllers/AuthController.php. Entre otras operaciones atiende registro, login, verificación, recuperación, logout y consulta del usuario autenticado. En el login obtiene el JSON mediante Request, normaliza los tipos básicos, llama a AuthValidator, delega el proceso real a AuthService y finalmente construye la respuesta con Response.

```
$data = $this->request->json();

$email = Validator::stringTrim($data["email"] ?? "");
$password = Validator::string($data["password"] ?? "");

AuthValidator::validateLogin(
    $email,
    $password
);

$data = $this->authService->login(
    $email,
    $password
);

Response::success(
    "LOGIN_SUCCESS",
    $data
);
```

El fragmento es el núcleo real de login() con los bloques catch y los comentarios omitidos para concentrarse en la coordinación. El Controller conoce la entrada HTTP y la respuesta final, pero no ejecuta password_verify(), no consulta users directamente y no modifica la tabla login_attempts.

AuthService.php: aplicar las reglas del login

AuthService.php está ubicado en backend/services/AuthService.php. Es la capa que coordina las reglas de autenticación: controla intentos fallidos, consulta el usuario, verifica la contraseña y el estado de la cuenta, limpia intentos anteriores y prepara la sesión autenticada. No decide el código HTTP que recibirá el navegador.

```
$ip = $_SERVER["REMOTE_ADDR"] ?? "";
$attempt = $this->checkLoginAttempts($ip, $email);
$user = User::findByEmail($this->pdo, $email);

if (!$user || !password_verify($password, $user["password_hash"])) {
    if ($attempt) {
        LoginAttempt::increment($this->pdo, $attempt["id"]);
    } else {
        LoginAttempt::create($this->pdo, $ip, $email);
    }

    throw new RuntimeException("INVALID_CREDENTIALS");
}

if (!$user["is_verified"]) {
    throw new RuntimeException("EMAIL_NOT_VERIFIED");
}

if ($attempt) {
    LoginAttempt::reset($this->pdo, $attempt["id"]);
}

Session::regenerate();
Session::set("user_id", $user["id"]);
$csrfToken = Session::generateCsrfToken();
```

En este punto se ve por qué AuthService es una capa de negocio: combina varios componentes para cumplir una regla completa de CineVerse. El límite de intentos se comprueba antes de validar la contraseña; una cuenta no verificada se rechaza; y sólo un login correcto regenera la sesión, guarda user_id y genera el token CSRF.

User.php: Model de la tabla users

User.php está ubicado en backend/models/User.php. Es el Model que agrupa las consultas SQL sobre la tabla users: comprobación de duplicados, alta, búsquedas por id/email/tokens, activación de cuenta y actualización de tokens o contraseña. Durante el login, AuthService utiliza findByEmail() para recuperar exactamente los datos que necesita para autenticar.

```
public static function findByEmail(
    PDO $pdo,
    string $email
): array|false {
    $stmt = $pdo->prepare("
        SELECT
            id,
            username,
            email,
            password_hash,
            is_verified,
            created_at
        FROM users
        WHERE email = :email
        LIMIT 1
    ");

    $stmt->execute([
        ":email" => $email
    ]);

    return $stmt->fetch();
}
```

El Model conoce la tabla y el SQL, pero no decide si una contraseña es válida ni qué respuesta HTTP debe devolverse. Esa separación permite reutilizar User::findByEmail() en login y recuperación sin mover reglas de autenticación al acceso a datos.

LoginAttempt.php: Model del límite de intentos

LoginAttempt.php está ubicado en backend/models/LoginAttempt.php y trabaja únicamente con la tabla login_attempts. Puede buscar un registro por IP + email, crear el primer fallo, incrementar el contador, eliminarlo después de un login correcto y borrar registros cuyo último intento tiene más de 15 minutos. El Model almacena esos datos; la decisión de bloquear a partir de tres intentos pertenece a AuthService.

```
public static function increment(PDO $pdo, int $id): void
{
    $stmt = $pdo->prepare("
        UPDATE login_attempts
        SET attempts = attempts + 1
        WHERE id = :id
    ");

    $stmt->execute([
        ":id" => $id
    ]);
}

public static function deleteExpired(PDO $pdo): void
{
    $stmt = $pdo->prepare("
        DELETE FROM login_attempts
        WHERE last_attempt < DATE_SUB(NOW(), INTERVAL 15 MINUTE)
    ");

    $stmt->execute();
}
```

AuthService::checkLoginAttempts() llama primero a deleteExpired(), busca el registro de la combinación IP + email y lanza LOGIN_TEMPORARILY_BLOCKED cuando el contador existente ya es 3 o mayor. Así el Model persiste el estado del control y el Service conserva la regla de negocio.

El identificador de sesión queda en la cookie de PHP; el frontend no necesita conocer su valor. El token CSRF sí se devuelve dentro del JSON y se conserva en Pinia porque las operaciones protegidas lo deben copiar a X-CSRF-Token.

El límite de intentos se aplica antes de validar la contraseña. Los tres primeros fallos se registran. Si al comenzar un intento ya existen tres fallos para esa combinación IP+email, el intento siguiente se bloquea y responde LOGIN_TEMPORARILY_BLOCKED antes de comprobar credenciales.

25.4 Sesión del usuario y cookie

Session.php: gestión de la sesión PHP

Session.php está ubicado en backend/core/auth/Session.php. Es una clase estática que se encarga de trabajar con la sesión de la aplicación para que Controllers, Services y middleware no tengan que manipular \$_SESSION directamente en distintos lugares del código. Entre sus responsabilidades se encuentran iniciar y configurar la sesión, guardar y recuperar valores, identificar al usuario autenticado, regenerar el identificador de sesión después del login, destruir la sesión durante el logout y generar o validar el token CSRF.

```
public static function getUserId(): ?int
{
    return self::has("user_id")
        ? (int) self::get("user_id")
        : null;
}

public static function generateCsrfToken(): string
{
    $token = bin2hex(
        random_bytes(32)
    );

    self::set("csrf_token", $token);

    return $token;
}

public static function validateCsrfToken(string $token): bool
{
    return hash_equals(
        self::getCsrfToken() ?? "",
        $token
    );
}
```

getUserId() ofrece una forma única de recuperar la identidad asociada a la sesión. El token CSRF se genera con random_bytes(32), se guarda en la misma sesión y hash_equals() realiza después la comparación con el valor recibido por CsrfMiddleware. La tabla siguiente resume además la configuración concreta de la cookie y la expiración por inactividad.

Protección	Comportamiento
lifetime	0: cookie de sesión.
path	/
HttpOnly	true
SameSite	Lax
Secure	Se activa cuando \$_SERVER["HTTPS"] no está vacío.
Inactividad	1800 segundos / 30 minutos.

Protección	Comportamiento
Session fixation	session_regenerate_id(true) después del login.

25.4.1 Qué pasa después de F5 si el usuario ya había iniciado sesión

Carlos pulsa F5 después de haber iniciado sesión. Pinia vuelve a arrancar en memoria, pero el navegador conserva la cookie PHP; por eso la SPA puede reconstruir la identidad consultando /user sin pedirle otra vez la contraseña.

```
nueva carga del frontend
-> el navegador conserva la cookie PHP
-> main.ts llama authStore.initialize()
-> GET /user con withCredentials=true
-> AuthMiddleware confirma user_id en $_SESSION
-> AuthController obtiene user_id + csrf_token de Session
-> User::findById(user_id)
-> AUTHENTICATED_USER { user, csrf_token }
-> Pinia reconstruye user y csrfToken
```

Por eso el store de autenticación no necesita persistirse en localStorage. La fuente real de la sesión es PHP; Pinia sólo reconstruye una representación útil para la interfaz.

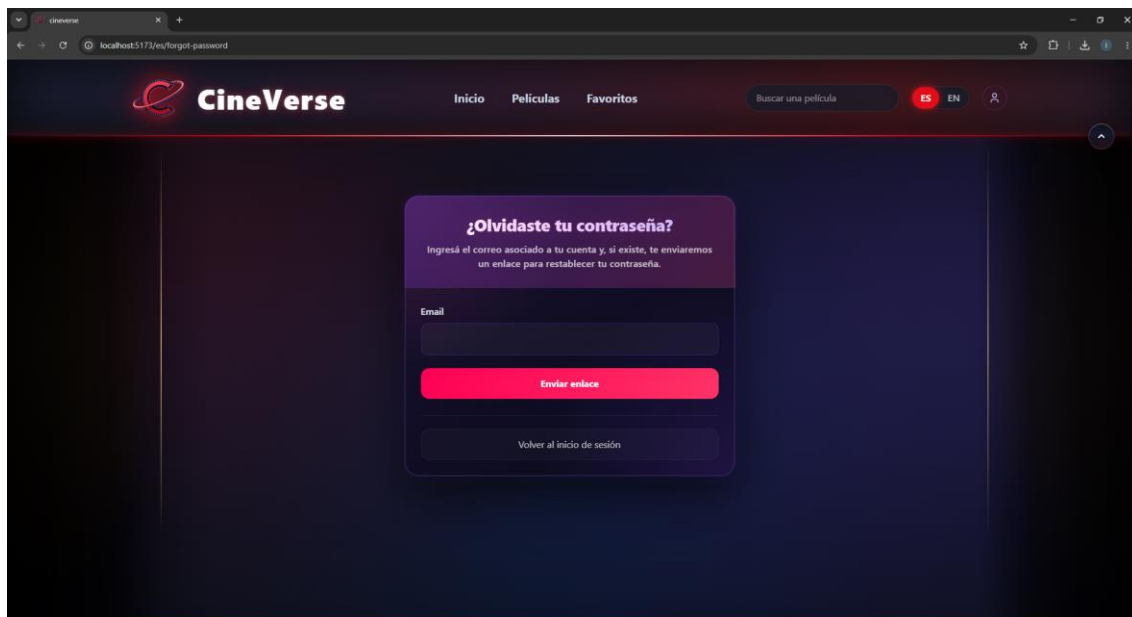
25.4.2 Cerrar sesión por dentro

Logout sí modifica estado y por eso POST /logout requiere sesión y CSRF. AuthService llama Session::destroy(): vacía \$_SESSION, expira la cookie si PHP usa cookies y destruye la sesión del servidor. Después el frontend debe limpiar su store para que la interfaz deje de considerar al usuario autenticado.

25.5 CSRF

CSRF protege operaciones que modifican datos frente a solicitudes iniciadas desde otro sitio aprovechando una sesión ya abierta. Session::generateCsrfToken() crea un valor aleatorio y lo guarda en la sesión. El frontend envía ese mismo valor en X-CSRF-Token y CsrfMiddleware lo compara con hash_equals antes de permitir la operación.

25.6 Recuperación de contraseña



Pantalla “¿Olvidaste tu contraseña?”. Es el primer paso del flujo: el usuario solicita un enlace sin que CineVerse revele si el email existe.

La recuperación no es una sola petición. CineVerse separa el proceso en tres momentos: solicitar el correo, validar el token recibido y, sólo si ese token es válido, guardar una contraseña nueva.

25.6.1 Ejemplo práctico: Carlos recupera su contraseña

Paso 1. Solicitar el enlace

Carlos no recuerda su contraseña. Desde Login pulsa “¿Olvidaste tu contraseña?”, Vue Router abre /es/forgot-password, escribe su email y pulsa “Enviar enlace”.

Piezas principales

Frontend: ForgotPasswordView.vue / ResetPasswordView.vue -> auth.service.ts

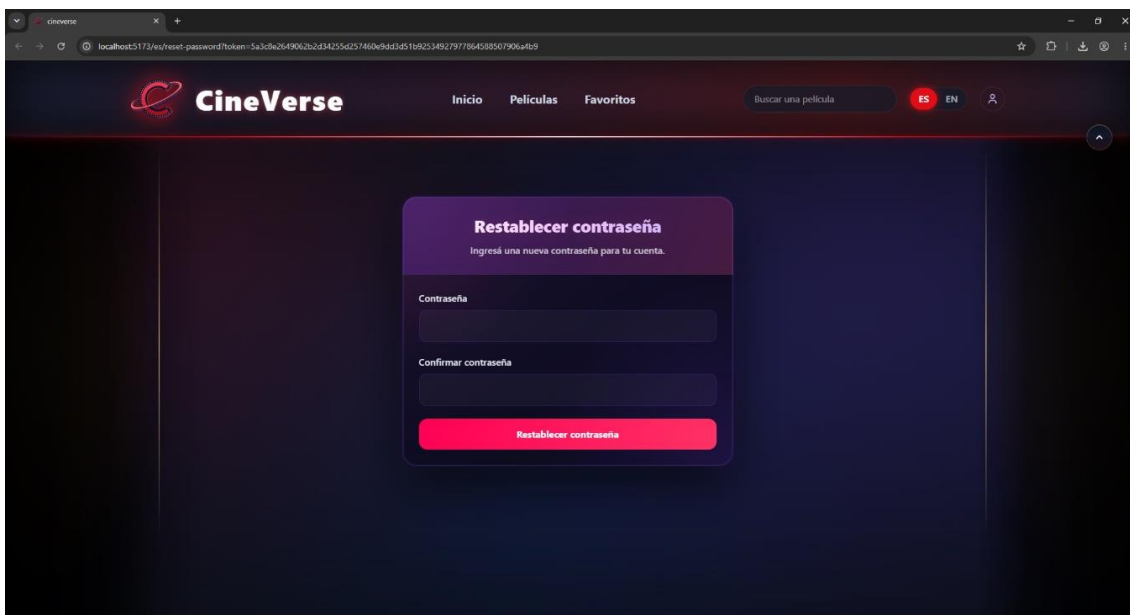
Backend: AuthController.php -> AuthService.php -> User.php -> MailService.php

```
Carlos pulsa "Enviar enlace"
-> ForgotPasswordView.sendResetEmail()
-> validateEmail()
-> forgotPassword()
-> POST /forgot-password { email, language }
-> AuthController::forgotPassword()
-> AuthService::forgotPassword()
-> si el email no existe: termina sin revelar ese dato
-> si existe: genera token con vencimiento de 1 hora
-> beginTransaction()
-> User::updatePasswordResetToken()
-> MailService -> PHPMailer -> SMTP
-> COMMIT
-> el frontend muestra una respuesta neutra
```

La respuesta neutra es deliberada. La pantalla dice “si existe” porque CineVerse evita que una persona pueda descubrir qué direcciones están registradas comparando respuestas distintas.

Paso 2. Abrir el enlace recibido

```
Carlos recibe el correo y pulsa el enlace
-> /es/reset-password?token=...
-> Vue Router monta ResetPasswordView.vue
-> onMounted() lee token de route.query
-> validateResetToken(token)
-> GET /reset-password/validate
-> backend comprueba que el token exista y no haya vencido
-> token válido: tokenStatus = "valid"
-> recién entonces aparece el formulario
```



Formulario para establecer una nueva contraseña. ResetPasswordView sólo lo muestra después de que el backend acepta el token recibido por email.

Paso 3. Establecer una contraseña nueva

Carlos escribe y confirma la nueva contraseña. La vista reutiliza las mismas reglas de contraseña del registro; si el formulario es válido, envía el token junto con la nueva contraseña al backend.

```
Carlos pulsa "Restablecer contraseña"
-> ResetPasswordView.sendResetPassword()
-> validatePassword() + confirmar coincidencia
-> resetPassword({ token, password })
-> POST /reset-password
-> AuthController::resetPassword()
-> AuthService::resetPassword()
-> volver a validar token y vencimiento
-> password_hash(nueva contraseña)
-> User::updatePassword()
-> UPDATE users
    password_hash = nuevo hash
    password_reset_token = NULL
    password_reset_expires_at = NULL
-> PASSWORD_RESET_SUCCESS
-> Carlos puede volver a Login con la nueva contraseña
```

25.6.2 Cómo se construye y envía el correo de recuperación

POST /forgot-password recibe email y language. Si el email no existe, AuthService termina sin hacer nada y el controlador devuelve igualmente PASSWORD_RESET_EMAIL_SENT; así no revela qué direcciones están registradas. Si la cuenta existe, se genera un token aleatorio de 32 bytes convertido a hexadecimal y una expiración de una hora.

AuthService inicia una transacción, guarda el token y llama a generatePasswordResetEmail(email, token, language). MailService construye FRONTEND_URL/<idioma>/reset-password?token=<token>, selecciona reset-password.es.php o reset-password.en.php, captura el HTML con ob_start()/ob_get_clean(), elige el asunto y reutiliza exactamente el mismo método send() de PHPMailer descrito para la verificación.

26. Reseñas y perfil

Las reseñas relacionan una cuenta de CineVerse con una película obtenida de TMDB. MySQL guarda user_id, movie_id, movie_title, rating y comment. Para las reseñas públicas el backend combina reviews con users para obtener también el username. Esa combinación entre tablas se realiza mediante un JOIN de SQL.

26.1 Reglas de reseña

Regla	Valor real
Una por película	Control en ReviewService + UNIQUE(user_id,movie_id) en MySQL.
Rating	Entero 1-5.
Comentario	20-1000 caracteres después de normalización.
Título de la película	Obligatorio; máximo 255.
Propiedad	Sólo el propietario puede editar/eliminar.
Paginación	12 reseñas por página.

Detalle de interfaz actual. El límite real del comentario es de 1000 caracteres en MovieReviews.vue, ReviewValidator.php y MySQL. Sin embargo, las claves commentTooLong de es.json y en.json todavía muestran un máximo de 2000. Esos textos deben decir 1000 para coincidir con la validación real.

26.2 Limpieza del comentario antes de guardarlo

Antes de validar el comentario, `ReviewValidator::normalizeComment()` elimina espacios sobrantes al principio y al final, normaliza saltos de línea y evita bloques excesivos de líneas vacías. Después comprueba la longitud y recién entonces el texto puede guardarse.

26.3 Crear

```
POST /reviews
X-CSRF-Token: <token>

{
  "movie_id": 550,
  "movie_title": "Fight Club",
  "rating": 5,
  "comment": "Una reseña válida de al menos veinte caracteres."
}
```

`ReviewService` comprueba primero si ya existe una reseña con el mismo `user_id` y `movie_id`. Si existe, devuelve `REVIEW_ALREADY_EXISTS`. Si no existe, `Review::create()` ejecuta el `INSERT`. La restricción `UNIQUE` de MySQL actúa como una segunda garantía frente a dos solicitudes concurrentes.

26.3.1 Qué ocurre en la interfaz cuando el usuario escribe una reseña

`MovieReviews` sólo muestra el botón `Escribir reseña` cuando `authStore.isAuthenticated` es `true`, no existe `userReview` y el formulario está cerrado. Al abrirlo, `formMode` pasa a `"create"`, `rating` vuelve a 0 y `comment` a texto vacío.

`StarRating` actualiza `formRating` mediante el evento `update:rating`. El textarea usa `v-model` para `formComment` y `handleCommentInput` normaliza saltos de línea mientras se escribe, evita más de una línea en blanco consecutiva y corta el texto a 1000 caracteres. `canSubmitForm` sólo queda habilitado con `rating > 0` y comentario normalizado entre 20 y 1000 caracteres.

Figura. Formulario de creación de una reseña con puntuación, comentario y contador de caracteres.

26.3.2 Ejemplo completo: Carlos reseña Fight Club

Carlos está viendo Fight Club. Abre la pestaña Reseñas, selecciona una puntuación y escribe su comentario. Cuando pulsa Guardar comienza el recorrido que termina persistiendo la reseña en MySQL y vuelve a leer el estado confirmado por el backend.

```
Película actual:
  movieId = 550
  movieTitle = "Fight Club"

Usuario escribe:
  rating = 5
  comment = "Una reseña válida ..."

MovieReviews.submitForm()
  -> toma csrfToken de useAuthStore
  -> normalizeLineBreaks(comment)
  -> createReview(payload, csrfToken)

POST /reviews
Cookie: sesión PHP
X-CSRF-Token: <token>
Body JSON:
{
  "movie_id": 550,
  "movie_title": "Fight Club",
  "rating": 5,
  "comment": "Una reseña válida ..."
}

Backend:
Router -> AuthMiddleware -> CsrfMiddleware
-> ReviewController::create()
-> Session::getUserId()
-> Request::json()
-> Validator::int/stringTrim
-> ReviewValidator::normalizeComment()
-> ReviewValidator::validateCreate()
-> ReviewService::create()
-> comprobar user_id + movie_id
-> Review::create()
-> INSERT INTO reviews (...) VALUES (...)
-> REVIEW_CREATED
```

La fila que termina en MySQL contiene user_id de la sesión; movie_id y movie_title de la película de TMDB; y rating/comment escritos por el usuario. No se guarda la reseña en Pinia ni en TMDB.

```
INSERT INTO reviews (
  user_id, movie_id, movie_title, rating, comment
) VALUES (
  :user_id, :movie_id, :movie_title, :rating, :comment
)
```

Cuando el POST termina correctamente, MovieReviews cierra el formulario y ejecuta fetchReviews(1). Es decir, no inventa localmente cómo debería quedar la pantalla: vuelve a leer desde el backend. Así se actualizan user_review, promedio, total y listado de comunidad con el estado confirmado en MySQL.

26.4 Editar y eliminar

Para editar o eliminar, el backend busca primero la reseña indicada por el id. Si no existe devuelve REVIEW_NOT_FOUND. Si existe pero user_id no coincide con el usuario de la sesión, devuelve NOT_REVIEW_OWNER. Sólo después de esa comprobación permite modificar o borrar el registro.

26.4.1 Editar una reseña

```

usuario pulsa Editar
-> openEditForm()
-> copia userReview.rating y userReview.comment al formulario
-> formMode = "edit"
-> PUT /reviews/{id}
    Body: { rating, comment }
    Cookie + X-CSRF-Token
-> ReviewController::update(id)
-> ReviewValidator::validateUpdate()
-> ReviewService::update()
-> Review::findReviewById(id)
-> comprobar review.user_id === Session user_id
-> UPDATE reviews SET rating=?, comment=? WHERE id=?
-> REVIEW_UPDATED
-> fetchReviews(1)

```

movie_id y movie_title no se modifican durante la edición. El backend sólo permite cambiar rating y comment. Si la reseña no existe responde REVIEW_NOT_FOUND; si pertenece a otro usuario responde NOT_REVIEW_OWNER.

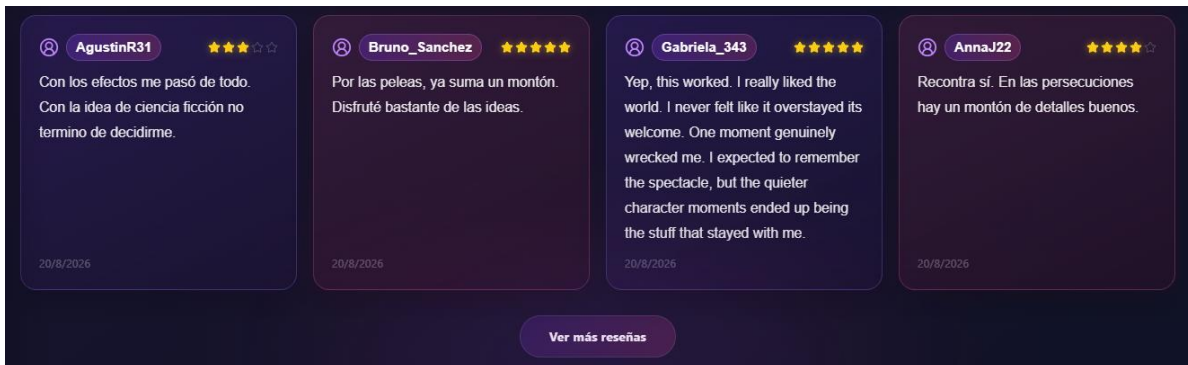
26.4.2 Eliminar una reseña

```

usuario pulsa Eliminar
-> aparece confirmación
-> confirmDelete()
-> DELETE /reviews/{id}
    Cookie + X-CSRF-Token
-> ReviewService busca la fila
-> comprueba propietario
-> DELETE FROM reviews WHERE id=:id
-> REVIEW_DELETED
-> fetchReviews(1)
-> user_review pasa a null
-> vuelve a aparecer "Escribir reseña"
-> promedio y total se recalculan

```

26.5 Reseñas públicas y estadísticas



Tarjetas de reseñas de la comunidad dentro del detalle de una película.

```

{
  "stats": {
    "average_rating": 4.2,
    "total_reviews": 110
  },
  "pagination": {
    "page": 1,
    "has_next": true
  },
  "user_review": null,
  "reviews": [ ... ]
}

```

En la primera página, si el usuario autenticado ya reseñó la película, su propia reseña se devuelve en `user_review` y se excluye del array comunitario para no mostrarla dos veces. Las estadísticas generales sí contemplan todas las reseñas.

26.5.1 Cuándo y cómo se obtienen las reseñas de una película

La consulta no se realiza al mismo tiempo que los endpoints de TMDb al abrir el detalle. MovieTabs comienza en Reparto. Cuando el usuario abre por primera vez Reseñas, Vue monta `MovieReviews` y `onMounted(loadInitial)` ejecuta `fetchReviews(1)`.

```
MovieReviews.onMounted(loadInitial)
-> getMovieReviews(movieId, 1)
-> GET /movies/{movieId}/reviews?page=1
-> endpoint público (auth=false, csrf=false)
-> ReviewController::getMovieReviews(movieId)
-> page desde Request::query()
-> Session::getId() puede ser null o un id
-> ReviewService::getMovieReviews()
-> MySQL
-> REVIEWS_FETCHED
```

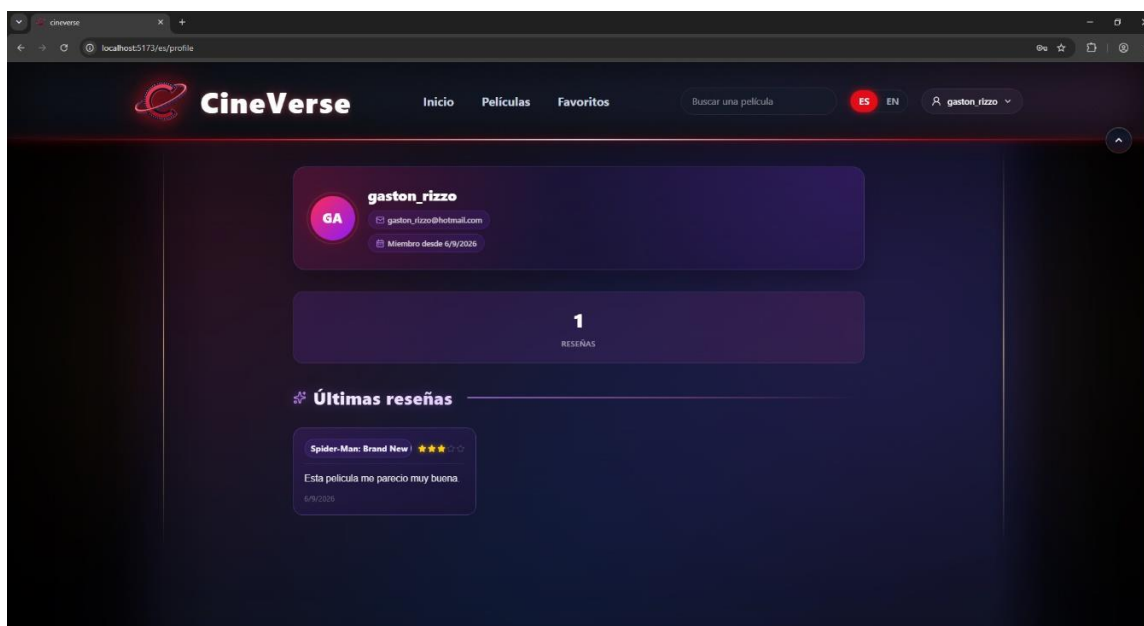
Aunque el endpoint sea público, `backendApi` usa `withCredentials:true`. Si existe una cookie válida, `Session::getId()` permite identificar al usuario. En la primera página `ReviewService` busca su reseña y la devuelve en `user_review`; luego excluye ese `user_id` del listado comunitario para no duplicarla.

Las reseñas comunitarias se obtienen con un `INNER JOIN` entre `reviews` y `users` para añadir `username`. Se ordenan por `created_at` `DESC` y se traen hasta 12 por página. `getMovieReviewStats` ejecuta `COUNT(*)` y `ROUND(AVG(rating),1)`; estas estadísticas incluyen todas las reseñas, incluida la propia. `countCommunityReviews` utiliza el mismo criterio de exclusión que la lista para calcular `has_next`.

26.5.2 Cargar más reseñas

`MovieReviews` mantiene `currentPage` y `hasNext`. Al pulsar Cargar más solicita `currentPage + 1`. Para `page > 1`, `fetchReviews` no reemplaza la lista: concatena `data.reviews` al array existente. `user_review` sólo se actualiza con la página 1 porque el backend únicamente la devuelve allí.

26.6 Perfil



Perfil: cantidad total de reseñas y últimas participaciones.

El perfil muestra la cantidad total de reseñas del usuario y sus seis participaciones más recientes. La constante `LATEST_REVIEWS_LIMIT` vale 6. La pantalla `MyReviews` utiliza `GET /reviews?page=N` para recorrer todas las reseñas propias en páginas de 12.

ProfileController.php: entrada HTTP del perfil

`ProfileController.php` está ubicado en `backend/controllers/ProfileController.php`. El endpoint `/profile` ya llega protegido por `AuthMiddleware`, por lo que el Controller obtiene el `user_id` desde `Session`, delega la construcción del resumen a `ProfileService` y devuelve `PROFILE_FETCHED` mediante `Response`.

```
$userId = Session::getUserId();
$profileSummary = $this->profileService->getProfile($userId);

Response::success(
    "PROFILE_FETCHED",
    $profileSummary
);
```

El fragmento muestra el núcleo real de `getProfile()` sin el manejo de excepciones. El Controller no escribe SQL ni decide cuántas reseñas recientes deben mostrarse: ambas responsabilidades se delegan a las capas siguientes.

ProfileService.php: regla de las seis reseñas recientes

`ProfileService.php` está ubicado en `backend/services/ProfileService.php`. Es deliberadamente pequeño: la regla `LATEST_REVIEWS_LIMIT = 6` pertenece a esta capa porque expresa una decisión de la aplicación, y el Service entrega ese límite al Model `Profile`.

```
private const LATEST_REVIEWS_LIMIT = 6;

public function getProfile(int $userId): array
{
    return Profile::getProfile(
        $this->pdo,
        $userId,
        self::LATEST_REVIEWS_LIMIT
    );
}
```

De esta forma el Model recibe el número de filas que debe consultar, pero no decide por sí solo que la interfaz muestre seis. Si la regla cambiara, el lugar natural para modificarla sería `ProfileService`.

Profile.php: consultas que construyen el resumen

`Profile.php` está ubicado en `backend/models/Profile.php`. Es un Model de lectura: recibe PDO, el id del usuario y el límite decidido por el Service; después ejecuta una consulta para contar todas las reseñas y otra para recuperar las más recientes.

```

$reviewsCountStmt = $pdo->prepare(
    "SELECT COUNT(*) AS total
    FROM reviews
    WHERE user_id = ?"
);
$reviewsCountStmt->execute([$userId]);
$reviewsCount = (int) $reviewsCountStmt->fetchColumn();

$latestReviewsStmt = $pdo->prepare(
    "SELECT
        id,
        movie_id,
        movie_title,
        rating,
        comment,
        created_at
    FROM reviews
    WHERE user_id = ?
    ORDER BY created_at DESC
    LIMIT " . $latestReviewsLimit
);
$latestReviewsStmt->execute([$userId]);
$latestReviews = $latestReviewsStmt->fetchAll(PDO::FETCH_ASSOC);

```

Las dos consultas utilizan el mismo `user_id`. La primera obtiene `reviews_count`; la segunda produce `latest_reviews` ordenadas desde la más nueva. `Profile::getProfile()` devuelve ambos resultados juntos y `ProfileService` los entrega al `Controller` sin que `Vue` tenga que ejecutar esas consultas por separado.

26.6.1 Perfil: resumen de actividad

```

ProfileView.onMounted(loadProfile)
-> GET /profile
-> AuthMiddleware
-> ProfileController::getProfile()
-> Session::getUserId()
-> ProfileService::getProfile()
-> Profile::getProfile()
    SELECT COUNT(*) FROM reviews WHERE user_id=?
    SELECT últimas 6 ORDER BY created_at DESC
-> PROFILE_FETCHED
-> reviewsCount + latestReviews

```

Las tarjetas del perfil usan `movie_id` para volver al detalle de la película. El perfil no consulta TMDb para reconstruir el título: utiliza `movie_title` persistido junto a cada reseña.

26.6.2 Mis reseñas: listado completo paginado

```

MyReviewsView.onMounted(loadReviews)
-> GET /reviews?page=1
-> AuthMiddleware
-> ReviewController::getUserReviews()
-> user_id desde Session
-> ReviewService::getUserReviews()
    offset = (page - 1) * 12
    SELECT ... WHERE user_id=? ORDER BY created_at DESC
    COUNT(*) total
    calcular has_next
-> REVIEWS_FETCHED
-> página 1 reemplaza el array
-> páginas siguientes se concatenan

```

Si estas pantallas reciben `AUTH_REQUIRED`, limpian el store de autenticación y redirigen a Login. Esto cubre el caso en que el frontend todavía creía tener usuario pero la sesión del servidor ya expiró.

26.7 Caso límite actual de page

`Validator::int()` comprueba que `page` sea entero, pero no exige explícitamente `page >= 1`. `ReviewService` calcula `offset = (page - 1) * 12`. Por eso una petición manual `page=0` puede producir un `offset` negativo. La interfaz normal

siempre comienza en 1, pero el backend debería validar el rango para endurecer el endpoint frente a llamadas manuales.

27. Validación y seguridad

CineVerse no confía en una sola comprobación. El frontend ayuda a evitar errores antes de enviar un formulario; el backend valida lo que realmente llega por HTTP; PDO utiliza consultas preparadas; y MySQL mantiene restricciones sobre los datos almacenados. Cada nivel cubre problemas diferentes.

```
DESARROLLO
TypeScript + vue-tsc
  → detectan errores de tipos antes de ejecutar

DURANTE LA EJECUCIÓN
Usuario
↓
Formulario Vue
↓
Solicitud HTTP
↓
Validator / AuthValidator / ReviewValidator
↓
Service: reglas de CineVerse
↓
Consultas preparadas con PDO
↓
MySQL: UNIQUE / FOREIGN KEY / CHECK
```

27.1 Protección de contraseñas

El backend nunca guarda en users la contraseña escrita por el usuario. `password_hash()` genera un hash y `password_verify()` comprueba posteriormente si una contraseña corresponde a ese hash.

`PASSWORD_DEFAULT` permite que PHP utilice el algoritmo recomendado por la versión instalada.

27.2 Inyección SQL

Una inyección SQL ocurre cuando texto controlado por un usuario llega a formar parte ejecutable de una consulta. Los modelos de CineVerse utilizan consultas preparadas de PDO: la estructura SQL se escribe por un lado y los valores se envían como parámetros. `LIMIT` y `OFFSET` se incorporan como números ya definidos o convertidos a entero, no como texto arbitrario recibido del usuario.

27.3 Sesión, CSRF y protección frente a la enumeración de cuentas

La enumeración de cuentas consiste en descubrir qué emails están registrados observando diferencias en las respuestas. Por eso los flujos de reenvío de verificación y recuperación de contraseña utilizan mensajes neutros cuando el email no existe.

- `HttpOnly` impide que JavaScript lea directamente la cookie de sesión.
- `SameSite=Lax` reduce determinadas solicitudes cross-site automáticas.
- `CSRF` añade una segunda prueba para operaciones que modifican datos.
- La regeneración del id reduce el riesgo de session fixation.
- Reenvío de verificación y forgot password mantienen respuestas neutras cuando el email no existe.

27.4 CORS

CORS es una política del navegador que controla qué origen web puede llamar a una API cuando frontend y backend están en direcciones diferentes. En desarrollo, `CorsMiddleware` permite `http://localhost:5173`,

credenciales, los headers Content-Type y X-CSRF-Token y los métodos utilizados por CineVerse. Las solicitudes OPTIONS de comprobación previa responden 204.

27.5 Mejoras de seguridad posibles

- Almacenar hash de tokens de verificación/reset en lugar de los tokens utilizables en claro.
- Aplicar rate limiting también a registro, reenvío y recuperación de contraseña si se publica en Internet.
- Revisar la detección de HTTPS cuando delante de PHP existe un servidor intermedio (reverse proxy) antes de confiar sólo en `$_SERVER["HTTPS"]`.
- Evitar permisos 0777 al crear la carpeta de logs en un entorno productivo.

27.6 Cómo se valida un POST /reviews en varias capas

1. Router
 - > exige auth y csrf según routes/api.php
2. AuthMiddleware
 - > comprueba `Session::has("user_id")`
3. CsrfMiddleware
 - > lee `HTTP_X_CSRF_TOKEN`
 - > `hash_equals(token de sesión, token recibido)`
4. `Request::json()`
 - > lee `php://input`
 - > `json_decode(..., true)`
5. Validator
 - > `int()` / `stringTrim()`
6. ReviewValidator
 - > `movie_id > 0`
 - > `movie_title obligatorio <= 255`
 - > `rating 1..5`
 - > `comment 20..1000` y normalizado
7. ReviewService
 - > una reseña por usuario/película
 - > propiedad al editar/borrar
8. PDO + MySQL
 - > consultas preparadas
 - > `UNIQUE` / `FOREIGN KEY` / `CHECK`

Cada nivel protege algo distinto. La validación del formulario mejora la experiencia del usuario, pero no es una frontera de seguridad porque una persona puede llamar la API sin usar Vue. Por eso el backend repite las comprobaciones y MySQL mantiene restricciones finales.

27.7 Por qué la contraseña no se normaliza igual que un email

`Validator::stringTrim()` se usa en datos como email, username o comentario antes de otras normalizaciones. La contraseña se lee con `Validator::string()` sin `trim()`, porque un espacio puede formar parte real del password. Después `AuthValidator` aplica las reglas de longitud y composición y `password_hash/password_verify` se ocupan del almacenamiento y comparación.

27.8 Consultas preparadas y valores numéricos de paginación

Los valores controlados por el usuario se envían como parámetros PDO, por ejemplo `:user_id`, `:movie_id` o `:comment`. `LIMIT` y `OFFSET` aparecen interpolados en algunas consultas, pero proceden de constantes del backend y de cálculos sobre un page convertido a entero; no se incorpora texto arbitrario del usuario a la instrucción SQL. Aun así, validar `page >= 1` eliminaría el caso límite de offset negativo descrito en la sección 26.

28. Estados de la interfaz y manejo de errores

CineVerse no muestra una pantalla diferente sólo cuando ocurre un error. La misma SPA también representa estados de carga, resultados vacíos, validaciones esperadas y rutas inexistentes. Para entender por qué aparece cada mensaje hay que seguir el estado desde la capa que lo produce hasta el componente que finalmente lo renderiza. Esta sección utiliza los archivos reales del frontend y del backend y, en cada ejemplo, identifica qué archivo participa, qué decisión toma y cómo esa decisión termina convertida en texto visible.

28.1 Carga, estado vacío, error esperado y fallo técnico

Antes de seguir el código conviene separar situaciones que visualmente pueden parecer similares, pero técnicamente significan cosas distintas.

Tipo de estado	Qué significa	Ejemplo real en CineVerse
Carga	La operación todavía no terminó.	“Cargando película”.
Estado vacío	La operación terminó correctamente, pero no hay elementos útiles para mostrar.	“No encontramos películas”, favoritos vacíos o perfil sin reseñas.
Error esperado	El sistema conoce la causa y puede expresarla con un código estable.	Credenciales incorrectas, rating inválido o CSRF inválido.
Fallo técnico	Falló una dependencia o ocurrió una excepción no esperada.	MySQL no disponible, SMTP o un fallo de TMDB.
Navegación inexistente	La URL no coincide con una vista válida de la SPA.	Pantalla 404 de Vue Router.

La diferencia es importante para diagnosticar el sistema. “No encontramos películas” puede aparecer después de una respuesta correcta de TMDB; un perfil con cero reseñas también es una respuesta válida; y una pantalla 404 de la SPA puede resolverse sin que PHP reciba ninguna petición. En cambio, una caída de MySQL o una excepción inesperada sí pertenecen a la categoría de fallo técnico.

28.2 Cómo leer los ejemplos: de la causa al texto visible

En Vue no suele existir un archivo HTML diferente para cada estado. Una ref, una computed, un store de Pinia, un estado de TanStack Vue Query o una respuesta HTTP cambia; Vue vuelve a evaluar el template; y v-if / v-else-if decide qué bloque queda en el DOM. Cuando el texto está internacionalizado, vue-i18n transforma una clave en el mensaje del idioma activo.

```

CAUSA
↓
archivo que detecta o produce la situación
↓
estado local / store / Vue Query / respuesta HTTP
↓
archivo que consume ese estado
↓
v-if / v-else-if decide qué bloque se renderiza
↓
vue-i18n obtiene el texto del idioma actual
↓
MENSAJE VISIBLE

```

Los ejemplos siguientes repiten este patrón. Los fragmentos de código llevan su ruta completa y, cuando corresponde, el método o bloque concreto. Así se puede ubicar cada pieza dentro del proyecto y no leer el código como fragmentos aislados.

28.3 Ejemplo 1: una URL inexistente termina en la pantalla 404

La pantalla 404 de la SPA se resuelve completamente en el navegador cuando la URL no coincide con ninguna ruta válida. PHP, MySQL y TMDb no intervienen en este recorrido.

Archivo: frontend/src/core/router/index.ts **Bloque:** rutas catch-all del final del router

```
{
  path: "/:lang(en|es)/:pathMatch(.*)"*,
  name: "NotFound",
  component: NotFoundView,
},
{
  path: "/:pathMatch(.*)"*,
  name: "NotFoundNoLang",
  component: NotFoundView,
}
```

El primer catch-all conserva un idioma válido al comienzo de la URL; el segundo captura cualquier otra dirección que tampoco haya coincidido con las rutas anteriores. Como ambas reglas están al final, Vue Router sólo llega a ellas después de intentar las rutas válidas.

Archivo: frontend/src/views/NotFoundView.vue **Bloque:** template de la vista 404

```
<h1 class="not-found-code">404</h1>
<h2 class="not-found-title">
  {{ t("notFound.title") }}
</h2>
<p class="not-found-sub">
  {{ t("notFound.message") }}
</p>
<RouterLink :to="`/${locale}`">
  {{ t("notFound.goHome") }}
</RouterLink>
```

```
/es/ruta-inexistente
↓
Vue Router compara todas las rutas válidas
↓
ninguna coincide
↓
catch-all → NotFoundView.vue
↓
t("notFound.title") + t("notFound.message")
↓
404 visible en pantalla
```

El router decide qué componente montar; NotFoundView decide qué se ve. Esa separación explica por qué un 404 de la SPA no necesita una respuesta HTTP 404 del backend.

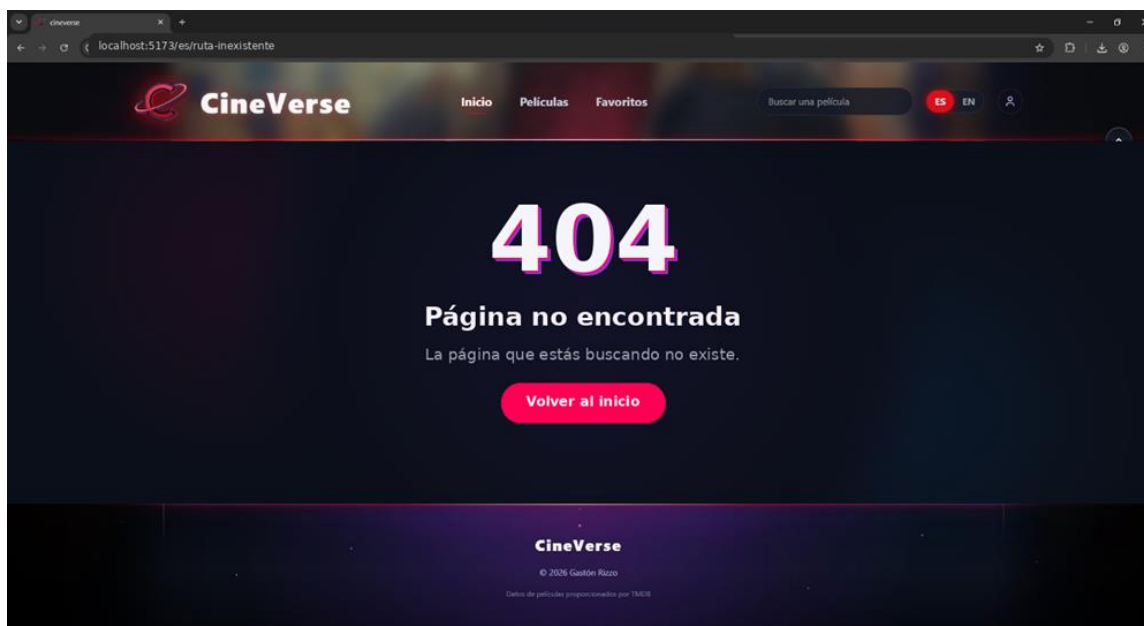


Figura. Pantalla 404 de la SPA. Vue Router no encontró una ruta válida y montó `NotFoundView` dentro de `RouterView`; el backend PHP no participa en este recorrido.

Hay una diferencia importante entre una ruta inválida y un recurso remoto inexistente. `/es/movies/abc` no cumple el patrón numérico del parámetro `id` y termina en el 404 del router. En cambio, `/es/movies/99999999` sí coincide con la ruta de detalle; `MovieDetailView` se monta y la consulta posterior a TMDB puede fallar. Ambos casos comunican “no encontrado”, pero nacen en capas distintas.

28.4 Ejemplo 2: por qué aparece “Cargando película”

El detalle de película no retira el overlay sólo porque haya terminado la petición. `MovieDetailView` combina dos condiciones: la consulta de TanStack Vue Query debe haber dejado de cargar y el componente `MovieHero` debe haber avisado que su póster ya puede mostrarse. Por eso el estado depende de varios archivos.

Archivo: `frontend/src/features/movies/views/MovieDetailView.vue` **Bloque:** estado de carga y handler `onHeroReady()`

```
const { data, isLoading, isError } = useMovieDetail(id, locale);
const heroReady = ref(false);

function onHeroReady() {
  heroReady.value = true;
}

<MovieHero
  v-if="movie"
  ...
  @ready="onHeroReady"
/>

<div v-if="isLoading || !heroReady" class="loading-overlay">
  ...
  <h1>{{ t('movieDetail.loadingTitle') }}</h1>
  <p>{{ t('movieDetail.loadingSubtitle') }}</p>
</div>
```

`isLoading` viene del composable `useMovieDetail()`. `heroReady`, en cambio, es una ref propia de la vista. La expresión `isLoading || !heroReady` significa: mantener el loader mientras todavía se obtienen datos o mientras `MovieHero` aún no haya emitido `ready`.

Archivo: `frontend/src/features/movies/composables/movie-detail/useMovieDetail.ts` **Bloque:** `useQuery()` que obtiene el detalle completo

```

return useQuery<MovieDetailResult>({
  queryKey: ["movie-detail", id, language],
  retry: false,
  queryFn: async () => {
    const movieData = await getMovieDetail(id, language.value);
    if (!movieData) throw new Error("Movie not found");
    const [creditsData, videosData, similarData] = await Promise.all([
      getMovieCredits(id, language.value),
      getMovieVideos(id, language.value),
      getSimilarMovies(id, 1),
    ]);
    ...
  },
});

```

Mientras esta query está pendiente, Vue Query mantiene `isLoading`. Cuando termina, ese estado deja de ser suficiente por sí solo: la vista sigue esperando el evento del hero.

Archivo: `frontend/src/features/movies/components/movie-detail/MovieHero.vue` **Bloque:** `onPosterLoad()` y `checkReady()`

```

function onPosterLoad() {
  posterLoaded.value = true;
  checkReady();
}

function checkReady() {
  if (posterLoaded.value) {
    setTimeout(() => {
      isReady.value = true;
      intro.value = true;
      emit("ready");
    }, 150);
  }
}

```

La implementación actual es concreta: `MovieHero` espera la carga del póster y luego aplica un delay de 150 ms antes de emitir `ready`. Si la película no tiene `poster_path`, el watcher considera el póster listo para no bloquear la interfaz. No existe una comprobación genérica de “todas las imágenes y el layout”; la condición real es el póster más ese pequeño retraso.

```

MovieDetailView monta
↓
useMovieDetail() → Vue Query → isLoading = true
↓
isLoading || !heroReady = true → aparece “Cargando película”
↓
terminan las consultas de TMDB → isLoading = false
↓
MovieHero carga el póster → espera 150 ms → emit("ready")
↓
onHeroReady() → heroReady = true
↓
false || false = false → el overlay desaparece

```

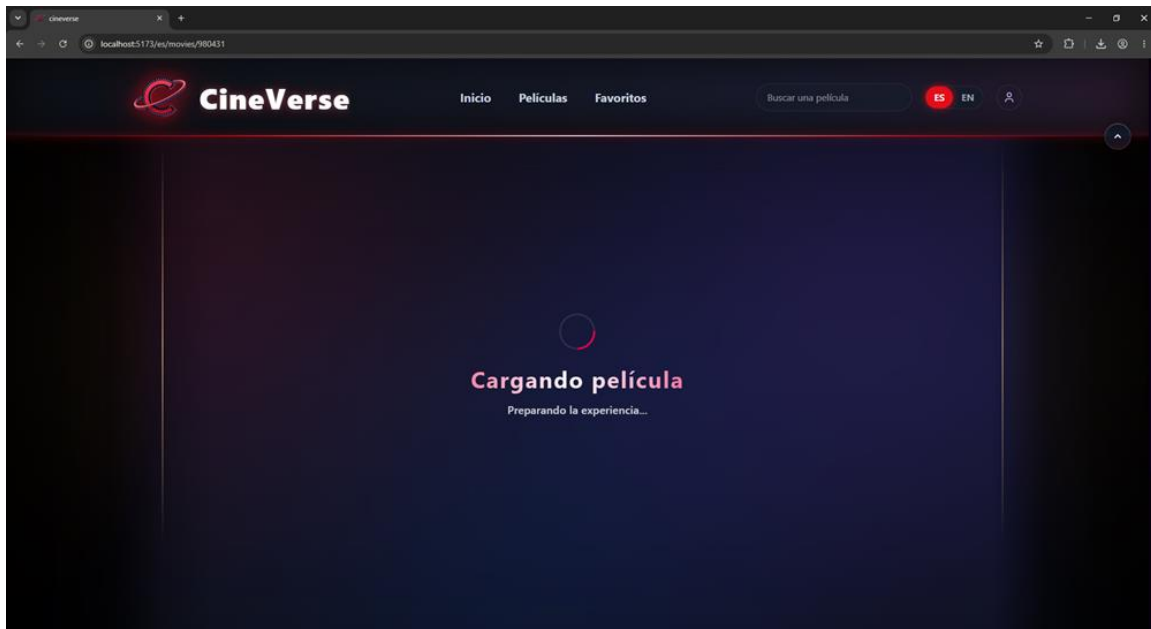


Figura. Estado de carga del detalle de película. El overlay se mantiene mientras la query siga cargando o MovieHero todavía no haya emitido ready después de cargar el póster.

28.5 Ejemplo 3: “No encontramos películas” es un estado vacío, no necesariamente un error

Este caso enseña la diferencia entre “la consulta falló” y “la consulta terminó, pero no quedó nada útil para mostrar”. MoviesView obtiene páginas mediante useMoviesQuery; MovieResults recibe esas páginas y recién después de procesarlas decide si muestra resultados, error o estado vacío.

Archivo: frontend/src/features/movies/composables/useMoviesQuery.ts **Bloque:** useInfiniteQuery() del explorador

useMoviesQuery utiliza TanStack Vue Query para producir data.pages, isLoading, error, hasNextPage e isFetchingNextPage. Según la búsqueda y los filtros, su queryFn llama a las funciones de movies.service.ts, que realizan las consultas a TMDB.

Archivo: frontend/src/features/movies/components/MovieResults.vue **Bloque:** watch de props.data

```
hasProcessedData.value = false;

const pages = data?.pages;
if (!pages || pages.length === 0) return;

const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);

hasProcessedData.value = true;
```

MovieResults aplanar todas las páginas y elimina películas cuyo poster_path no es utilizable. hasProcessedData evita que el estado vacío se muestre mientras la lista todavía está siendo recalculada.

Archivo: frontend/src/features/movies/components/MovieResults.vue **Bloque:** carga/error y contenedor de resultados/estado vacío

```
<div v-if="isLoading" class="cinema-loading">...</div>
<div v-else-if="error" class="empty-state">...</div>

<div name="movie">
  <div v-if="validMovies.length">...</div>
  <div v-else-if="hasProcessedData && validMovies.length === 0"
    class="empty-state">
```

```

    {{ t("common.emptyResultsTitle") }}
  </div>
</div>

```

```

MoviesView / filtros
↓
useMoviesQuery.ts → useInfiniteQuery()
↓
movies.service.ts → TMDB
↓
data.pages
↓
MovieResults.vue → flatMap + filtro de poster_path
↓
hasProcessedData = true
↓
validMovies.length === 0
↓
“No encontramos películas”

```

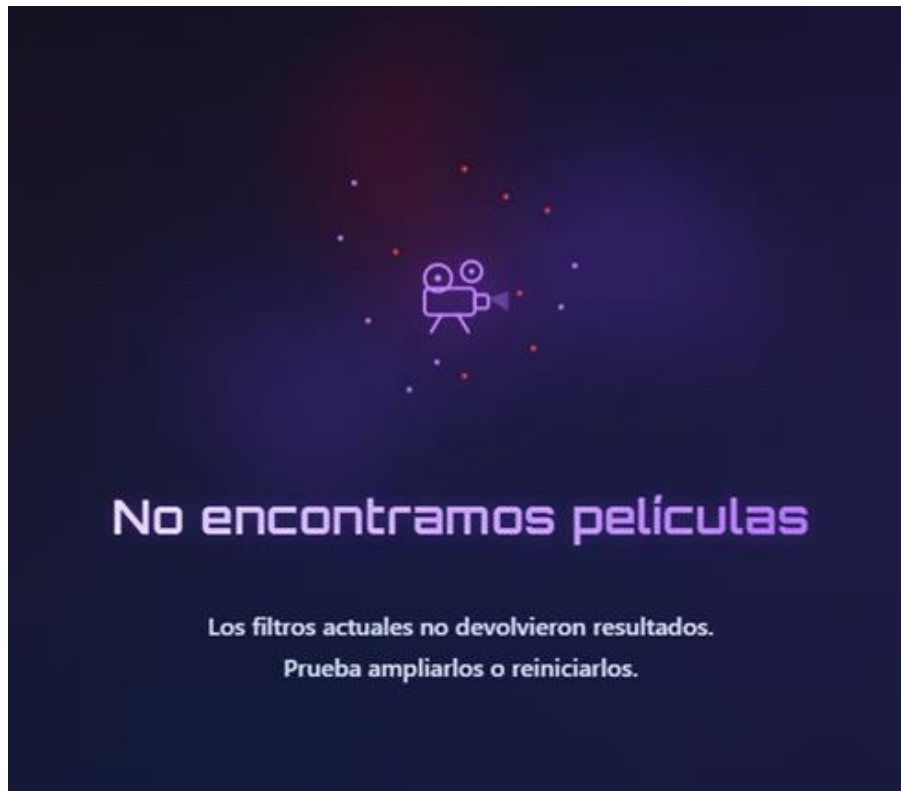


Figura. Estado vacío del explorador. La consulta ya fue procesada, pero la lista final de películas utilizables quedó vacía; por eso Vue muestra “No encontramos películas”.

Qué no significa este estado: no demuestra que TMDB esté caído. Si `props.error` existe, `MovieResults` usa las claves de error general. El mensaje de “No encontramos películas” pertenece al camino en el que no hay error y la colección procesada queda vacía.

28.6 Ejemplo 4: favoritos vacíos sin ninguna petición HTTP

Favoritos es el ejemplo más simple para comprobar que una pantalla vacía no necesita servidor. El estado nace en Pinia, se persiste en `localStorage` y `MoviesFavoritesView` lo consume directamente.

Archivo: `frontend/src/main.ts` **Bloque:** registro de `pinia-plugin-persistedstate`

```

const pinia = createPinia();
pinia.use(piniaPluginPersistedState);

const app = createApp(App);
app.use(pinia);

```

Archivo: frontend/src/features/movies/stores/useMoviesFavoritesStore.ts **Bloque:** store favorites y persistencia

```
const favorites = ref<Movie[]>([]);

return {
  favorites,
  isFavorite,
  addFavorite,
  removeFavorite,
  clearFavorites,
  favoritesCount
};

{
  persist: true,
}
```

Archivo: frontend/src/features/movies/views/MoviesFavoritesView.vue **Bloque:** estado empty-hero

```
<div v-if="favoritesStore.favorites.length === 0" class="empty-hero">
  <h1>{{ t('favorites.emptyTitle') }}</h1>
  <p>{{ t('favorites.emptySubtitle') }}</p>
</div>
```

```
localStorage
↓
pinia-plugin-persistedstate restaura el store
↓
useMoviesFavoritesStore.ts → favorites = []
↓
MoviesFavoritesView.vue
↓
favorites.length === 0
↓
Vue muestra "Aún no tienes favoritos"
```

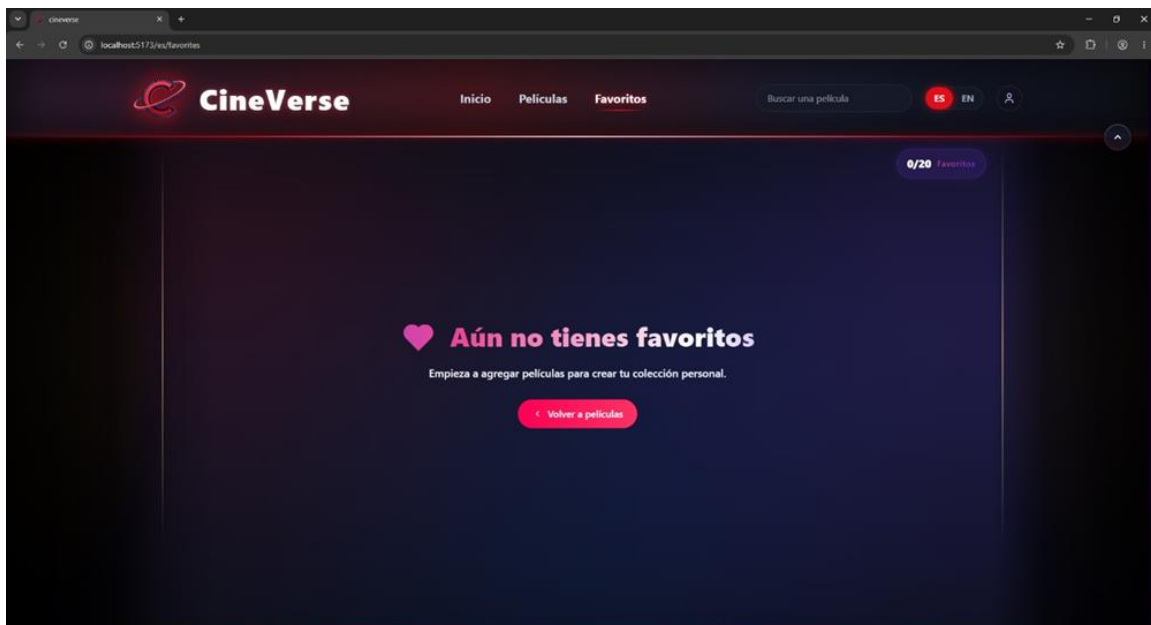


Figura. Favoritos vacíos. El estado nace en Pinia/localStorage y se resuelve completamente en el navegador; no se ejecutan Axios, PHP, MySQL ni TMDB para mostrar este mensaje.

28.7 Ejemplo 5: perfil válido con cero reseñas

El perfil contrasta con Favoritos porque aquí sí participa toda la cadena frontend → backend → MySQL. Una cuenta sin reseñas no genera un error: el backend responde correctamente con `reviews_count = 0` y `latest_reviews = []`. Vue interpreta ese array vacío y muestra el mensaje correspondiente.

Archivo: frontend/src/features/profile/services/profile.service.ts **Bloque:** `getProfile()`

```
export const getProfile = async (): Promise<ProfileResponse> => {
  const { data } = await backendApi.get<ProfileResponse>("/profile");
  return data;
};
```

Archivo: backend/routes/api.php **Bloque:** GET /profile

```
"/profile" => [
  "handler" => [ProfileController::class, "getProfile"],
  "auth" => true,
  "csrf" => false
]
```

La ruta exige sesión válida antes de llegar al Controller. ProfileController obtiene el user_id de la sesión y llama a ProfileService::getProfile(); el service fija que el perfil muestre como máximo las últimas seis reseñas y delega las consultas al modelo.

Archivo: backend/models/Profile.php **Bloque:** Profile::getProfile()

```
$reviewsCountStmt = $pdo->prepare(
  "SELECT COUNT(*) AS total FROM reviews WHERE user_id = ?"
);
...
$latestReviewsStmt = $pdo->prepare(
  "SELECT id, movie_id, movie_title, rating, comment, created_at
  FROM reviews
  WHERE user_id = ?
  ORDER BY created_at DESC
  LIMIT " . $latestReviewsLimit
);

return [
  "reviews_count" => $reviewsCount,
  "latest_reviews" => $latestReviews
];
```

Archivo: frontend/src/features/profile/views/ProfileView.vue **Bloque:** loadProfile() y estado sin reseñas

```
const response = await getProfile();

reviewsCount.value = response.data.reviews_count;
latestReviews.value = response.data.latest_reviews;

<div v-if="latestReviews.length === 0" class="no-reviews">
  <p>{{ t("profile.noReviewsYet") }}</p>
</div>
```

```
ProfileView.vue
  ↓ getProfile()
profile.service.ts → backendApi.get("/profile")
  ↓
routes/api.php → ProfileController::getProfile()
  ↓
ProfileService::getProfile()
  ↓
Profile::getProfile() → MySQL
  ↓
reviews_count = 0 / latest_reviews = []
  ↓ respuesta JSON
ProfileView → latestReviews.length === 0
  ↓
"“Todavía no has escrito ninguna reseña.”"
```

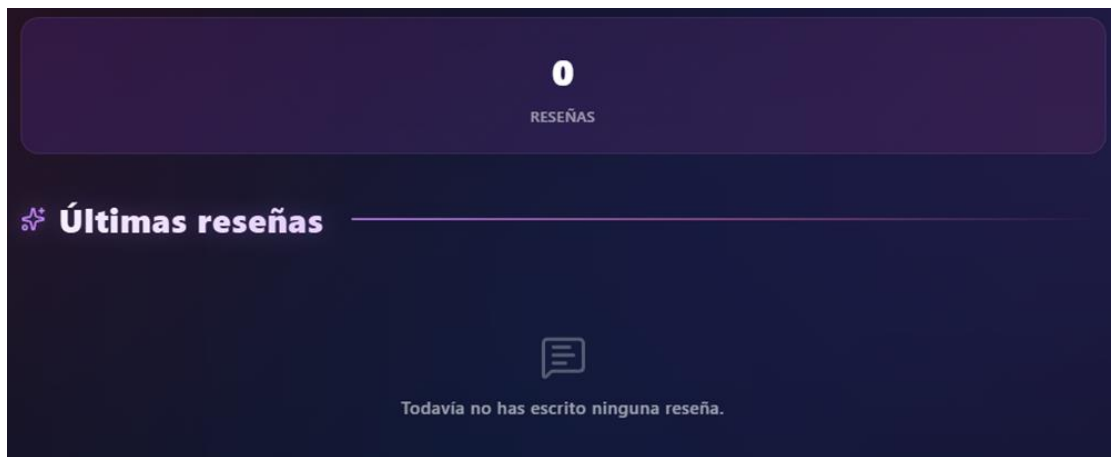


Figura. Perfil sin reseñas. El backend respondió correctamente; `latest_reviews` llegó como array vacío y la condición del template activó el mensaje de estado vacío.

28.8 Ejemplo 6: credenciales incorrectas desde el formulario hasta el banner de Login

El login permite seguir un error controlado de punta a punta. La vista valida primero lo que puede comprobar sin servidor; si el formulario supera esa validación, el frontend envía `POST /login`. La decisión sobre si la contraseña coincide sólo puede tomarse en el backend, porque allí está el `password_hash` de MySQL.

Archivo: `frontend/src/features/auth/services/auth.service.ts` **Bloque:** `loginUser()`

```
export const loginUser = async (payload: LoginPayload): Promise<LoginResponse> => {
  const { data } = await backendApi.post<LoginResponse>("/login", payload);
  return data;
};
```

Archivo: `backend/services/AuthService.php` **Bloque:** `AuthService::login()`

```
$user = User::findByEmail($this->pdo, $email);

if (!$user || !password_verify($password, $user["password_hash"])) {
  ... // crea o incrementa login_attempts
  throw new RuntimeException("INVALID_CREDENTIALS");
}
```

`AuthService` también registra el intento fallido por combinación IP + email. La excepción no se envía cruda al navegador: `AuthController` la traduce a la respuesta pública estándar de la API.

Archivo: `backend/controllers/AuthController.php` **Bloque:** `catch de AuthController::login()`

```
if ($e->getMessage() === "INVALID_CREDENTIALS") {
  Response::error(
    $e->getMessage(),
    401
  );
  return;
}
```

Archivo: `frontend/src/features/auth/views/LoginView.vue` **Bloque:** `catch de login()`

```
const code = error.response?.data?.code as string;

switch (code) {
  case "INVALID_CREDENTIALS":
    formError.value = "invalidCredentials";
    break;
  ...
}

<div v-if="formError" class="form-error-banner">
  {{ t(`auth.errors.${formError}`) }}
</div>
```

La clave final se encuentra en frontend/src/core/i18n/es.json: `auth.errors.invalidCredentials = "Usuario o contraseña incorrectos."`. Por eso el backend comunica una causa estable y el frontend decide cómo presentarla.

```

LoginView valida el formulario
↓
auth.service.ts → POST /login
↓
AuthController → AuthService → User::findByEmail() / password_verify()
↓
INVALID_CREDENTIALS
↓
HTTP 401 + JSON { success:false, code:"INVALID_CREDENTIALS" }
↓
Axios rechaza la Promise → catch de LoginView
↓
formError = "invalidCredentials"
↓
vue-i18n
↓
"Usuario o contraseña incorrectos."

```

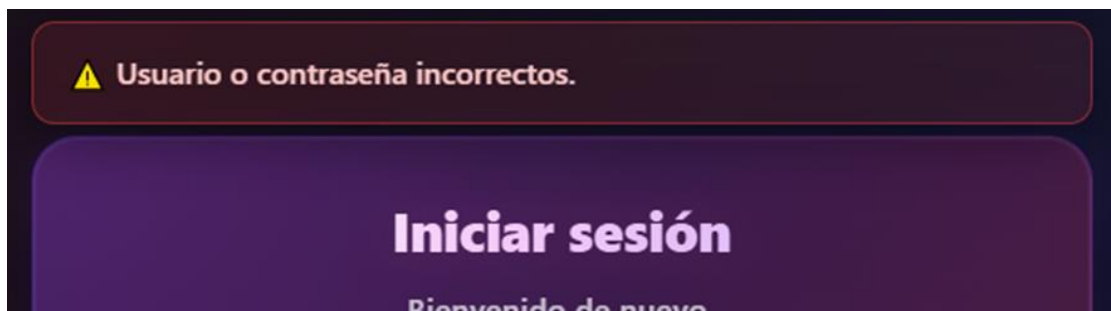


Figura. Banner de credenciales incorrectas. El texto visible es el último paso de una cadena que nace en la comprobación de AuthService, pasa por HTTP 401 y termina en formError dentro de LoginView.

Este ejemplo también separa validación frontend y autenticación backend. Un campo vacío o un formato inválido puede detectarse antes de enviar la petición; comprobar una contraseña requiere consultar el usuario almacenado en el servidor.

28.9 Ejemplo 7: por qué aparece “Enlace inválido” al restablecer la contraseña

ResetPasswordView distingue tres estados: checking, valid e invalid. El caso es especialmente útil porque existen dos caminos para llegar a invalid: uno completamente local, cuando la URL no trae token, y otro remoto, cuando el token existe pero la validación contra el backend falla.

Archivo: frontend/src/features/auth/views/ResetPasswordView.vue **Bloque:** onMounted() que valida el token

```

const tokenStatus = ref<"checking" | "valid" | "invalid">("checking");

onMounted(async () => {
  if (!token) {
    tokenStatus.value = "invalid";
    return;
  }

  try {
    await validateResetToken(token);
    tokenStatus.value = "valid";
  } catch {
    tokenStatus.value = "invalid";
  }
});

```

Si no existe token, el retorno ocurre antes de cualquier HTTP. Si sí existe, la función `validateResetToken()` del service envía GET `/reset-password/validate`.

Archivo: frontend/src/features/auth/services/auth.service.ts **Bloque:** `validateResetToken()`

```
export const validateResetToken = async (token: string) => {
  const { data } = await backendApi.get("/reset-password/validate", {
    params: { token }
  });
  return data;
};
```

backend/routes/api.php deriva ese GET a AuthController::validateResetToken(). El Controller normaliza el parámetro y llama al servicio. AuthService no duplica la validación: delega en un método privado compartido también por el cambio efectivo de contraseña.

Archivo: backend/services/AuthService.php **Bloque:** getValidPasswordResetUser()

```
$user = User::findByPasswordResetToken($this->pdo, $token);

if (!$user) {
  throw new RuntimeException("INVALID_PASSWORD_RESET_TOKEN");
}

if (strtotime($user["password_reset_expires_at"]) < time()) {
  throw new RuntimeException("PASSWORD_RESET_TOKEN_EXPIRED");
}
```

Archivo: frontend/src/features/auth/views/ResetPasswordView.vue **Bloque:** selección del template según tokenStatus

```
<template v-if="tokenStatus === 'checking'"> ... </template>
<template v-else-if="tokenStatus === 'valid'"> ... </template>
<template v-else>
  <h1>{{ t("auth.resetPassword.invalidTitle") }}</h1>
  <p>{{ t("auth.resetPassword.invalidMessage") }}</p>
</template>
```

CASO A: falta ?token=...

ResetPasswordView → tokenStatus = "invalid" → no hay HTTP

CASO B: hay token

ResetPasswordView

↓ validateResetToken()

auth.service.ts → GET /reset-password/validate

↓

AuthController → AuthService → getValidPasswordResetUser()

↓

token inexistente / expirado → respuesta de error

↓

Axios rechaza → catch de onMounted

↓

tokenStatus = "invalid"

↓

"Enlace inválido"

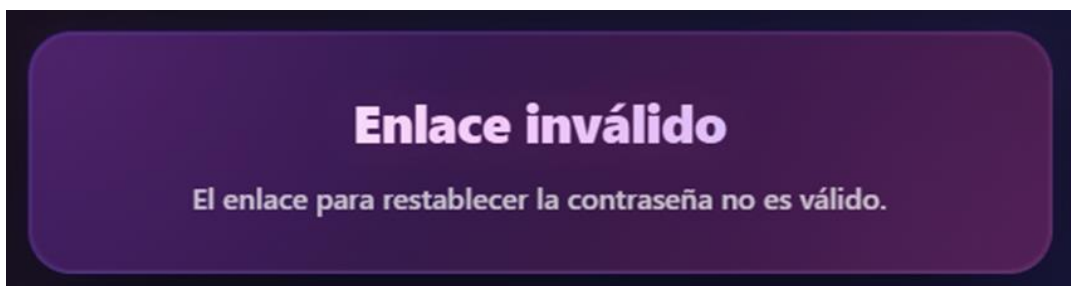


Figura. Estado "Enlace inválido" de ResetPasswordView. Puede aparecer sin llamar al backend si falta el token, o después de una validación remota rechazada.

Limitación actual. El catch del onMounted no inspecciona error.response.data.code: cualquier rechazo de validateResetToken(), incluso una caída de infraestructura durante esa comprobación, termina en tokenStatus = "invalid". Por eso la pantalla no distingue visualmente entre token inválido y determinados fallos técnicos.

28.10 Tres significados distintos de “404” o “no encontrado”

CineVerse puede encontrarse con la idea de “no encontrado” en capas diferentes. Distinguir las evita atribuir el problema al sistema equivocado.

Origen	Ejemplo	Qué ocurre
Vue Router	/es/ruta-inexistente	No coincide ninguna ruta de la SPA y se monta NotFoundView. No se consulta el backend.
Backend PHP	GET a un endpoint inexistente	El Router PHP puede devolver HTTP 404 con ROUTE_NOT_FOUND.
TMDB	/movie/{id} inexistente	La consulta remota falla; Vue Query marca isError y la vista de detalle muestra su estado de error.

El número HTTP o la frase “no encontrado” no explican por sí solos qué pantalla aparecerá. Primero hay que identificar qué capa produjo el estado y qué componente del frontend consume ese resultado.

28.11 Cómo el backend expresa errores controlados

Cuando el problema nace en la API propia, CineVerse combina dos niveles: el status HTTP describe la categoría general y el campo code identifica la causa estable que el frontend puede interpretar. La construcción central está en backend/core/http/Response.php.

Archivo: backend/core/http/Response.php **Bloque:** Response::error()

```
public static function error(string $code, int $statusCode): void {
    self::json(
        $statusCode,
        false,
        $code
    );
}
```

Situación	HTTP	code
JSON inválido	400	INVALID_JSON_BODY
Sin autenticación	401	AUTH_REQUIRED
Credenciales incorrectas	401	INVALID_CREDENTIALS
CSRF inválido	403	INVALID_CSRF_TOKEN
Recurso inexistente	404	REVIEW_NOT_FOUND / ROUTE_NOT_FOUND
Reseña duplicada	409	REVIEW_ALREADY_EXISTS
Validación	422	INVALID_RATING, COMMENT_TOO_SHORT, etc.
Rate limit de login	429	LOGIN_TEMPORARILY_BLOCKED
Error inesperado	500	INTERNAL_SERVER_ERROR
MySQL no disponible	503	DATABASE_UNAVAILABLE

Ejemplo de validación esperada: un rating igual a 6 no significa que el servidor esté averiado. ReviewValidator rechaza el dato y el backend responde con una causa conocida.

```
POST /reviews
{
```

```

"movie_id": 550,
"movie_title": "Fight Club",
"rating": 6,
"comment": "Comentario con longitud suficiente."
}

→ HTTP 422
→ { "success": false, "code": "INVALID_RATING" }

```

CSRF es otro corte temprano: si una escritura protegida no contiene un X-CSRF-Token válido, `CsrfMiddleware` responde HTTP 403 con `INVALID_CSRF_TOKEN` antes de que se ejecute el Controller de la operación.

28.12 Fallos técnicos, dependencias externas y clasificación imperfecta

Un fallo técnico sigue otro recorrido. Si MySQL no está disponible durante el arranque, `backend/public/index.php` captura la `PDOException` antes de despachar la ruta, `AppLogger` conserva el detalle técnico y `Response::error()` devuelve al navegador un código seguro.

Archivo: `backend/public/index.php` **Bloque:** conexión inicial con `Database::connect()`

```

try {
    $pdo = Database::connect();
} catch (PDOException $exception) {
    AppLogger::error($exception->getMessage(), ...);
    Response::error("DATABASE_UNAVAILABLE", 503);
    exit;
}

Detalle técnico que puede quedar en el log:
SQLSTATE[HY000] [1049] Unknown database 'cineverse2'

Respuesta pública:
HTTP 503
{ "success": false, "code": "DATABASE_UNAVAILABLE" }

```

TMDB también es una dependencia externa y puede fallar aunque PHP y MySQL sigan funcionando. En la implementación actual `MovieDetailView` y `PersonDetailView` utilizan estados generales de error de Vue Query y no distinguen visualmente todas las variantes posibles de un fallo remoto (por ejemplo 404, 429 o 5xx).

El correo presenta otra clasificación imperfecta: `EMAIL_DELIVERY_FAILED` se traduce de forma específica en `RegisterView`, mientras otros flujos pueden caer en `unknownError`. Del mismo modo, como se vio en 28.9, la validación inicial del reset puede convertir un fallo de infraestructura en `tokenStatus = "invalid"`. El documento describe el comportamiento actual, no un comportamiento idealizado.

28.13 Qué se registra y qué no

La presencia de un mensaje negativo en pantalla no implica que deba escribirse un log de nivel `ERROR`. Los estados normales y las validaciones previstas forman parte del funcionamiento esperado; los logs técnicos se reservan para fallos que requieren diagnóstico.

Situación	¿LogERROR?	Motivo
Búsqueda sin resultados	No	Estado vacío válido.
Favoritos vacíos	No	Estado local normal.
Perfil con 0 reseñas	No	Respuesta correcta de la API.
Credenciales incorrectas	Normalmente no	Resultado esperado de autenticación.
Rating inválido	No	Validación controlada.
MySQL no disponible	Sí	Fallo técnico de infraestructura.

Situación	¿LogERROR?	Motivo
Excepción SMTP inesperada	Sí	Fallo técnico que requiere diagnóstico.

La implementación interna de AppLogger y Monolog -ruta, RotatingFileHandler, LineFormatter y rotación diaria- se desarrolla en la sección 29. Aquí importa la separación entre el estado seguro que recibe el usuario y el detalle técnico que puede quedar registrado.

28.14 Mapa final: de la causa a la pantalla

La tabla resume los ejemplos anteriores y permite reconocer rápidamente qué capa origina cada estado y qué decisión termina haciéndolo visible.

Pantalla / estado	Origen principal	¿HTTP?	Condición o decisión final
404 de la SPA	Vue Router	No	catch-all → NotFoundView
Cargando película	Vue Query + MovieHero	Sí, TMDB en curso	isLoading !heroReady
No encontramos películas	MovieResults	Sí, TMDB	hasProcessedData && validMovies.length === 0
Sin favoritos	Pinia + localStorage	No	favorites.length === 0
Perfil sin reseñas	PHP/MySQL + Vue	Sí	latestReviews.length === 0
Credenciales incorrectas	PHP/MySQL + LoginView	Sí	INVALID_CREDENTIALS → formError
Enlace inválido	ResetPasswordView y, a veces, PHP/MySQL	A veces	tokenStatus === "invalid"

La forma práctica de leer cualquier caso de CineVerse es siempre la misma: localizar la causa, identificar el archivo que la produce, seguir el dato o código que cruza las capas, encontrar el estado reactivo que recibe Vue y, por último, observar qué condición del template selecciona el texto. Con ese recorrido se puede explicar no sólo qué mensaje aparece, sino por qué aparece y qué partes reales del sistema participaron para llegar a él.

29. Logs: registro y diagnóstico de errores

Un log es un registro técnico que el backend conserva para poder investigar fallos sin mostrar detalles internos al usuario. CineVerse no escribe directamente en un archivo desde cada Controller. Utiliza una clase propia, AppLogger, como puerta de entrada y delega la escritura, rotación y formato en Monolog.

Esta sección sigue el recorrido completo desde AppLogger::error() hasta el archivo diario. Primero se explican AppLogger y Monolog; después se muestra cómo se configura el logger y recién al final se analizan entradas reales.

```
Controller / bootstrap
-> AppLogger::error(...)
-> AppLogger::get()
-> Logger de Monolog
-> RotatingFileHandler
-> LineFormatter
-> cineverse-backend-AAAA-MM-DD.log
```

29.1 AppLogger y Monolog: dos responsabilidades distintas

AppLogger.php pertenece al código de CineVerse. Su objetivo es ofrecer un punto único para registrar errores y esconder al resto del backend los detalles de configuración. Los Controllers pueden llamar AppLogger::error() sin

saber en qué carpeta se guarda el archivo, cuántos archivos rotativos se conservan o qué formatter utiliza Monolog.

Monolog, en cambio, es una librería externa de logging. Dentro de CineVerse intervienen varias clases de esa librería: `Logger` recibe el registro; `RotatingFileHandler` decide escribirlo en archivos rotativos; `LineFormatter` convierte cada registro a texto; y `Level::Error` establece el nivel mínimo aceptado por el handler.

```
namespace App\Core\logging;

use Monolog\Formatter\LineFormatter;
use Monolog\Handler\RotatingFileHandler;
use Monolog\Level;
use Monolog\Logger;

final class AppLogger
{
    private static ?Logger $logger = null;
    ...
}
```

Las instrucciones `use` no crean esos objetos; únicamente permiten referirse a las clases de Monolog mediante nombres cortos. La propiedad `$logger` será la instancia compartida que `AppLogger` configura la primera vez que se necesita registrar algo.

29.2 Cómo entra Monolog en el proyecto

Monolog no forma parte de PHP. Está declarado como dependencia en `backend/composer.json`. Al ejecutar `composer install`, Composer instala la librería dentro de `vendor` y genera `vendor/autoload.php`. `bootstrap.php` carga ese autoloader al comienzo de cada solicitud.

```
"require": {
    "vlucas/phpdotenv": "^5.6",
    "monolog/monolog": "^3.10",
    "phpmailer/phpmailer": "^7.1"
}

// core/bootstrap.php
require_once __DIR__ . '/../vendor/autoload.php';
```

Gracias al autoloader de Composer, `AppLogger` puede crear `Logger`, `RotatingFileHandler` y `LineFormatter` sin utilizar `require_once` para cada archivo de Monolog. Este mismo mecanismo carga también las clases propias del backend mediante la configuración PSR-4 explicada en la sección 6.

29.3 Cómo se construye el Logger dentro de AppLogger

29.3.1 Una instancia compartida durante la ejecución

```
private static ?Logger $logger = null;

public static function get(): Logger
{
    if (self::$logger === null) {
        self::$logger = new Logger("app");
        ...
    }

    return self::$logger;
}
```

`$logger` comienza en `null` porque todavía no existe una instancia. La primera llamada a `get()` entra en el `if` y construye el `Logger`. Las siguientes llamadas durante esa ejecución de PHP reutilizan la misma instancia. "app" es el nombre o canal del `Logger`; no es el nombre del archivo.

29.3.2 RotatingFileHandler: archivo diario, retención y nivel

```
$logPath = self::resolveLogPath();

$handler = new RotatingFileHandler(
    $logPath,
    30,
    Level::Error
);
```

El primer argumento indica el archivo base. El segundo fija en 30 la cantidad máxima de archivos rotativos que deben conservarse. El tercero establece Level::Error como nivel mínimo. Por lo tanto, este handler está preparado para registrar ERROR y niveles superiores; no funciona como un historial general de mensajes INFO de la aplicación. Como la rotación configurada es diaria, normalmente esos archivos corresponden a fechas distintas, pero el parámetro limita archivos, no garantiza una retención expresada en días.

RotatingFileHandler agrega la fecha al nombre base y cambia automáticamente de archivo al rotar. Cuando se supera el máximo configurado, Monolog puede eliminar los archivos más antiguos para evitar crecimiento indefinido.

29.3.3 LineFormatter: cómo se ve cada entrada

```
$handler->setFormatter(
    new LineFormatter(
        "[%datetime%] %level_name%: %message%\n",
        "Y-m-d H:i:s",
        true,
        true
    )
);

self::$logger->pushHandler($handler);
```

El primer argumento define la estructura textual. %datetime% se reemplaza por la fecha y hora, %level_name% por el nivel y %message% por el mensaje enviado a Monolog. "Y-m-d H:i:s" produce fechas como 2026-09-07 07:14:16.

El primer true permite saltos de línea dentro del mensaje, algo necesario porque AppLogger agrega el contexto JSON debajo del texto principal. El segundo true evita imprimir context y extra vacíos del formato automático de Monolog. Finalmente pushHandler() conecta el handler ya configurado con el Logger; antes de esa llamada el handler existía, pero todavía no estaba asociado al logger.

29.4 De backend/.env al archivo físico: CINEVERSE_LOG_PATH

CINEVERSE_LOG_PATH es una variable de entorno definida por CineVerse en backend/.env. No es una constante de PHP ni una opción nativa de Monolog. bootstrap.php carga .env mediante phpdotenv y deja el valor disponible en \$_ENV; AppLogger lo utiliza cuando necesita calcular la carpeta de destino.

```
# backend/.env
CINEVERSE_LOG_PATH=storage/logs

// AppLogger::resolveLogPath()
$basePath = dirname(__DIR__, 2);
$relativePath =
    $_ENV["CINEVERSE_LOG_PATH"] ?? "storage/logs";
```

AppLogger.php se encuentra dentro de backend/core/logging. dirname(__DIR__, 2) sube dos niveles y obtiene la raíz física del backend. Si .env contiene storage/logs, esa ruta relativa se combina con la raíz y termina normalmente en backend/storage/logs. El operador ?? deja storage/logs como defensa interna de AppLogger si la variable no estuviera disponible; sin embargo, en el arranque normal bootstrap.php declara CINEVERSE_LOG_PATH como obligatoria mediante phpdotenv, por lo que una configuración incompleta debería detectarse antes de llegar a este fallback.

```

$isAbsolutePath = preg_match(
    '/^(?:[A-Za-z]:[\\\/]|\\\/)/',
    $relativePath
) === 1;

$logDir = $isAbsolutePath
    ? $relativePath
    : $basePath . DIRECTORY_SEPARATOR . $relativePath;

$logPath = rtrim($logDir, "\\")
    . DIRECTORY_SEPARATOR
    . "cineverse-backend.log";

```

La comprobación permite dos formas de configuración. Una ruta relativa, como `storage/logs`, se interpreta dentro del backend. Una ruta absoluta de Windows o Linux se utiliza directamente. `DIRECTORY_SEPARATOR` evita escribir manualmente el separador de carpetas del sistema operativo.

```

CINEVERSE_LOG_PATH=storage/logs
-> backend/storage/logs
-> cineverse-backend.log
-> cineverse-backend-2026-09-07.log

CINEVERSE_LOG_PATH=D:\logs\cineverse
-> D:\logs\cineverse
-> cineverse-backend-AAAA-MM-DD.log

```

El código ejecutado utiliza `cineverse-backend.log` como nombre base. Un comentario antiguo dentro de `AppLogger.php` todavía menciona `app-AAAA-MM-DD.log`; la documentación sigue la implementación efectiva, que genera `cineverse-backend-AAAA-MM-DD.log`.

29.4.1 Creación automática de la carpeta

```

$logDir = dirname($logPath);

if (!is_dir($logDir)) {
    mkdir($logDir, 0777, true);
}

return $logPath;

```

`is_dir()` comprueba si la carpeta existe. Si no existe, `mkdir()` intenta crearla. El tercer argumento `true` permite crear carpetas intermedias. El permiso `0777` es amplio y resulta cómodo en desarrollo, pero no es una elección recomendable para producción; allí deberían asignarse únicamente los permisos que necesita el usuario con el que se ejecuta PHP.

29.5 AppLogger::error(): convertir mensaje y contexto en una entrada

El resto del backend no llama directamente al Logger de Monolog. Utiliza `AppLogger::error(message, context)`. `message` contiene la causa técnica principal y `context` es un array opcional con datos útiles para localizar la operación.

```

public static function error(
    string $message,
    array $context = []
): void {
    self::get()->error(
        $message . "\n" .
        json_encode(
            $context,
            JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE
        )
    );
}

```

`self::get()` devuelve el Logger ya configurado y `->error()` crea un registro de nivel `ERROR`. Antes de llamar a Monolog, `AppLogger` convierte el array de contexto a JSON y lo concatena al mensaje. `JSON_PRETTY_PRINT` agrega indentación y saltos de línea; `JSON_UNESCAPED_UNICODE` mantiene caracteres Unicode legibles.

Esta es una decisión concreta del proyecto: el array no se entrega como segundo argumento estructurado de Monolog. El contexto se convierte manualmente a texto y queda dentro de %message%. Por eso LineFormatter permite saltos de línea y la entrada se ve como un mensaje seguido por un bloque JSON.

29.6 Qué termina escrito en el archivo

```
[2026-09-07 07:14:16] ERROR: mensaje técnico
{
  "controller": "...",
  "method": "...",
  "status": 500
}
```

La fecha proviene de %datetime%, ERROR de %level_name% y el resto de %message%. El bloque JSON existe porque AppLogger::error() lo agregó al mensaje antes de entregarlo a Monolog. El archivo concreto depende del día: por ejemplo, cineverse-backend-2026-09-07.log.

29.7 Ejemplo real: MySQL no disponible durante el arranque

public/index.php abre la conexión a MySQL antes de crear el Router. Si Database::connect() lanza PDOException, index.php registra la causa técnica y devuelve al cliente un código estable que no expone el SQLSTATE.

```
try {
    $pdo = Database::connect();
} catch (\PDOException $exception) {
    AppLogger::error($exception->getMessage(), [
        "controller" => "Application",
        "method" => "bootstrap",
        "status" => 503,
        "pdo_code" => $exception->getCode(),
    ]);

    Response::error("DATABASE_UNAVAILABLE", 503);
    exit;
}
```

Si MySQL devuelve el error 1049 porque la base configurada no existe, el archivo puede contener una entrada como la siguiente:

```
[2026-09-07 07:14:16] ERROR: SQLSTATE[HY000] [1049] Unknown database 'cineverse2'
{
  "controller": "Application",
  "method": "bootstrap",
  "status": 503,
  "pdo_code": 1049
}
```

El navegador no recibe ese detalle. Response::error("DATABASE_UNAVAILABLE", 503) devuelve una respuesta segura. Así se separan dos públicos distintos: el desarrollador conserva información de diagnóstico en el log y el cliente recibe un código estable para manejar el error.

29.8 Ejemplo desde un Controller

Los Controllers utilizan el mismo mecanismo cuando aparece un Throwable que no corresponde a un error esperado de validación o negocio. ReviewController::create() agrega contexto relacionado con la operación que estaba intentando ejecutar.

```

catch (Throwable $e) {
    AppLogger::error($e->getMessage(), [
        "controller" => "ReviewController",
        "method" => "create",
        "user_id" => $userId,
        "movie_id" => $movieId
    ]);

    Response::error(
        "INTERNAL_SERVER_ERROR",
        500
    );
}

```

El contexto no es fijo para todos los errores. En bootstrap resulta útil `status` y `pdo_code`; al crear una reseña resultan útiles `controller`, `method`, `user_id` y `movie_id`. La idea es registrar datos suficientes para localizar el problema sin convertir el log en una copia indiscriminada de toda la solicitud.

29.9 Ejemplo especial: conservar la causa original de un error SMTP

MailService transforma ciertos fallos de PHPMailer en `RuntimeException("EMAIL_DELIVERY_FAILED")` y conserva la excepción original como `previous`. AuthController utiliza `mailExceptionMessage()` para registrar la causa técnica real en lugar de perderla detrás del código genérico.

```

private function mailExceptionMessage(
    RuntimeException $e
): string {
    return $e->getPrevious()?->getMessage()
        ?? $e->getMessage();
}

AppLogger::error(
    $this->mailExceptionMessage($e),
    [
        "controller" => "AuthController",
        "method" => "register",
        "email" => $email
    ]
);

```

De esta forma el cliente puede recibir `EMAIL_DELIVERY_FAILED` mientras el archivo conserva, por ejemplo, un mensaje de autenticación SMTP producido por PHPMailer. La diferencia entre respuesta segura y diagnóstico interno se conecta con el manejo de errores explicado en la sección 28.

29.10 Qué conviene registrar, privacidad y permisos

No todos los resultados negativos deben registrarse como `ERROR`. Un rating igual a 6, una contraseña incorrecta o una reseña duplicada son situaciones esperadas que el backend sabe clasificar y responder. En cambio, una `PDOException` inesperada, una caída de SMTP o un `Throwable` no previsto son candidatos naturales para `AppLogger`.

- Registrar contexto útil para localizar la operación: `controller`, `method`, ids relacionados, código técnico o email cuando realmente ayuda al diagnóstico.
- No registrar contraseñas, tokens CSRF, tokens de verificación ni tokens de recuperación de contraseña.
- Mantener storage/logs fuera de la raíz pública del servidor para que los archivos no puedan descargarse por HTTP.
- En producción, sustituir permisos amplios como `0777` por los mínimos necesarios para que PHP pueda crear y rotar archivos.
- Recordar que el sistema actual está configurado para `ERROR` o superior; no es un historial de cada acción normal del usuario.

El recorrido completo queda entonces: una parte del backend detecta un fallo técnico, llama `AppLogger::error()`, `AppLogger` obtiene o reutiliza el `Logger`, `Monolog` entrega el registro al `RotatingFileHandler`, `LineFormatter`

produce el texto y el archivo diario queda disponible para diagnóstico. El usuario recibe por separado una respuesta HTTP controlada.

30. Instalación local de CineVerse

Al clonar o descargar CineVerse desde GitHub, las carpetas backend y frontend ya forman parte del proyecto. No hay que crear ninguno de los dos proyectos desde cero. Lo que sí debe reconstruirse o configurarse en cada computadora son las dependencias instaladas, los archivos .env y los servicios locales que utilizará la aplicación.

Los archivos .gitignore actuales reflejan ese flujo: backend/vendor y backend/.env no se versionan; en el frontend se excluyen node_modules, dist, dist-ssr y .env. En cambio, composer.json, composer.lock, package.json, package-lock.json y los archivos .env.example permiten reconstruir una instalación nueva.

Elemento	¿Se espera en GitHub?	Qué ocurre en una instalación nueva
backend/ y frontend/	Sí	Contienen el código fuente de CineVerse y ya existen al clonar o descargar el repositorio.
backend/vendor/	No	Composer la reconstruye al ejecutar composer install.
frontend/node_modules/	No	npm la reconstruye al ejecutar npm install.
frontend/dist/	No	Es un resultado de compilación; se genera cuando se construye el frontend.
.env.example	Sí	Sirve como plantilla con los nombres de las variables requeridas.
.env	No	Se crea localmente con valores propios de esa computadora y puede contener credenciales.

30.1 Programas y servicios necesarios

Componente	Uso en CineVerse
Visual Studio Code	Es el editor utilizado en esta guía para abrir backend y frontend y trabajar con la terminal integrada. Otro editor es posible, pero las capturas y los pasos siguen VS Code.
PHP 8.1 o superior	Ejecuta el backend PHP y sus dependencias. La versión puede comprobarse con <code>php -v</code> o con el ejecutable de PHP instalado en la computadora.
Composer	Instala las dependencias PHP declaradas en composer.json y composer.lock y genera vendor/autoload.php.
Node.js + npm	Instala las dependencias del frontend y permite ejecutar Vite y las herramientas de TypeScript. El proyecto resuelve Vite 7.3.6, que requiere Node.js ^20.19.0 o >=22.12.0.
MySQL 8	Motor que guarda users, reviews y login_attempts. El archivo SQL de CineVerse utiliza características de MySQL 8.
MySQL Workbench	Herramienta gráfica utilizada en esta guía para abrir e importar el archivo SQL y comprobar la base de datos. No es necesaria para que CineVerse funcione una vez instalada.
XAMPP / Apache	En el entorno de desarrollo documentado, Apache de XAMPP sirve backend/public mediante cineverse.local.

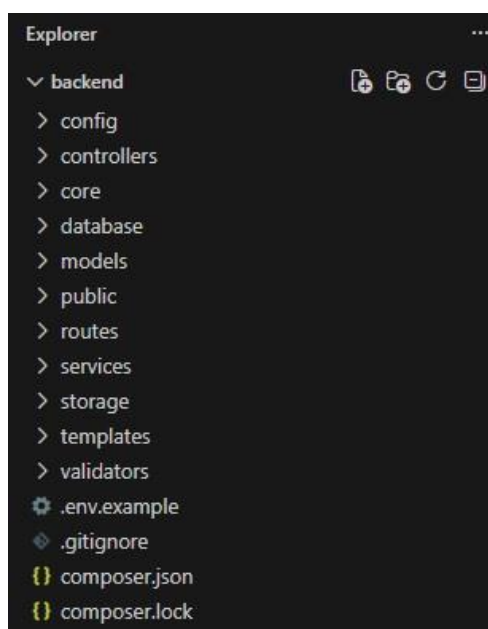
Componente	Uso en CineVerse
TMDB Read Access Token	Credencial que el frontend utiliza para consultar la API de TMDB.
Servidor SMTP	Necesario si se quieren probar los correos de verificación y recuperación de contraseña.
Postman (opcional)	Herramienta utilizada en esta documentación para probar manualmente los endpoints del backend PHP sin depender de la interfaz Vue. Permite inspeccionar métodos, JSON, cabeceras, cookies y respuestas.

El backend no utiliza Laravel ni otro framework, por lo que no existe un comando `php artisan serve`. En el entorno principal de desarrollo documentado, Apache de XAMPP sirve directamente `backend/public` y `.htaccess` reescribe las rutas para que lleguen a `public/index.php`.

30.2 Abrir el backend, instalar sus dependencias y configurar .env

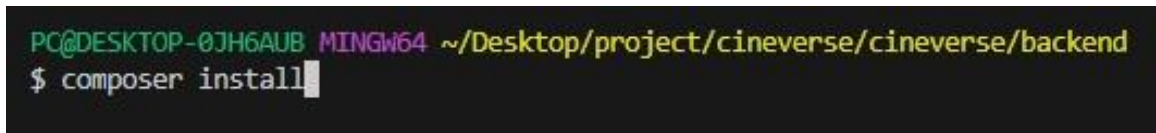
La carpeta `backend` ya contiene el código PHP de CineVerse. Para instalar el proyecto en una computadora nueva no hay que crear un backend nuevo: hay que abrir esa carpeta, instalar sus dependencias y completar la configuración local.

Paso 1. Abrir Visual Studio Code y elegir Archivo > Abrir carpeta. Seleccionar la carpeta `backend` que vino con el proyecto.



Backend de CineVerse abierto como carpeta de proyecto en Visual Studio Code.

Paso 2. Con backend abierto, elegir Terminal > Nueva terminal. La terminal integrada debe quedar situada dentro de la carpeta `backend`. Si la ruta termina en `/backend`, no hace falta ejecutar `cd backend`.



Terminal integrada situada dentro de backend antes de ejecutar Composer.

Paso 3. Ejecutar Composer para reconstruir las dependencias PHP declaradas por el proyecto:

```
composer install
```

Composer lee `composer.json` y `composer.lock`, descarga las versiones correspondientes y crea vendor. Entre las dependencias utilizadas por CineVerse se encuentran `phpdotenv` para leer variables de entorno, `Monolog` para

logging y PHPMailer para correo. Composer también genera vendor/autoload.php, utilizado por el backend para cargar clases y librerías.

Paso 4. Crear backend/.env tomando backend/.env.example como plantilla. .env contiene valores propios de esa computadora y no debe publicarse ni quedar accesible por HTTP.

El archivo debe definir el entorno de ejecución, la conexión al servidor MySQL, la ruta de logs, el servidor SMTP y la URL del frontend. Un ejemplo neutro para una instalación local es:

```
APP_ENV=local
APP_DEBUG=true

DB_HOST=localhost
DB_PORT=3306
DB_NAME=cineverse
DB_USER=usuario
DB_PASSWORD=contraseña
DB_CHARSET=utf8mb4

CINEVERSE_LOG_PATH=storage/logs

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=usuario
MAIL_PASSWORD=contraseña
MAIL_FROM=no-reply@example.com
MAIL_FROM_NAME=CineVerse

FRONTEND_URL=http://localhost:5173
```

Los valores anteriores son ilustrativos. DB_HOST, DB_PORT, DB_USER y DB_PASSWORD deben coincidir con el servidor MySQL que realmente utilice esa instalación; los datos SMTP también deben reemplazarse por credenciales válidas si se van a probar correos.

Caso alternativo. Si en Visual Studio Code se abrió una carpeta raíz que contiene backend y frontend en lugar de abrir backend directamente, entonces la terminal sí estará un nivel más arriba y habrá que ejecutar `cd backend` antes de `composer install`.

30.3 Base de datos

CineVerse incluye un archivo SQL preparado para construir la base que utiliza el backend y cargar los datos de prueba del proyecto. Dentro del repositorio, el archivo se encuentra en `database/cineverse.sql`. No es necesario crear manualmente la base ni las tablas: el propio script contiene las instrucciones necesarias.

El script crea la base cineverse si todavía no existe, la selecciona y reconstruye las tablas activas `users`, `login_attempts` y `reviews`. Después carga los usuarios y las reseñas de prueba. También contiene `DROP TABLE IF EXISTS` para eliminar versiones anteriores de esas tablas y algunas tablas heredadas que ya no utiliza el backend PHP actual. Por ese motivo, este procedimiento está pensado para preparar o reconstruir un entorno local de pruebas y no debe ejecutarse sobre una base cuyos datos se quieran conservar.

El servidor debe ser MySQL 8 y debe ser el mismo al que después se conectará el backend. Los valores DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD y DB_CHARSET de backend/.env deben describir esa misma instalación de MySQL.

30.3.1 Preparar la base de datos con MySQL Workbench

MySQL Workbench se utiliza en esta guía como cliente gráfico para conectarse al servidor MySQL, abrir el script y ejecutar sus instrucciones. Workbench no contiene la base ni reemplaza al servidor: actúa sobre la conexión que el desarrollador seleccione.

Paso 1. Localizar cineverse.sql en la carpeta raíz del proyecto, junto a backend y frontend. Ese es el script de base de datos que debe utilizarse para esta instalación. No hace falta copiar sus instrucciones manualmente ni crear la base o las tablas una por una.

Paso 2. Iniciar el servidor MySQL 8 que utilizará CineVerse. Abrir MySQL Workbench y conectarse a ese servidor con el host, el puerto, el usuario y la contraseña correspondientes. El usuario empleado para ejecutar el script debe tener permisos para crear la base, crear y eliminar tablas e insertar datos.

Paso 3. Con la conexión correcta ya abierta, seleccionar File > Open SQL Script y elegir el archivo localizado en el Paso 1. Workbench mostrará su contenido en una pestaña del editor SQL.

Paso 4. Antes de ejecutar nada, comprobar nuevamente que la pestaña del script está asociada al servidor MySQL destinado al entorno local de CineVerse. El archivo elimina tablas existentes antes de reconstruirlas, por lo que no debe ejecutarse sobre una base cuyos datos se quieran conservar. No es necesario crear previamente un schema llamado cineverse: el propio script ejecuta CREATE DATABASE IF NOT EXISTS cineverse y después USE cineverse.

Paso 5. Ejecutar todo el contenido del script. Para evitar ejecutar únicamente la instrucción donde se encuentre el cursor, seleccionar el contenido completo y utilizar la opción de Workbench que ejecuta la selección o el script completo. Durante este proceso se crea y selecciona cineverse, se reconstruyen users, login_attempts y reviews y se cargan los usuarios y las reseñas de prueba.

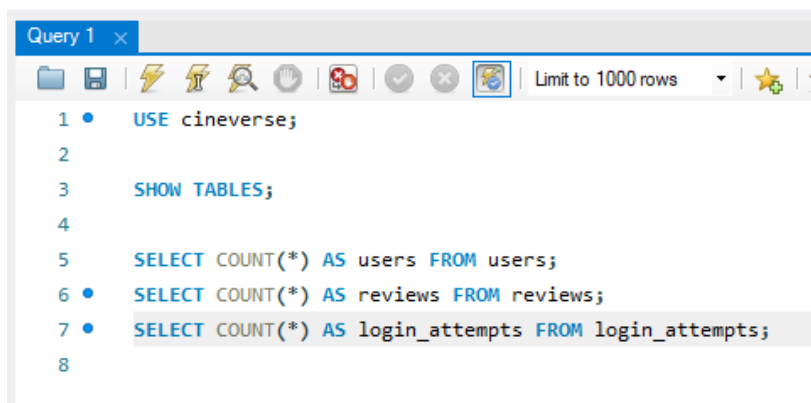
Paso 6. Cuando la ejecución termine sin errores, actualizar el panel Schemas de MySQL Workbench. Debe aparecer la base cineverse y, dentro de Tables, las tablas users, login_attempts y reviews.

Paso 7. Ejecutar las siguientes consultas de control. SHOW TABLES comprueba la estructura visible del schema y los COUNT(*) permiten verificar cuántos registros quedaron cargados en cada tabla:

```
USE cineverse;

SHOW TABLES;

SELECT COUNT(*) AS users FROM users;
SELECT COUNT(*) AS reviews FROM reviews;
SELECT COUNT(*) AS login_attempts FROM login_attempts;
```



Consultas utilizadas para verificar la estructura y los datos después de ejecutar el archivo SQL de CineVerse.

Tabla	Resultado esperado después de una carga limpia
users	5.000
reviews	24.322
login_attempts	0

Después de una carga limpia, el resultado esperado es 5.000 filas en users, 24.322 en reviews y 0 en login_attempts. El archivo SQL no carga registros iniciales en login_attempts: esa tabla comienza a utilizarse cuando el backend registra intentos fallidos de inicio de sesión durante el uso de la aplicación.

Paso 8. Comprobar que backend/.env apunta al mismo servidor MySQL utilizado en Workbench. Una base correctamente creada no conecta automáticamente el backend: host, puerto y credenciales deben corresponder a esa instalación, y DB_NAME debe ser cineverse.

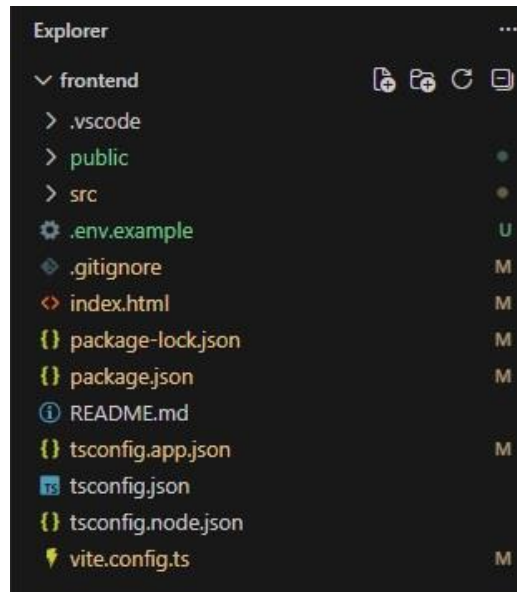
```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=cineverse
DB_USER=<usuario de MySQL>
DB_PASSWORD=<contraseña de MySQL>
DB_CHARSET=utf8mb4
```

Los valores de DB_HOST, DB_PORT, DB_USER y DB_PASSWORD dependen de la instalación local. DB_NAME=cineverse coincide con la base creada y seleccionada por el script. DB_CHARSET=utf8mb4 hace que la conexión PDO utilice utf8mb4, en consonancia con el juego de caracteres definido por el esquema. Si Workbench se conectó a otro host o puerto, esos mismos datos deben reflejarse aquí.

30.4 Abrir el frontend, instalar sus dependencias y configurar .env

La carpeta frontend también forma parte del proyecto descargado desde GitHub. Contiene el código Vue, package.json y package-lock.json. Lo que normalmente no viene versionado es node_modules, porque puede reconstruirse a partir de esos archivos.

Paso 1. Abrir Visual Studio Code y elegir Archivo > Abrir carpeta. Seleccionar la carpeta frontend de CineVerse.



Frontend de CineVerse abierto como carpeta de proyecto en Visual Studio Code.

Paso 2. Elegir Terminal > Nueva terminal. Si frontend se abrió directamente como carpeta de proyecto, la terminal ya debe quedar situada dentro de frontend y no hace falta ejecutar cd frontend.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse/frontend (main)
$ npm install
```

Terminal integrada situada dentro de frontend antes de instalar dependencias.

Paso 3. Instalar las dependencias declaradas en package.json y fijadas en package-lock.json:

```
npm ci
```

npm ci instala exactamente las versiones fijadas en package-lock.json y crea node_modules desde una instalación limpia, por lo que es el comando preferido para reproducir el proyecto. npm install sigue siendo útil cuando se agregan o actualizan dependencias. Ninguno de estos comandos mantiene el frontend ejecutándose; la puesta en marcha diaria se explica en la sección 31.

Paso 4. Antes de completar frontend/.env, crear o iniciar sesión en TMDB y abrir Account Settings > API. Allí se obtiene el API Read Access Token. Después, copiar frontend/.env.example a frontend/.env y completar los valores necesarios para esa instalación. La sección 3.3-3.5 desarrolla este proceso con capturas y el recorrido completo de la credencial.

```
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="token"
VITE_BACKEND_URL="/api"
```

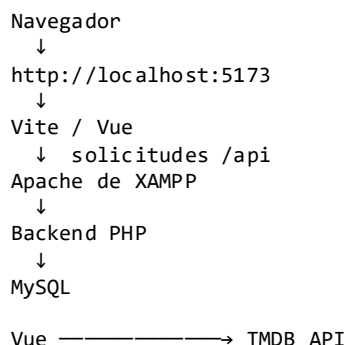
VITE_TMDB_BASE_URL indica dónde está la API v3 de TMDB. VITE_TMDB_TOKEN debe contener el API Read Access Token obtenido desde la configuración API de la cuenta; tmdbApi.ts lo envía como Authorization: Bearer. VITE_BACKEND_URL indica cómo llega Vue a la API propia de CineVerse; durante el desarrollo /api se resuelve mediante el proxy de Vite.

Caso alternativo. Si se abrió una carpeta raíz que contiene frontend y backend, entonces sí se ejecuta cd frontend antes de npm install. En esta guía el flujo principal consiste en abrir frontend directamente en Visual Studio Code.

31. Cómo ejecutar CineVerse en desarrollo

Una vez instalada la aplicación, el desarrollo local depende de varias piezas que cumplen funciones distintas. Vite sirve el frontend Vue, Apache de XAMPP sirve el backend PHP y MySQL proporciona la base configurada en backend/.env. TMDB es un servicio externo y se consulta directamente desde el frontend.

Hay que distinguir la configuración inicial de la puesta en marcha diaria. El VirtualHost y el archivo hosts normalmente se configuran una sola vez por computadora. En cambio, Apache, el servidor MySQL configurado y Vite deben estar disponibles cuando se quiera trabajar con CineVerse.



Parte	Dirección o referencia en desarrollo
Frontend Vue / Vite	http://localhost:5173
Backend PHP / Apache	http://cineverse.local
API propia usada por Vue	/api
Ejemplo de endpoint backend	http://cineverse.local/user
TMDB API	https://api.themoviedb.org/3

31.1 Configurar el VirtualHost de Apache paso a paso

El VirtualHost permite que Apache relacione el nombre cineverse.local con la carpeta pública del backend. Esta configuración normalmente se realiza una sola vez en cada computadora, salvo que después se mueva el proyecto a otra ruta.

Paso 1. Localizar la ruta absoluta de backend/public en la computadora. Es la única carpeta del backend que Apache debe publicar directamente.

Paso 2. Abrir la configuración de VirtualHosts de Apache. En una instalación habitual de XAMPP para Windows se encuentra en C:\xampp\apache\conf\extra\httpd-vhosts.conf. También puede abrirse desde XAMPP Control Panel > Apache > Config > Apache (httpd-vhosts.conf).

Paso 3. Agregar un bloque para CineVerse y reemplazar la ruta de ejemplo por la ubicación real de backend/public:

```
<VirtualHost *:80>
    ServerName cineverse.local

    DocumentRoot "C:/ruta/al/proyecto/cineverse/backend/public"

    <Directory "C:/ruta/al/proyecto/cineverse/backend/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

```
<VirtualHost *:80>

    ServerName cineverse.local

    DocumentRoot "C:/Users/PC/Desktop/project/cineverse/cineverse/backend/public"

    # Header always set Access-Control-Allow-Origin "*"
    # Header always set Access-Control-Allow-Methods "GET, POST, PUT, DELETE, OPTIONS"
    # Header always set Access-Control-Allow-Headers "*"

    <Directory "C:/Users/PC/Desktop/project/cineverse/cineverse/backend/public">
        AllowOverride All
        Require all granted
    </Directory>

</VirtualHost>
```

VirtualHost real de CineVerse en el entorno de desarrollo utilizado para estas capturas.

Nota sobre la captura: las líneas Header always set visibles con # están comentadas, por lo que están inactivas y no forman parte de la configuración necesaria del VirtualHost. CineVerse gestiona CORS desde backend/core/middleware/CorsMiddleware.php; no hay que descomentarlas para seguir esta guía.

La ruta visible en la captura pertenece a la computadora donde se desarrolló CineVerse. No debe copiarse literalmente en otra máquina: cada instalación debe apuntar a su propia carpeta backend/public.

ServerName define el nombre que recibirá Apache. DocumentRoot indica la carpeta pública que puede servir. Apuntar sólo a backend/public evita exponer directamente controllers, services, templates, storage, .env y otros archivos internos. AllowOverride All permite que .htaccess aplique sus reglas y Require all granted autoriza a Apache a atender esa carpeta.

Paso 4. Comprobar en C:\xampp\apache\conf\httpd.conf que Apache cargue el archivo de VirtualHosts y tenga habilitado mod_rewrite:

```
Include conf/extra/httpd-vhosts.conf
LoadModule rewrite_module modules/mod_rewrite.so
```

Si alguna de esas líneas está comentada con #, Apache no utilizará esa parte de la configuración hasta habilitarla y reiniciarse. mod_rewrite es necesario para que las reglas de backend/public/.htaccess puedan enviar las rutas de la API a public/index.php.

Paso 5. Guardar los cambios. Después de modificar httpd-vhosts.conf o httpd.conf hay que reiniciar Apache para que vuelva a cargar la configuración.

31.2 Configurar cineverse.local en el archivo hosts de Windows

cineverse.local es un nombre local utilizado sólo durante el desarrollo. No existe en Internet ni está registrado en un DNS público. Windows necesita saber que ese nombre debe resolverse hacia la propia computadora.

La dirección 127.0.0.1 representa la máquina local, también llamada localhost. El archivo hosts permite asociar manualmente un nombre con una dirección IP antes de recurrir a un servidor DNS.

Paso 1. Abrir Bloc de notas como administrador. Esto es necesario porque hosts es un archivo protegido por Windows.

Paso 2. Elegir Archivo > Abrir y navegar hasta C:\Windows\System32\drivers\etc. Si el archivo no aparece, cambiar el filtro de Documentos de texto a Todos los archivos.

Paso 3. Abrir hosts y agregar al final la asociación utilizada por CineVerse:

```
127.0.0.1 cineverse.local
```



Archivo hosts de Windows asociando cineverse.local con 127.0.0.1.

Paso 4. Guardar el archivo. A partir de ese momento Windows puede resolver cineverse.local como la propia computadora.

El archivo hosts no decide qué aplicación debe responder ni dónde está el backend. Su única función es convertir cineverse.local en 127.0.0.1. El VirtualHost de la sección anterior completa el recorrido al indicarle a Apache que ese nombre corresponde a backend/public.

```

cineverse.local
    ↓
archivo hosts
    ↓
127.0.0.1
    ↓
Apache
    ↓
VirtualHost
    ↓
backend/public
  
```

Modificar hosts no requiere por sí solo reiniciar Apache. Apache debe reiniciarse cuando cambió su propia configuración, por ejemplo httpd-vhosts.conf o httpd.conf.

31.3 Cómo utiliza CineVerse el archivo .htaccess

backend/public/.htaccess ya forma parte del proyecto. El desarrollador no tiene que crearlo durante la instalación. Su función es permitir que las URLs de la API lleguen al único punto de entrada PHP aunque no exista un archivo físico con el mismo nombre que la ruta.

Por ejemplo, cuando Vue solicita http://cineverse.local/user no existe un archivo backend/public/user ni user.php. Apache debe enviar esa URL hacia backend/public/index.php, y desde allí el Router PHP decide qué código ejecutar.

```

RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d

RewriteRule ^ index.php [QSA,L]
  
```

RewriteEngine On activa el motor de reescritura. Las dos condiciones indican que la regla sólo debe aplicarse cuando la petición no corresponde a un archivo (-f) ni a una carpeta (-d) existentes. RewriteRule envía entonces la ruta internamente a index.php y QSA conserva los parámetros de la URL.



El mismo mecanismo permite que `/login`, `/reviews` o `/reviews/15` lleguen al Router PHP sin que exista un archivo separado para cada URL. Para que funcione, Apache debe tener habilitado `mod_rewrite` y el VirtualHost debe permitir `AllowOverride All`.

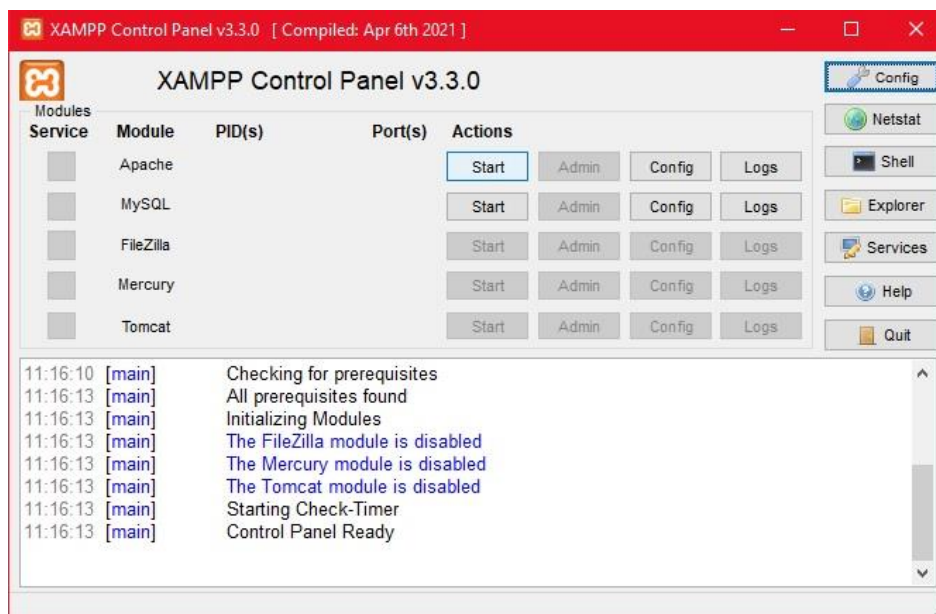
31.4 Iniciar el backend con Apache de XAMPP

En el entorno de desarrollo documentado, el backend PHP no se inicia con `npm`, `Composer` ni con un comando propio de CineVerse. Apache es quien sirve `backend/public` y actualmente se utiliza el Apache incluido con XAMPP. Si Apache está detenido, `cineverse.local` no puede llegar al backend.

Paso 1. Comprobar que el servidor MySQL indicado por `DB_HOST` y `DB_PORT` en `backend/.env` está funcionando y que la base indicada por `DB_NAME` existe.

Ese servidor MySQL no tiene por qué ser el módulo MySQL de XAMPP. CineVerse se conecta al servidor definido por las variables `DB_*`. En este entorno XAMPP se utiliza principalmente para Apache.

Paso 2. Abrir XAMPP Control Panel. En la fila Apache debe aparecer el botón `Start` si el servidor todavía está detenido.



XAMPP Control Panel antes de iniciar Apache.

Paso 3. Presionar `Start` en la fila Apache. Cuando inicia correctamente, XAMPP muestra los procesos y los puertos utilizados y cambia el estado visual del módulo.

Modules	Service	Module	PID(s)	Port(s)	Actions
☐	Apache				Start

Antes: Apache detenido y botón Start disponible.

Modules	Service	Module	PID(s)	Port(s)	Actions
☒	Apache		11324 20576	80, 443	Stop

Después: Apache activo y atendiendo los puertos configurados.

Paso 4. Con Apache activo, probar una ruta del backend. Si la configuración de VirtualHost y hosts es correcta, `http://cineverse.local/user` debe llegar al código PHP.

No hace falta ejecutar `php -S` cuando se utiliza este método. Apache permanece atendiendo `http://cineverse.local` y dirige las rutas hacia `backend/public` mientras el módulo esté iniciado.

31.5 Iniciar el frontend Vue con Vite

El frontend se ejecuta de forma distinta. Apache mantiene disponible el backend PHP, mientras que Vue necesita que Vite sirva el proyecto durante el desarrollo.

Paso 1. Abrir la carpeta frontend en Visual Studio Code, como se hizo durante la instalación, y elegir Terminal > Nueva terminal.

Paso 2. Comprobar que la ruta de la terminal termina en `/frontend`. Si se abrió frontend directamente, no es necesario ejecutar `cd frontend`.

Paso 3. Iniciar el servidor de desarrollo de Vite:

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse/frontend (main)
$ npm run dev
```

Comando `npm run dev` ejecutado desde la carpeta frontend.

```
npm run dev
```

`npm` lee el script `dev` de `package.json`, cuyo valor es `vite`. Vite inicia entonces el servidor de desarrollo. La terminal debe permanecer abierta mientras se trabaja con el frontend.

Paso 4. Esperar a que Vite muestre la dirección local. En este proyecto normalmente aparece `http://localhost:5173/`. Esa es la URL que se abre en el navegador para utilizar CineVerse.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse/frontend (main)

VITE v7.3.6 ready in 3657 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Vite iniciado correctamente y mostrando `http://localhost:5173/`.

Diferencia importante: `npm install` instala o reconstruye las dependencias en `node_modules`; `npm run dev` inicia Vite. Son dos pasos distintos.

Durante el uso normal de desarrollo, el navegador abre `localhost:5173`. `cineverse.local` queda como dirección del backend a la que Vite reenviará las solicitudes que comienzan con `/api`.

31.6 Cómo viaja una solicitud /api desde Vue hasta PHP

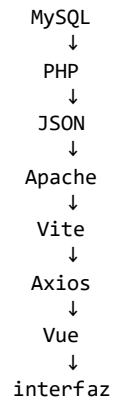
Con frontend y backend en ejecución se puede seguir una petición real para entender cómo se conectan todas las piezas. Supongamos que CineVerse necesita conocer el usuario autenticado y ejecuta `backendApi.get("/user")`.

`backendApi.ts` utiliza `VITE_BACKEND_URL=/api` como `baseUrl`. Por eso el navegador, que está ejecutando el frontend en `localhost:5173`, solicita primero `http://localhost:5173/api/user`.

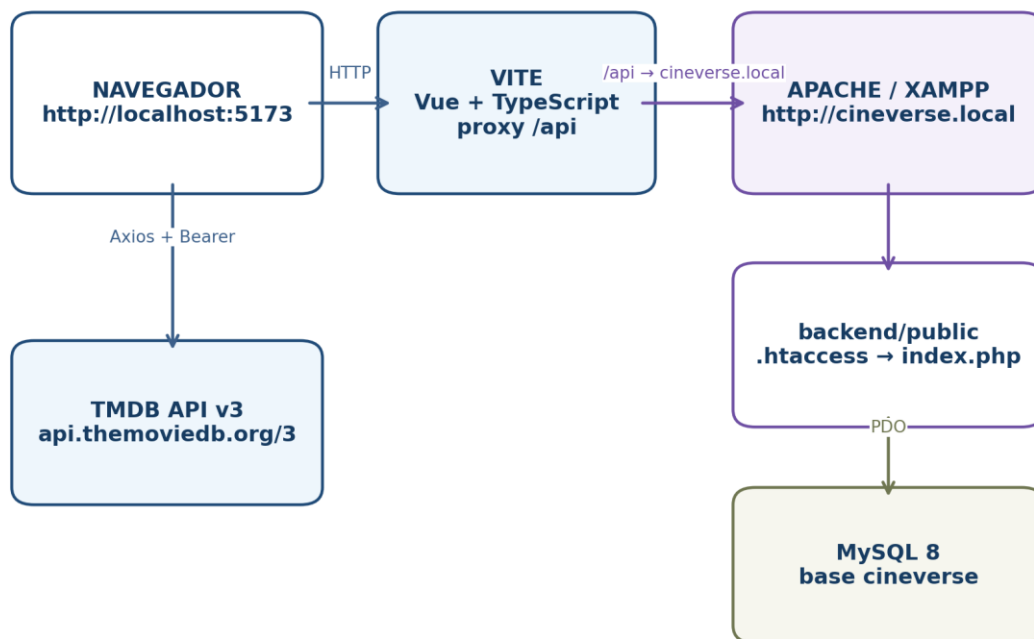
`vite.config.ts` contiene una regla de proxy para `/api`. Vite reconoce ese prefijo, lo elimina y reenvía la petición al destino configurado para el backend: `http://cineverse.local`. La URL termina siendo `http://cineverse.local/user`.



La respuesta vuelve en sentido inverso: MySQL entrega los datos a PHP, el backend genera JSON, Apache devuelve la respuesta, Vite la entrega al navegador y Axios la pone a disposición del código Vue.



Ejecución en desarrollo



El usuario abre la SPA en localhost:5173. Vite reenvía /api al VirtualHost de Apache; el frontend consulta TMDB directamente.

Flujo de ejecución local: navegador, Vite, Apache, backend y MySQL.

TMDB no utiliza este recorrido. Las consultas de películas y personas salen desde el frontend mediante `tmdbApi.ts` directamente hacia la API de TMDB; el proxy `/api`, Apache y el backend PHP intervienen sólo cuando Vue necesita la API propia de CineVerse.

31.7 Alternativa opcional: servidor incorporado de PHP

Para pruebas rápidas también puede utilizarse el servidor incorporado de PHP, pero no es el método principal documentado para CineVerse. PHP no interpreta `.htaccess`, por lo que `public/index.php` debe indicarse como router al iniciar el servidor desde la carpeta `backend`:

```
C:\xampp\php\php.exe -S 127.0.0.1:8000 -t public public/index.php
```

Si el ejecutable php correcto ya está disponible en PATH, puede utilizarse `php -S 127.0.0.1:8000 -t public public/index.php`. En este modo el backend queda en `http://127.0.0.1:8000` y el target del proxy `/api` de Vite tendría que apuntar temporalmente a esa dirección en lugar de `cineverse.local`.

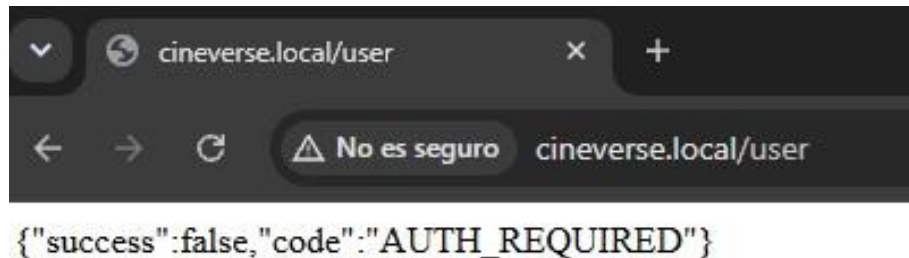
31.8 Comprobación final y diagnóstico rápido

Una vez iniciados el servidor MySQL configurado, Apache y Vite, conviene comprobar el entorno por partes. Esto permite saber con rapidez qué componente falla si CineVerse no funciona como se espera.

Prueba	Qué confirma	Si falla, revisar primero
Abrir <code>http://localhost:5173</code>	Vite está sirviendo el frontend Vue.	Terminal de Vite, <code>npm run dev</code> y errores del frontend.
Abrir Inicio/Películas o realizar una búsqueda	El frontend puede consultar TMDB.	<code>VITE_TMDB_BASE_URL</code> , <code>VITE_TMDB_TOKEN</code> y conexión a Internet.

Prueba	Qué confirma	Si falla, revisar primero
Abrir <code>http://cineverse.local/user</code>	Windows resuelve el nombre y Apache entrega la petición al backend PHP.	hosts, Apache, VirtualHost, <code>mod_rewrite</code> y <code>.htaccess</code> .
Iniciar sesión	Sesiones, backend y conexión MySQL funcionan juntos.	DB_*, estado del servidor MySQL y logs del backend.
Crear o consultar una reseña	Funciona el recorrido completo Vue → PHP → MySQL → respuesta.	API de reviews, sesión, PDO y datos de la base.

Una comprobación especialmente útil es abrir `http://cineverse.local/user` sin haber iniciado sesión. La respuesta esperada es JSON con `AUTH_REQUIRED`. Ese resultado no significa que la instalación esté rota: demuestra que la petición llegó correctamente a una ruta protegida del backend y que el sistema rechazó el acceso porque no existe una sesión autenticada.



Respuesta `AUTH_REQUIRED` de `/user` sin una sesión autenticada.

Si en cambio aparece un error del navegador porque `cineverse.local` no existe, hay que revisar hosts y Apache. Si aparece un 404 de Apache, hay que revisar el VirtualHost, `mod_rewrite` y `.htaccess`. Si el backend responde `DATABASE_UNAVAILABLE`, la petición ya llegó a PHP y el problema está en la conexión con el servidor MySQL configurado.

32. Cómo publicar CineVerse en un servidor

Publicar la aplicación significa pasar del entorno local de desarrollo a un servidor accesible desde Internet. Una organización clara es servir el frontend desde el dominio principal, por ejemplo `https://midominio.com`, y el backend desde un subdominio como `https://api.midominio.com`. El frontend ya compilado se sirve como archivos estáticos y PHP continúa ejecutándose en el servidor de la API.

32.1 Qué debe ofrecer el servidor

El servidor debe permitir PHP 8.1 o superior, MySQL 8, HTTPS, reescritura de URLs, escritura en `storage/logs` y salida SMTP. También debe ser posible instalar las dependencias de Composer o subir una carpeta `vendor` compatible con la versión de PHP utilizada por el hosting.

Las variables `VITE_*` se incorporan al JavaScript generado durante la compilación. Por esa razón `VITE_TMDB_TOKEN` puede observarse desde el navegador y no debe considerarse un secreto de servidor. Una credencial que realmente deba permanecer privada tendría que utilizarse desde una API del lado del servidor.

32.2 Generar la versión del frontend para producción

Antes de generar la versión de producción se configuran las variables `VITE_*` con las direcciones reales. Si el backend se publica en un subdominio, `VITE_BACKEND_URL` debe contener la URL completa de esa API:

```
VITE_TMDB_BASE_URL=https://api.themoviedb.org/3
VITE_TMDB_TOKEN=<token_tmdb>
VITE_BACKEND_URL=https://api.midominio.com
```

```
cd frontend
npm install
npm run build
```

npm run build genera frontend/dist, que contiene HTML, JavaScript, CSS y demás archivos listos para publicar. Esos archivos pueden copiarse al directorio público del dominio. Node.js no necesita quedar ejecutándose para servir el resultado final de Vite.

32.3 Configurar el servidor para las rutas de Vue Router

Vue Router utiliza URLs normales mediante createWebHistory. Si el usuario abre directamente /es/movies/550 o recarga esa dirección, el servidor del frontend debe devolver index.html para que Vue Router pueda decidir qué pantalla mostrar. Sin esa regla, Apache podría responder 404 antes de que Vue llegue a ejecutarse.

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
```

32.4 Publicar el backend PHP sin exponer archivos privados

El servidor web no debería exponer todo el proyecto PHP. Siempre que el hosting lo permita, el DocumentRoot de la API debe apuntar únicamente a backend/public. DocumentRoot es la carpeta que el servidor considera accesible por HTTP; así .env, logs, templates, services, models y otros archivos internos quedan fuera de la web pública.

```
/home/usuario/cineverse-backend/
├── controllers/
├── core/
├── database/
├── models/
├── routes/
├── services/
├── storage/
├── templates/
├── validators/
├── vendor/
├── .env
└── public/ ← DocumentRoot de api.midominio.com
```

32.4.1 Compatibilidad con Linux y PSR-4

Windows normalmente no distingue mayúsculas y minúsculas en nombres de carpeta, mientras que Linux sí. Los namespaces de CineVerse usan nombres como App\Core\Http y App\Core\Auth, pero las carpetas actuales aparecen como core/http y core/auth. Los desajustes observados están en auth, http, middleware y validation. La carpeta core/logging no presenta ese problema: AppLogger.php declara actualmente namespace App\Core\logging. Si se quisiera normalizarlo a App\Core\Logging, habría que renombrar la carpeta y actualizar el namespace y los imports al mismo tiempo.

```
core/auth      -> core/Auth
core/http      -> core/Http
core/middleware -> core/Middleware
core/validation -> core/Validation
```

Después de corregir los nombres de carpeta debe regenerarse el autoload de Composer con composer dump-autoload -o o mediante composer install --optimize-autoloader. Esto actualiza el mapa que Composer utiliza para localizar las clases.

32.5 Instalar las dependencias PHP en producción

```
cd ~/cineverse-backend
composer install --no-dev --optimize-autoloader
```

Si el hosting permite Composer, conviene ejecutar composer install --no-dev --optimize-autoloader dentro del backend. Si no ofrece acceso por consola o SSH, es decir, una terminal remota para ejecutar comandos en el

servidor, vendor puede prepararse previamente. En ese caso hay que comprobar que las dependencias sean compatibles con la versión de PHP del servidor de destino.

32.6 Crear e importar la base MySQL

En un hosting compartido es habitual crear primero la base y el usuario MySQL desde el panel de administración del proveedor, por ejemplo cPanel o Plesk. Después se importa el esquema. El archivo SQL actual también contiene datos de prueba y órdenes de recreación, por lo que no debe tratarse como una migración inocua sobre una base con datos reales.

Si el proveedor obliga a utilizar un nombre prefijado, por ejemplo `usuario_cineverse`, hay que adaptar `CREATE DATABASE cineverse` y `USE cineverse` antes de importar. La variable `DB_NAME` del backend debe coincidir con el nombre real creado en el servidor.

32.7 Configurar el backend para producción

```
APP_ENV=production
APP_DEBUG=false

DB_HOST=localhost
DB_PORT=3306
DB_NAME=usuario_cineverse
DB_USER=usuario_cv
DB_PASSWORD=<contraseña>
DB_CHARSET=utf8mb4

CINEVERSE_LOG_PATH=storage/logs

MAIL_HOST=smtp.midominio.com
MAIL_PORT=587
MAIL_USERNAME=no-reply@midominio.com
MAIL_PASSWORD=<contraseña>
MAIL_FROM=no-reply@midominio.com
MAIL_FROM_NAME=CineVerse

FRONTEND_URL=https://midominio.com
```

En producción `FRONTEND_URL` debe contener la dirección HTTPS real del frontend porque `MailService` la utiliza para construir los enlaces de verificación y recuperación. `APP_DEBUG` debe mantenerse en `false`. Las contraseñas y demás secretos permanecen únicamente en el `.env` del servidor.

32.8 CORS, HTTPS y cookies

El CORS actual permite `http://localhost:5173` y debe adaptarse al dominio real del frontend. Lo más mantenible sería obtener ese origen desde `FRONTEND_URL` en lugar de dejarlo escrito directamente en `CorsMiddleware`. Como la API utiliza cookies y `Access-Control-Allow-Credentials: true`, no corresponde permitir cualquier origen con `*` para este flujo.

El frontend y el backend deberían publicarse con HTTPS para cifrar el tráfico. `Session` activa la opción `Secure` de la cookie cuando PHP detecta HTTPS. Si el hosting utiliza un servidor intermedio (proxy) o una CDN por delante de PHP, hay que comprobar que esa información llegue correctamente a PHP para no crear cookies con una configuración incorrecta.

32.9 Base path y estructura de las URLs

`public/index.php` contiene `/cineverse/backend/public` como `basePath` fijo. Cuando el dominio o subdominio ya apunta a `backend/public`, una ruta como `/login` no contiene ese prefijo y funciona igualmente. Si la API se publica realmente dentro de una subcarpeta diferente, ese valor debe revisarse o convertirse en una opción configurable.

32.10 Logs y permisos

El usuario con el que se ejecuta PHP necesita permiso para escribir en storage/logs, pero no necesita permisos universales. En producción deben utilizarse los permisos mínimos que permitan crear y rotar los archivos. La carpeta de logs debe permanecer fuera de la raíz pública.

32.11 Verificación después de publicar la aplicación

- Abrir el dominio principal y comprobar /es y /en.
- Recargar directamente una ruta interna como /es/movies/550 para verificar la regla de respaldo de la SPA que devuelve index.html.
- Realizar una búsqueda y un Discover contra TMDB.
- Registrar una cuenta y comprobar la recepción del correo de verificación.
- Iniciar y cerrar sesión; comprobar la cookie y el CSRF en operaciones protegidas.
- Crear, editar y eliminar una reseña.
- Probar recuperación de contraseña y los enlaces contruidos con FRONTEND_URL.
- Confirmar que una ruta inexistente de la API devuelve JSON y que una ruta inexistente del frontend muestra el 404 de Vue.
- Confirmar que storage/logs puede escribir y que los detalles técnicos no aparecen en la interfaz.

32.12 Atribución y condiciones de TMDB

TMDB indica en su documentación oficial que la Developer API puede utilizarse sin costo para usos no comerciales siempre que se atribuya a TMDB como fuente de los datos y/o imágenes. También solicita identificar el uso de la API con su logo aprobado y mostrar el aviso de atribución correspondiente en una sección tipo About o Credits. Si CineVerse se publica, deben revisarse los términos vigentes de TMDB en ese momento, especialmente si el proyecto pasa a tener finalidad comercial.

Referencia oficial: [TMDB FAQ -atribución y uso de la API](#)

33. Pruebas y verificación del proyecto

CineVerse se verifica en varias capas porque ninguna prueba aislada demuestra por sí sola que todo el sistema funcione. Una acción puede verse correctamente en Vue y, sin embargo, no haberse guardado en MySQL; un archivo PHP puede tener sintaxis válida y aun así fallar al conectarse a la base; y un build correcto no demuestra que la autenticación responda como se espera. Por eso esta sección combina pruebas funcionales, pruebas directas de la API, análisis del código y verificación de datos persistidos.

Tipo de comprobación	Qué permite demostrar
Prueba funcional	Comprobar manualmente, desde la interfaz, que una función de CineVerse responde correctamente cuando el usuario la utiliza. Por ejemplo, iniciar sesión y verificar que la aplicación reconoce al usuario autenticado.
Prueba de API con Postman	Comprobar que el backend PHP recibe solicitudes y devuelve las respuestas esperadas sin depender de la interfaz Vue.
Análisis estático	Comprobar que los archivos PHP tienen sintaxis válida y que el código TypeScript respeta los tipos declarados.
Compilación	Comprobar que Vite puede generar correctamente los archivos de producción del frontend.
Verificación en MySQL	Comprobar que una operación realizada desde la aplicación produjo realmente el cambio esperado en la base de datos.

Tipo de comprobación	Qué permite demostrar
Comprobación de seguridad	Comprobar que la sesión, la protección CSRF y los validadores rechazan solicitudes que no cumplen las reglas previstas.
Pruebas automatizadas	Ejecutar automáticamente un conjunto de pruebas cada vez que sea necesario para comprobar que las funciones del proyecto siguen comportándose correctamente después de modificar el código. CineVerse no incluye actualmente una suite completa de este tipo.

33.1 Pruebas funcionales

Las pruebas funcionales permiten comprobar manualmente que las principales funciones de CineVerse se comportan como se espera cuando se utilizan desde la interfaz. En este tipo de comprobación interesa el resultado que observa el usuario: que pueda iniciar sesión correctamente, agregar o eliminar una película de favoritos, consultar información de una película o crear, modificar y eliminar una reseña.

Estas pruebas se realizan utilizando la aplicación de la misma forma que lo haría un usuario. Por ejemplo, después de iniciar sesión se comprueba que CineVerse reconozca la cuenta autenticada; al agregar una película a favoritos se verifica que el cambio aparezca en la interfaz y se conserve después de recargar la página; y al publicar una reseña se comprueba que esta aparezca asociada a la película correspondiente.

Las pruebas funcionales descritas en esta sección son comprobaciones manuales. No forman parte de una suite automatizada y, por sí solas, no permiten volver a ejecutar automáticamente las mismas comprobaciones después de cada modificación del código. Por ese motivo se complementan con las pruebas directas de la API mediante Postman, el análisis estático, la comprobación del build y las consultas de control sobre MySQL.

33.2 Pruebas manuales de la API con Postman

Postman es una herramienta para enviar solicitudes HTTP directamente a una API y observar su respuesta. En CineVerse permite probar el backend PHP sin pasar por Vue: se elige el método, se escribe la URL, se define el JSON o las cabeceras necesarias y se inspeccionan el estado HTTP, las cookies y el cuerpo JSON devuelto.

Por qué es útil

Si una operación funciona en Postman pero falla desde Vue, el problema probablemente está en la integración del frontend. Si también falla en Postman, puede investigarse directamente el Router, los middleware, controllers, services, modelos o la infraestructura del backend.

33.2.1 Login: comprobar autenticación y respuesta del backend

La primera prueba envía POST /login con una cuenta preparada para el entorno de desarrollo. Una respuesta correcta contiene code = LOGIN_SUCCESS, el objeto user y csrf_token. Postman también conserva la cookie de sesión que PHP envía en la respuesta. Los valores visibles en la captura pertenecen al conjunto de datos de prueba utilizado para documentar la API.

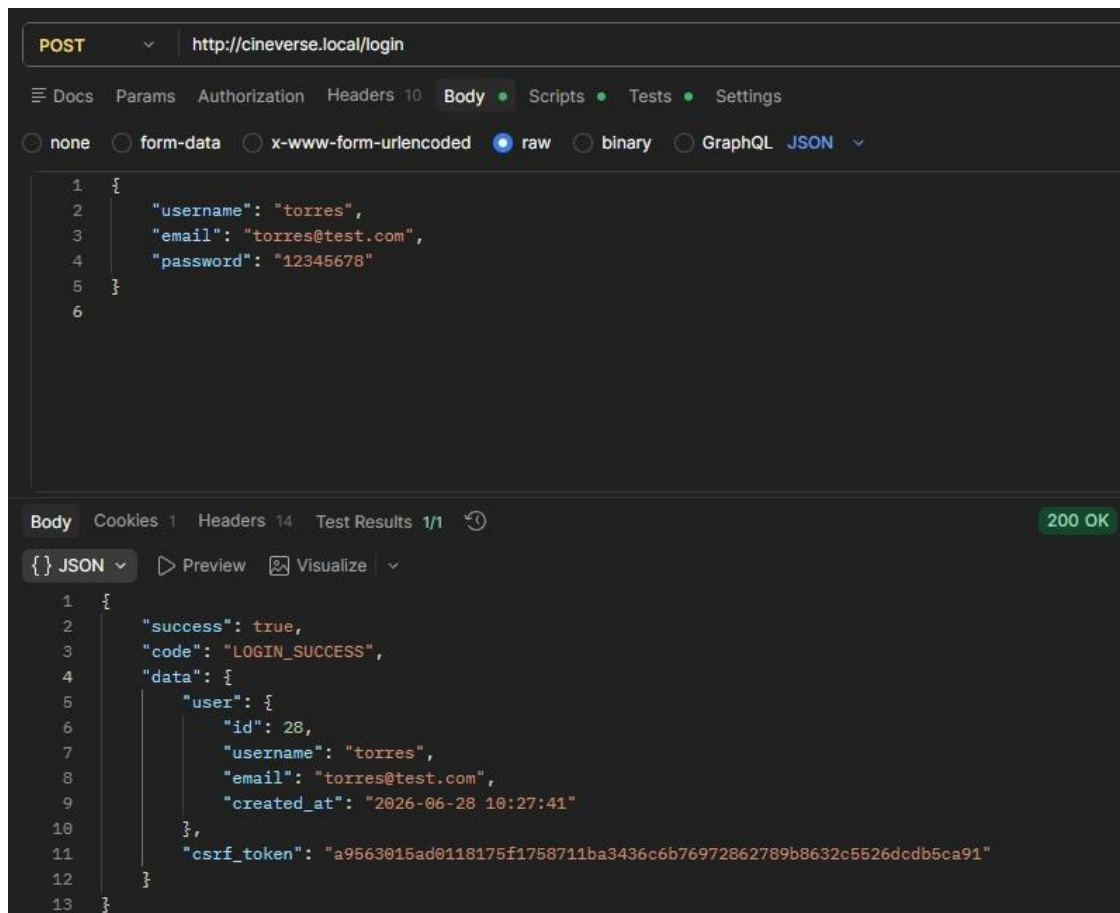


Figura. Prueba de POST /login en Postman: respuesta 200 OK y LOGIN_SUCCESS.

Qué demuestra esta prueba: que la ruta POST /login llega al backend, que las credenciales pueden validarse contra MySQL, que PHP crea la sesión y que el backend devuelve el usuario junto con el token CSRF que utilizarán las operaciones protegidas.

33.2.2 GET /user: comprobar que la sesión continúa activa

Después del login, GET /user permite verificar que una segunda solicitud sigue asociada al mismo usuario. Postman reutiliza la cookie de sesión almacenada y el backend responde AUTHENTICATED_USER. Esta prueba demuestra que la autenticación no depende de que Pinia conserve el usuario: la sesión existe en PHP y puede reconstruirse mediante el endpoint.

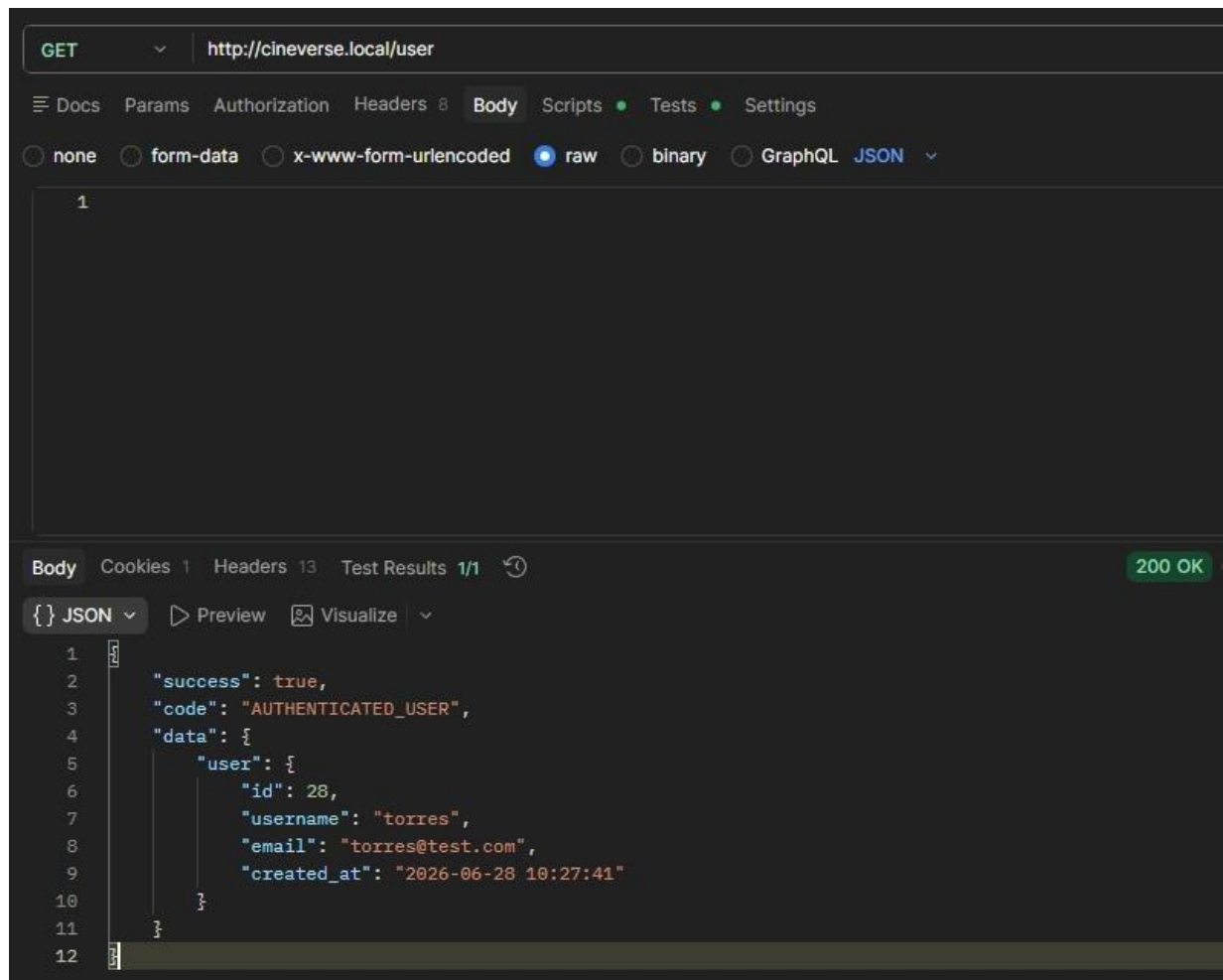


Figura. GET /user después del login: el backend reconoce la sesión existente.

Qué demuestra esta prueba: que la sesión PHP sobrevive entre solicitudes independientes y que GET /user puede reconstruir el usuario autenticado sin depender del estado que Pinia tuviera previamente en memoria.

33.2.3 Reseñas: estadísticas, paginación y contenido

La solicitud GET /movies/550/reviews?page=1 prueba un endpoint público. La respuesta incluye stats, pagination, user_review y reviews. page indica la página recibida y has_next informa si existen páginas posteriores. user_review puede ser null cuando la sesión actual no tiene una reseña propia para esa película.

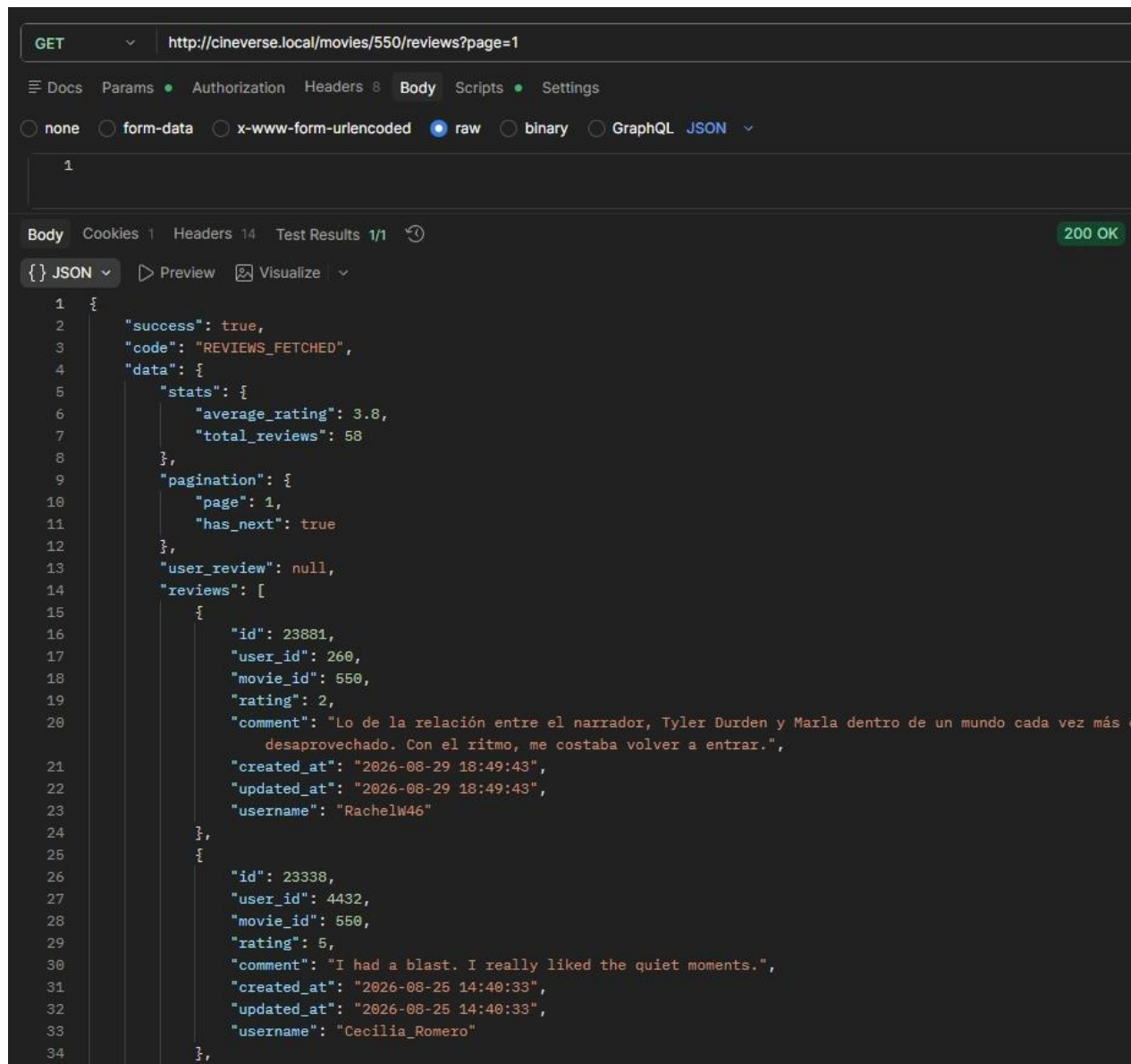


Figura. GET /movies/550/reviews?page=1: e estadísticas, paginación y primera página de reseñas.

Qué demuestra esta prueba: que un endpoint público puede devolver estadísticas, paginación y reseñas directamente desde el backend y MySQL, y que `user_review` se adapta a la sesión disponible sin convertir la ruta en un endpoint protegido.

33.2.4 Logout protegido con CSRF

Las operaciones que modifican estado requieren algo más que la cookie de sesión. En POST /logout se envía X-CSRF-Token con el valor obtenido durante la autenticación. Si la sesión y el token son válidos, el backend responde LOGOUT_SUCCESS. Omitir o alterar el token debe producir HTTP 403 + INVALID_CSRF_TOKEN.

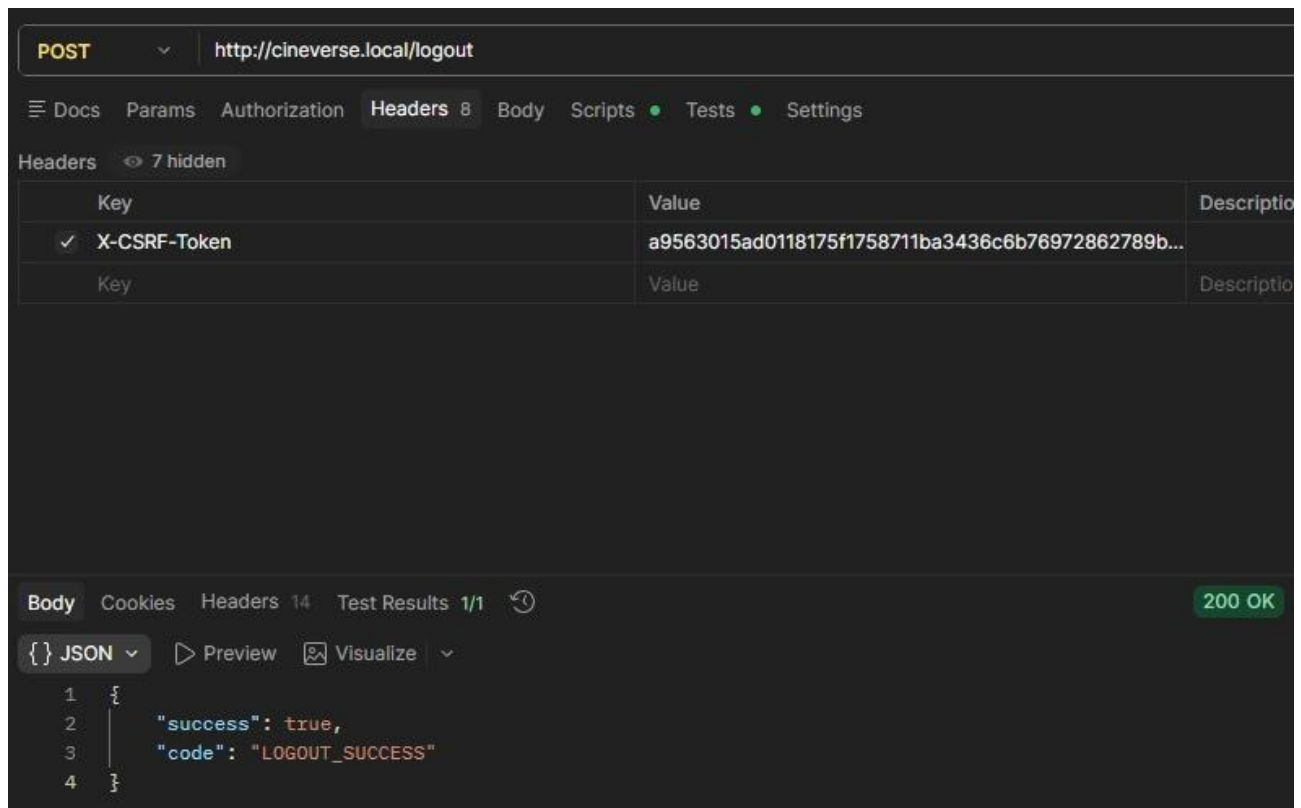


Figura. POST /logout protegido mediante la cabecera X-CSRF-Token.

Qué demuestra esta prueba: que una operación que modifica estado necesita dos elementos diferentes, la cookie de sesión y el X-CSRF-Token. Si el token no coincide con el almacenado en la sesión, `CsrfMiddleware` debe cortar la petición antes de ejecutar `logout`.

33.2.5 Pruebas negativas: enviar datos inválidos

Postman también permite demostrar que la API rechaza entradas incorrectas. Por ejemplo, enviar `GET /movies/550/reviews?page=a` debe devolver un error controlado con `code = MUST_BE_INTEGER`. Este tipo de prueba verifica que los validadores actúan antes de construir una consulta SQL con un valor inválido.

```
GET /movies/550/reviews?page=a
```

```

{
  "success": false,
  "code": "MUST_BE_INTEGER"
}

```

33.2.6 Probar TMDB de forma independiente

Una prueba útil para separar problemas del frontend, del backend propio y del servicio externo consiste en invocar directamente TMDB con el mismo tipo de credencial que utiliza CineVerse. En Postman puede elegirse `Authorization > Bearer Token` y enviar una solicitud de detalle.

```

GET https://api.themoviedb.org/3/movie/550?language=es-ES
Authorization: Bearer <API Read Access Token>
Accept: application/json

```

Un HTTP 200 con JSON confirma que la credencial y el acceso a TMDB funcionan fuera de CineVerse. Si esta prueba funciona pero la pantalla Vue falla, conviene revisar `tmdbApi.ts`, el `service`, el `composable` y el manejo del resultado. Si falla también en Postman, el problema está antes de Vue: credencial, conectividad, parámetros o disponibilidad del servicio externo.

33.3 Verificaciones automáticas del código

Estas verificaciones no ejecutan un recorrido funcional completo, pero detectan problemas importantes antes de probar la aplicación manualmente.

33.3.1 Sintaxis PHP

php -l analiza un archivo PHP sin ejecutar la aplicación. Detecta llaves mal cerradas, errores de sintaxis e instrucciones inválidas. No comprueba que una consulta SQL sea correcta ni que MySQL esté disponible.

```
php -l public/index.php
php -l controllers/AuthController.php
# repetir para los archivos PHP propios
```

En la revisión realizada sobre el código entregado, los archivos PHP propios pasan esta comprobación sin errores de sintaxis.

33.3.2 Tipos de TypeScript

vue-tsc aplica las comprobaciones de TypeScript también sobre los componentes .vue. Permite detectar incompatibilidades de tipos, propiedades inexistentes y otros errores estáticos antes de abrir la aplicación en el navegador.

```
cd frontend
node node_modules/vue-tsc/bin/vue-tsc.js --noEmit
```

La revisión del frontend actual completa esta comprobación correctamente. El resultado confirma coherencia estática, pero no valida en tiempo de ejecución que TMDB o el backend hayan enviado exactamente la estructura declarada.

33.3.3 Compilación del frontend para producción

npm run build comprueba que Vite pueda generar la versión publicable del frontend. Para que la prueba sea reproducible conviene instalar las dependencias desde package.json y package-lock.json en el sistema donde se ejecuta el build, en lugar de reutilizar un node_modules creado en otro sistema operativo.

```
cd frontend
npm install
npm run build
```

33.4 Pruebas completas por funcionalidad

Las siguientes pruebas combinan varias capas y permiten observar no sólo la respuesta inmediata, sino también sus efectos posteriores.

33.4.1 Favoritos y límite de 20

- Guardar una película, comprobar que el contador aumenta y recargar F5 para verificar que siga presente.
- Eliminar el favorito, volver a recargar y confirmar que no se restaure.
- Guardar 20 películas e intentar agregar una número 21; el store debe respetar MAX_FAVORITES y la interfaz debe informar el límite según el componente.

33.4.2 Crear, editar, eliminar y paginar reseñas

- Crear una reseña desde la pestaña Reseñas y confirmar que cambien user_review, el promedio y el total.
- Editar rating y comment y verificar que movie_id y movie_title permanezcan sin cambios.
- Eliminar la reseña y confirmar que la interfaz vuelva a permitir Escribir reseña.
- Utilizar una película con más de una página de reseñas, cargar la siguiente y confirmar que se concatene sin duplicar elementos.

33.4.3 Sesión y CSRF

- Iniciar sesión, recargar la SPA y confirmar que GET /user reconstruya user y csrfToken a partir de la cookie.
- Cerrar sesión y confirmar que una ruta requiresAuth vuelva a redirigir a Login.

- Repetir una escritura protegida sin X-CSRF-Token y verificar HTTP 403 + INVALID_CSRF_TOKEN.

33.4.4 Correo, TMDb y logs

- Registrar una cuenta con SMTP operativo y comprobar idioma, enlace y recepción del correo de verificación.
- Probar una configuración SMTP inválida y confirmar que el frontend reciba un código seguro mientras el detalle técnico queda en logs.
- Buscar una película, cambiar ES/EN y confirmar que la solicitud a TMDb cambia language.
- Provocar un error de infraestructura controlado y verificar el recorrido explicado en la sección 29: AppLogger debe registrar el detalle técnico en el archivo diario mientras la respuesta HTTP evita exponer ese detalle al usuario.

33.5 Verificar en MySQL una reseña creada desde la interfaz

Ver una reseña en pantalla no demuestra por sí solo que quedó persistida. Después de crearla desde la interfaz se puede ejecutar una consulta directa sobre reviews para comprobar que el backend la guardó realmente y que los campos coinciden con lo mostrado.

```
USE cineverse;

SELECT
    id, user_id, movie_id, movie_title,
    rating, comment, created_at, updated_at
FROM reviews
WHERE user_id = <id_usuario>
    AND movie_id = <id_pelicula>;
```

El resultado debe corresponder con la reseña visible. Esta prueba conecta de forma explícita MovieReviews -> POST /reviews -> ReviewController -> ReviewService -> Review::create() -> PDO -> MySQL.

33.6 Estado actual de las pruebas automatizadas

El proyecto no incluye actualmente una suite completa de pruebas automatizadas. Las verificaciones anteriores combinan análisis estático, comprobación del build, pruebas manuales desde la interfaz, pruebas directas de la API con Postman y consultas de control sobre MySQL.

Para incorporar pruebas automatizadas al backend PHP podría utilizarse **PHPUnit**, una herramienta diseñada para escribir y ejecutar pruebas sobre código PHP. Actualmente CineVerse no incluye PHPUnit en backend/composer.json, por lo que sería necesario instalarlo como dependencia de desarrollo:

```
composer require --dev phpunit/phpunit
```

Composer agregaría PHPUnit a las dependencias de desarrollo del backend. La versión de PHPUnit debe elegirse de acuerdo con la versión de PHP utilizada por el entorno de desarrollo. Una vez instalado, podría ejecutarse desde vendor/bin/phpunit.

Una estructura posible sería crear una carpeta dedicada a las pruebas:

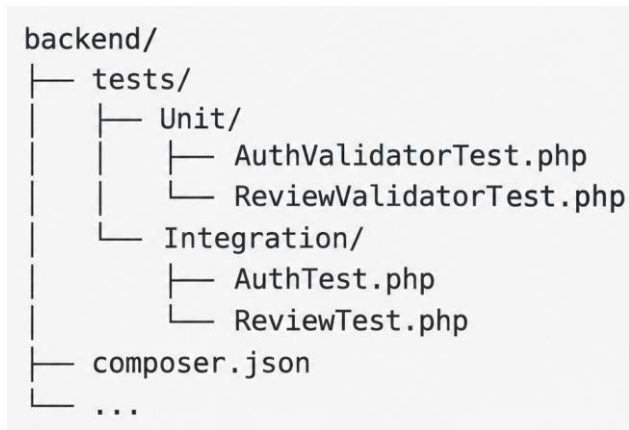


Figura. Estructura propuesta para las pruebas automatizadas del backend.

En esta propuesta, la carpeta tests se divide en Unit e Integration. Unit contendría pruebas destinadas a comprobar componentes concretos de forma aislada, como los validadores, mientras que Integration reuniría pruebas que verifican el funcionamiento conjunto de varias capas del backend. Esta estructura corresponde a una posible incorporación futura de PHPUnit y no forma parte actualmente del proyecto CineVerse.

Las pruebas unitarias comprobarían piezas concretas del código de forma aislada. Por ejemplo, podría probarse que AuthValidator rechace una dirección de correo inválida o que ReviewValidator no acepte una puntuación fuera del rango permitido.

Un ejemplo simplificado con PHPUnit sería:

```

<?php

use InvalidArgumentException;
use PHPUnit\Framework\TestCase;
use App\Validators\ReviewValidator;

final class ReviewValidatorTest extends TestCase
{
    public function test_rechaza_rating_fuera_de_rango(): void
    {
        $this->expectException(InvalidArgumentException::class);
        $this->expectExceptionMessage("INVALID_RATING");

        ReviewValidator::validateCreate(
            550,
            "Fight Club",
            11,
            "Comentario de prueba suficientemente largo"
        );
    }
}
  
```

La prueba indica a PHPUnit que espera una `InvalidArgumentException` con el mensaje `INVALID_RATING`. Después ejecuta `ReviewValidator::validateCreate()` con un rating de 11. Si el validador aceptara ese valor o produjera un resultado diferente al esperado, PHPUnit marcaría la prueba como fallida.

Las pruebas podrían ejecutarse desde la carpeta backend con:

```
vendor/bin/phpunit tests
```

Con ese comando, PHPUnit recorrería la carpeta tests y ejecutaría los archivos de prueba que encuentre allí, mostrando cuáles pasaron y cuáles fallaron.

También podrían incorporarse pruebas de integración, que comprueban varias capas trabajando juntas. Por ejemplo, una prueba del inicio de sesión podría preparar un usuario de prueba, ejecutar el flujo de autenticación y verificar que una contraseña incorrecta produzca el resultado esperado.

Conceptualmente:

```

Preparar usuario de prueba
↓
ejecutar login con contraseña incorrecta
↓
AuthController
↓
AuthService
↓
User / MySQL
↓
comprobar INVALID_CREDENTIALS

```

En el frontend podrían añadirse herramientas específicas para Vue y TypeScript, como **Vitest** para pruebas unitarias y de componentes. Por ejemplo, podría comprobarse automáticamente que una función del explorador seleccione correctamente el camino top_rated, discover o búsqueda por texto según los filtros recibidos.

Por lo tanto, como mejora futura sería conveniente incorporar:

- PHPUnit en el backend para validadores, servicios y pruebas de integración.
- Vitest en el frontend para funciones, composables y lógica de Vue.
- Pruebas específicas de los recorridos principales: autenticación, reseñas y explorador de películas.

Estas herramientas permitirían ejecutar automáticamente las mismas comprobaciones después de modificar el código y detectar cuándo un cambio hace que una función que antes funcionaba correctamente deje de hacerlo.

Alcance

Postman, php -l, vue-tsc y npm run build comprueban problemas diferentes. Sin embargo, no reemplazan un conjunto de pruebas automatizadas capaz de verificar, después de cada cambio, que las funciones existentes continúen funcionando correctamente.